Learn Being Forward

COURSES · CAMPUS PARTNERSHIPS · PLACEMENTS

STUDY NOTES · BASIC TO ADVANCED

Robotics Fundamentals
Sense, Think, Act: Build Real Robots

Complete notes for the Learn Being Forward Robotics Fundamentals syllabus: electricity and electronics, components, tools, mechanics, programming, microcontrollers, sensors, motors and drivers, kinematics, PID control, mobile robots, communication, robotic arms, vision, navigation, ROS 2, simulation, AI, drones, IoT, CAD, industrial automation and careers, explained with diagrams and tested programs, followed by basic, intermediate and advanced questions with fully worked solutions.

26	3	54	650
modules	levels	minimum days	questions and programs

How to use these notes

These notes follow the **Learn Being Forward Robotics Fundamentals syllabus** exactly: the same 26 modules, in the same order, with every topic and subtopic explained step by step. Each subtopic number (for example **12.3.1**) matches the syllabus, so you can always find where you are.

Level	Modules	Minimum	Outcome
1. Beginner	1 to 7	11 days	Understand robots and their parts, apply basic electricity, electronics and mechanics, use tools safely, program in blocks and code, and work with microcontrollers
2. Intermediate	8 to 16	20 days	Use sensors, motors and drivers, build chassis and drive systems, apply PID control, build autonomous mobile robots, connect devices, build robotic arms and write reliable embedded code
3. Advanced	17 to 26	23 days	Add computer vision, navigation and path planning, use ROS 2 and simulators, apply AI, understand drones and IoT robots, design parts in CAD, learn industrial automation and complete a capstone robot
Total	1 to 26	54 days	About 9 weeks at 6 study days per week

1 · Understand
Read the concept and draw the circuit, mechanism or flowchart before building.

2 · Build
Build it on real hardware or in a simulator such as Wokwi or Tinkercad, changing one thing at a time.

3 · Practise
Solve the basic, intermediate and advanced questions and programs before looking at the solutions.

4 · Check and extend
Compare with the solutions, complete the practice checkpoint and record a short demo with notes on problems solved.

Every topic follows the same pattern: the concept in plain words, diagrams, worked calculations, annotated programs, **shortcut tricks** and safety notes. Each module then gives about 25 practice questions in three levels, **basic**, **intermediate** and **advanced**: concept and calculation questions plus programming tasks. Arduino sketches were compiled and run on a simulated Arduino core, and Python programs were run with sensors, motors and the clock simulated, so the output shown beside each program is its real output. Panels marked TYPICAL OUTPUT show representative output of tools that need real hardware. All calculations were checked by program.

HOW TO READ THE BOXES

Gold-bordered boxes hold **shortcut tricks**, worked examples and notes, red boxes warn about common mistakes and hardware dangers, and navy boxes hold the key rule to remember. Grey panels under a program show its output or a simulated run with timed hardware events.

Course plan at a glance

Module	Topic	Level	Min days
01	Introduction to robotics	Beginner	1
02	Basic electricity and electronics	Beginner	2

Module	Topic	Level	Min days
03	Electronic components	Beginner	2
04	Tools, safety and prototyping	Beginner	1
05	Mechanical basics for robots	Beginner	2
06	Programming foundations for robotics	Beginner	2
07	Microcontrollers and development boards	Beginner	1
08	Sensors	Intermediate	3
09	Actuators and motors	Intermediate	2
10	Motor drivers and power systems	Intermediate	2
11	Robot chassis, locomotion and kinematics	Intermediate	2
12	Control systems and PID	Intermediate	2
13	Building mobile robots	Intermediate	3
14	Communication between devices	Intermediate	2
15	Robotic arms and manipulators	Intermediate	2
16	Embedded programming patterns for robots	Intermediate	2
17	Robot vision and perception	Advanced	3
18	Localisation, mapping and navigation	Advanced	3
19	Robot Operating System (ROS 2)	Advanced	3
20	Robot simulation	Advanced	3
21	Artificial intelligence and machine learning in robotics	Advanced	2
22	Drones and aerial robotics	Advanced	2
23	IoT robotics and cloud connectivity	Advanced	2
24	Mechanical design, CAD and 3D printing	Advanced	2
25	Industrial robotics, automation, safety and ethics	Advanced	2
26	Competitions, capstone projects and careers	Advanced	2

Table of contents

LEVEL 1 · BEGINNER LEVEL · MODULES 1 TO 7

Module 01 · Introduction to robotics 9

1.1 What is a robot? 9
1.2 History of robotics 9
1.3 Parts of a robot 9
1.4 Types of robots 9
1.5 Why do we need robotics? 9
1.6 Where is robotics used? 9
1.7 Robotics as a multidisciplinary field 9
1.8 Practice questions 9
1.9 Solutions 9

Module 02 · Basic electricity and electronics 21

2.1 Electrical quantities 21
2.2 Laws 21
2.3 Circuits 21
2.4 AC vs DC 21
2.5 Reading circuit diagrams 21
2.6 Digital vs analog signals 21
2.7 Practice questions 21
2.8 Solutions 21

Module 03 · Electronic components 32

3.1 Passive components 32
3.2 Semiconductors 32
3.3 Switches and relays 32
3.4 Integrated circuits 32
3.5 Datasheets 32
3.6 Practice questions 32
3.7 Solutions 32

Module 04 · Tools, safety and prototyping 45

4.1 Lab safety 45
4.2 Prototyping 45
4.3 Measurement tools 45
4.4 Hand tools 45
4.5 Soldering 45
4.6 Online circuit simulators 45
4.7 Practice questions 45
4.8 Solutions 45

Module 05 · Mechanical basics for robots 56

5.1 Forces and motion 56
5.2 Torque and power 56
5.3 Simple machines 56
5.4 Mechanisms 56
5.5 Structure and materials 56

5.6 Degrees of freedom (introduction) 56
5.7 Practice questions 56
5.8 Solutions 56

Module 06 · Programming foundations for robotics 67

6.1 Computational thinking 67
6.2 Block-based programming 67
6.3 Text programming basics (C/C++ and Python) 67
6.4 Programming for hardware 67
6.5 Practice questions 67
6.6 Solutions 67

Module 07 · Microcontrollers and development boards 80

7.1 Microcontroller basics 80
7.2 Popular boards 80
7.3 Pins and interfaces 80
7.4 Choosing the right controller for a robot 80
7.5 First program: blinking an LED 80
7.6 Practice questions 80
7.7 Solutions 80

LEVEL 2 · INTERMEDIATE LEVEL · MODULES 8 TO 16

Module 08 · Sensors 92

8.1 Sensor basics 92
8.2 Distance and obstacle sensors 92
8.3 Line and colour sensors 92
8.4 Motion and orientation sensors 92
8.5 Environmental sensors 92
8.6 Touch, force and position sensors 92
8.7 Filtering sensor data 92
8.8 Practice questions 92
8.9 Solutions 92

Module 09 · Actuators and motors 107

9.1 DC motors 107
9.2 Servo motors 107
9.3 Stepper motors 107
9.4 Brushless DC (BLDC) motors and ESCs 107
9.5 Other actuators 107
9.6 Choosing motors 107
9.7 Practice questions 107
9.8 Solutions 107

Module 10 · Motor drivers and power systems 119

10.1 Why motor drivers 119
10.2 H-bridge 119
10.3 Stepper drivers 119
10.4 Power sources 119
10.5 Power distribution 119
10.6 Protection 119
10.7 Practice questions 119

10.8 Solutions 119

Module 11 · Robot chassis, locomotion and kinematics 132

11.1 Chassis design 132
11.2 Drive systems 132
11.3 Legged and other locomotion (overview) 132
11.4 Kinematics of a differential-drive robot 132
11.5 Odometry with wheel encoders 132
11.6 Practice questions 132
11.7 Solutions 132

Module 12 · Control systems and PID 143

12.1 Control basics 143
12.2 On-off (bang-bang) control 143
12.3 PID control 143
12.4 Applications 143
12.5 Practice questions 143
12.6 Solutions 143

Module 13 · Building mobile robots 156

13.1 Line follower robot 156
13.2 Obstacle-avoiding robot 156
13.3 Edge-avoiding (table-top) robot 156
13.4 Light-following robot 156
13.5 Bluetooth-controlled robot 156
13.6 Maze-solving robot 156
13.7 Testing and debugging robots 156
13.8 Practice questions 156
13.9 Solutions 156

Module 14 · Communication between devices 172

14.1 Wired communication protocols 172
14.2 Wireless communication 172
14.3 Communication between robots 172
14.4 Data formats 172
14.5 Practice questions 172
14.6 Solutions 172

Module 15 · Robotic arms and manipulators 184

15.1 Arm anatomy 184
15.2 Arm configurations 184
15.3 End effectors 184
15.4 Kinematics 184
15.5 Trajectory planning 184
15.6 Building a servo arm 184
15.7 Practice questions 184
15.8 Solutions 184

Module 16 · Embedded programming patterns for robots 200

16.1 Non-blocking code 200
16.2 Interrupts 200
16.3 State machines 200
16.4 Code organisation 200

16.5 Real-time operating systems (overview) 200
16.6 Debugging embedded systems 200
16.7 Practice questions 200
16.8 Solutions 200

LEVEL 3 · ADVANCED LEVEL · MODULES 17 TO 26

Module 17 · Robot vision and perception 217
17.1 Cameras for robots 217
17.2 Image basics 217
17.3 Computer vision with OpenCV 217
17.4 Vision-based robot behaviours 217
17.5 AI vision models (overview) 217
17.6 Practice questions 217
17.7 Solutions 217

Module 18 · Localisation, mapping and navigation 232
18.1 Localisation 232
18.2 Mapping 232
18.3 Path planning 232
18.4 Navigation behaviours 232
18.5 Practice questions 232
18.6 Solutions 232

Module 19 · Robot Operating System (ROS 2) 249
19.1 Introduction to ROS 2 249
19.2 Core concepts 249
19.3 Tools 249
19.4 Writing ROS 2 programs 249
19.5 Robot description 249
19.6 Navigation and manipulation stacks (overview) 249
19.7 Microcontrollers with ROS 2 249
19.8 Practice questions 249
19.9 Solutions 249

Module 20 · Robot simulation 269
20.1 Why simulate 269
20.2 Circuit simulators 269
20.3 Robot simulators 269
20.4 Simulating robots 269
20.5 Simulation vs reality gap 269
20.6 Practice questions 269
20.7 Solutions 269

Module 21 · Artificial intelligence and machine learning in robotics 283
21.1 AI concepts for robotics 283
21.2 Machine learning on robots 283
21.3 Edge AI 283
21.4 Reinforcement learning (overview) 283
21.5 Voice and language interfaces 283
21.6 Practice questions 283
21.7 Solutions 283

Module 22 · Drones and aerial robotics 298
22.1 Drone basics 298
22.2 Drone components 298
22.3 Flight control 298
22.4 Safety and regulations 298
22.5 Applications 298
22.6 Practice questions 298
22.7 Solutions 298
Module 23 · IoT robotics and cloud connectivity 312
23.1 Internet of Things concepts 312
23.2 IoT protocols 312
23.3 IoT platforms 312
23.4 Remote robot control 312
23.5 Security for connected robots 312
23.6 Practice questions 312
23.7 Solutions 312
Module 24 · Mechanical design, CAD and 3D printing 327
24.1 Design process 327
24.2 CAD software 327
24.3 3D printing 327
24.4 Other fabrication methods 327
24.5 PCB design (overview) 327
24.6 Practice questions 327
24.7 Solutions 327
Module 25 · Industrial robotics, automation, safety and ethics 342
25.1 Industrial robots 342
25.2 Industrial automation 342
25.3 Robot safety 342
25.4 Ethics and society 342
25.5 Practice questions 342
25.6 Solutions 342
Module 26 · Competitions, capstone projects and careers 356
26.1 Robotics competitions 356
26.2 Capstone projects 356
26.3 Project documentation 356
26.4 Careers in robotics 356
26.5 Next steps 356
26.6 Practice questions 356
26.7 Solutions 356

LEVEL 1

Beginner level

Understand robots and their parts, apply basic electricity, electronics and mechanics, use tools safely, program in blocks and code, and work with microcontrollers

7 MODULES · 11 DAYS MINIMUM

Module 01 · Introduction to robotics	**1 day**
Module 02 · Basic electricity and electronics	**2 days**
Module 03 · Electronic components	**2 days**
Module 04 · Tools, safety and prototyping	**1 day**
Module 05 · Mechanical basics for robots	**2 days**
Module 06 · Programming foundations for robotics	**2 days**
Module 07 · Microcontrollers and development boards	**1 day**

MODULE 01 · BEGINNER

Introduction to robotics

Minimum duration **1 day** | Level **Beginner**

This module explains what robots are, how they work and where they are used, before any building begins: the definition and the sense–think–act cycle, robots versus simple machines and automation, the history from automatons to AI humanoids, the six parts of every robot, the main types of robots, why we need them, where they are used, and why robotics combines mechanical, electrical, electronic and computer engineering, followed by 25 solved questions.

IN THIS MODULE

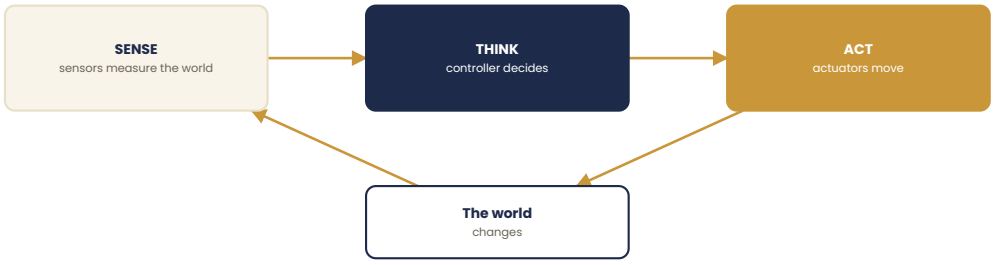
1.1 What is a robot?
1.2 History of robotics
1.3 Parts of a robot
1.4 Types of robots
1.5 Why do we need robotics?
1.6 Where is robotics used?
1.7 Robotics as a multidisciplinary field
1.8 Practice questions
1.9 Solutions

1.1 What is a robot?

1.1.1 Definition: a programmable machine that senses, thinks (processes) and acts in the physical world

A **robot** is a programmable machine that **senses** its surroundings, **thinks** (processes that information and decides) and **acts** on the physical world, often without a person controlling every step. A robot vacuum feels a wall with its bumper (sense), its microcontroller decides to turn (think) and its wheel motors turn it (act).

1.1.2 The sense–think–act cycle



THE SENSE–THINK–ACT LOOP RUNS CONTINUOUSLY, MANY TIMES PER SECOND

The cycle repeats continuously, typically 10 to 1000 times per second. Because each action changes the world and the next reading shows the result, the robot can correct itself: this is **feedback**. A simulated

robot driving towards a wall shows the loop at work:

SENSE_THINK_ACT.PY

```
distance = 1.20           # metres to the wall (the "world")
speed = 0.0
for step in range(8):
    reading = distance                        # SENSE
    if reading > 0.65:                        # THINK
        speed = 0.20
        decision = "cruise"
    elif reading > 0.25:
        speed = 0.08
        decision = "slow down"
    else:
        speed = 0.0
        decision = "stop"
    distance -= speed * 1.0                   # ACT: drive for 1 second
    print(f"step {step}: sensed {reading:.2f} m -> {decision:9} -> now {distance:.2f} m")
```

OUTPUT

```
step 0: sensed 1.20 m -> cruise    -> now 1.00 m
step 1: sensed 1.00 m -> cruise    -> now 0.80 m
step 2: sensed 0.80 m -> cruise    -> now 0.60 m
step 3: sensed 0.60 m -> slow down -> now 0.52 m
step 4: sensed 0.52 m -> slow down -> now 0.44 m
step 5: sensed 0.44 m -> slow down -> now 0.36 m
step 6: sensed 0.36 m -> slow down -> now 0.28 m
step 7: sensed 0.28 m -> slow down -> now 0.20 m
```

1.1.3 Robots vs simple machines vs automation

	Simple machine	Automation	Robot
Example	lever, pulley, bicycle	washing machine cycle, traffic light timer, conveyor	robot vacuum, welding arm, Mars rover
Senses?	no	sometimes, a little	yes
Decides?	no	follows a fixed sequence	yes, reacts to what it senses
Reprogrammable for new tasks?	no	limited	yes

SHORTCUT TRICK

Ask three questions: **Does it sense? Does it decide? Can it be reprogrammed?** Three yeses make it a robot. A toaster with a timer is automation; a robot lawn mower that finds the edge of the grass is a robot.

1.2 History of robotics

1.2.1 Early automatons

Long before electronics, inventors built **automatons**: self-operating mechanical figures driven by water, weights, cams and springs. Al-Jazari's water-powered musicians (1206), Leonardo da Vinci's mechanical knight (about 1495) and Jacquet-Droz's writing boy (1770s), which wrote text programmed by a wheel of cams, all followed a fixed sequence.

1.2.2 The word "robot" (Karel Čapek, 1920) and Asimov's Three Laws

The word **robot** comes from the Czech *robota* (forced labour) and was introduced in Karel Čapek's play **R.U.R.** (1920). Science-fiction writer Isaac Asimov later proposed the **Three Laws of Robotics** (1942), still a starting point for discussing robot ethics:

1. A robot may not injure a human being or, through inaction, allow a human being to come to harm.
2. A robot must obey orders given by human beings, except where such orders conflict with the First Law.
3. A robot must protect its own existence as long as this does not conflict with the First or Second Law.

1.2.3 Unimate, the first industrial robot (1961)

Unimate, designed by George Devol and Joseph Engelberger, began work at a General Motors plant in 1961, lifting hot die-cast parts: dangerous work for people. It began the industrial robot era; today several million industrial robots work in factories worldwide.

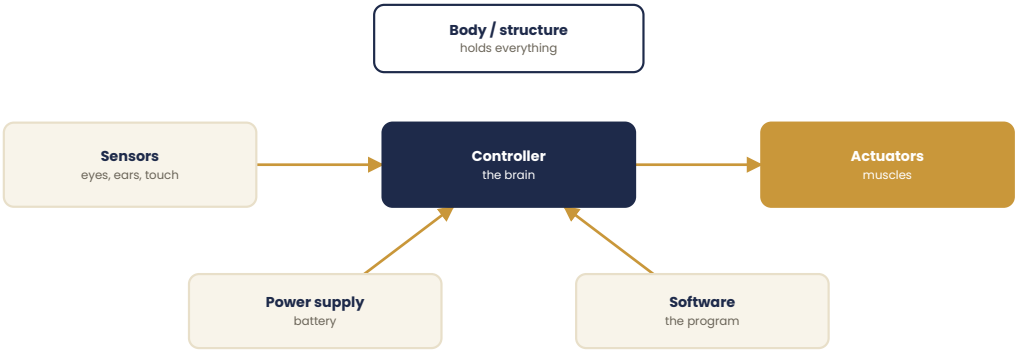
1.2.4 Mobile robots, humanoids and Mars rovers

Year	Milestone
1966–72	Shakey (SRI): first mobile robot to reason about its actions, with a camera and range finder
1997	Sojourner, the first rover on Mars
2000	Honda ASIMO walking humanoid
2002	iRobot Roomba brings robots into homes
2004–12	Spirit, Opportunity and Curiosity explore Mars; Opportunity drove over 45 km
2005	DARPA Grand Challenge: self-driving cars complete a desert course
2021	Perseverance rover and the Ingenuity helicopter on Mars

1.2.5 Modern AI-powered robots and humanoids

Today robots combine cheap powerful computers, cameras, LiDAR and **AI**: warehouse robots move millions of shelves, quadrupeds inspect factories, and companies are developing general-purpose humanoids that learn tasks from demonstrations and language instructions. Robotics and AI are converging rapidly, making the field one of the fastest-growing in engineering.

1.3 Parts of a robot



THE SIX PARTS OF EVERY ROBOT

Part	Job	Examples
1.3.1 Body and mechanical structure	holds the parts together and gives shape and strength	chassis, arm links, frame, wheels, grippers
1.3.2 Sensors	measure the robot and its surroundings	ultrasonic, IR, camera, encoder, IMU, touch switch
1.3.3 Controller (the brain)	runs the program: reads sensors, decides, commands actuators	Arduino, ESP32, Raspberry Pi, Pico
1.3.4 Actuators	create motion or action	DC motor, servo, stepper, pump, solenoid
1.3.5 Power supply	supplies energy	Li-ion battery, AA cells, adapter, regulators
1.3.6 Software	the instructions: behaviour, control, intelligence	Arduino sketch, Python program, ROS 2 nodes

1.4 Types of robots

Type	How it works	Examples
1.4.1 Mobile robots: wheeled, tracked, legged	move through their environment	delivery robots, tanks for rough ground, quadrupeds
1.4.2 Robotic arms and manipulators	fixed base, jointed arm with a tool (end effector)	welding, painting, pick-and-place
1.4.3 Aerial robots (drones)	propellers generate lift	survey, agriculture spraying, delivery
1.4.4 Underwater robots	ROVs (tethered) and AUVs (autonomous)	pipeline inspection, ocean research
1.4.5 Humanoids	human-like body with arms and legs	research, service, future general-purpose work

Type	How it works	Examples
1.4.6 Collaborative robots (cobots)	work safely beside people, stop on contact	assembly assistants, lab automation

1.4.7 Autonomous vs remote-controlled robots

A **remote-controlled** (teleoperated) robot is driven by a person: bomb-disposal robots, surgical robots and most underwater ROVs. An **autonomous** robot decides for itself from its sensors: robot vacuums, warehouse robots, Mars rovers between commands. Many robots are **semi-autonomous**: the person gives goals ("go to the kitchen") and the robot works out how.

> WORKED EXAMPLE
> **Why Mars rovers must be autonomous**
> Radio signals travel at 300 000 km/s. When Mars is 225 million km away, one signal takes 225 000 000 ÷ 300 000 = **750 s = 12.5 minutes**. A command and its result take 25 minutes round trip, so live joystick control is impossible: the rover must avoid hazards on its own.

1.5 Why do we need robotics?

1.5.1 Doing dangerous, dirty and dull work

Robots take on the "**three Ds**": **dangerous** (bomb disposal, nuclear clean-up, mines, fire), **dirty** (sewers, tank cleaning) and **dull** (repeating the same movement thousands of times a day). This protects workers and frees people for more interesting work.

1.5.2 Precision, speed and consistency

An industrial arm can repeat a position to within a few hundredths of a millimetre, work 24 hours a day and perform exactly the same weld every time. Surgical robots scale down a surgeon's hand movements and filter tremor.

1.5.3 New industries and careers

Robotics creates jobs in design, programming, maintenance, AI, testing and training, and enables new businesses: drone services, agricultural robots, automated warehouses and robotics education.

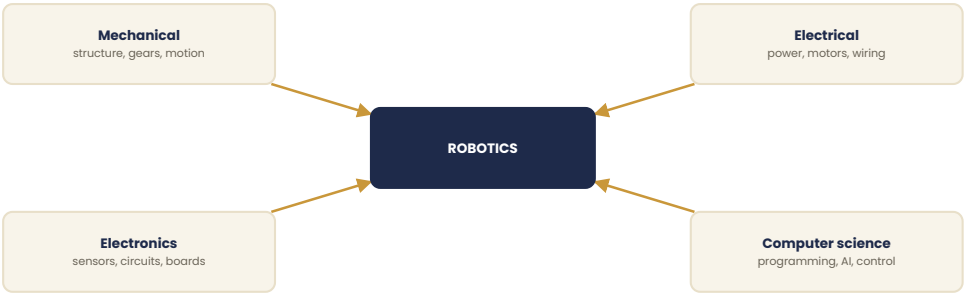
1.6 Where is robotics used?

Area	Examples
1.6.1 Manufacturing and warehouses	car assembly, welding, painting, electronics assembly; warehouse robots that bring shelves to pickers
1.6.2 Healthcare and surgery	robot-assisted surgery, rehabilitation exoskeletons, hospital delivery, pharmacy automation
1.6.3 Agriculture	automated tractors, fruit-picking, weeding robots, crop-spraying drones

Area	Examples
1.6.4 Defence, space exploration and disaster response	bomb disposal, Mars rovers, ISRO's Pragyan rover on the Moon (Chandrayaan-3, 2023), search-and-rescue robots
1.6.5 Homes and education	robot vacuums and lawn mowers, educational kits, STEM labs
1.6.6 Self-driving vehicles	cameras, radar and LiDAR with AI: robotaxis, autonomous trucks, delivery bots

1.7 Robotics as a multidisciplinary field

1.7.1 Mechanical, electrical, electronics and computer science together



ROBOTICS BRINGS SEVERAL ENGINEERING FIELDS TOGETHER

A robot fails if any one part is weak: perfect code cannot fix a motor too small to move the robot, and a strong chassis is useless with a flat battery. Robotics teams therefore combine specialists, and good roboticists understand enough of every field to see how the parts interact. Mathematics (geometry, trigonometry, calculus) and physics connect them all.

PRACTICE CHECKPOINT

Identify the sensors, controller, actuators and power source in five real robots, and draw the sense–think–act cycle for one of them.

Robot	Sensors	Controller	Actuators	Power
Robot vacuum	bumper, cliff IR, wheel encoders, LiDAR or camera	microcontroller / SoC	wheel motors, brush and fan motors	Li-ion pack
Line-following robot	IR line sensors	Arduino	2 DC motors	AA or Li-ion
Industrial arm	joint encoders, force sensor, camera	robot controller (industrial PC)	6 servo motors, gripper	mains
Mars rover	cameras, IMU, wheel encoders, science instruments	radiation-hardened computer	6 wheel motors, steering, arm	RTG or solar panels + batteries
Drone	IMU, barometer, GPS, camera	flight controller	4 brushless motors	LiPo pack

1.8 Practice questions

1.8.1 Basic level

Q1 What are the three steps of the robot cycle?

(a) see, walk, talk
(b) input, output, store
(c) sense, think, act
(d) plan, build, sell

Q2 Who introduced the word "robot"?

(a) Karel Čapek, in the play R.U.R. (1920)
(b) Isaac Asimov, in 1942
(c) George Devol, in 1961
(d) Leonardo da Vinci, in 1495

Q3 What was Unimate?

(a) the first Mars rover
(b) a robot vacuum
(c) a humanoid
(d) the first industrial robot, at General Motors in 1961

Q4 Which part of a robot is its "brain"?

(a) the battery **(b)** the controller **(c)** the chassis **(d)** the wheels

Q5 Which of these is an actuator?

(a) an ultrasonic sensor
(b) a servo motor
(c) a battery
(d) a microcontroller

Q6 Which of these is a sensor?

(a) a DC motor
(b) a gripper
(c) a solenoid
(d) a wheel encoder

Q7 What is a cobot?

(a) a collaborative robot designed to work safely beside people
(b) a robot that cooks
(c) a robot made of cobalt
(d) a remote-controlled toy

Q8 What are the "three Ds" robots are good for?

(a) design, drawing and drama
(b) data, devices and drones
(c) dangerous, dirty and dull work
(d) driving, digging and diving only

1.8.2 Intermediate level

Q9 How is a robot different from simple automation like a timer-controlled washing machine?

(a) a robot never uses motors
(b) automation always uses AI
(c) there is no difference
(d) a robot senses its surroundings and changes its actions in response

Q10 What does Asimov's First Law say?

(a) a robot must obey all orders
(b) a robot may not injure a human or, through inaction, allow a human to come to harm
(c) a robot must protect itself first
(d) robots must be made of metal

Q11 Which robot type uses propellers to fly?

(a) aerial robots (drones)
(b) cobots
(c) manipulators
(d) AUVs

Q12 What is an AUV?

(a) a type of motor driver
(b) an aerial drone
(c) an autonomous underwater vehicle
(d) a wheeled robot base

Q13 A surgeon controls every movement of a surgical robot. This robot is

(a) teleoperated (remote-controlled)
(b) fully autonomous
(c) a simple machine
(d) an automaton

Q14 Which Indian rover explored the Moon's surface in 2023?

(a) Curiosity
(b) Pragyan (Chandrayaan-3)
(c) Sojourner
(d) Opportunity

Q15 Why is robotics called multidisciplinary?

(a) it is taught in many languages
(b) robots have many colours
(c) it uses many batteries
(d) it combines mechanical, electrical, electronics and computer engineering

Q16 Which robot type has a fixed base and a jointed arm with a tool?

(a) a mobile robot
(b) a drone
(c) a manipulator (robotic arm)
(d) an AUV

1.8.3 Advanced level

Q17 Mars is 300 million km away. How long does a radio signal take one way (c = 300 000 km/s)?

(a) 100 s
(b) 10 s
(c) 1000 s (about 16.7 minutes)
(d) 1 hour

Q18 A robot loop runs 50 times per second. How long is each cycle?

(a) 50 ms **(b)** 2 ms **(c)** 0.5 s **(d)** 20 ms

Q19 A robot vacuum bumps a wall and turns. Which mapping is correct?

(a) motors = sense, bumper = act
(b) bumper = sense, controller decides = think, wheel motors turn = act
(c) battery = think
(d) wheels = think

Q20 Which robot most needs autonomy rather than remote control?

(a) a deep-space rover with long signal delays
(b) a toy RC car in a park
(c) a bomb-disposal robot 20 m away
(d) a camera crane on a film set

Q21 An arm repeats a position to ±0.02 mm. What is the total spread of positions?

(a) 0.02 mm (b) 0.04 mm (c) 0.2 mm (d) 2 mm

Q22 Simulate a robot that sees a wall at 2.0 m and moves 0.3 m per step until the sensor reads less than 0.5 m. Print each step and how many steps it took.

Q23 Classify devices as robot or not robot from three yes/no answers (senses, decides, reprogrammable) and test five devices.

Q24 Which list correctly gives a drone's parts?

(a) IMU and GPS (sensors), flight controller (brain), brushless motors (actuators), LiPo (power)
(b) propellers (sensors), GPS (actuator)
(c) battery (brain), camera (power)
(d) motors (sensors), IMU (actuators)

Q25 Why does the sense–think–act loop need feedback?

(a) to charge the battery
(b) to make the robot heavier
(c) each new reading shows the result of the last action, so the robot can correct errors
(d) feedback is not needed

1.9 Solutions

Q1 **Answer: (c) sense, think, act**
Sense → think → act, repeated continuously.

Q2 **Answer: (a) Karel Čapek, in the play R.U.R. (1920)**
From Czech *robota*, forced labour.

Q3 **Answer: (d) the first industrial robot, at General Motors in 1961**
Lifting hot die-cast parts.

Q4 **Answer: (b) the controller**
It runs the program.

Q5 **Answer: (b) a servo motor**
Actuators create motion.

Q6 **Answer: (d) a wheel encoder**

It measures wheel rotation.

Q7 **Answer: (a) a collaborative robot designed to work safely beside people**

Collaborative robot.

Q8 **Answer: (c) dangerous, dirty and dull work**

Work that harms or bores people.

Q9 **Answer: (d) a robot senses its surroundings and changes its actions in response**

Sensing + decision + reprogrammability.

Q10 **Answer: (b) a robot may not injure a human or, through inaction, allow a human to come to harm**

Human safety comes first.

Q11 **Answer: (a) aerial robots (drones)**

Drones.

Q12 **Answer: (c) an autonomous underwater vehicle**

Works underwater without a tether.

Q13 **Answer: (a) teleoperated (remote-controlled)**

A human is in control.

Q14 **Answer: (b) Pragyan (Chandrayaan-3)**

ISRO's Pragyan.

Q15 **Answer: (d) it combines mechanical, electrical, electronics and computer engineering**

All fields together.

Q16 **Answer: (c) a manipulator (robotic arm)**

The tool is the end effector.

Q17 **Answer: (c) 1000 s (about 16.7 minutes)**

300 000 000 ÷ 300 000 = **1000 s**.

Q18 **Answer: (d) 20 ms**

1 ÷ 50 = 0.02 s = **20 ms**.

Q19 **Answer: (b) bumper = sense, controller decides = think, wheel motors turn = act**

Each part has its role.

Q20 **Answer: (a) a deep-space rover with long signal delays**

Delays make live control impossible.

Q21 **Answer: (b) 0.04 mm**

±0.02 means from −0.02 to +0.02: **0.04 mm**.

Q22 **Answer: Solution program**

A while loop repeats sense–think–act until the condition fails: 200, 170 … 50 cm → 6 moves, stopping at 20 cm. Whole centimetres avoid floating-point surprises (2.0 − 5 × 0.3 is 0.4999…, not exactly 0.5).

```
d, steps = 200, 0                    # work in centimetres (whole numbers)
while d >= 50:
    print(f"step {steps}: distance {d} cm -> move")
    d -= 30
    steps += 1
print(f"stopped at {d} cm after {steps} steps")
```

OUTPUT

```
step 0: distance 200 cm -> move
step 1: distance 170 cm -> move
step 2: distance 140 cm -> move
step 3: distance 110 cm -> move
step 4: distance 80 cm -> move
step 5: distance 50 cm -> move
stopped at 20 cm after 6 steps
```

Q23 **Answer: Solution program**

Only devices that sense, decide and can be reprogrammed count as robots in this definition.

```
devices = {"robot vacuum": (1, 1, 1), "toaster": (0, 0, 0), "traffic light timer": (0, 0, 1),
           "line follower": (1, 1, 1), "automatic door": (1, 0, 0)}
for name, (s, d, r) in devices.items():
    print(f"{name:20} -> {'ROBOT' if s and d and r else 'not a robot'}")
```

OUTPUT

```
robot vacuum         -> ROBOT
toaster              -> not a robot
traffic light timer  -> not a robot
line follower        -> ROBOT
automatic door       -> not a robot
```

Q24 **Answer: (a) IMU and GPS (sensors), flight controller (brain), brushless motors (actuators), LiPo (power)**

Match each part to its role.

Q25 **Answer: (c) each new reading shows the result of the last action, so the robot can correct errors**

Closed-loop control.

A robot senses, thinks and acts in a feedback loop and can be reprogrammed. Its six parts: body, sensors, controller, actuators, power and software. Types: mobile (wheeled, tracked, legged), arms, drones, underwater, humanoids, cobots; teleoperated or autonomous. Robots do dangerous, dirty and dull work with precision, and robotics combines mechanical, electrical, electronics and computer science.

MODULE 02 · BEGINNER

Basic electricity and electronics

Minimum duration **2 days** | Level **Beginner**

Robots run on electricity, and this module builds the essential understanding: voltage, current, resistance, power and energy with their units and prefixes; Ohm's law, the power formula and Kirchhoff's laws; open, closed and short circuits, series and parallel circuits and voltage dividers; AC versus DC; schematic symbols; and digital versus analog signals, logic levels, binary numbers and logic gates, followed by 25 solved questions and programs.

IN THIS MODULE

2.1 Electrical quantities
2.2 Laws
2.3 Circuits
2.4 AC vs DC
2.5 Reading circuit diagrams
2.6 Digital vs analog signals
2.7 Practice questions
2.8 Solutions

2.1 Electrical quantities

2.1.1 Voltage, current and resistance

Quantity	Symbol	Unit	Meaning	Water analogy
Voltage	V	volt (V)	electrical "pressure" that pushes charge	water pressure
Current	I	ampere (A)	rate of flow of charge	litres per second
Resistance	R	ohm (Ω)	opposition to current	a narrow pipe

Current only flows in a **closed loop**: from the battery's + terminal through the circuit and back to its − terminal (by convention current flows from + to −; electrons actually move the other way). Voltage is always measured **between two points**, usually relative to ground (0 V, GND).

2.1.2 Power and energy

Power (P, watts) is how fast energy is used: P = V × I. **Energy** (E) is power × time: watt-hours (Wh) or joules (1 Wh = 3600 J). Battery capacity is given in milliamp-hours (mAh); multiply by voltage to get energy: a 3.7 V, 2000 mAh cell stores 3.7 × 2 = 7.4 Wh.

2.1.3 Units and prefixes (mA, kΩ, µF)

Prefix	Symbol	Factor	Example
mega	M	10^6	1 MΩ = 1 000 000 Ω

Prefix	Symbol	Factor	Example
kilo	k	10^{3}	4.7 kΩ = 4700 Ω
milli	m	10^{-3}	20 mA = 0.020 A
micro	μ	10^{-6}	100 μF = 0.0001 F
nano	n	10^{-9}	100 nF = 0.1 μF
pico	p	10^{-12}	22 pF

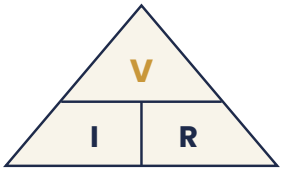
SHORTCUT TRICK

Convert to base units (V, A, Ω) **before** calculating, then convert the answer back. 5 V ÷ 220 Ω = 0.0227 A = **22.7 mA**. Resistor values are often written with the prefix as the decimal point: 4k7 = 4.7 kΩ, 2R2 = 2.2 Ω.

2.2 Laws

2.2.1 Ohm's law

$$V = I \times R \quad I = V \div R \quad R = V \div I$$



COVER THE QUANTITY YOU WANT: V = I × R, I = V/R, R = V/I

WORKED EXAMPLE

LED resistor

A red LED needs about 2.0 V and 15 mA. From a 5 V Arduino pin the resistor must drop 5 - 2 = 3 V at 0.015 A: R = 3 ÷ 0.015 = **200 Ω**. Choose the next standard value up, **220 Ω** (current 3 ÷ 220 = 13.6 mA).

2.2.2 Power formula (P = V × I)

Combining with Ohm's law: $P = V \times I = I^2 \times R = V^2 \div R$. Use it to check that components survive: the 220 Ω LED resistor dissipates 3 × 0.0136 = 0.041 W, fine for a ¼ W (0.25 W) resistor. A motor drawing 1.5 A at 12 V uses 18 W.

2.2.3 Kirchhoff's current and voltage laws (introduction)

Kirchhoff's current law (KCL): the current flowing into a junction equals the current flowing out (charge is not lost). **Kirchhoff's voltage law (KVL)**: around any closed loop, the voltage rises (batteries) equal the voltage drops (components). In the LED circuit: 5 V = 3 V (resistor) + 2 V (LED).

2.3 Circuits

2.3.1 Open, closed and short circuits

Circuit	What happens
Closed	a complete loop: current flows and the circuit works
Open	a break (switch off, broken wire): no current
Short	a path with almost no resistance across the supply: very large current, heat, fire, damaged battery or board. Always avoid.

WATCH OUT
Never connect a battery's terminals directly together, or 5 V straight to GND on a breadboard. Lithium batteries can deliver tens of amps into a short circuit and catch fire. Use a fuse in every battery-powered robot.

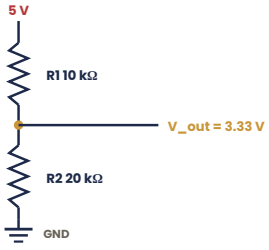
2.3.2 Series and parallel circuits

	Series (one path)	Parallel (several paths)
Current	the same through every component	splits; total = sum of branch currents
Voltage	shared; total = sum of drops	the same across every branch
Total resistance	R = R1 + R2 + …	1/R = 1/R1 + 1/R2 + … (two: R1·R2 ÷ (R1+R2))
If one part fails open	everything stops	the other branches keep working

SHORTCUT TRICK
Parallel resistance is always **smaller than the smallest** resistor. Two equal resistors in parallel give half the value: 10 kΩ ‖ 10 kΩ = 5 kΩ.

2.3.3 Voltage dividers

V_out = V_in × R2 ÷ (R1 + R2) (R1 on top, R2 to ground)



A VOLTAGE DIVIDER TURNS 5 V INTO 3.33 V FOR A 3.3 V INPUT

Dividers read sensors (an LDR or thermistor as one resistor gives a voltage that changes with light or temperature) and reduce 5 V signals to safe 3.3 V levels. They must not power loads: drawing current changes the output.

CIRCUITS.PY

```
def series(*rs):   return sum(rs)
def parallel(*rs): return 1 / sum(1 / r for r in rs)
def divider(vin, r1, r2): return vin * r2 / (r1 + r2)

print(f"series 220 + 330 + 1000 = {series(220, 330, 1000)} ohm")
print(f"parallel 1k || 1k      = {parallel(1000, 1000):.0f} ohm")
print(f"parallel 1k || 2.2k    = {parallel(1000, 2200):.1f} ohm")
print(f"divider 5 V, 10k/20k   = {divider(5, 10e3, 20e3):.2f} V")
print(f"LDR divider 5 V, 10k fixed, LDR 1k (bright) .. 50k (dark): "
      f"{divider(5, 10e3, 1e3):.2f} V .. {divider(5, 10e3, 50e3):.2f} V")
```

OUTPUT

```
series 220 + 330 + 1000 = 1550 ohm
parallel 1k || 1k      = 500 ohm
parallel 1k || 2.2k    = 687.5 ohm
divider 5 V, 10k/20k   = 3.33 V
LDR divider 5 V, 10k fixed, LDR 1k (bright) .. 50k (dark): 0.45 V .. 4.17 V
```

2.4 AC vs DC

	DC (direct current)	AC (alternating current)
Direction	always the same	reverses many times per second
Sources	batteries, USB, solar cells, adapters	mains sockets (230 V, 50 Hz in India)
Used by	electronics, robots, microcontrollers, most motors in robots	homes, industry; converted to DC by adapters
Danger	low-voltage DC (≤ 24 V) is safe to touch	mains AC can kill



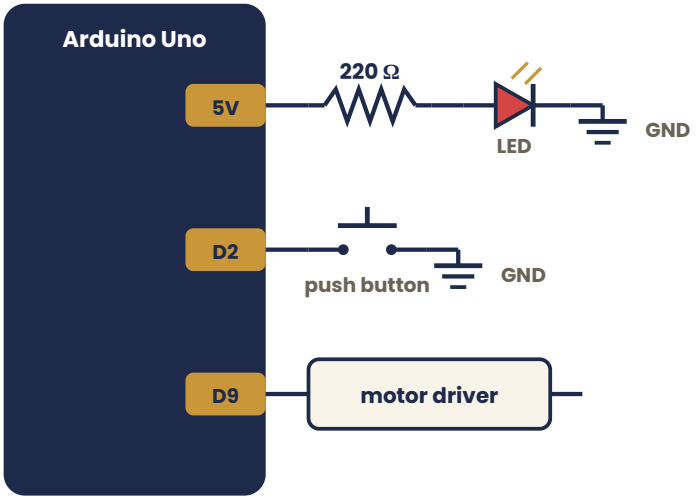
DIGITAL DC SIGNALS SWITCH BETWEEN TWO LEVELS; AC MAINS IS A SMOOTH SINE WAVE THAT CHANGES POLARITY

Robots run on **DC**. A wall adapter (power supply) converts mains AC to low-voltage DC; inside, a transformer or switching converter lowers the voltage and diodes (a rectifier) turn AC into DC.

2.5 Reading circuit diagrams

2.5.1 Common schematic symbols

A **schematic** shows how components connect electrically, not where they sit physically. Lines are wires; a dot where lines cross means they are joined; lines crossing without a dot are not connected.



SCHEMATIC SYMBOLS USED IN THIS BOOK: RESISTOR, LED, PUSH BUTTON, GROUND, SUPPLY AND A BLOCK FOR A MODULE

Symbol	Component
zig-zag (or rectangle)	resistor
triangle with bar and arrows	LED (bar = cathode, the negative side)
two parallel plates	capacitor (one curved or marked + for electrolytic)
long and short lines	battery (long line = +)
three shrinking lines	ground (0 V)
circle with M	motor
break with a lever	switch / push button

2.6 Digital vs analog signals

An **analog** signal can have any value in a range (a temperature sensor giving 0.72 V, 0.73 V…). A **digital** signal has only two meaningful states, HIGH and LOW (1 and 0). Microcontrollers read analog signals with an ADC (analog-to-digital converter) and produce "analog-like" outputs with PWM.

2.6.1 HIGH and LOW logic levels (5 V and 3.3 V logic)

Logic family	HIGH	LOW	Boards
5 V logic	about 5 V (read as HIGH above ~3 V)	0 V (below ~1.5 V)	Arduino Uno, Nano, Mega
3.3 V logic	about 3.3 V (HIGH above ~2 V)	0 V (below ~0.8 V)	ESP32, Pico, Raspberry Pi, STM32

WATCH OUT

Never feed a 5 V signal into a 3.3 V board's pin: use a voltage divider or level shifter. A 3.3 V output into a 5 V input usually works, because 3.3 V is above the 5 V board's HIGH threshold.

2.6.2 Binary numbers basics

Computers store everything as **bits** (0/1). Each binary place is worth double the one to its right: 1, 2, 4, 8, 16, 32, 64, 128. 8 bits = 1 **byte** = 0–255. $1011_2 = 8 + 0 + 2 + 1 = 11$. A 10-bit ADC gives $2^{10} = 1024$ values (0–1023).

BINARY.PY

```
for n in (5, 11, 200, 255):
    print(f"{n:3d} = {n:08b} (binary) = 0x{n:02X} (hex)")
print("0b1011 =", 0b1011, "| 2**10 =", 2 ** 10, "| int('11001000', 2) =", int("11001000", 2))
```

OUTPUT

```
  5 = 00000101 (binary) = 0x05 (hex)
 11 = 00001011 (binary) = 0x0B (hex)
200 = 11001000 (binary) = 0xC8 (hex)
255 = 11111111 (binary) = 0xFF (hex)
0b1011 = 11 | 2**10 = 1024 | int('11001000', 2) = 200
```

2.6.3 Logic gates: AND, OR, NOT, NAND, NOR, XOR

LOGIC_GATES.PY

```
gates = {"AND": lambda a, b: a & b, "OR": lambda a, b: a | b, "NAND": lambda a, b: 1 - (a & b),
         "NOR": lambda a, b: 1 - (a | b), "XOR": lambda a, b: a ^ b}
print("A B | " + " ".join(f"{g:>4}" for g in gates) + " | NOT A")
for a in (0, 1):
    for b in (0, 1):
        print(f"{a} {b} | " + " ".join(f"{f(a, b):>4}" for f in gates.values()) + f" |   {1 - a}")
```

OUTPUT

```
A B |  AND   OR NAND  NOR  XOR | NOT A
0 0 |    0    0    1    1    0 |   1
0 1 |    0    1    1    0    1 |   1
1 0 |    0    1    1    0    1 |   0
1 1 |    1    1    0    0    0 |   0
```

Robots use logic all the time, in code or in chips: "drive forward if the path is clear AND the battery is OK"; "stop if the left bumper OR the right bumper is pressed"; NAND and NOR are "universal": any logic circuit can be built from them alone.

PRACTICE CHECKPOINT

Calculate the resistor needed for an LED, solve ten Ohm's-law problems, and draw schematics for series, parallel and voltage-divider circuits.

2.7 Practice questions

2.7.1 Basic level

Q1 What is the unit of resistance?

(a) volt (V) **(b)** ampere (A) **(c)** watt (W) **(d)** ohm (Ω)

Q2 Which formula is Ohm's law?

(a) P = V ÷ I **(b)** R = V × I **(c)** I = V × R **(d)** V = I × R

Q3 What is 4.7 kΩ in ohms?

(a) 470 Ω **(b)** 47 000 Ω **(c)** 4700 Ω **(d)** 4.7 Ω

Q4 What is 25 mA in amps?

(a) 0.025 A **(b)** 0.25 A **(c)** 2.5 A **(d)** 25 A

Q5 Which kind of current do batteries supply?

(a) AC **(b)** both alternately
(c) neither **(d)** DC

Q6 What happens in a short circuit?

(a) no current flows
(b) a very large current flows, causing heat and possible damage or fire
(c) the voltage doubles safely
(d) the circuit works normally

Q7 What is the logic HIGH voltage of an Arduino Uno?

(a) 3.3 V **(b)** 5 V **(c)** 12 V **(d)** 1.8 V

Q8 Which gate outputs 1 only when both inputs are 1?

(a) OR **(b)** XOR **(c)** NOR **(d)** AND

2.7.2 Intermediate level

Q9 A 12 V supply drives 0.5 A through a motor. What power does it use?

(a) 6 W **(b)** 24 W **(c)** 0.04 W **(d)** 12.5 W

Q10 What current flows through a 1 kΩ resistor with 5 V across it?

(a) 5 A **(b)** 0.5 mA **(c)** 5 mA **(d)** 50 mA

Q11 What is the total of 220 Ω, 330 Ω and 470 Ω in series?

(a) 1020 kΩ **(b)** 102 Ω **(c)** 330 Ω **(d)** 1020 Ω

Q12 What is 2 kΩ in parallel with 2 kΩ?

(a) 4 kΩ **(b)** 1 kΩ **(c)** 2 kΩ **(d)** 0.5 kΩ

Q13 A divider has R1 = 1 kΩ (top) and R2 = 2 kΩ on 9 V. What is V_out?

(a) 6 V **(b)** 3 V **(c)** 4.5 V **(d)** 9 V

Q14 What is binary 1101 in decimal?

(a) 11 **(b)** 14 **(c)** 13 **(d)** 1101

Q15 Which law says currents into a junction equal currents out?

(a) Kirchhoff's current law **(b)** Ohm's law
(c) Kirchhoff's voltage law **(d)** Newton's first law

Q16 XOR gives 1 when

(a) both inputs are 1 **(b)** the inputs are different
(c) both inputs are 0 **(d)** always

2.7.3 Advanced level

Q17 What resistor is needed for a 2.1 V LED at 20 mA from 5 V?

(a) 250 Ω **(b)** 105 Ω
(c) 2.9 Ω **(d)** 145 Ω (use 150 Ω)

Q18 How much energy does a 7.4 V, 2200 mAh battery store?

(a) 2200 Wh **(b)** 7.4 Wh **(c)** 16.28 Wh **(d)** 297 Wh

Q19 A robot draws 1.1 A from that battery. Roughly how long will it run (ignoring losses)?

(a) about 20 hours **(b)** about 12 minutes
(c) about 2 hours **(d)** about 7 hours

Q20 What is 100 Ω ∥ 220 Ω ∥ 470 Ω (to the nearest ohm)?

(a) 790 Ω **(b)** 100 Ω **(c)** 30 Ω **(d)** 60 Ω

Q21 Write ohm(V=None, I=None, R=None) that computes whichever value is missing, and solve: V for 0.02 A and 220 Ω; I for 9 V and 450 Ω; R for 3.3 V and 15 mA.

Q22 Find two standard resistors (from 1k, 1.5k, 2.2k, 3.3k, 4.7k, 10k) for a divider that turns 5 V into the value closest to 3.3 V without exceeding 3.4 V.

Q23 Print the truth table of a robot safety rule: MOTOR = (START AND NOT BUMPER) AND BATTERY_OK.

Q24 A sensor outputs 0–5 V and must connect to an ESP32 (3.3 V). What is the simplest safe solution?

(a) connect it directly
(b) a voltage divider such as 1 kΩ / 2 kΩ so 5 V becomes 3.33 V
(c) use a 12 V supply
(d) add a capacitor only

Q25 Why must a voltage divider not be used to power a motor?

(a) the load current changes the output voltage and wastes power in the resistors
(b) dividers only work with AC
(c) motors need 3.3 V logic
(d) it would make the motor too fast

2.8 Solutions

Q1 **Answer: (d) ohm (Ω)**

Ω.

Q2 **Answer: (d) V = I × R**

V = IR.

Q3 **Answer: (c) 4700 Ω**

k = 1000.

Q4 **Answer: (a) 0.025 A**

m = 1/1000.

Q5 **Answer: (d) DC**

Direct current.

Q6 **Answer: (b) a very large current flows, causing heat and possible damage or fire**

Avoid shorts; use fuses.

Q7 **Answer: (b) 5 V**

5 V logic.

Q8 **Answer: (d) AND**

AND.

Q9 **Answer: (a) 6 W**

P = 12 × 0.5 = **6 W**.

Q10 **Answer: (c) 5 mA**

5 ÷ 1000 = 0.005 A.

Q11 **Answer: (d) 1020 Ω**

Add them.

Q12 **Answer: (b) 1 kΩ**

Equal resistors in parallel: half.

Q13 **Answer: (a) 6 V**

9 × 2/3 = **6 V**.

Q14 **Answer: (c) 13**

8 + 4 + 0 + 1 = 13.

Q15 **Answer: (a) Kirchhoff's current law**

KCL.

Q16 **Answer: (b) the inputs are different**

Exclusive OR.

Q17 **Answer: (d) 145 Ω (use 150 Ω)**

(5 − 2.1) ÷ 0.02 = **145 Ω**.

Q18 **Answer: (c) 16.28 Wh**

7.4 × 2.2 = **16.28 Wh**.

Q19 **Answer: (c) about 2 hours**

2.2 Ah ÷ 1.1 A = **2 h**.

Q20 **Answer: (d) 60 Ω**

1/R = 0.01 + 0.004545 + 0.002128 = 0.016673 → **60 Ω**.

Q21 **Answer: Solution program**

One function, three rearrangements of V = IR.

```
def ohm(V=None, I=None, R=None):
    if V is None: return I * R
    if I is None: return V / R
    return V / I
print(f"V = {ohm(I=0.02, R=220):.2f} V")
print(f"I = {ohm(V=9, R=450) * 1000:.1f} mA")
print(f"R = {ohm(V=3.3, I=0.015):.0f} ohm")
```

OUTPUT

```
V = 4.40 V
I = 20.0 mA
R = 220 ohm
```

Q22 **Answer: Solution program**

Trying every pair is quick for a small list. The pairs that give about 3.4 V (1 k/2.2 k = 3.44 V, 2.2 k/4.7 k = 3.41 V, 4.7 k/10 k = 3.401 V) all just exceed the 3.4 V limit, so the best allowed pair is 1 k/1.5 k at 3.0 V, still read as HIGH by a 3.3 V board. Relax the limit slightly and 4.7 k/10 k is the usual choice.

```
E = [1000, 1500, 2200, 3300, 4700, 10000]
best = None
for r1 in E:
    for r2 in E:
        v = 5 * r2 / (r1 + r2)
        if v <= 3.4 and (best is None or abs(v - 3.3) < abs(best[0] - 3.3)):
            best = (v, r1, r2)
v, r1, r2 = best
print(f"R1 = {r1} ohm, R2 = {r2} ohm -> {v:.3f} V")
```

OUTPUT

```
R1 = 1000 ohm, R2 = 1500 ohm -> 3.000 V
```

Q23 **Answer: Solution program**

Only one combination runs the motor: started, no bumper hit, battery OK.

```
from itertools import product
print("START BUMPER BAT_OK | MOTOR")
for s, b, ok in product((0, 1), repeat=3):
    motor = s and not b and ok
    print(f"  {s}      {b}      {ok}    |   {int(motor)}")
```

OUTPUT

```
START BUMPER BAT_OK | MOTOR
  0      0      0    |   0
  0      0      1    |   0
  0      1      0    |   0
  0      1      1    |   0
  1      0      0    |   0
  1      0      1    |   1
  1      1      0    |   0
  1      1      1    |   0
```

Q24 **Answer: (b) a voltage divider such as 1 kΩ / 2 kΩ so 5 V becomes 3.33 V**

Divide down to 3.3 V.

Q25 **Answer: (a) the load current changes the output voltage and wastes power in the resistors**

Use a regulator or driver.

V = I × R and P = V × I (= I^2R = V^2/R); convert prefixes to base units first. KCL: currents in = out; KVL: rises = drops round a loop. Series: R adds, current same; parallel: voltage same, R smaller than the smallest. Divider: $V_{out} = V_{in} \cdot R2/(R1+R2)$. Robots run on DC; avoid shorts, fuse batteries. 5 V vs 3.3 V logic; binary places 1, 2, 4, 8…; AND, OR, NOT, NAND, NOR, XOR.

MODULE 03 · BEGINNER

Electronic components

Minimum duration **2 days** | Level **Beginner**

Every robot is built from a small set of common components. This module covers resistors and their colour codes, potentiometers, ceramic and electrolytic capacitors, inductors; diodes, rectifiers, LEDs and Zener diodes; BJT and MOSFET transistors as switches; voltage regulators (7805, LM317, buck converters); push buttons, toggle and limit switches, relays; the 555 timer and op-amps; and how to read the key values in a datasheet, followed by 25 solved questions and programs.

IN THIS MODULE

3.1 Passive components
3.2 Semiconductors
3.3 Switches and relays
3.4 Integrated circuits
3.5 Datasheets
3.6 Practice questions
3.7 Solutions

3.1 Passive components

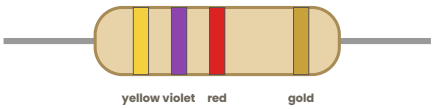
Passive components (resistors, capacitors, inductors) do not amplify or switch; they resist, store or filter energy.

3.1.1 Resistors and colour codes

A **resistor** limits current or sets voltages. Its value is printed as coloured bands: on a 4-band resistor the first two bands are digits, the third is the multiplier (number of zeros) and the fourth the tolerance. Also check the **power rating**: common through-hole resistors are ¼ W.

Colour	Digit	Multiplier	Colour	Digit	Multiplier
black	0	×1	green	5	×100 k
brown	1	×10	blue	6	×1 M
red	2	×100	violet	7	×10 M
orange	3	×1 k	grey	8	—
yellow	4	×10 k	white	9	—

Tolerance band: **gold ±5 %**, silver ±10 %, brown ±1 % (5-band precision resistors have three digit bands). Gold as a *multiplier* means ×0.1.



YELLOW-VIOLET-RED-GOLD = 4, 7, ×100 = 4700 Ω = 4.7 KΩ ±5 %

SHORTCUT TRICK

Memory aid for the colour order 0–9: "**B**ad **B**eer **R**ots **O**ur **Y**oung **G**uts **B**ut **V**odka **G**oes **W**ell" = black, brown, red, orange, yellow, green, blue, violet, grey, white. Or simply learn that the middle colours follow the rainbow (red, orange, yellow, green, blue, violet).

COLOUR_CODE.PY

```
COLORS = ["black", "brown", "red", "orange", "yellow", "green", "blue", "violet", "grey", "white"]
TOL = {"gold": "5 %", "silver": "10 %", "brown": "1 %", "red": "2 %"}

def decode(bands):
    *digits, mult, tol = bands
    value = int("".join(str(COLORS.index(c)) for c in digits))
    m = 0.1 if mult == "gold" else 0.01 if mult == "silver" else 10 ** COLORS.index(mult)
    ohms = value * m
    pretty = f"{ohms / 1e6:g} MΩ" if ohms >= 1e6 else f"{ohms / 1e3:g} kΩ" if ohms >= 1e3 else f"
{ohms:g} Ω"
    return f"{pretty} ±{TOL[tol]}"

for b in (["yellow", "violet", "red", "gold"], ["brown", "black", "orange", "gold"],
          ["red", "red", "brown", "gold"], ["brown", "black", "black", "red", "brown"],
          ["orange", "orange", "gold", "gold"]):
    print(f"{'-'.join(b):34} {decode(b)}")
```

OUTPUT

```
yellow-violet-red-gold             4.7 kΩ ±5 %
brown-black-orange-gold            10 kΩ ±5 %
red-red-brown-gold                 220 Ω ±5 %
brown-black-black-red-brown        10 kΩ ±1 %
orange-orange-gold-gold            3.3 Ω ±5 %
```

3.1.2 Variable resistors and potentiometers

A **potentiometer** ("pot") has three terminals: the two ends of a resistive track and a **wiper** that slides along it. Connected between 5 V and GND, the wiper gives any voltage from 0 to 5 V: a knob for speed or position input. A **trimmer** is a small pot set once with a screwdriver; a **rheostat** uses just two terminals as a variable resistor. Servos contain a pot to sense their shaft angle.

3.1.3 Capacitors (ceramic, electrolytic) and their uses

A **capacitor** stores charge, measured in farads (F; practical values in µF, nF, pF). It blocks steady DC but passes changes. Uses in robots: **decoupling** (0.1 µF ceramic next to each chip smooths fast supply noise), **bulk smoothing** (100–1000 µF electrolytic across a motor or servo supply absorbs current surges), timing (with a resistor) and filtering.

Type	Values	Notes
Ceramic	1 pF – 10 µF	not polarised, small, great for decoupling (code 104 = 10 × 10^4 pF = 100 nF)
Electrolytic	1 µF – 10 000 µF	**polarised**: the stripe marks the - lead; reversed or over-voltage, it can burst
Tantalum / film	various	stable values; film for audio and timing

RC time constant $\tau = R \times C$: a capacitor charging through a resistor reaches 63 % of the supply after τ and is effectively full after 5τ.

WORKED EXAMPLE

RC timing

10 kΩ and 100 µF: $\tau = 10\,000 \times 0.0001 =$ **1 s**; fully charged after about 5 s. This is how simple delay and debounce circuits work.

3.1.4 Inductors (overview)

An **inductor** is a coil that stores energy in a magnetic field and resists changes in current (henry, H). It appears in switching regulators (buck converters), filters and inside every motor and relay coil. When current through an inductor is switched off, it produces a high voltage spike: that is why motors and relays need flyback diodes (3.2.1).

3.2 Semiconductors

3.2.1 Diodes and rectification

A **diode** lets current flow in one direction only, from anode (+) to cathode (- , marked by a stripe), with a forward voltage drop of about **0.7 V** (silicon) or 0.3 V (Schottky). Uses: **reverse-polarity protection** (a battery connected backwards does no damage), **rectification** (four diodes in a bridge turn AC into pulsing DC), and **flyback protection** across motors and relay coils.

3.2.2 LEDs

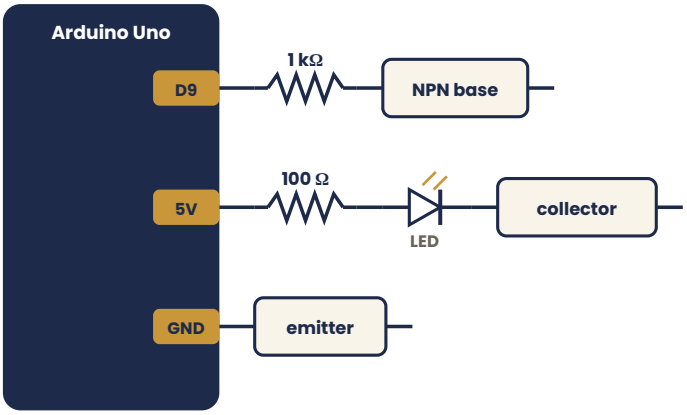
An **LED** (light-emitting diode) glows when forward current flows. The longer leg is the anode (+); the flat side of the rim and the shorter leg are the cathode. It **always needs a series resistor** to limit current (typically 5–20 mA). Forward voltage depends on colour: red ≈ 1.8–2.2 V, green/yellow ≈ 2.0–2.4 V, blue/white ≈ 2.8–3.4 V.

3.2.3 Zener diodes

A **Zener diode** is used in reverse: once the voltage reaches its rated Zener voltage (3.3 V, 5.1 V…) it conducts and holds that voltage. With a series resistor it makes a simple reference or clamps an input so it cannot exceed a safe level, protecting microcontroller pins from spikes.

3.2.4 Transistors (BJT and MOSFET) as switches

A **transistor** lets a small signal control a large current. An **NPN BJT** (BC547 for ~100 mA, 2N2222 for ~600 mA) is controlled by base **current**; an **N-channel MOSFET** (IRLZ44N, AO3400; choose "logic level") is controlled by gate **voltage** and handles amps with little heat. Both switch the load on its ground side (low-side switching).



NPN TRANSISTOR SWITCH: D9 → 1 KΩ → BASE; LOAD FROM +5 V TO COLLECTOR; EMITTER TO GND

WORKED EXAMPLE

Base resistor for a BJT switch

Load 100 mA, gain $h_{FE} \approx 100$ → base current 1 mA; drive it 3–5× harder to saturate the transistor fully: about 4 mA. $R_B = (5 - 0.7) \div 0.004 = 1075\ \Omega$ → **1 kΩ**.

TRANSISTORSWITCH.INO

```
// A small signal on pin 9 switches a bigger load through a transistor
const int SWITCH_PIN = 9;     // to the transistor base through 1 kOhm
const int BUTTON = 2;

void setup() {
  pinMode(SWITCH_PIN, OUTPUT);
  pinMode(BUTTON, INPUT_PULLUP);
  Serial.begin(9600);
}

void loop() {
  bool pressed = digitalRead(BUTTON) == LOW;
  digitalWrite(SWITCH_PIN, pressed ? HIGH : LOW);   // transistor ON while pressed
  static bool last = false;
  if (pressed != last) {
    Serial.println(pressed ? "load ON" : "load OFF");
    last = pressed;
  }
  delay(10);
}
```

SERIAL MONITOR

```
load ON
load OFF
```

```
load ON
load OFF
```

3.2.5 Voltage regulators (7805, LM317, buck converters)

Regulator	Type	Output	Notes
7805	linear	fixed 5 V, up to 1 A	needs at least ~7 V in; wastes $(V_{in} - 5) \times I$ as heat
LM317	linear, adjustable	$V_{out} = 1.25 \times (1 + R2/R1)$, 1.25–37 V	two resistors set the voltage
AMS1117-3.3	low-dropout linear	3.3 V, 1 A	common on boards; dropout about 1.1 V
Buck converter (LM2596, MP1584)	switching step-down	adjustable, 2–5 A	85–95 % efficient: best for batteries
Boost converter	switching step-up	raises voltage (e.g. 3.7 V → 5 V)	used in power banks

WORKED EXAMPLE

Why buck converters win

Powering a 5 V, 1 A Raspberry Pi from 12 V: a 7805 wastes (12 - 5) × 1 = **7 W** as heat (efficiency 5/12 = 42 %) and needs a large heatsink. A 90 %-efficient buck converter draws 5 ÷ 0.9 = 5.56 W, wasting only **0.56 W**.

3.3 Switches and relays

3.3.1 Push buttons, toggle and limit switches

Switch	Action	Robot use
Push button (momentary)	conducts only while pressed	start, mode select
Toggle / slide switch	stays in the position set	main power switch
Limit (micro) switch	lever pressed by a moving part	end-stops on arms and 3D printers, bumpers
Reed switch	closed by a magnet	door sensors, wheel revolution counting

Switch terms: **SPST** (single pole, single throw: on/off), **SPDT** (one input, two outputs), and NO/NC contacts (normally open/closed). Connect inputs with a pull-up or pull-down resistor (often the board's internal pull-up) and debounce them in software.

3.3.2 Relays for high-power loads

A **relay** is an electrically controlled switch: a small current through its coil (via a transistor, with a flyback diode) pulls metal contacts together, fully isolating the control circuit from the load. Relay modules include the transistor, diode and an LED. Use them for loads such as lamps or pumps; mains voltage work must be left to qualified people.

3.4 Integrated circuits

3.4.1 555 timer

The **555** is a famous timer chip. In **astable** mode it oscillates continuously (blinking LEDs, tones, PWM); in **monostable** mode it produces one pulse of fixed length when triggered.

Astable 555: f = 1.44 ÷ ((R1 + 2·R2) × C); monostable pulse: t = 1.1 × R × C

TIMER555.PY

```
def astable(r1, r2, c):
    f = 1.44 / ((r1 + 2 * r2) * c)
    high = 0.693 * (r1 + r2) * c
    low = 0.693 * r2 * c
    return f, high / (high + low)

for r1, r2, c in [(1e3, 10e3, 10e-6), (10e3, 47e3, 1e-6), (1e3, 6.8e3, 10e-9)]:
    f, duty = astable(r1, r2, c)
    print(f"R1={r1:>7.0f} R2={r2:>7.0f} C={c:.0e} F -> {f:9.2f} Hz, duty {duty:.0%}")
print(f"monostable 100 k, 10 uF: pulse {1.1 * 100e3 * 10e-6:.1f} s")
```

OUTPUT

```
R1=   1000 R2=  10000 C=1e-05 F ->      6.86 Hz, duty 52%
R1=  10000 R2=  47000 C=1e-06 F ->     13.85 Hz, duty 55%
R1=   1000 R2=   6800 C=1e-08 F ->   9863.01 Hz, duty 53%
monostable 100 k, 10 uF: pulse 1.1 s
```

3.4.2 Op-amps (overview)

An **operational amplifier** (LM358, LM741) amplifies the difference between its two inputs. With resistors it makes amplifiers (boosting a tiny sensor signal, gain = 1 + R_f/R_1 for a non-inverting amplifier), comparators (output HIGH when a sensor passes a threshold, used in many IR and sound sensor modules) and filters.

3.5 Datasheets

3.5.1 Reading key values: voltage, current, pinout

A **datasheet** is the manufacturer's specification. Read the first page summary, then check:

Item	Example (IRLZ44N MOSFET)	Why it matters
Absolute maximum ratings	V_{DS} 55 V, I_D 47 A	exceeding them destroys the part: stay well below
Operating conditions	$R_{DS(on)}$ 0.035 Ω at V_{GS} = 4 V	logic-level suitability; heat = $I^2 \times R$
Pinout / package	TO-220: gate, drain, source (left to right)	wiring it correctly
Thermal data	junction-to-ambient resistance	whether it needs a heatsink

Item	Example (IRLZ44N MOSFET)	Why it matters
Typical application circuit	example schematic	a proven starting design

WORKED EXAMPLE

Heat in a MOSFET

Switching a 5 A motor with $R_{DS(on)}$ = 0.035 Ω: P = I² × R = 25 × 0.035 = **0.875 W**: warm but manageable in TO-220 without a heatsink in open air; at 10 A it would be 3.5 W and need one.

PRACTICE CHECKPOINT

Identify 20 components by sight and value, read resistor colour codes, and build a transistor switch that turns on an LED from a small input signal.

3.6 Practice questions

3.6.1 Basic level

Q1 What value is brown-black-red-gold?

(a) 100 Ω ±5 % **(b)** 1 kΩ ±5 % **(c)** 10 kΩ ±5 % **(d)** 12 Ω ±10 %

Q2 Which capacitor type is polarised?

(a) electrolytic **(b)** ceramic **(c)** film **(d)** all capacitors

Q3 What is the typical forward voltage of a silicon diode?

(a) 0.07 V **(b)** 7 V **(c)** 0.7 V **(d)** 3.3 V

Q4 Which leg of an LED is the anode (+)?

(a) the shorter leg **(b)** either
(c) the flat side **(d)** the longer leg

Q5 Which regulator gives a fixed 5 V output?

(a) LM317 only **(b)** LM358 **(c)** 555 **(d)** 7805

Q6 What does a potentiometer's wiper provide?

(a) a fixed 5 V
(b) a clock signal
(c) an adjustable voltage between the two ends
(d) a PWM output

Q7 Which switch type is used as an end-stop on a robot arm?

(a) limit (micro) switch **(b)** toggle switch
(c) reed relay coil **(d)** potentiometer

Q8 What does the 555 timer do in astable mode?

(a) regulates voltage
(b) amplifies audio only
(c) stores charge
(d) oscillates continuously, producing a square wave

3.6.2 Intermediate level

Q9 What value is red-red-brown-gold?

(a) 22 Ω **(b)** 220 Ω **(c)** 2.2 kΩ **(d)** 221 Ω

Q10 What value is green-blue-orange-gold?

(a) 5.6 kΩ **(b)** 56 kΩ **(c)** 563 Ω **(d)** 560 kΩ

Q11 Which colours mark 4.7 kΩ?

(a) yellow-violet-orange **(b)** orange-violet-red
(c) yellow-grey-red **(d)** yellow-violet-red

Q12 What is the RC time constant of 47 kΩ and 10 µF?

(a) 0.47 s (b) 4.7 s (c) 0.047 s (d) 470 s

Q13 What does a flyback diode protect against?

(a) reverse battery only
(b) overheating
(c) the voltage spike when an inductive load is switched off
(d) static electricity

Q14 Why is a logic-level MOSFET preferred for switching motors from an Arduino?

(a) it needs no ground
(b) it works only with AC
(c) it is a type of relay
(d) it turns fully on with a 5 V (or 3.3 V) gate and wastes little power

Q15 What does ceramic capacitor code 104 mean?

(a) 104 pF (b) 100 nF (0.1 µF)
(c) 10 µF (d) 1 nF

Q16 What does a Zener diode do in reverse at its rated voltage?

(a) conducts and holds that voltage (b) blocks completely
(c) emits light (d) amplifies

3.6.3 Advanced level

Q17 A 7805 supplies 0.8 A from a 12 V battery. How much heat does it waste?

(a) 4 W (b) 9.6 W (c) 5.6 W (d) 0.8 W

Q18 LM317 with R1 = 240 Ω and R2 = 720 Ω gives what output?

(a) 5 V (b) 3 V (c) 1.25 V (d) 3.75 V

Q19 Base resistor for an NPN switch: 5 V drive, V_BE 0.7 V, base current 4.3 mA?

(a) 100 Ω (b) 1 kΩ (c) 10 kΩ (d) 4.3 kΩ

Q20 A 555 astable uses R1 = 1 kΩ, R2 = 10 kΩ, C = 10 µF. What is the frequency?

(a) about 68.6 Hz (b) about 0.69 Hz (c) about 144 Hz (d) about 6.86 Hz

Q21 Write a function that turns a resistance in ohms into the 4-band colour code (two digits + multiplier + gold) and test 220, 4700, 100 000 and 1 000 000 Ω.

Q22 Pick the standard E12 resistor (values 10, 12, 15, 18, 22, 27, 33, 39, 47, 56, 68, 82 × powers of ten) just above the value needed for LEDs of 2.0 V, 2.2 V and 3.2 V at 15 mA from 5 V, and print the actual current.

Q23 Simulate an RC circuit charging: 5 V supply, R = 10 kΩ, C = 100 µF. Print the capacitor voltage at 0–5 time constants.

Q24 Which is the best way to power a 5 V, 2 A robot computer from a 3S (11.1 V) LiPo?

(a) a 7805 without a heatsink (b) a voltage divider
(c) a 5 V buck converter rated above 2 A (d) a Zener diode alone

Q25 A MOSFET with R_DS(on) = 0.02 Ω switches 8 A. How much heat does it produce?

(a) 0.16 W **(b)** 12.8 W **(c)** 1.28 W **(d)** 0.64 W

3.7 Solutions

Q1 **Answer: (b) 1 kΩ ±5 %**

1, 0, ×100 = 1000 Ω.

Q2 **Answer: (a) electrolytic**

Mind the stripe (-).

Q3 **Answer: (c) 0.7 V**

≈ 0.7 V (Schottky ≈ 0.3 V).

Q4 **Answer: (d) the longer leg**

Long leg = +.

Q5 **Answer: (d) 7805**

78xx: last digits = voltage.

Q6 **Answer: (c) an adjustable voltage between the two ends**

A voltage divider you can turn.

Q7 **Answer: (a) limit (micro) switch**

Pressed by the moving part.

Q8 **Answer: (d) oscillates continuously, producing a square wave**

Free-running oscillator.

Q9 **Answer: (b) 220 Ω**

2, 2, ×10.

Q10 **Answer: (b) 56 kΩ**

5, 6, ×1000.

Q11 **Answer: (d) yellow-violet-red**

4, 7, ×100.

Q12 **Answer: (a) 0.47 s**

47 000 × 0.00001 = 0.47 s.

Q13 **Answer: (c) the voltage spike when an inductive load is switched off**

Placed across motors and relay coils.

Q14 **Answer: (d) it turns fully on with a 5 V (or 3.3 V) gate and wastes little power**

Low $R_{DS(on)}$ at logic voltages.

Q15 **Answer: (b) 100 nF (0.1 µF)**

10×10^4 pF = 100 000 pF.

Q16 **Answer: (a) conducts and holds that voltage**

Voltage reference / clamp.

Q17 **Answer: (c) 5.6 W**

(12 - 5) × 0.8 = **5.6 W**.

Q18 **Answer: (a) 5 V**

1.25 × (1 + 720/240) = 1.25 × 4 = **5 V**.

Q19 **Answer: (b) 1 kΩ**

(5 - 0.7) ÷ 0.0043 = **1000 Ω**.

Q20 **Answer: (d) about 6.86 Hz**

1.44 ÷ (21 000 × 0.00001) = **6.86 Hz**.

Q21 **Answer: Solution program**

Divide by 10 until two digits remain; the number of divisions is the multiplier band.

```
COLORS = ["black", "brown", "red", "orange", "yellow", "green", "blue", "violet", "grey", "white"]
def to_bands(ohms):
    zeros = 0
    while ohms >= 100:
        ohms //= 10
        zeros += 1
    d1, d2 = divmod(int(ohms), 10)
    return [COLORS[d1], COLORS[d2], COLORS[zeros], "gold"]
for r in (220, 4700, 100_000, 1_000_000):
    print(f"{r:>9} ohm -> {'-'.join(to_bands(r))}")
```

OUTPUT

```
      220 ohm -> red-red-brown-gold
     4700 ohm -> yellow-violet-red-gold
   100000 ohm -> brown-black-yellow-gold
  1000000 ohm -> brown-black-green-gold
```

Q22 **Answer: Solution program**

Always round the resistor up so the current stays at or below the target.

```
E12 = [10, 12, 15, 18, 22, 27, 33, 39, 47, 56, 68, 82]
def next_e12(r):
    decade = 1
    while True:
        for v in E12:
            if v * decade >= r:
                return v * decade
        decade *= 10
for vf in (2.0, 2.2, 3.2):
    need = (5 - vf) / 0.015
    r = next_e12(need)
    print(f"Vf {vf} V: need {need:.0f} ohm -> use {r} ohm, current {(5 - vf) / r * 1000:.1f} mA")
```

OUTPUT

```
Vf 2.0 V: need 200 ohm -> use 220 ohm, current 13.6 mA
Vf 2.2 V: need 187 ohm -> use 220 ohm, current 12.7 mA
Vf 3.2 V: need 120 ohm -> use 120 ohm, current 15.0 mA
```

Q23 **Answer: Solution program**

V = V_s(1 - e^(-t/RC)): 63 % after one τ, over 99 % after five.

```
import math
V, R, C = 5.0, 10e3, 100e-6
tau = R * C
for k in range(6):
    v = V * (1 - math.exp(-k))
    print(f"t = {k * tau:.0f} s ({k} tau): {v:.2f} V ({v / V:.0%})")
```

OUTPUT

```
t = 0 s (0 tau): 0.00 V (0%)
t = 1 s (1 tau): 3.16 V (63%)
t = 2 s (2 tau): 4.32 V (86%)
t = 3 s (3 tau): 4.75 V (95%)
t = 4 s (4 tau): 4.91 V (98%)
t = 5 s (5 tau): 4.97 V (99%)
```

Q24 **Answer: (c) a 5 V buck converter rated above 2 A**

Efficient switching regulation.

Q25 **Answer: (c) 1.28 W**

$I^2R = 64 \times 0.02 =$ **1.28 W**.

Resistor bands: digit, digit, multiplier, tolerance (gold ±5 %). Capacitors: ceramic for decoupling, electrolytic (polarised) for bulk; $\tau = RC$. Diodes: one-way, 0.7 V drop, flyback protection; LEDs need a series resistor; Zeners clamp. Transistors switch big loads: BJT by base current, logic-level MOSFET by gate voltage. Prefer buck converters to 7805s. 555: $f = 1.44/((R1+2R2)C)$. Read datasheets for maximum ratings and pinouts.

MODULE 04 · BEGINNER

Tools, safety and prototyping

Minimum duration **1 day** | Level **Beginner**

Safe, correct use of tools is the foundation of every hardware project. This module covers electrical, lithium-battery, soldering and ESD safety; breadboards, jumper wires, connectors, perfboards and PCBs; the multimeter (voltage, current, resistance, continuity), oscilloscope, logic analyser and bench power supply; hand tools, glue guns and fasteners; soldering and desoldering; and the Tinkercad Circuits and Wokwi online simulators, followed by 25 solved questions and programs.

IN THIS MODULE

4.1 Lab safety
4.2 Prototyping
4.3 Measurement tools
4.4 Hand tools
4.5 Soldering
4.6 Online circuit simulators
4.7 Practice questions
4.8 Solutions

4.1 Lab safety

4.1.1 Electrical safety

- Work with low-voltage DC (under 24 V) for robots. Never open mains-powered equipment or work on mains wiring.
- Disconnect power before changing wiring; check for shorts before reconnecting.
- Use a fuse near the battery in every robot, rated a little above the normal maximum current.
- Keep liquids away from the bench; work on a non-conductive surface.
- If something smells hot, smokes or a component swells, disconnect power immediately.

4.1.2 Battery safety (especially lithium batteries)

Lithium-ion and LiPo batteries store a lot of energy and can catch fire if shorted, over-charged, over-discharged, punctured or crushed.

Rule	Why
Use the correct charger and never leave charging batteries unattended	over-charging causes heat and fire
Never discharge below ~3.0 V per cell; use a BMS or low-voltage alarm	deep discharge damages cells and makes them unsafe
Insulate terminals; store in a fire-resistant bag	shorts deliver huge currents
Do not use swollen ("puffy"), dented or punctured batteries	internal damage can lead to fire
Fuse the battery lead	limits damage from wiring faults

WORKED EXAMPLE

Battery C rating

A 2200 mAh LiPo rated 25C can deliver 2.2 A × 25 = **55 A**. That is why a short circuit is so dangerous, and why wires and fuses must be sized for the real current.

4.1.3 Soldering and tool safety

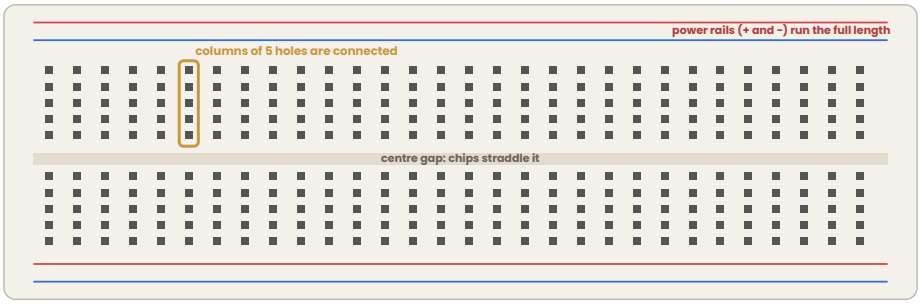
A soldering iron tip is at 300–400 °C. Always return it to its stand, never touch the tip or freshly soldered joints, work in a ventilated area (flux fumes irritate the lungs; use a fume extractor or fan), wash hands after handling solder, and wear safety glasses when cutting wires or leads (clippings fly). Cut away from your body and keep fingers clear of blades.

4.1.4 ESD (static electricity) protection

Static discharge from your body (thousands of volts, though harmless to you) can destroy microcontrollers and MOSFETs. Touch a grounded metal object before handling boards, use an ESD wrist strap and mat for valuable work, store chips in anti-static bags, and hold boards by their edges.

4.2 Prototyping

4.2.1 Breadboard layout and use



SOLDERLESS BREADBOARD: EACH SHORT COLUMN OF 5 HOLES IS ONE CONNECTION; THE LONG RAILS CARRY POWER

Insert components so their legs are in **different** columns (legs in the same column are shorted together). Put chips across the centre gap. Use the red rail for + and blue for GND, and on long boards check whether the rails are split in the middle. Breadboards suit currents up to about 1 A and are unreliable for vibrating robots: move proven circuits to perfboard.

4.2.2 Jumper wires and connectors

Jumper wires come male-male (breadboard), male-female (board header to breadboard) and female-female (module to module). Use colours consistently: **red = +**, **black = GND**, others for signals. For permanent robots use crimped connectors: JST-XH (signals), Dupont headers, XT30/XT60 (battery power, high current), screw terminals (motors).

4.2.3 Perfboards and PCBs

Board	Description	Best for
Breadboard	push-in, no solder	experiments, quick changes

Board	Description	Best for
Perfboard / stripboard	holes on a grid; components soldered and wired by hand (stripboard has copper strips)	one-off permanent circuits, shields
PCB (printed circuit board)	designed in KiCad/EasyEDA, made by a factory	reliable, compact, repeatable builds

4.3 Measurement tools

4.3.1 Multimeter: voltage, current, resistance, continuity

Measure	Dial	How to connect	Watch out
DC voltage	V⎓	probes **across** the component (in parallel), circuit powered	black probe on GND
Current	A or mA	meter **in series**: break the circuit and route current through it; red lead in the A/mA socket	never across a supply: it is a short circuit and blows the fuse
Resistance	Ω	across the component, circuit **unpowered**	other parts in the circuit affect the reading
Continuity	))) beep	across a wire or joint, unpowered	beeps when connected: finds breaks and shorts

WATCH OUT
After measuring current, move the red probe back to the V/Ω socket. Measuring voltage with the probe still in the A socket shorts the circuit through the meter.

4.3.2 Oscilloscope and logic analyser (overview)

An **oscilloscope** draws voltage against time, showing PWM signals, noise and ripple that a multimeter averages out. A **logic analyser** (cheap 8-channel USB ones work with PulseView) records many digital lines at once and decodes I2C, SPI and UART messages: invaluable when a sensor "does not answer".



WHAT AN OSCILLOSCOPE SHOWS THAT A MULTIMETER CANNOT

4.3.3 Bench power supply

A **bench power supply** provides an adjustable voltage with an adjustable **current limit**. Set the limit just above the expected current before connecting a new circuit: if there is a short, the supply limits the current instead of letting parts burn. It also shows how much current your robot really draws.

4.4 Hand tools

4.4.1 Wire strippers, cutters, pliers, screwdrivers

Tool	Use
Wire strippers	remove insulation without nicking the copper (choose the right gauge hole)
Flush side cutters	trim component leads and cable ties flush
Needle-nose pliers	bend leads, hold small parts, insert into tight spots
Precision screwdrivers / hex keys	M2/M3 screws on robot chassis and modules
Crimping tool	attach Dupont, JST and ring terminals properly
Calipers	measure parts for mounting and CAD

4.4.2 Glue guns and fasteners

Hot glue is quick for holding wires and light parts, but softens with heat and is not structural. Use **M3 screws and nuts** (with nylon-insert lock nuts or thread-locker against vibration), brass standoffs for boards, cable ties for wiring, and double-sided foam tape for light sensors. Design robots so the battery and boards can be removed without tools.

4.5 Soldering

4.5.1 Soldering iron, solder and flux

Use a temperature-controlled iron (about 330–370 °C for common solder), 0.6–0.8 mm **rosin-core** solder (lead-free SAC, or 60/40 tin-lead which flows more easily but contains lead), a damp sponge or brass wool to clean the tip, and extra flux for difficult joints. **Tin** the tip (coat it with fresh solder) so heat transfers well.

4.5.2 Making good joints and desoldering

1. Clean and heat **both** the pad and the lead with the tip for 1–2 seconds.
2. Feed solder into the joint (not onto the iron); it should flow around the lead.
3. Remove the solder, then the iron; do not move the part while it cools.
4. Inspect: a good joint is shiny (lead-free is satin) and shaped like a small volcano/cone; a dull ball or a crack is a "cold joint" to redo.
5. Desolder with a solder sucker (pump) or desoldering braid (wick) and flux.

> **SHORTCUT TRICK**
> Most soldering problems come from not enough heat on the **pad**. Touch the iron to where the pad and lead meet, count to two, then add solder. Never carry a blob of solder on the tip to the joint.

4.6 Online circuit simulators

4.6.1 Tinkercad Circuits and Wokwi

Simulator	Strengths
Tinkercad Circuits (Autodesk)	beginner-friendly breadboard view, Arduino Uno, code blocks or text, multimeter, many basic components; free in a browser
Wokwi	Arduino, ESP32, Raspberry Pi Pico (MicroPython/C), STM32; servos, OLEDs, sensors, Wi-Fi simulation, logic analyser; great for testing firmware before hardware arrives

Simulators let you test wiring and code with no risk of burning parts, and share a project link with a teacher. They do not show every real-world problem (noise, weak batteries, loose wires), so always finish on real hardware.

PRACTICE CHECKPOINT

Build three circuits on a breadboard, measure their voltages and currents with a multimeter, and solder a simple LED circuit on a perfboard.

4.7 Practice questions

4.7.1 Basic level

Q1 Which voltage range is considered safe for robot projects?

(a) 230 V AC mains
(b) 110 V DC
(c) any voltage
(d) low-voltage DC, under about 24 V

Q2 How is a multimeter connected to measure current?

(a) in parallel across the supply
(b) in series with the circuit
(c) with the circuit off
(d) across a resistor only

Q3 How is a multimeter connected to measure voltage?

(a) across (in parallel with) the component
(b) in series
(c) with the circuit unpowered
(d) between two separate circuits

Q4 What does the continuity setting do?

(a) measures battery capacity
(b) beeps when two points are connected
(c) measures temperature
(d) shows a waveform

Q5 In a breadboard, which holes are connected?

(a) each short column of 5 holes, and each long power rail
(b) all holes on the board
(c) no holes
(d) only diagonal holes

Q6 What colour is normally used for GND wires?

(a) red
(b) yellow
(c) black
(d) green

Q7 What does ESD stand for?

(a) electronic soldering device
(b) energy storage diode
(c) emergency stop device
(d) electrostatic discharge

Q8 Which simulator supports ESP32 and Raspberry Pi Pico?

(a) Tinkercad only
(b) Excel
(c) Paint
(d) Wokwi

Q9 What is the main danger of measuring current across a power supply?

(a) the meter reads zero
(b) the meter acts as a short circuit and its fuse blows
(c) it charges the battery
(d) nothing

4.7.2 Intermediate level

Q10 Why should resistance be measured with the circuit unpowered?

(a) the meter needs the circuit's power
(b) resistors only work unpowered
(c) the circuit's own voltage gives false readings and can damage the meter
(d) it is faster

Q11 What is a cold solder joint?

(a) a dull, lumpy or cracked joint that did not flow properly
(b) a joint made with lead-free solder
(c) a joint cooled with water
(d) a perfect joint

Q12 Why set a current limit on a bench supply before testing a new circuit?

(a) to make the circuit faster
(b) to charge capacitors
(c) it is required for AC
(d) a short circuit is limited instead of burning parts

Q13 What should you do with a swollen LiPo battery?

(a) charge it fully
(b) stop using it and dispose of it safely
(c) puncture it to release gas
(d) keep using it carefully

Q14 What can an oscilloscope show that a multimeter cannot?

(a) resistance
(b) how a voltage changes over time, such as a PWM waveform
(c) continuity
(d) DC voltage

Q15 Which tool decodes I2C and SPI messages on several lines at once?

(a) a multimeter
(b) a glue gun
(c) a bench supply
(d) a logic analyser

Q16 Which connector suits a high-current battery lead?

(a) XT60
(b) Dupont jumper
(c) JST-SH 1 mm
(d) a breadboard hole

Q17 Why are breadboards poor for a finished mobile robot?

(a) they are too heavy
(b) they cannot carry 5 V
(c) vibration loosens wires and contacts
(d) they block Wi-Fi

4.7.3 Advanced level

Q18 A 1500 mAh 30C LiPo can deliver what maximum current?

(a) 1.5 A
(b) 30 A
(c) 450 A
(d) 45 A

Q19 A 5 V PWM signal at 40 % duty is measured with a multimeter on DC volts. About what does it read?

(a) 5 V
(b) 2 V
(c) 0.4 V
(d) 3.3 V

Q20 Measuring a 3.7 V battery, the meter shows -3.7 V. What happened?

(a) the probes are reversed
(b) the battery is dead
(c) the meter is on AC
(d) the battery is 7.4 V

Q21 Pick a fuse for a robot: given device currents (A) drawn at the same time, add 25 % margin and choose the next standard fuse (1, 2, 3, 5, 7.5, 10, 15, 20 A).

Q22 Simulate a multimeter reading voltages in a 9 V series circuit with 1 kΩ, 2.2 kΩ and 4.7 kΩ resistors: print the current and the voltage across each resistor, and check KVL.

Q23 Estimate how long a 2000 mAh battery lasts at currents of 250 mA, 500 mA and 1.2 A if only 80 % of capacity is used (to protect the cells).

Q24 Which is the safe soldering order?

(a) melt solder on the tip and carry it to the joint
(b) blow on the joint to cool it quickly
(c) heat pad and lead, feed solder into the joint, remove solder, then the iron, hold still
(d) move the part while the solder sets

Q25 A robot circuit works in Wokwi but not on the bench. Which is the most likely reason?

(a) real-world issues the simulator ignores, such as a loose wire, weak battery or noise
(b) Wokwi changes the code
(c) real Arduinos run different C++
(d) simulators always lie

4.8 Solutions

Q1 **Answer: (d) low-voltage DC, under about 24 V**

Stay with low-voltage DC.

Q2 **Answer: (b) in series with the circuit**

Current must flow through the meter.

Q3 **Answer: (a) across (in parallel with) the component**

Voltage is measured between two points.

Q4 **Answer: (b) beeps when two points are connected**

Finds breaks and shorts.

Q5 **Answer: (a) each short column of 5 holes, and each long power rail**

Columns of 5 and the rails.

Q6 **Answer: (c) black**

Red = +, black = GND.

Q7 **Answer: (d) electrostatic discharge**

Static can kill chips.

Q8 **Answer: (d) Wokwi**

Wokwi.

Q9 **Answer: (b) the meter acts as a short circuit and its fuse blows**

Current mode has almost zero resistance.

Q10 **Answer: (c) the circuit's own voltage gives false readings and can damage the meter**

Ohms mode supplies its own small current.

Q11 **Answer: (a) a dull, lumpy or cracked joint that did not flow properly**

Reheat with flux.

Q12 **Answer: (d) a short circuit is limited instead of burning parts**

Safe first power-up.

Q13 **Answer: (b) stop using it and dispose of it safely**

Swelling means internal damage.

Q14 **Answer: (b) how a voltage changes over time, such as a PWM waveform**

Time-based view.

Q15 **Answer: (d) a logic analyser**

Digital protocol decoding.

Q16 **Answer: (a) XT60**

Rated for tens of amps.

Q17 **Answer: (c) vibration loosens wires and contacts**

Move to perfboard or PCB.

Q18 **Answer: (d) 45 A**

1.5 × 30 = **45 A**.

Q19 **Answer: (b) 2 V**

Average = 5 × 0.4 = **2 V**.

Q20 **Answer: (a) the probes are reversed**

Red on +, black on -.

Q21 **Answer: Solution program**

The fuse must carry normal current without blowing but stop a short: 4.4 A × 1.25 = 5.5 A → 7.5 A fuse.

```
loads = {"2 drive motors": 2 * 1.2, "servo": 0.8, "Raspberry Pi": 1.0, "sensors": 0.2}
total = sum(loads.values())
need = total * 1.25
fuse = next(f for f in (1, 2, 3, 5, 7.5, 10, 15, 20) if f >= need)
print(f"total {total:.1f} A, with margin {need:.2f} A -> fuse {fuse} A")
```

OUTPUT

```
total 4.4 A, with margin 5.50 A -> fuse 7.5 A
```

Q22 **Answer: Solution program**

The same current flows through each resistor; drops are proportional to resistance and add up to the supply.

```
V, rs = 9.0, [1000, 2200, 4700]
I = V / sum(rs)
drops = [I * r for r in rs]
print(f"current {I * 1000:.3f} mA")
for r, d in zip(rs, drops):
    print(f"  across {r} ohm: {d:.2f} V")
print(f"sum of drops {sum(drops):.2f} V = supply {V} V")
```

OUTPUT

```
current 1.139 mA
  across 1000 ohm: 1.14 V
  across 2200 ohm: 2.51 V
  across 4700 ohm: 5.35 V
sum of drops 9.00 V = supply 9.0 V
```

Q23 **Answer: Solution program**

Runtime = usable capacity ÷ current; never plan to drain lithium cells completely.

```
cap, usable = 2000, 0.8
for ma in (250, 500, 1200):
    h = cap * usable / ma
    print(f"{ma:5d} mA -> {h:.1f} h ({h * 60:.0f} min)")
```

OUTPUT

```
  250 mA -> 6.4 h (384 min)
  500 mA -> 3.2 h (192 min)
 1200 mA -> 1.3 h (80 min)
```

Q24 **Answer: (c) heat pad and lead, feed solder into the joint, remove solder, then the iron, hold still**

Heat the joint, not the solder.

Q25 **Answer: (a) real-world issues the simulator ignores, such as a loose wire, weak battery or noise**

Check wiring, power and connections with a multimeter.

Work on low-voltage DC, fuse batteries, treat lithium cells with care (correct charger, no deep discharge, no swollen packs). Use breadboards for experiments, perfboard/PCB for robots. Multimeter: volts across, amps in series, ohms and continuity unpowered. Oscilloscopes show waveforms; logic analysers decode buses; bench supplies limit current. Solder by heating pad and lead, then feeding solder. Test ideas in Tinkercad or Wokwi first.

MODULE 05 · BEGINNER

Mechanical basics for robots

Minimum duration **2 days** | Level **Beginner**

Robots move, so understanding forces, motion and mechanisms is essential. This module covers force, mass, weight and Newton's laws, friction, speed, velocity and acceleration; torque and the relation between torque, speed and power; levers, wheels and axles, pulleys, inclined planes and screws; spur, bevel and worm gears, gear ratios, belts, chains, linkages and cams; materials, centre of gravity and stability; and degrees of freedom, followed by 25 solved questions and programs.

IN THIS MODULE

5.1	Forces and motion	5.2	Torque and power
5.3	Simple machines	5.4	Mechanisms
5.5	Structure and materials	5.6	Degrees of freedom (introduction)
5.7	Practice questions	5.8	Solutions

5.1 Forces and motion

5.1.1 Force, mass, weight and Newton's laws

Mass (kg) is how much matter an object has; **weight** is the force of gravity on it: $W = m \times g$, with $g = 9.81$ m/s² on Earth. A 2 kg robot weighs $2 \times 9.81 = 19.6$ N (newtons).

Law	Statement	In a robot
1st (inertia)	an object stays at rest or keeps moving in a straight line unless a force acts	a robot keeps rolling after the motors stop; brakes or reverse torque are needed
2nd	$F = m \times a$: force = mass × acceleration	a heavier robot needs more force (motor torque) to accelerate equally
3rd	every action has an equal and opposite reaction	wheels push the floor back; the floor pushes the robot forward; propellers push air down to lift a drone

5.1.2 Friction

Friction opposes sliding: $F_{friction} = \mu \times N$, where μ is the coefficient of friction and N the normal force (usually the weight). Wheels need friction to grip and push; too little and they spin, too much (skid-steer turning) wastes power. Rubber on concrete has $\mu \approx 0.6$–0.8; plastic on smooth tile ≈ 0.2–0.3. Rolling resistance (wheels) is far smaller than sliding friction.

5.1.3 Speed, velocity and acceleration

Speed = distance ÷ time (m/s). **Velocity** is speed in a given direction. **Acceleration** = change in velocity ÷ time (m/s²). A robot that goes from 0 to 0.5 m/s in 2 s accelerates at 0.25 m/s². For a wheel, linear speed = wheel circumference × revolutions per second: v = π × D × rpm ÷ 60.

FORCES.PY

```
import math
g = 9.81
mass, accel = 2.5, 0.4                    # kg, m/s^2
print(f"weight {mass * g:.1f} N; force to accelerate at {accel} m/s^2: {mass * accel:.1f} N")
for mu, surface in ((0.7, "rubber on concrete"), (0.25, "plastic on tile")):
    print(f"max grip force on {surface}: {mu * mass * g:.1f} N")
D, rpm = 0.065, 200
v = math.pi * D * rpm / 60
print(f"65 mm wheel at {rpm} rpm -> {v:.2f} m/s ({v * 3.6:.2f} km/h)")
```

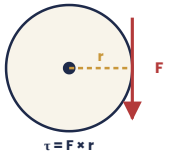
OUTPUT

```
weight 24.5 N; force to accelerate at 0.4 m/s^2: 1.0 N
max grip force on rubber on concrete: 17.2 N
max grip force on plastic on tile: 6.1 N
65 mm wheel at 200 rpm -> 0.68 m/s (2.45 km/h)
```

5.2 Torque and power

5.2.1 Torque and its importance for motors

Torque is turning force: τ = F × r (newton-metres, N·m), force times the distance from the axis. A motor's torque turns the wheel; the wheel pushes the ground with force F = τ ÷ r. Motor datasheets often give torque in kg·cm (1 kg·cm = 0.0981 N·m). **Stall torque** is the maximum torque when the shaft is held still (the motor then draws its stall current and heats quickly); design for at most about 25–50 % of stall torque in normal running.



TORQUE = FORCE × RADIUS

WORKED EXAMPLE

Torque to climb a ramp

A 3 kg robot on a 10° ramp: the force along the slope is m·g·sin θ = 3 × 9.81 × 0.174 = 5.11 N. Add rolling resistance (say 0.6 N) → 5.7 N. With 65 mm wheels (r = 0.0325 m) the total torque is 5.7 × 0.0325 = 0.185 N·m, so each of two motors needs **0.093 N·m ≈ 0.95 kg·cm**; choose motors with at least 2–4 kg·cm stall torque.

5.2.2 Relation between torque, speed and power

Mechanical power $P = \tau \times \omega$, where ω (rad/s) = rpm × 2π ÷ 60. For a given power, more torque means less speed.

A DC motor's torque falls linearly from stall torque (at zero speed) to zero at its free-running speed; mechanical power peaks at half the free speed. Gears trade speed for torque without (ideally) changing power.

TORQUE_SPEED.PY

```
import math
stall_t, free_rpm = 0.25, 300            # N*m, rpm (example gear motor)
print(" rpm   torque N*m   power W")
for rpm in range(0, 301, 50):
    t = stall_t * (1 - rpm / free_rpm)
    p = t * rpm * 2 * math.pi / 60
    print(f"{rpm:4d}   {t:9.3f}   {p:7.2f}")
```

OUTPUT

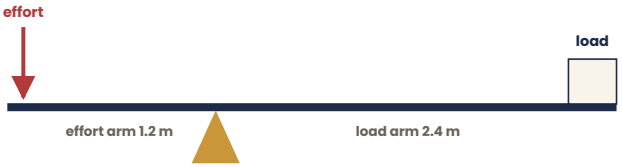
```
 rpm   torque N*m   power W
   0       0.250      0.00
  50       0.208      1.09
 100       0.167      1.75
 150       0.125      1.96
 200       0.083      1.75
 250       0.042      1.09
 300       0.000      0.00
```

5.3 Simple machines

5.3.1 Levers, wheels and axles, pulleys, inclined planes, screws

Simple machines trade distance for force. **Mechanical advantage** (MA) = output force ÷ input force. You never get more work out than you put in: a larger force always moves through a smaller distance.

Machine	Mechanical advantage	Robot example
Lever	effort arm ÷ load arm	gripper jaws, arm links
Wheel and axle	wheel radius ÷ axle radius	wheels, steering knobs
Pulley system	number of supporting rope sections	lifting mechanisms, cable-driven grippers
Inclined plane	slope length ÷ height	ramps; climbing robots
Screw	2π × lever radius ÷ pitch	leadscrews in linear actuators and 3D printers



A LEVER: MA = EFFORT ARM ÷ LOAD ARM (HERE 1.2 ÷ 2.4 = 0.5, TRADING FORCE FOR DISTANCE AND SPEED)

5.4 Mechanisms

5.4.1 Gears: spur, bevel, worm

Gear	Shafts	Notes
Spur	parallel	simplest, efficient (≈ 95 % per stage)
Bevel	at 90°	turns the drive round a corner
Worm and wheel	at 90°	very high ratio in one stage (e.g. 30:1); usually self-locking (holds position without power), but inefficient
Planetary	in line	compact high ratios, used in many gear motors and robot joints
Rack and pinion	rotation ↔ straight line	steering, linear slides

5.4.2 Gear ratios: trading speed for torque

Gear ratio = driven teeth ÷ driver teeth. Output speed = input speed ÷ ratio; output torque ≈ input torque × ratio × efficiency.

WORKED EXAMPLE

A two-stage gearbox

Stage 1: 10-tooth pinion drives a 40-tooth gear (4:1). Stage 2: 12 teeth drive 36 (3:1). Total ratio 4 × 3 = **12:1**. A motor at 6000 rpm and 0.02 N·m gives 6000 ÷ 12 = **500 rpm** and 0.02 × 12 × 0.95² = **0.217 N·m** at the output.

GEARS.PY

```
def gear_train(motor_rpm, motor_torque, stages, eff=0.95):
    ratio, torque = 1.0, motor_torque
    for driver, driven in stages:
        ratio *= driven / driver
        torque *= driven / driver * eff
    return ratio, motor_rpm / ratio, torque

ratio, rpm, torque = gear_train(6000, 0.02, [(10, 40), (12, 36)])
print(f"ratio {ratio:.0f}:1 -> {rpm:.0f} rpm, {torque:.3f} N*m")
ratio, rpm, torque = gear_train(6000, 0.02, [(10, 50), (10, 50), (12, 24)])
print(f"ratio {ratio:.0f}:1 -> {rpm:.0f} rpm, {torque:.3f} N*m")
```

OUTPUT

```
ratio 12:1 -> 500 rpm, 0.217 N*m
ratio 50:1 -> 120 rpm, 0.857 N*m
```

5.4.3 Belts and chains

Timing belts (GT2 in 3D printers) transmit motion between distant shafts quietly and without slipping; the ratio is pulley teeth driven ÷ driver, as with gears. **Chains** carry larger forces (bicycles, big robots). Both need correct tension: too loose and they skip, too tight and bearings wear.

5.4.4 Linkages and cams

A **linkage** joins rigid bars with pivots to convert one motion into another: the four-bar linkage (wipers, walking robot legs), the crank-slider (rotation ↔ back-and-forth, as in an engine) and parallel linkages that keep a gripper level. A **cam** is a shaped wheel whose rotation pushes a follower in a programmed pattern: automatons and many toys are cam-driven.

5.5 Structure and materials

5.5.1 Acrylic, aluminium, wood, 3D-printed plastics

Material	Pros	Cons
Acrylic (laser-cut)	cheap, neat, easy to cut	brittle; cracks at screw holes
Aluminium (sheet, extrusion)	strong, light, stiff; 2020 extrusion for frames	needs cutting and drilling tools
Plywood / MDF	cheap, easy to work with hand tools	heavier, affected by moisture
3D-printed PLA / PETG	any shape, fast iteration	weaker along layers; PLA softens around 55–60 °C
Carbon fibre	very stiff and light (drones)	expensive; conductive dust when cut

5.5.2 Centre of gravity and stability

The **centre of gravity** (CG) is the point where the robot's weight acts. A robot stays upright while the vertical line through its CG falls inside its **support polygon** (the area between its wheels or feet). Keep heavy parts (battery, motors) **low and central**, and widen the wheelbase for stability; arms reaching out or climbing slopes shift the CG and can tip the robot.

WORKED EXAMPLE

Tipping on a slope

A robot with its CG 8 cm above the ground and wheels 12 cm apart (6 cm each side of the centre) tips sideways when $\tan\theta > 6 \div 8 = 0.75$, i.e. on a slope steeper than **36.9°**. Lowering the CG to 5 cm raises the limit to $\tan^{-1}(1.2)$ = **50.2°**.

5.6 Degrees of freedom (introduction)

Degrees of freedom (DOF) count the independent ways a body or mechanism can move. A free object in space has 6 DOF: 3 translations (x, y, z) and 3 rotations (roll, pitch, yaw). A wheeled robot on the floor moves in x, y and rotates about z (3 DOF in the plane, though a differential-drive robot cannot slide sideways). A robot arm has one DOF per joint: 6 joints let its gripper reach any position and orientation within its workspace.

PRACTICE CHECKPOINT

Calculate gear ratios for a robot drive, build a simple geared mechanism, and explain why a heavier robot needs more motor torque.

5.7 Practice questions

5.7.1 Basic level

Q1 What is the weight of a 3 kg robot on Earth (g = 9.81 m/s²)?

(a) 3 N **(b)** 9.81 N **(c)** 294 N **(d)** 29.4 N

Q2 Which Newton's law is F = m × a?

(a) the first **(b)** the third **(c)** the second **(d)** none

Q3 What is torque?

(a) speed of rotation
(b) electrical power
(c) turning force: force × distance from the axis
(d) a type of gear

Q4 What is the unit of torque?

(a) watt **(b)** newton per second
(c) rpm **(d)** newton-metre (N·m)

Q5 Which gear changes the drive direction by 90°?

(a) spur gear **(b)** bevel gear **(c)** idler gear **(d)** pulley

Q6 What does a higher gear ratio (reduction) give at the output?

(a) more torque, less speed **(b)** more speed, less torque
(c) more of both **(d)** less of both

Q7 Where should heavy parts be placed for stability?

(a) high at the front **(b)** low and central
(c) on one side **(d)** on top

Q8 How many degrees of freedom does a free object in 3-D space have?

(a) 6 **(b)** 3 **(c)** 2 **(d)** 12

5.7.2 Intermediate level

Q9 Which gear type is usually self-locking?

(a) spur **(b)** bevel **(c)** worm and wheel **(d)** planetary

Q10 What is stall torque?

(a) the torque at full speed
(b) torque with no load
(c) the torque of a servo at 90°
(d) the maximum torque when the motor shaft is held stopped

Q11 Why does a robot keep rolling after the motors stop?

(a) gravity pulls it forward **(b)** friction pushes it
(c) the battery is still on **(d)** inertia (Newton's first law)

Q12 How does friction help a wheeled robot?

(a) it makes motors faster
(b) it reduces weight
(c) it lets the wheels grip the floor and push the robot
(d) it is never useful

Q13 A 15-tooth gear drives a 45-tooth gear. What is the ratio?

(a) 3:1 **(b)** 1:3 **(c)** 30:1 **(d)** 60:1

Q14 What converts rotation into straight-line motion?

(a) bevel gear **(b)** spur gear pair
(c) worm wheel only **(d)** rack and pinion (or a leadscrew)

Q15 What is the mechanical advantage of a lever with 40 cm effort arm and 10 cm load arm?

(a) 0.25 **(b)** 4 **(c)** 50 **(d)** 400

Q16 Why should a motor run well below its stall torque?

(a) it is faster at stall
(b) near stall it draws high current and overheats
(c) stall torque is always too low to measure
(d) gears break at low torque

5.7.3 Advanced level

Q17 A 70 mm wheel turns at 150 rpm. What is the robot speed?

(a) about 5.5 m/s **(b)** about 0.055 m/s
(c) about 1.5 m/s **(d)** about 0.55 m/s

Q18 A motor gives 0.05 N·m at 3000 rpm through a 20:1 gearbox at 90 % efficiency. What is the output torque?

(a) 0.9 N·m **(b)** 1 N·m **(c)** 0.05 N·m **(d)** 0.0025 N·m

Q19 What force is needed to accelerate a 4 kg robot at 0.5 m/s²?

(a) 8 N **(b)** 0.125 N **(c)** 2 N **(d)** 19.6 N

Q20 A wheel of radius 3 cm is driven with 0.12 N·m. What push force does it give at the ground?

(a) 0.36 N **(b)** 0.04 N **(c)** 40 N **(d)** 4 N

Q21 A 2.5 kg robot must climb a 15° ramp with 50 mm wheels and two motors. Compute the torque per motor (in N·m and kg·cm), including 0.5 N rolling resistance and a 2× safety factor.

Q22 Find a two-stage gear train (each stage from 10/20/30/40/50 driven by 10 or 12 teeth) giving a ratio as close as possible to 18:1, and print the result.

Q23 Compute the slope at which a robot tips sideways for CG heights of 4, 6, 8 and 10 cm with a 14 cm track width.

Q24 A robot arm must hold a heavy part still when the power fails. Which gearing helps most?

(a) a direct drive with no gears **(b)** a worm gear (self-locking)
(c) a belt drive **(d)** a spur gear with high efficiency

Q25 Why does a heavier robot need more motor torque?

(a) F = m × a: more mass needs more force to accelerate and to climb, and force comes from torque ÷ wheel radius

(b) heavier robots have bigger batteries

(c) weight reduces friction

(d) it does not need more torque

5.8 Solutions

Q1 **Answer: (d) 29.4 N**

W = mg = 29.43 N.

Q2 **Answer: (c) the second**

2nd law.

Q3 **Answer: (c) turning force: force × distance from the axis**

$\tau = F \times r$.

Q4 **Answer: (d) newton-metre (N·m)**

N·m (or kg·cm).

Q5 **Answer: (b) bevel gear**

Bevel (and worm) gears.

Q6 **Answer: (a) more torque, less speed**

Trade speed for torque.

Q7 **Answer: (b) low and central**

Low CG.

Q8 **Answer: (a) 6**

3 translations + 3 rotations.

Q9 **Answer: (c) worm and wheel**

The wheel cannot drive the worm backwards.

Q10 **Answer: (d) the maximum torque when the motor shaft is held stopped**

Also maximum current: avoid running there.

Q11 **Answer: (d) inertia (Newton's first law)**

It needs a braking force.

Q12 **Answer: (c) it lets the wheels grip the floor and push the robot**

No friction = wheel spin.

Q13 **Answer: (a) 3:1**

45 ÷ 15 = 3.

Q14 **Answer: (d) rack and pinion (or a leadscrew)**

Rotary → linear.

Q15 **Answer: (b) 4**

40 ÷ 10 = 4.

Q16 **Answer: (b) near stall it draws high current and overheats**

Design for 25–50 % of stall.

Q17 **Answer: (d) about 0.55 m/s**

π × 0.07 × 150 ÷ 60 = **0.55 m/s**.

Q18 **Answer: (a) 0.9 N·m**

0.05 × 20 × 0.9 = **0.9 N·m**.

Q19 **Answer: (c) 2 N**

F = ma = 4 × 0.5.

Q20 **Answer: (d) 4 N**

F = τ ÷ r = 0.12 ÷ 0.03 = **4 N**.

Q21 **Answer: Solution program**

Slope force = mg sin θ; torque = force × wheel radius, shared between motors, then doubled for margin.

```
import math
m, theta, D, motors = 2.5, 15, 0.05, 2
force = m * 9.81 * math.sin(math.radians(theta)) + 0.5
torque_each = force * (D / 2) / motors * 2          # x2 safety factor
print(f"force along slope {force:.2f} N")
print(f"torque per motor {torque_each:.3f} N*m = {torque_each / 0.0981:.2f} kg*cm")
```

OUTPUT

```
force along slope 6.85 N
torque per motor 0.171 N*m = 1.75 kg*cm
```

Q22 **Answer: Solution program**

Stage ratios multiply. No combination of these teeth gives exactly 18:1 (it would need 10→30 then 10→60, or 12→36 then 10→60), so the search returns the closest: 12→50 twice = 17.36:1, about 3.5 % lower.

```
best = None
drivers, driven = (10, 12), (10, 20, 30, 40, 50)
for a in drivers:
    for b in driven:
        for c in drivers:
            for d in driven:
                r = (b / a) * (d / c)
                if best is None or abs(r - 18) < abs(best[0] - 18):
                    best = (r, a, b, c, d)
r, a, b, c, d = best
print(f"{a}->{b} then {c}->{d}: ratio {r:.2f}:1")
```

OUTPUT

```
12->50 then 12->50: ratio 17.36:1
```

Q23 **Answer: Solution program**

Tipping angle = atan(half track ÷ CG height): a lower centre of gravity tolerates much steeper slopes.

```
import math
half_track = 14 / 2
for h in (4, 6, 8, 10):
    angle = math.degrees(math.atan(half_track / h))
    print(f"CG height {h:2d} cm -> tips at {angle:.1f} degrees")
```

OUTPUT

```
CG height  4 cm -> tips at 60.3 degrees
CG height  6 cm -> tips at 49.4 degrees
CG height  8 cm -> tips at 41.2 degrees
CG height 10 cm -> tips at 35.0 degrees
```

Q24 **Answer: (b) a worm gear (self-locking)**

Self-locking holds the load.

Q25 **Answer: (a) F = m × a: more mass needs more force to accelerate and to climb, and force comes from torque ÷ wheel radius**

More mass → more force → more torque.

Weight = mg; F = ma; friction = μN. Wheel speed v = πD·rpm/60. Torque τ = F × r; push force = τ/r; power = τ × ω. Gear ratio = driven ÷ driver teeth: speed ÷ ratio, torque × ratio × efficiency. Worm gears self-lock. Keep the centre of gravity low and inside the wheelbase. Free bodies have 6 DOF; each arm joint adds one.

MODULE 06 · BEGINNER

Programming foundations for robotics

Minimum duration **2 days** | Level **Beginner**

Robots are controlled by software, and this module builds the logical and programming foundation: computational thinking (decomposition, patterns, abstraction, algorithms), flowcharts and pseudo-code, block-based programming, and text programming in both C/C++ (Arduino) and Python: variables, data types, operators, conditions, loops, functions and arrays, then programming for hardware with inputs, outputs, timing and serial debugging, followed by 25 solved questions and programs.

IN THIS MODULE

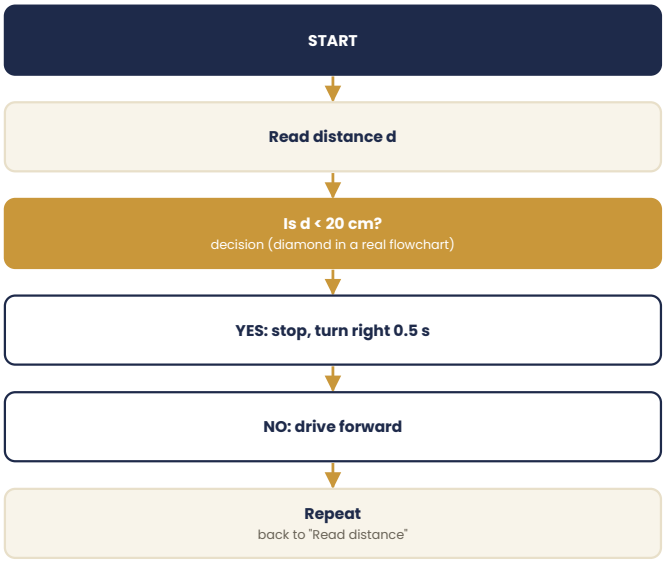
6.1 Computational thinking
6.2 Block-based programming
6.3 Text programming basics (C/C++ and Python)
6.4 Programming for hardware
6.5 Practice questions
6.6 Solutions

6.1 Computational thinking

6.1.1 Decomposition, patterns, abstraction, algorithms

Skill	Meaning	Line-follower example
Decomposition	break a big problem into small parts	read sensors · decide direction · drive motors · detect the end
Pattern recognition	spot what repeats	the same "read → decide → drive" happens every few milliseconds
Abstraction	hide details behind simple names	a function drive(left, right) hides the motor-driver pins
Algorithm	exact step-by-step instructions	if the left sensor sees the line, turn left; if the right does, turn right; else go straight

6.1.2 Flowcharts and pseudo-code



FLOWCHART OF AN OBSTACLE AVOIDER

THE SAME ALGORITHM AS PSEUDO-CODE

```
LOOP forever
    d ← read ultrasonic distance
    IF d < 20 THEN
        stop motors
        turn right for 0.5 s
    ELSE
        drive forward
    END IF
END LOOP
```

Flowchart shape	Meaning
oval	start / end
rectangle	process (an action)
diamond	decision (yes/no question)
parallelogram	input / output (read a sensor, print)
arrow	flow of control

6.2 Block-based programming

6.2.1 Scratch, mBlock or Tinkercad codeblocks for beginners

Block languages let beginners build programs by snapping together coloured blocks, so there are no syntax errors to fight. **Scratch** teaches logic; **mBlock** (Scratch-based) programs Arduino and mBot robots; **Tinkercad Codeblocks** program a simulated Arduino and can show the equivalent C++ code; **MakeCode** serves micro:bit and similar boards. Every block maps to code: "forever" = loop(), "if … then … else" = if/else, "wait 1 seconds" = delay(1000).

Block	Arduino C++	Python
when started / forever	void setup() {} / void loop() {}	while True:
set pin 13 HIGH	digitalWrite(13, HIGH);	led.on()
wait 0.5 seconds	delay(500);	time.sleep(0.5)
if distance < 20 then … else …	if (d < 20) { … } else { … }	if d < 20: … else: …
repeat 10	for (int i = 0; i < 10; i++) { … }	for i in range(10):

SHORTCUT TRICK
Prototype the logic in blocks, then use the "show code" view to learn the matching C++. Moving to text early pays off: real robots need libraries, functions and debugging that blocks make awkward.

6.3 Text programming basics (C/C++ and Python)

Arduino-style microcontrollers are programmed in **C/C++**; Raspberry Pi robots, ROS 2 and AI use **Python**. The ideas are identical; the syntax differs: C++ needs declared types, semicolons and braces { }; Python uses indentation.

6.3.1 Variables and data types

Type	C/C++ (Arduino Uno)	Python	Example
integer	int (-32 768…32 767), long (±2.1 billion)	int (unlimited)	count = 5
decimal	float (≈ 7 digits)	float	speed = 0.45
true/false	bool	bool (True/False)	blocked = false
character / text	char, String	str	"LEFT"
byte	byte (0–255)	int	pwm = 200

WATCH OUT
On an Arduino Uno an `int` is only 16 bits: 300 × 200 = 60 000 overflows to a negative number. Use `long` for large values and `unsigned long` for millis() timestamps.

6.3.2 Operators

Arithmetic + - * / %, comparison == != < > <= >=, logical (C++: && || !, Python: and or not), assignment = += -=. Integer division differs: in C++ `7 / 2` is 3 (both ints); in Python `7 / 2` is 3.5 and `7 // 2` is 3.

6.3.3 Conditions (if-else)

6.3.4 Loops (for, while)

6.3.5 Functions

6.3.6 Arrays and lists

The four ideas together, first in Arduino C++ and then the same program in Python:

BASICS.INO

```
int readings[5] = {42, 18, 35, 9, 27};      // distances in cm (an array)

String decide(int d) {                      // a function returning text
  if (d < 10) return "STOP";
  else if (d < 25) return "SLOW";
  else return "GO";
}

void setup() {
  Serial.begin(9600);
  int total = 0;
  for (int i = 0; i < 5; i++) {             // for loop over the array
    Serial.print(readings[i]);
    Serial.print(" cm -> ");
    Serial.println(decide(readings[i]));
    total += readings[i];
  }
  Serial.print("average: ");
  Serial.println(total / 5.0);
  int countdown = 3;
  while (countdown > 0) {                   // while loop
    Serial.print(countdown--);
    Serial.print(" ");
  }
  Serial.println("go!");
}

void loop() {}
```

SERIAL MONITOR

```
42 cm -> GO
18 cm -> SLOW
35 cm -> GO
9 cm -> STOP
27 cm -> GO
average: 26.20
3 2 1 go!
```

BASICS.PY

```
readings = [42, 18, 35, 9, 27]          # a list

def decide(d):                          # a function
    if d < 10:
        return "STOP"
    elif d < 25:
        return "SLOW"
    else:
        return "GO"

for d in readings:                      # for loop over the list
    print(d, "cm ->", decide(d))
print("average:", sum(readings) / len(readings))
countdown = 3
while countdown > 0:                    # while loop
    print(countdown, end=" ")
    countdown -= 1
print("go!")
```

OUTPUT

```
42 cm -> GO
18 cm -> SLOW
35 cm -> GO
9 cm -> STOP
27 cm -> GO
average: 26.2
3 2 1 go!
```

6.4 Programming for hardware

6.4.1 Reading inputs and controlling outputs

Every robot program has the same skeleton: **configure** pins once (setup), then **repeatedly read inputs, decide, and set outputs** (loop). Digital inputs give HIGH/LOW (buttons, IR sensors); analog inputs give numbers (0–1023 on an Uno); outputs switch LEDs and drivers or produce PWM.

INPUTOUTPUT.INO

```
const int BUTTON = 2, LED = 13, POT = A0, MOTOR_PWM = 9;

void setup() {
  pinMode(BUTTON, INPUT_PULLUP);    // pressed = LOW
  pinMode(LED, OUTPUT);
  pinMode(MOTOR_PWM, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  bool run = digitalRead(BUTTON) == LOW;          // input: digital
  int knob = analogRead(POT);                     // input: analog 0..1023
  int speed = run ? map(knob, 0, 1023, 0, 255) : 0;
  analogWrite(MOTOR_PWM, speed);                  // output: PWM 0..255
  digitalWrite(LED, run);                         // output: digital
  Serial.print("run="); Serial.print(run);
  Serial.print(" knob="); Serial.print(knob);
  Serial.print(" pwm="); Serial.println(speed);
  delay(250);
}
```

SERIAL MONITOR

```
run=0 knob=200 pwm=0
run=0 knob=512 pwm=0
run=1 knob=512 pwm=127
run=1 knob=900 pwm=224
run=1 knob=900 pwm=224
run=1 knob=1023 pwm=255
```

6.4.2 Timing and delays

`delay(ms)` / `time.sleep(s)` pause everything: the robot cannot read sensors while waiting. For multitasking use the **millis() pattern**: remember when something last happened and act when enough time has passed, so the loop never blocks.

NONBLOCKING.INO

```
const int LED = 13;
unsigned long lastBlink = 0, lastPrint = 0;
bool ledOn = false;

void setup() { pinMode(LED, OUTPUT); Serial.begin(9600); }

void loop() {
  unsigned long now = millis();
  if (now - lastBlink >= 500) {        // task 1: toggle LED every 500 ms
    lastBlink = now;
    ledOn = !ledOn;
    digitalWrite(LED, ledOn);
  }
  if (now - lastPrint >= 1200) {       // task 2: report every 1.2 s
    lastPrint = now;
    Serial.print(now); Serial.print(" ms: LED is ");
    Serial.println(ledOn ? "on" : "off");
  }
  // sensors could be read here every pass: nothing blocks
}
```

SERIAL MONITOR

```
1200 ms: LED is off
2400 ms: LED is off
3600 ms: LED is on
```

6.4.3 Debugging with serial print

`Serial.print()` (Arduino) and `print()` (Python) are the first debugging tools: print sensor values, decisions and timings to see what the program really does. Label every value, print at a sensible rate (not thousands of lines per second), and use the Arduino IDE's **Serial Plotter** to graph numbers. Remove or reduce prints in the final version; they take time.

SHORTCUT TRICK

When a robot misbehaves, print **inputs, decisions and outputs** on one line per loop: "d=18 decision=SLOW pwm=120". The bug is almost always visible in the first few lines.

PRACTICE CHECKPOINT

Draw flowcharts for a traffic light, a line follower and an obstacle avoider, and write equivalent programs in block code and in C/C++ or Python.

6.5 Practice questions

6.5.1 Basic level

Q1 Breaking a big problem into smaller parts is called

(a) abstraction (b) iteration (c) decomposition (d) compilation

Q2 Which flowchart shape is a decision?

(a) diamond (b) rectangle (c) oval (d) parallelogram

Q3 Which Arduino function runs over and over?

(a) setup() (b) loop() (c) main() (d) repeat()

Q4 Which language is normally used on an Arduino?

(a) HTML (b) SQL (c) Scratch only (d) C/C++

Q5 What does delay(1000) do?

(a) repeats 1000 times
(b) sets a pin HIGH
(c) pauses the program for 1 second
(d) waits 1000 seconds

Q6 Which data type stores true/false?

(a) int (b) float (c) bool (d) char

Q7 What does Serial.println() help with?

(a) driving motors
(b) reading sensors
(c) charging the battery
(d) debugging by printing values to the computer

Q8 Which block environment programs Arduino and mBot robots?

(a) Excel (b) mBlock (c) Photoshop (d) Kodi

Q9 Which shape marks start and end in a flowchart?

(a) diamond (b) rectangle (c) arrow (d) oval

6.5.2 Intermediate level

Q10 Hiding the details of motor pins behind drive(left, right) is

(a) abstraction (b) decomposition (c) debugging (d) recursion

Q11 In C++ on an Arduino, what is 7 / 2 (both ints)?

(a) 3.5
(b) 3
(c) 4
(d) 3.0 exactly as float

Q12 In Python, what is 7 // 2?

(a) 3 (b) 3.5 (c) 4 (d) 2

Q13 Why can delay() be a problem in robots?

(a) it uses too much memory

(b) it damages the board
(c) the program cannot read sensors or react while it waits
(d) it only works once

Q14 What range does analogRead() return on an Uno?

(a) 0–255 (b) 0–5 (c) 0–4095 (d) 0–1023

Q15 What range does analogWrite() take on an Uno?

(a) 0–1023 (b) 0–100 (c) 0–5 (d) 0–255

Q16 Which loop is best when the number of repeats is known?

(a) while (b) do-while only (c) for (d) if

Q17 What does map(512, 0, 1023, 0, 255) return?

(a) 127 (b) 512 (c) 255 (d) 64

6.5.3 Advanced level

Q18 An Arduino int holds a maximum of

(a) 65 535 (b) 255 (c) 2 147 483 647 (d) 32 767

Q19 What happens on an Uno to int x = 250 * 200;?

(a) x = 50 000
(b) it overflows and gives a wrong (negative) value
(c) compile error
(d) x = 250

Q20 Write an Arduino sketch that prints the numbers 1 to 10 and, for each, whether it is EVEN or ODD.

Q21 Write an Arduino sketch that finds the largest and smallest values in an array of 6 sensor readings and prints them.

Q22 Write a Python traffic-light controller that cycles GREEN 3 s, YELLOW 1 s, RED 3 s twice, printing the time each light turns on.

Q23 Write a Python line-follower decision function for three sensors (left, centre, right; 1 = line seen) and test all eight combinations.

Q24 Write an Arduino sketch that blinks an LED on pin 13 every 300 ms AND prints a count every second without using delay().

Q25 Which is the correct order of a robot program's main loop?

(a) set outputs → stop → read inputs
(b) read inputs → decide → set outputs → repeat
(c) decide → read inputs once → end
(d) print → delay(10000) → read

6.6 Solutions

Q1 **Answer: (c) decomposition**

Decomposition.

Q2 **Answer: (a) diamond**

Yes/no questions.

Q3 **Answer: (b) loop()**

setup() runs once.

Q4 **Answer: (d) C/C++**

Arduino C/C++.

Q5 **Answer: (c) pauses the program for 1 second**

Milliseconds.

Q6 **Answer: (c) bool**

Boolean.

Q7 **Answer: (d) debugging by printing values to the computer**

Serial monitor output.

Q8 **Answer: (b) mBlock**

Scratch-based mBlock.

Q9 **Answer: (d) oval**

Terminator.

Q10 **Answer: (a) abstraction**

Abstraction.

Q11 **Answer: (b) 3**

Integer division truncates.

Q12 **Answer: (a) 3**

Floor division.

Q13 **Answer: (c) the program cannot read sensors or react while it waits**

Use millis() instead.

Q14 **Answer: (d) 0–1023**

10-bit ADC.

Q15 **Answer: (d) 0–255**

8-bit PWM.

Q16 **Answer: (c) for**

for (i = 0; i < n; i++).

Q17 **Answer: (a) 127**

512 × 255 ÷ 1023 = 127.6 → 127 (integer).

Q18 **Answer: (d) 32 767**

16-bit signed.

Q19 **Answer: (b) it overflows and gives a wrong (negative) value**

50 000 > 32 767.

Q20 **Answer: Solution program**

The remainder operator % 2 is 0 for even numbers; the ternary operator chooses the text.

```
void setup() {
  Serial.begin(9600);
  for (int i = 1; i <= 10; i++) {
    Serial.print(i);
    Serial.println(i % 2 == 0 ? " EVEN" : " ODD");
  }
}
void loop() {}
```

SERIAL MONITOR

```
1 ODD
2 EVEN
3 ODD
4 EVEN
5 ODD
6 EVEN
7 ODD
8 EVEN
9 ODD
10 EVEN
```

Q21 **Answer: Solution program**

Start both with the first element, then compare each remaining one.

```
int r[6] = {512, 330, 870, 120, 640, 455};
void setup() {
  Serial.begin(9600);
  int lo = r[0], hi = r[0];
  for (int i = 1; i < 6; i++) {
    if (r[i] < lo) lo = r[i];
    if (r[i] > hi) hi = r[i];
  }
  Serial.print("min "); Serial.print(lo);
  Serial.print(" max "); Serial.println(hi);
}
void loop() {}
```

SERIAL MONITOR

```
min 120 max 870
```

Q22 **Answer: Solution program**

A list of (light, duration) pairs makes the sequence easy to change. (The clock is simulated, so the run took no real time.)

```
import time
SEQ = [("GREEN", 3), ("YELLOW", 1), ("RED", 3)]
t0 = time.monotonic()
for cycle in range(2):
    for light, secs in SEQ:
        print(f"{time.monotonic() - t0:4.1f} s: {light}")
        time.sleep(secs)
```

OUTPUT

```
 0.0 s: GREEN
 3.0 s: YELLOW
 4.0 s: RED
 7.0 s: GREEN
10.0 s: YELLOW
11.0 s: RED
```

Q23 **Answer: Solution program**

Covering every combination (including "lost" and "junction") is what makes a robot behave reliably.

```
from itertools import product
def steer(l, c, r):
    if c and not l and not r: return "straight"
    if l and not r: return "turn left"
    if r and not l: return "turn right"
    if l and c and r: return "junction: stop"
    return "line lost: search"
for l, c, r in product((0, 1), repeat=3):
    print(l, c, r, "->", steer(l, c, r))
```

OUTPUT

```
0 0 0 -> line lost: search
0 0 1 -> turn right
0 1 0 -> straight
0 1 1 -> turn right
1 0 0 -> turn left
1 0 1 -> line lost: search
1 1 0 -> turn left
1 1 1 -> junction: stop
```

Q24 **Answer: Solution program**

Two independent timers in one loop: the millis() pattern lets a robot do several things at once.

```
unsigned long tLed = 0, tCount = 0;
int count = 0;
bool on = false;
void setup() { pinMode(13, OUTPUT); Serial.begin(9600); }
void loop() {
  unsigned long now = millis();
  if (now - tLed >= 300) { tLed = now; on = !on; digitalWrite(13, on); }
  if (now - tCount >= 1000) { tCount = now; Serial.print("count "); Serial.println(++count); }
}
```

SERIAL MONITOR

```
count 1
count 2
count 3
```

Q25 **Answer: (b) read inputs → decide → set outputs → repeat**

Sense, think, act.

Think first: decompose, find patterns, abstract with functions, write the algorithm as a flowchart or pseudo-code. Blocks map one-to-one to code. In C/C++ and Python use variables (mind int overflow on Uno), operators, if/else, for/while, functions and arrays/lists. Hardware programs: configure pins, then loop read → decide → output; use millis() instead of long delays; debug with labelled serial prints.

MODULE 07 · BEGINNER

Microcontrollers and development boards

Minimum duration **1 day** | Level **Beginner**

The microcontroller is the brain of most small robots. This module explains microcontrollers versus microprocessors, CPU, flash, RAM, EEPROM, I/O pins and clock speed; compares the Arduino Uno, Nano and Mega, ESP32 and ESP8266, Raspberry Pi Pico, STM32 and single-board computers such as Raspberry Pi and NVIDIA Jetson; covers digital I/O, ADC inputs, PWM outputs and UART, I2C and SPI pins; shows how to choose a controller for a robot; and runs a first blink program, followed by 25 solved questions and programs.

IN THIS MODULE

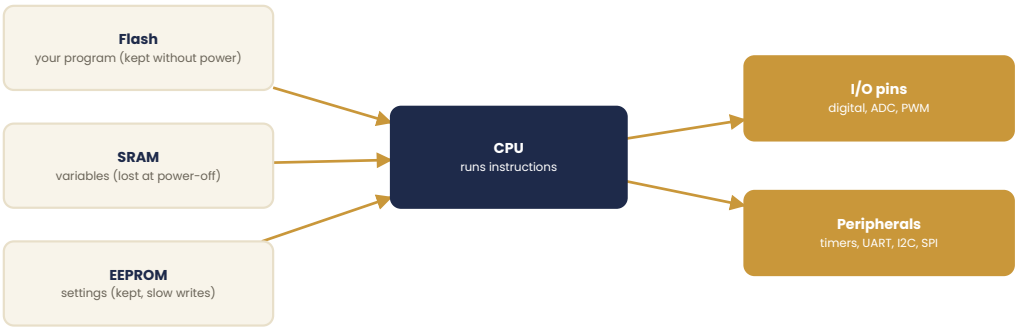
7.1 Microcontroller basics
7.2 Popular boards
7.3 Pins and interfaces
7.4 Choosing the right controller for a robot
7.5 First program: blinking an LED
7.6 Practice questions
7.7 Solutions

7.1 Microcontroller basics

7.1.1 Microcontroller vs microprocessor

	Microcontroller (MCU)	Microprocessor (MPU) / SBC
Contains	CPU + RAM + flash + I/O peripherals on one chip	CPU only; RAM, storage and I/O are separate chips
Software	one program, no operating system (or a small RTOS)	a full OS such as Linux
Start-up	instant	10–60 s
Timing	precise, real-time	can be delayed by the OS
Power	milliwatts	watts
Examples	ATmega328P (Uno), ESP32, RP2040, STM32	Raspberry Pi, Jetson, laptops

7.1.2 CPU, memory (flash, RAM, EEPROM), I/O pins



INSIDE A MICROCONTROLLER

Memory	Holds	Uno size	Survives power-off?
Flash	the compiled program	32 KB (0.5 KB used by the bootloader)	yes
SRAM	variables, the stack, buffers	2 KB	no
EEPROM	calibration values, settings	1 KB (about 100 000 writes per cell)	yes

SHORTCUT TRICK
On an Uno the 2 KB of SRAM runs out first. Keep constant text in flash with `Serial.println(F("text"))`, avoid big arrays and the String class in long-running code, and watch the "global variables use…" line when you compile.

7.1.3 Clock speed

The **clock** sets how many basic steps the CPU does per second: the Uno runs at 16 MHz (16 million cycles per second; most AVR instructions take 1–2 cycles), the ESP32 at up to 240 MHz with two cores, the Pico at 133 MHz (Pico 2: 150 MHz). Higher clocks mean faster maths and more complex tasks, at the cost of more power.

7.2 Popular boards

Board	Chip / clock	Memory	I/O	Logic	Wireless
7.2.1 Arduino Uno R3	ATmega328P, 16 MHz, 8-bit	32 KB flash, 2 KB RAM	14 digital (6 PWM), 6 analog	5 V	—
Arduino Nano	same as Uno, tiny board	same	14 digital, 8 analog	5 V	—
Arduino Mega 2560	ATmega2560, 16 MHz	256 KB flash, 8 KB RAM	54 digital (15 PWM), 16 analog, 4 UARTs	5 V	—
7.2.2 ESP32	dual-core 240 MHz, 32-bit	520 KB RAM, 4 MB flash	about 34 GPIO, 12-bit ADC, touch, DAC	3.3 V	Wi-Fi + Bluetooth
ESP8266 (NodeMCU)	80/160 MHz, single core	~80 KB RAM, 4 MB flash	about 11 GPIO, 1 ADC	3.3 V	Wi-Fi

Board	Chip / clock	Memory	I/O	Logic	Wireless
7.2.3 Raspberry Pi Pico	RP2040, dual 133 MHz	264 KB RAM, 2 MB flash	26 GPIO, 3 ADC, PIO	3.3 V	Pico W: Wi-Fi + BT
7.2.4 STM32 (e.g. Nucleo, Blue Pill)	Arm Cortex-M, 72–480 MHz	wide range	many timers, CAN, USB	3.3 V	some models

7.2.5 Single-board computers: Raspberry Pi and NVIDIA Jetson (overview)

When a robot needs a camera with computer vision, mapping, a web interface or AI, add a **single-board computer**: a **Raspberry Pi** (Linux, Python, ROS 2) or an **NVIDIA Jetson** (Orin Nano and similar, with a powerful GPU for deep-learning vision). Typically the SBC thinks and a microcontroller drives the motors in real time, connected by USB serial or UART.

7.3 Pins and interfaces

7.3.1 Digital I/O

Each digital pin can be an **output** (HIGH or LOW, driving an LED or a driver input; on an Uno about 20 mA recommended, 40 mA absolute maximum) or an **input** (reading a button or sensor, with optional internal pull-up). `pinMode()` chooses the direction.

7.3.2 Analog inputs (ADC)

An **ADC** converts a voltage into a number. Uno: 10-bit, 0–1023 for 0–5 V (4.9 mV per step). ESP32 and Pico: 12-bit, 0–4095 for 0–3.3 V (the ESP32 ADC is noticeably non-linear near its ends). Value → voltage: $V = \text{reading} \times V_{ref} \div (2^{bits} - 1)$.

7.3.3 PWM outputs

PWM switches a pin on and off rapidly; the duty cycle sets average power for LED brightness and motor speed. On the Uno, pins 3, 5, 6, 9, 10, 11 (marked ~) support analogWrite(0–255) at about 490 Hz (980 Hz on pins 5 and 6). The ESP32 and Pico can make PWM on almost any pin with adjustable frequency and resolution.

7.3.4 Communication pins (UART, I2C, SPI)

Bus	Uno pins	Typical robot devices
UART (serial)	D0 RX, D1 TX (shared with USB)	Bluetooth HC-05, GPS, Raspberry Pi link
I2C	A4 SDA, A5 SCL	MPU6050 IMU, OLED, PCA9685 servo driver, VL53L0X
SPI	D11 MOSI, D12 MISO, D13 SCK, D10 SS	SD card, nRF24L01 radio, fast displays

7.4 Choosing the right controller for a robot

Need	Good choice	Why
first robot, line follower, obstacle avoider	Arduino Uno / Nano	5 V, huge community, simple
many servos and sensors (hexapod, arm)	Arduino Mega or a PCA9685 with any board	lots of pins / dedicated servo driver
phone or Wi-Fi control, IoT robot	ESP32	Wi-Fi + Bluetooth built in, fast
MicroPython, precise timing, cheap	Raspberry Pi Pico (W)	PIO, dual core
vision, mapping, ROS 2	Raspberry Pi (+ a microcontroller for motors)	Linux, cameras, Python
deep-learning vision	NVIDIA Jetson	GPU acceleration

SHORTCUT TRICK
Count your pins first: list every sensor and actuator with the pins and buses it needs, add 20 % spare, then choose the smallest board that fits, checking logic voltage (5 V vs 3.3 V) for every module.

7.5 First program: blinking an LED

The classic first program blinks the built-in LED (pin 13 on the Uno, labelled L). Upload it with the Arduino IDE: choose the board and port, click Upload, and the LED blinks once a second. The panel below is the simulated run with pin changes logged.

BLINK.INO

```
void setup() {
  pinMode(LED_BUILTIN, OUTPUT);     // pin 13 on the Uno
}

void loop() {
  digitalWrite(LED_BUILTIN, HIGH);  // on
  delay(500);
  digitalWrite(LED_BUILTIN, LOW);   // off
  delay(500);
}
```

SIMULATED RUN (pin changes and Serial output)

```
[     0 ms] pin 13 -> HIGH
[   500 ms] pin 13 -> LOW
[  1000 ms] pin 13 -> HIGH
[  1500 ms] pin 13 -> LOW
[  2000 ms] pin 13 -> HIGH
[  2500 ms] pin 13 -> LOW
```

The same in MicroPython on a Pico:

BLINK.PY (MICROPYTHON)

```
from machine import Pin
import time
led = Pin(25, Pin.OUT)      # "LED" on Pico W
while True:
    led.toggle()
    time.sleep(0.5)
```

PRACTICE CHECKPOINT

Compare five development boards in a table, choose one for three robot ideas with reasons, and blink an LED on a real or simulated board.

7.6 Practice questions

7.6.1 Basic level

Q1 What does a microcontroller contain on one chip?

(a) CPU, RAM, flash memory and I/O peripherals
(b) only the CPU
(c) a GPU and a hard disk
(d) a full Linux PC

Q2 Which chip is on the Arduino Uno?

(a) ESP32 (b) RP2040 (c) ATmega328P (d) BCM2712

Q3 Where is the Arduino program stored?

(a) SRAM (b) EEPROM only (c) the USB cable (d) flash memory

Q4 Which memory loses its contents when power is removed?

(a) flash (b) SRAM (c) EEPROM (d) all of them

Q5 What is the clock speed of an Uno?

(a) 16 MHz (b) 240 MHz (c) 1 GHz (d) 16 kHz

Q6 Which board has built-in Wi-Fi and Bluetooth?

(a) Arduino Uno (b) Arduino Mega (c) ESP32 (d) Arduino Nano

Q7 Which Uno pins can do PWM?

(a) 3, 5, 6, 9, 10, 11
(b) all digital pins
(c) A0–A5
(d) 0 and 1

Q8 What logic level does the ESP32 use?

(a) 5 V (b) 3.3 V (c) 12 V (d) 1.8 V

Q9 Which board would you pick to learn MicroPython with precise timing?

(a) Raspberry Pi Pico
(b) Arduino Mega
(c) NVIDIA Jetson
(d) Arduino Uno

7.6.2 Intermediate level

Q10 What is EEPROM used for?

(a) running the program
(b) storing temporary variables
(c) generating PWM
(d) saving settings that must survive power-off

Q11 Which board suits a robot with 18 servos and many sensors?

(a) Arduino Nano alone with no driver
(b) ESP8266 alone
(c) Arduino Mega or a board with a PCA9685 servo driver

(d) a 555 timer

Q12 Which Uno pins are I2C?

(a) D0 and D1 (b) D11 and D13
(c) A4 (SDA) and A5 (SCL) (d) D2 and D3

Q13 Why add a Raspberry Pi to a robot that already has an Arduino?

(a) to supply 5 V
(b) because Arduinos cannot read buttons
(c) to make it lighter
(d) for vision, mapping, networking or AI that the microcontroller cannot handle

Q14 What resolution is the ESP32 ADC?

(a) 10-bit (0–1023) (b) 12-bit (0–4095)
(c) 8-bit (d) 16-bit

Q15 What does F("text") do in Arduino code?

(a) keeps the text in flash instead of using SRAM
(b) formats a float
(c) makes the text bold
(d) flashes the LED

Q16 Which NVIDIA board is used for deep-learning vision on robots?

(a) GeForce (b) Jetson (c) Uno (d) Tegra Nano Pro

Q17 Which Uno pins are shared with the USB serial connection?

(a) D0 and D1 (b) D12 and D13 (c) A0 and A1 (d) D9 and D10

7.6.3 Advanced level

Q18 A 10-bit ADC with a 5 V reference reads 614. What is the voltage?

(a) about 6.14 V (b) about 1.5 V (c) about 3.0 V (d) about 2.4 V

Q19 A 12-bit ADC with a 3.3 V reference reads 2048. What is the voltage?

(a) about 3.3 V (b) about 2.05 V (c) about 0.8 V (d) about 1.65 V

Q20 How long is one clock cycle at 16 MHz?

(a) 16 ns (b) 625 ns (c) 6.25 µs (d) 62.5 ns

Q21 Compute the voltage resolution (mV per step) of 8-, 10- and 12-bit ADCs with 3.3 V and 5 V references.

Q22 Help choose a board: given a robot's needs (digital pins, analog pins, wireless, Linux), print the first suitable board from Uno, Mega, ESP32, Pico W, Raspberry Pi.

Q23 Write an Arduino sketch that blinks the built-in LED in an SOS pattern once (3 short, 3 long, 3 short), printing each symbol.

Q24 Store a calibration value (an int offset of -12) in EEPROM, read it back and print it.

Q25 Which pairing is best for a mapping robot with a camera?

(a) a single Arduino Uno for everything
(b) an ESP8266 for vision
(c) Raspberry Pi for vision and mapping + Arduino or Pico for motor control
(d) a 555 timer and relays

7.7 Solutions

Q1 **Answer: (a) CPU, RAM, flash memory and I/O peripherals**
System on one chip.

Q2 **Answer: (c) ATmega328P**
ATmega328P.

Q3 **Answer: (d) flash memory**
Kept without power.

Q4 **Answer: (b) SRAM**
Variables live in SRAM.

Q5 **Answer: (a) 16 MHz**
16 MHz.

Q6 **Answer: (c) ESP32**
ESP32.

Q7 **Answer: (a) 3, 5, 6, 9, 10, 11**
Marked with ~.

Q8 **Answer: (b) 3.3 V**
3.3 V: not 5 V tolerant.

Q9 **Answer: (a) Raspberry Pi Pico**
MicroPython + PIO.

Q10 **Answer: (d) saving settings that must survive power-off**
Calibration, preferences.

Q11 **Answer: (c) Arduino Mega or a board with a PCA9685 servo driver**
Many pins / dedicated driver.

Q12 **Answer: (c) A4 (SDA) and A5 (SCL)**

A4/A5.

Q13 **Answer: (d) for vision, mapping, networking or AI that the microcontroller cannot handle**

Pi thinks; MCU acts.

Q14 **Answer: (b) 12-bit (0–4095)**

12-bit.

Q15 **Answer: (a) keeps the text in flash instead of using SRAM**

Saves scarce RAM.

Q16 **Answer: (b) Jetson**

Jetson family.

Q17 **Answer: (a) D0 and D1**

Avoid using them for other things while uploading.

Q18 **Answer: (c) about 3.0 V**

614 × 5 ÷ 1023 = **3.0 V**.

Q19 **Answer: (d) about 1.65 V**

2048 × 3.3 ÷ 4095 = **1.65 V**.

Q20 **Answer: (d) 62.5 ns**

1 ÷ 16 000 000 = **62.5 ns**.

Q21 **Answer: Solution program**

Resolution = V_ref ÷ (2^bits − 1): each extra bit halves the step size.

```
for bits in (8, 10, 12):
    steps = 2 ** bits - 1
    print(f"{bits:2d}-bit: 3.3 V -> {3300 / steps:6.2f} mV/step   5 V -> {5000 / steps:6.2f}
mV/step")
```

OUTPUT

```
 8-bit: 3.3 V ->  12.94 mV/step   5 V ->  19.61 mV/step
10-bit: 3.3 V ->   3.23 mV/step   5 V ->   4.89 mV/step
12-bit: 3.3 V ->   0.81 mV/step   5 V ->   1.22 mV/step
```

Q22 **Answer: Solution program**

Matching requirements against a table of boards is exactly the selection process engineers use; when no single board fits, combine them.

```
BOARDS = [("Arduino Uno", 14, 6, False, False), ("Arduino Mega", 54, 16, False, False),
          ("ESP32", 34, 15, True, False), ("Pico W", 26, 3, True, False), ("Raspberry Pi 5", 26, 0,
True, True)]
def choose(dig, ana, wireless=False, linux=False):
    for name, d, a, w, l in BOARDS:
        if d >= dig and a >= ana and (w or not wireless) and (l or not linux):
            return name
    return "combine boards (e.g. Pi + microcontroller)"
print("line follower:", choose(8, 5))
print("hexapod:", choose(40, 2))
print("phone-controlled car:", choose(8, 1, wireless=True))
print("vision robot with 4 analog sensors:", choose(10, 4, wireless=True, linux=True))
```

OUTPUT

```
line follower: Arduino Uno
hexapod: Arduino Mega
phone-controlled car: ESP32
vision robot with 4 analog sensors: combine boards (e.g. Pi + microcontroller)
```

Q23 **Answer: Solution program**

A function removes repetition; the string encodes the pattern, so changing the message needs no new code.

```
void blinkOnce(int ms) {
  digitalWrite(LED_BUILTIN, HIGH); delay(ms);
  digitalWrite(LED_BUILTIN, LOW); delay(200);
}
void setup() {
  pinMode(LED_BUILTIN, OUTPUT);
  Serial.begin(9600);
  const char *pattern = "...---...";
  for (int i = 0; pattern[i]; i++) {
    Serial.print(pattern[i]);
    blinkOnce(pattern[i] == '.' ? 200 : 600);
  }
  Serial.println();
}
void loop() {}
```

SERIAL MONITOR

```
...---...
```

Q24 **Answer: Solution program**

EEPROM.put/get handle multi-byte types; the value survives power-off.

```
#include <EEPROM.h>
void setup() {
  Serial.begin(9600);
  int offset = -12;
  EEPROM.put(0, offset);           // write (only when it changes, to limit wear)
  int back = 0;
  EEPROM.get(0, back);
  Serial.print("stored offset: ");
  Serial.println(back);
}
void loop() {}
```

SERIAL MONITOR

```
stored offset: -12
```

Q25 **Answer: (c) Raspberry Pi for vision and mapping + Arduino or Pico for motor control**

Divide high-level and real-time work.

An MCU puts CPU, flash (program), SRAM (variables), EEPROM (settings) and peripherals on one chip and runs one program instantly and in real time. Uno: ATmega328P, 16 MHz, 5 V, 6 PWM, 6 analog; Mega for many pins; ESP32 for Wi-Fi/Bluetooth (3.3 V); Pico for MicroPython; STM32 for performance; Pi/Jetson for vision and AI. Count pins and check logic voltages before choosing.

LEVEL 2

Intermediate level

Use sensors, motors and drivers, build chassis and drive systems, apply PID control, build autonomous mobile robots, connect devices, build robotic arms and write reliable embedded code

9 MODULES · 20 DAYS MINIMUM

Module 08 · Sensors	**3 days**
Module 09 · Actuators and motors	**2 days**
Module 10 · Motor drivers and power systems	**2 days**
Module 11 · Robot chassis, locomotion and kinematics	**2 days**
Module 12 · Control systems and PID	**2 days**
Module 13 · Building mobile robots	**3 days**
Module 14 · Communication between devices	**2 days**
Module 15 · Robotic arms and manipulators	**2 days**
Module 16 · Embedded programming patterns for robots	**2 days**

MODULE 08 · INTERMEDIATE

Sensors

Minimum duration **3 days** | Level **Intermediate**

Sensors let robots perceive their environment. This module covers analog versus digital sensors, range, resolution, accuracy, noise and calibration; ultrasonic, IR, time-of-flight and LiDAR distance sensors; IR line arrays and colour sensors; accelerometers, gyroscopes, magnetometers, IMUs and sensor fusion, and wheel encoders; temperature, humidity, light, gas and sound sensors; touch, force and position sensors; and filtering noisy data with averaging and moving averages, followed by 25 solved questions and programs.

IN THIS MODULE

8.1 Sensor basics
8.2 Distance and obstacle sensors
8.3 Line and colour sensors
8.4 Motion and orientation sensors
8.5 Environmental sensors
8.6 Touch, force and position sensors
8.7 Filtering sensor data
8.8 Practice questions
8.9 Solutions

8.1 Sensor basics

8.1.1 Analog vs digital sensors

	Analog sensor	Digital sensor
Output	a voltage that varies continuously	HIGH/LOW, pulses, or data over a bus (I2C, SPI, UART, 1-Wire)
Read with	analogRead() (ADC)	digitalRead(), pulseIn(), libraries
Examples	LDR, potentiometer, thermistor, Sharp IR distance sensor	button, PIR, HC-SR04, MPU6050, DHT22, VL53L0X
Notes	affected by noise and supply voltage	often includes its own ADC and calibration

8.1.2 Range, resolution, accuracy and noise

Term	Meaning	HC-SR04 example
Range	smallest to largest value it can measure	2 cm – 400 cm
Resolution	smallest change it can detect	about 0.3 cm
Accuracy	how close readings are to the true value	about ±3 mm in good conditions
Precision (repeatability)	how close repeated readings are to each other	varies with surface and angle

Term	Meaning	HC-SR04 example
Noise	random variation in readings	occasional wrong echoes
Update rate	readings per second	up to about 40 per second

8.1.3 Calibration

Calibration compares sensor readings with known true values and corrects them. Often a straight-line (linear) correction is enough: true = gain × reading + offset. Measure at two or more known points (for example 10 cm and 50 cm on a ruler) and compute gain and offset; for non-linear sensors use a lookup table.

CALIBRATE.PY

```
# two-point calibration of a distance sensor against a ruler
readings = [(10.0, 10.9), (50.0, 52.3)]          # (true cm, sensor cm)
(t1, r1), (t2, r2) = readings
gain = (t2 - t1) / (r2 - r1)
offset = t1 - gain * r1
print(f"true = {gain:.4f} x reading {offset:+.3f}")
for raw in (10.9, 31.6, 52.3, 80.0):
    print(f"raw {raw:5.1f} cm -> calibrated {gain * raw + offset:5.1f} cm")
```

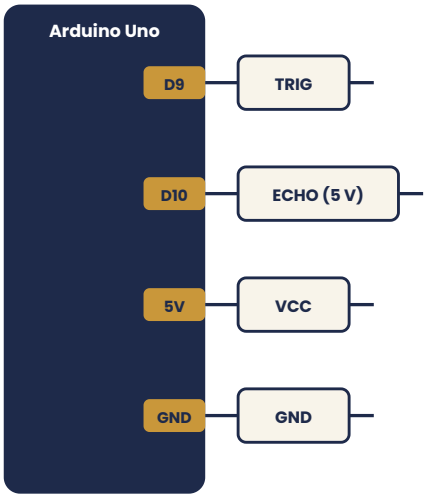
OUTPUT

```
true = 0.9662 x reading -0.531
raw  10.9 cm -> calibrated  10.0 cm
raw  31.6 cm -> calibrated  30.0 cm
raw  52.3 cm -> calibrated  50.0 cm
raw  80.0 cm -> calibrated  76.8 cm
```

8.2 Distance and obstacle sensors

8.2.1 Ultrasonic sensor (HC-SR04)

The HC-SR04 sends a 40 kHz sound burst after a 10 µs pulse on TRIG and holds ECHO HIGH for the round-trip time. Sound travels about 343 m/s (0.0343 cm/µs) at 20 °C: **distance (cm) = echo time (µs) × 0.0343 ÷ 2 ≈ µs ÷ 58**. It works on most surfaces but misses soft, angled or very thin objects and has a cone about 15° wide.



HC-SR04 ON AN UNO (ON 3.3 V BOARDS DIVIDE THE ECHO LINE DOWN)

ULTRASONIC.INO

```
const int TRIG = 9, ECHO = 10;

long readCm() {
  digitalWrite(TRIG, LOW);  delayMicroseconds(2);
  digitalWrite(TRIG, HIGH); delayMicroseconds(10);   // 10 us trigger pulse
  digitalWrite(TRIG, LOW);
  long us = pulseIn(ECHO, HIGH, 30000);              // timeout 30 ms (about 5 m)
  if (us == 0) return -1;                            // no echo
  return us / 58;
}

void setup() {
  pinMode(TRIG, OUTPUT); pinMode(ECHO, INPUT);
  Serial.begin(9600);
}

void loop() {
  long d = readCm();
  if (d < 0) Serial.println("out of range");
  else { Serial.print(d); Serial.println(" cm"); }
  delay(60);                                         // let echoes die away
}
```

SERIAL MONITOR

```
10 cm
30 cm
50 cm
out of range
```

8.2.2 Infrared (IR) proximity sensors

IR modules shine infrared light and measure the reflection. **Digital IR obstacle modules** (with a comparator and adjustment pot) output LOW when something is within a set distance (2–30 cm). **Sharp GP2Y0A21** sensors output an analog voltage that falls non-linearly with distance (10–80 cm). Dark surfaces reflect less and sunlight interferes, so IR suits indoor, short-range detection.

8.2.3 Time-of-flight laser sensors (VL53L0X)

A **time-of-flight** (ToF) sensor such as the VL53L0X (up to about 2 m) or VL53L1X (4 m) times a tiny infrared laser pulse. It has a narrow beam, millimetre resolution and works on most colours; it talks over I2C (default address 0x29). Several on one bus need their XSHUT pins used to give each a new address at start-up.

8.2.4 LiDAR (overview)

A **LiDAR** spins a laser rangefinder to measure hundreds of distances around the robot every rotation (5–10 Hz, up to 8–12 m for low-cost 2-D units such as RPLIDAR or LD19). It is the main sensor for mapping and navigation (module 18); 3-D LiDARs are used on self-driving cars.

Sensor	Range	Beam	Weak points
Ultrasonic HC-SR04	2–400 cm	wide (~15°)	soft or angled surfaces, slow
IR module	2–30 cm	medium	dark surfaces, sunlight
Sharp analog IR	10–80 cm	narrow	non-linear, noisy
ToF VL53L0X	3–200 cm	narrow (~25°)	bright sunlight reduces range
2-D LiDAR	0.15–12 m, 360°	scanning line	cost, glass and mirrors

8.3 Line and colour sensors

8.3.1 IR line sensor arrays

A line sensor shines IR at the floor: white reflects a lot, black very little. An **array** of 3–8 sensors (TCRT5000-based modules, QTR-8A) shows where the line is under the robot. With analog readings you can compute a **weighted position**, giving smooth steering for PID line following (module 12).

LINEPOSITION.INO

```
const int N = 5;
const int pins[N] = {A0, A1, A2, A3, A4};

void setup() { Serial.begin(9600); }

void loop() {
  long weighted = 0, total = 0;
  for (int i = 0; i < N; i++) {
    int v = 1023 - analogRead(pins[i]);      // black line = low reading -> invert
    weighted += (long)v * i * 1000;
    total += v;
  }
  long pos = weighted / total;               // 0 (far left) .. 4000 (far right)
  int error = pos - 2000;                    // 0 = centred
  Serial.print("position "); Serial.print(pos);
  Serial.print("  error "); Serial.println(error);
}
```

SERIAL MONITOR

```
position 2000  error 0
position 1430  error -570
position 732  error -1268
```

8.3.2 Colour sensors (TCS3200, TCS34725)

The **TCS3200** has an array of photodiodes with red, green, blue and clear filters and outputs a square wave whose frequency is proportional to the light of the selected colour. The **TCS34725** gives 16-bit R, G, B and clear values over I2C and has an IR-blocking filter and a white LED for consistent lighting. Calibrate against white and black samples, then classify colours by their normalised ratios.

COLOUR_CLASSIFY.PY

```
SAMPLES = {"red": (210, 60, 55), "green": (70, 190, 80), "blue": (50, 80, 200), "yellow": (220, 210,
70), "white": (240, 240, 235)}
def classify(r, g, b):
    s = r + g + b
    rn, gn, bn = r / s, g / s, b / s                  # ratios ignore overall brightness
    def dist(ref):
        t = sum(ref)
        return (rn - ref[0] / t) ** 2 + (gn - ref[1] / t) ** 2 + (bn - ref[2] / t) ** 2
    return min(SAMPLES, key=lambda k: dist(SAMPLES[k]))
for reading in [(105, 31, 28), (60, 150, 70), (180, 175, 60), (40, 70, 160), (120, 121, 118)]:
    print(reading, "->", classify(*reading))
```

OUTPUT

```
(105, 31, 28) -> red
(60, 150, 70) -> green
(180, 175, 60) -> yellow
(40, 70, 160) -> blue
(120, 121, 118) -> white
```

8.4 Motion and orientation sensors

8.4.1 Accelerometers and gyroscopes (MPU6050)

An **accelerometer** measures acceleration including gravity, so when still it shows which way is down (tilt). A **gyroscope** measures rotation rate (°/s); integrating it over time gives the angle turned. The **MPU6050** (I2C, 0x68) combines a 3-axis accelerometer (±2 g: 16 384 per g) and a 3-axis gyroscope (±250 °/s: 131 per °/s).

8.4.2 Magnetometers (compass)

A **magnetometer** (QMC5883L, or inside a 9-axis IMU) measures the Earth's magnetic field to give heading relative to magnetic north: heading = atan2(y, x). Motors, steel and batteries nearby distort it, so calibrate in the robot (hard- and soft-iron correction) and mount it away from motors.

8.4.3 IMUs and sensor fusion (overview)

An **IMU** (inertial measurement unit) combines these sensors. Each has a weakness: accelerometers are noisy and disturbed by motion; gyros drift over time; magnetometers are disturbed by metal. **Sensor fusion** combines them so each covers the others' weaknesses. The simple **complementary filter** trusts

the gyro over short times and the accelerometer over long times; Kalman filters do this optimally; chips like the BNO055 do fusion internally.

Complementary filter: angle = α × (angle + gyro_rate × dt) + (1 - α) × accel_angle, with $\alpha \approx 0.98$

COMPLEMENTARY_FILTER.PY

```
from smbus2 import SMBus
import math, time

def s16(h, l):
    v = (h << 8) | l
    return v - 65536 if v > 32767 else v

bus = SMBus(1); bus.write_byte_data(0x68, 0x6B, 0)        # wake the MPU6050
angle, alpha, dt = 0.0, 0.98, 0.02
for step in range(1, 151):
    d = bus.read_i2c_block_data(0x68, 0x3B, 14)
    ax, az = s16(d[0], d[1]) / 16384, s16(d[4], d[5]) / 16384
    gy = s16(d[10], d[11]) / 131                              # deg/s about the y axis
    accel_angle = math.degrees(math.atan2(ax, az))
    angle = alpha * (angle + gy * dt) + (1 - alpha) * accel_angle
    if step % 30 == 0:
        print(f"t={step * dt:.1f} s  accel says {accel_angle:5.1f} deg  fused {angle:5.1f} deg")
    time.sleep(dt)
```

OUTPUT

```
t=0.6 s  accel says  30.0 deg  fused  13.6 deg
t=1.2 s  accel says  30.0 deg  fused  21.1 deg
t=1.8 s  accel says  30.0 deg  fused  25.1 deg
t=2.4 s  accel says  30.0 deg  fused  27.3 deg
t=3.0 s  accel says  30.0 deg  fused  28.6 deg
```

The board was held still at 30°; the fused angle starts at 0 and converges smoothly to the accelerometer's 30° while ignoring short-term accelerometer noise. When the robot rotates, the gyro term follows the motion instantly.

8.4.4 Wheel encoders

An **encoder** produces pulses as a wheel turns: optical slotted discs or magnetic Hall-effect encoders on gear motors. Counting pulses gives distance; counting per second gives speed. **Quadrature** encoders have two channels (A, B) 90° apart so the direction can be detected. Count with interrupts so no pulse is missed.

ENCODER.INO

```
volatile long ticks = 0;
const int TICKS_PER_REV = 20;          // slotted disc with 20 slots
const float WHEEL_CM = 21.0;           // wheel circumference

void onTick() { ticks++; }             // interrupt service routine: keep it tiny

void setup() {
  Serial.begin(9600);
  pinMode(2, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(2), onTick, FALLING);
}

void loop() {
  static long last = 0;
  delay(500);
  noInterrupts(); long t = ticks; interrupts();       // read a multi-byte volatile safely
  float cm = t * WHEEL_CM / TICKS_PER_REV;
  float speed = (t - last) * WHEEL_CM / TICKS_PER_REV / 0.5;
  last = t;
  Serial.print("ticks "); Serial.print(t);
  Serial.print("  distance "); Serial.print(cm, 1);
  Serial.print(" cm  speed "); Serial.print(speed, 1); Serial.println(" cm/s");
}
```

SERIAL MONITOR

```
ticks 20  distance 21.0 cm  speed 42.0 cm/s
ticks 40  distance 42.0 cm  speed 42.0 cm/s
ticks 60  distance 63.0 cm  speed 42.0 cm/s
```

8.5 Environmental sensors

8.5.1 Temperature and humidity (DHT11/DHT22)

DHT22.INO

```
#include <DHT.h>
DHT dht(4, DHT22);

void setup() { Serial.begin(9600); dht.begin(); }

void loop() {
  float t = dht.readTemperature();
  float h = dht.readHumidity();
  if (isnan(t) || isnan(h)) { Serial.println("read failed"); return; }
  Serial.print(t, 1); Serial.print(" C  "); Serial.print(h, 0); Serial.println(" %");
  delay(2000);                         // DHT22: at most one reading every 2 s
}
```

SERIAL MONITOR

```
26.4 C  58 %
26.6 C  57 %
```

8.5.2 Light (LDR)

An **LDR** (light-dependent resistor) drops from about 1 MΩ in darkness to 1 kΩ or less in bright light. In a divider with a 10 kΩ resistor it gives an analog voltage that rises with light (LDR on the GND side: falls). Use it for light-seeking or light-avoiding robots and day/night detection.

8.5.3 Gas and smoke sensors

MQ-series sensors (MQ-2 smoke/LPG, MQ-135 air quality, MQ-7 CO) contain a heated element whose resistance changes with gas concentration. They need a 24–48 hour burn-in and a warm-up of a few minutes after power-on, draw about 150 mA for the heater, and give only rough readings unless calibrated. Use them for detection and alarms, not precise measurement.

8.5.4 Sound sensors

Microphone modules give an analog sound level and often a digital output that goes HIGH above a threshold set by a pot: clap-controlled robots and noise monitors. Measure loudness as the peak-to-peak or RMS value over a short window, not a single sample.

8.6 Touch, force and position sensors

8.6.1 Touch and limit switches, force-sensitive resistors, potentiometers

Sensor	Output	Robot use
Bump / limit switch	digital (pressed / released)	collision detection, arm end-stops, homing
Capacitive touch pad (TTP223, ESP32 touch pins)	digital	touch buttons on the robot body
Force-sensitive resistor (FSR)	resistance falls as force rises (analog, via a divider)	gripper force sensing, foot contact
Potentiometer	analog angle	joint position (inside servos), joysticks
Load cell + HX711	24-bit digital weight	measuring payload weight

8.7 Filtering sensor data

8.7.1 Averaging and moving-average filters

Real readings are noisy. An **average** of N quick readings reduces random noise by about $\sqrt{N}$. A **moving average** keeps the last N readings and outputs their mean after each new one: smooth, but it lags behind changes by about N/2 samples. A **median** filter rejects single wild readings (spikes), common with ultrasonic sensors. An **exponential** filter ($y = y + \alpha(x - y)$) needs no buffer.

FILTERS.PY

```
import random, statistics
random.seed(2)
true = [50] * 10 + [30] * 10                        # obstacle moves from 50 cm to 30 cm
raw = [t + random.gauss(0, 3) for t in true]
raw[6] = 120                                        # a spike (missed echo)

def moving_avg(x, n=5):
    return [sum(x[max(0, i - n + 1):i + 1]) / len(x[max(0, i - n + 1):i + 1]) for i in
range(len(x))]
def median_f(x, n=5):
    return [statistics.median(x[max(0, i - n + 1):i + 1]) for i in range(len(x))]
def expo(x, a=0.3):
    y, out = x[0], []
    for v in x:
        y += a * (v - y); out.append(y)
    return out

ma, md, ex = moving_avg(raw), median_f(raw), expo(raw)
print(" i   true    raw   mov.avg  median   expo")
for i in (4, 5, 6, 7, 8, 12, 15, 19):
    print(f"{i:2d}  {true[i]:5.0f}  {raw[i]:6.1f}  {ma[i]:7.1f}  {md[i]:6.1f}  {ex[i]:6.1f}")
```

OUTPUT

```
 i   true    raw   mov.avg  median   expo
 4     50    52.5     51.8    51.2    52.5
 5     50    45.8     49.6    50.4    50.5
 6     50   120.0     64.0    51.2    71.3
 7     50    47.7     63.3    50.4    64.3
 8     50    46.8     62.6    47.7    59.0
12     30    27.3     35.8    29.1    37.5
15     30    20.4     27.3    28.4    29.5
19     30    30.8     28.3    28.8    29.9
```

The spike at sample 6 pulls the moving average and exponential filters far off for several samples, while the median ignores it. After the step change, every filter takes a few samples to catch up: that is the price of smoothing.

PRACTICE CHECKPOINT

Connect and read six different sensors, calibrate an ultrasonic sensor against a ruler, and smooth noisy readings with a moving-average filter.

8.8 Practice questions

8.8.1 Basic level

Q1 How does the HC-SR04 measure distance?

(a) by measuring reflected colour
(b) by a laser triangle
(c) by magnetic field
(d) by timing an ultrasonic echo

Q2 What does an accelerometer measure when the robot is still?

(a) rotation speed
(b) gravity, which shows tilt
(c) temperature
(d) magnetic north

Q3 What does a gyroscope measure?

(a) position
(b) rotation rate (degrees per second)
(c) altitude
(d) light level

Q4 Which sensor detects a black line on a white floor?

(a) DHT22
(b) HC-SR04
(c) MQ-2
(d) IR reflectance sensor

Q5 Which bus does the VL53L0X ToF sensor use?

(a) I2C
(b) UART only
(c) analog voltage
(d) PWM

Q6 What does a magnetometer give a robot?

(a) distance
(b) speed of sound
(c) compass heading
(d) colour

Q7 What does a wheel encoder count?

(a) obstacles
(b) battery charge
(c) light flashes
(d) wheel rotation pulses

Q8 Which filter is best at removing single spikes?

(a) moving average
(b) median filter
(c) no filter
(d) doubling the value

Q9 Which sensor gives 360° distance scans for mapping?

(a) HC-SR04
(b) TCS34725
(c) LiDAR
(d) FSR

8.8.2 Intermediate level

Q10 What is sensor resolution?

(a) the smallest change it can detect
(b) the maximum distance
(c) how often it reads
(d) its price

Q11 Why do quadrature encoders have two channels?

(a) to double the voltage
(b) for redundancy only
(c) to detect the direction of rotation
(d) to measure temperature

Q12 Why combine accelerometer and gyro readings (sensor fusion)?

(a) the gyro drifts and the accelerometer is noisy; together they give a stable, responsive angle
(b) to save power
(c) because each alone gives no data
(d) to read colour

Q13 What is the main weakness of ultrasonic sensors?

(a) they only work in the dark
(b) soft or angled surfaces may not reflect the sound back
(c) they need a magnet
(d) they cannot measure over 5 cm

Q14 Why must MQ gas sensors warm up?

(a) to charge a battery
(b) to calibrate Wi-Fi
(c) they never need it
(d) their heated element needs time to stabilise

Q15 What does a force-sensitive resistor do as force increases?

(a) its resistance increases
(b) it emits light
(c) its resistance decreases
(d) it measures temperature

Q16 What is the disadvantage of a moving average with a large window?

(a) it adds spikes
(b) it uses no memory
(c) it lags behind real changes
(d) it amplifies noise

Q17 Why read encoders with interrupts?

(a) interrupts make wheels faster
(b) to save flash
(c) encoders need 12 V
(d) so no pulses are missed while the program does other work

8.8.3 Advanced level

Q18 An HC-SR04 echo lasts 1450 µs. What is the distance?

(a) about 50 cm
(b) about 25 cm
(c) about 14.5 cm
(d) about 2.5 cm

Q19 An encoder gives 20 ticks per revolution on a 6.5 cm wheel. How far is 140 ticks?

(a) about 143 cm
(b) about 45.5 cm
(c) about 14 cm
(d) about 910 cm

Q20 Raw readings 10.9 and 52.3 correspond to true 10 and 50. What is the calibration gain?

(a) about 1.035
(b) about 0.966
(c) 1.0
(d) about 0.5

Q21 Write an Arduino sketch that takes 5 ultrasonic readings, discards the highest and lowest, and prints the average of the middle three.

Q22 Write a moving-average class with a fixed window that accepts readings one at a time; feed it [12, 14, 13, 30, 13, 12, 14] with a window of 3.

Q23 Convert MPU6050 raw accelerometer readings (x = 8192, y = 0, z = 14189) into g and compute the

tilt angle.

Q24 Read an LDR on A0 and print DARK, DIM or BRIGHT (thresholds 300 and 700) for four readings.

Q25 A robot must follow a black line, avoid obstacles and know how far it has driven. Which sensor set fits?

(a) IR line array + ultrasonic or ToF sensor + wheel encoders

(b) DHT22 + MQ-2 + LDR

(c) magnetometer only

(d) one potentiometer

8.9 Solutions

Q1 **Answer: (d) by timing an ultrasonic echo**

Time of flight of sound.

Q2 **Answer: (b) gravity, which shows tilt**

It senses the 1 g of gravity.

Q3 **Answer: (b) rotation rate (degrees per second)**

Integrate for angle.

Q4 **Answer: (d) IR reflectance sensor**

Black reflects little IR.

Q5 **Answer: (a) I2C**

I2C, address 0x29.

Q6 **Answer: (c) compass heading**

atan2(y, x).

Q7 **Answer: (d) wheel rotation pulses**

Distance and speed.

Q8 **Answer: (b) median filter**

Median ignores outliers.

Q9 **Answer: (c) LiDAR**

Scanning laser rangefinder.

Q10 **Answer: (a) the smallest change it can detect**

Resolution.

Q11 **Answer: (c) to detect the direction of rotation**

A/B phase order.

Q12 **Answer: (a) the gyro drifts and the accelerometer is noisy; together they give a stable, responsive angle**

Complementary strengths.

Q13 **Answer: (b) soft or angled surfaces may not reflect the sound back**

Echo lost.

Q14 **Answer: (d) their heated element needs time to stabilise**

Heater element.

Q15 **Answer: (c) its resistance decreases**

Read via a divider.

Q16 **Answer: (c) it lags behind real changes**

Smoothing vs delay.

Q17 **Answer: (d) so no pulses are missed while the program does other work**

Fast pulses.

Q18 **Answer: (b) about 25 cm**

1450 ÷ 58 = **25 cm**.

Q19 **Answer: (a) about 143 cm**

140/20 = 7 rev × π × 6.5 = **142.9 cm**.

Q20 **Answer: (b) about 0.966**

(50 − 10) ÷ (52.3 − 10.9) = 40 ÷ 41.4 = **0.966**.

Q21 **Answer: Solution program**

A trimmed mean rejects one spike in each direction: readings 20, 21, 120, 19, 22 cm give (20 + 21 + 19 + 22 + 120 − 19 − 120) ÷ 3 = 21 cm.

```
long readUs() { return pulseIn(10, HIGH, 30000); }
void setup() {
  Serial.begin(9600);
  pinMode(9, OUTPUT); pinMode(10, INPUT);
  long r[5];
  for (int i = 0; i < 5; i++) {
    digitalWrite(9, HIGH); delayMicroseconds(10); digitalWrite(9, LOW);
    r[i] = readUs() / 58;
  }
  long lo = r[0], hi = r[0], sum = 0;
  for (int i = 0; i < 5; i++) { sum += r[i]; if (r[i] < lo) lo = r[i]; if (r[i] > hi) hi = r[i]; }
  Serial.print("trimmed mean: "); Serial.print((sum - lo - hi) / 3.0, 1); Serial.println(" cm");
}
void loop() {}
```

SERIAL MONITOR

```
trimmed mean: 21.0 cm
```

Q22 **Answer: Solution program**

deque(maxlen=n) drops the oldest value automatically; note how the spike 30 lifts three outputs.

```
from collections import deque
class MovingAverage:
    def __init__(self, n):
        self.buf = deque(maxlen=n)
    def add(self, x):
        self.buf.append(x)
        return sum(self.buf) / len(self.buf)
ma = MovingAverage(3)
for x in [12, 14, 13, 30, 13, 12, 14]:
    print(f"in {x:2d} -> avg {ma.add(x):5.2f}")
```

OUTPUT

```
in 12 -> avg 12.00
in 14 -> avg 13.00
in 13 -> avg 13.00
in 30 -> avg 19.00
in 13 -> avg 18.67
in 12 -> avg 18.33
in 14 -> avg 13.00
```

Q23 **Answer: Solution program**

16384 counts = 1 g at ±2 g range; atan2(0.5, 0.866) = 30°.

```
import math
ax, ay, az = 8192 / 16384, 0 / 16384, 14189 / 16384
pitch = math.degrees(math.atan2(ax, math.sqrt(ay ** 2 + az ** 2)))
print(f"ax={ax:.3f} g ay={ay:.3f} g az={az:.3f} g -> tilt {pitch:.1f} degrees")
```

OUTPUT

```
ax=0.500 g ay=0.000 g az=0.866 g -> tilt 30.0 degrees
```

Q24 **Answer: Solution program**

An if/else-if chain maps the analog value to three bands.

```
void setup() { Serial.begin(9600); }
void loop() {
  int v = analogRead(A0);
  Serial.print(v);
  if (v < 300) Serial.println(" DARK");
  else if (v < 700) Serial.println(" DIM");
  else Serial.println(" BRIGHT");
  delay(500);
}
```

SERIAL MONITOR

```
120 DARK
450 DIM
880 BRIGHT
710 BRIGHT
```

Q25 **Answer: (a) IR line array + ultrasonic or ToF sensor + wheel encoders**

Match each task to a sensor.

Know each sensor's range, resolution, accuracy and noise, and calibrate (true = gain × reading + offset). Distance: ultrasonic (µs ÷ 58 = cm), IR, ToF (I2C), LiDAR. Line arrays give a weighted line position; colour sensors classify by ratios. IMU: accelerometer (tilt), gyro (rate), magnetometer (heading), fused with a complementary filter. Count encoders with interrupts. Filter noise: average, moving average (lags), median (removes spikes), exponential.

MODULE 09 · INTERMEDIATE

Actuators and motors

Minimum duration **2 days** | Level **Intermediate**

Actuators turn electrical signals into movement. This module explains how DC motors work and how PWM and H-bridges control their speed and direction, geared and BO motors; servo position control, standard versus continuous-rotation servos and torque ratings; stepper steps, step angle, unipolar and bipolar types and microstepping; brushless motors and ESCs; solenoids, linear actuators, pneumatics and hydraulics; and how to choose motors from torque, speed, voltage and current, followed by 25 solved questions and programs.

IN THIS MODULE

9.1 DC motors
9.2 Servo motors
9.3 Stepper motors
9.4 Brushless DC (BLDC) motors and ESCs
9.5 Other actuators
9.6 Choosing motors
9.7 Practice questions
9.8 Solutions

9.1 DC motors

9.1.1 How DC motors work

A brushed **DC motor** has a rotating coil (armature) between permanent magnets. Current in the coil creates a magnetic force that turns it; the **commutator** and brushes reverse the coil current every half turn so it keeps spinning. Speed rises with voltage; torque rises with current. Reverse the supply polarity and it spins the other way.

Rating	Meaning	Typical small motor
Rated voltage	designed operating voltage	3–12 V
No-load speed	rpm with nothing attached	10 000–20 000 rpm (bare motor)
No-load current	current when spinning freely	50–200 mA
Stall torque / current	when the shaft is held still	highest torque; current 5–10× running
Rated torque	continuous safe torque	about 25 % of stall

9.1.2 Speed control with PWM

A microcontroller pin cannot power a motor (module 10), but it can send a **PWM** signal to a driver. The motor's inertia smooths the pulses, so a 50 % duty cycle gives roughly half the average voltage and a lower speed. Below a certain duty the motor stalls (it cannot overcome friction); find this "dead band" and start from it.

9.1.3 Direction control

Direction needs an **H-bridge** (module 10): two inputs choose the polarity. With an L298N or TB6612FNG: IN1 = HIGH, IN2 = LOW → forward; IN1 = LOW, IN2 = HIGH → reverse; both LOW → coast; both HIGH → brake. PWM on the enable pin sets the speed.

MOTORDIRECTION.INO

```
const int ENA = 9, IN1 = 7, IN2 = 8;     // L298N channel A

void drive(int speed) {                    // -255 .. 255
  digitalWrite(IN1, speed > 0);
  digitalWrite(IN2, speed < 0);
  analogWrite(ENA, abs(speed));
}

void setup() {
  pinMode(ENA, OUTPUT); pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT);
  drive(150);  delay(1000);     // forward at about 59 %
  drive(255);  delay(1000);     // full speed forward
  drive(0);    delay(300);      // stop before reversing
  drive(-180); delay(1000);     // reverse
  drive(0);
}

void loop() {}
```

SIMULATED RUN (pin changes and Serial output)

```
[     0 ms] pin 7 -> HIGH
[     0 ms] pin 8 -> LOW
[     0 ms] PWM pin 9 = 150
[  1000 ms] pin 7 -> HIGH
[  1000 ms] pin 8 -> LOW
[  1000 ms] PWM pin 9 = 255
[  2000 ms] pin 7 -> LOW
[  2000 ms] pin 8 -> LOW
[  2000 ms] PWM pin 9 = 0
[  2300 ms] pin 7 -> LOW
[  2300 ms] pin 8 -> HIGH
[  2300 ms] PWM pin 9 = 180
[  3300 ms] pin 7 -> LOW
[  3300 ms] pin 8 -> LOW
[  3300 ms] PWM pin 9 = 0
```

9.1.4 Geared DC motors and BO motors

Bare motors spin too fast with too little torque for wheels, so robots use **geared motors**. The yellow plastic **BO motors** (3–6 V, about 48:1 ratio, ~150–250 rpm) are the standard hobby robot drive; metal-gear motors such as N20 or JGA25-370 (often with encoders) are stronger and more precise.

9.2 Servo motors

9.2.1 Position control with pulse width

A hobby **servo** contains a DC motor, gearbox, potentiometer and control circuit: it turns to the angle commanded by a pulse every 20 ms (50 Hz): about **1 ms = 0°, 1.5 ms = 90°, 2 ms = 180°** (many servos accept 0.5–2.5 ms). The internal control loop holds the angle against loads. Wires: brown/black = GND, red = power (4.8–6 V), orange/yellow = signal.

SERVOSWEEP.INO

```
#include <Servo.h>
Servo arm;

void setup() {
  Serial.begin(9600);
  arm.attach(9);
  int angles[] = {0, 45, 90, 135, 180, 90};
  for (int a : angles) {
    arm.write(a);                    // move to angle (degrees)
    Serial.print("angle "); Serial.println(a);
    delay(400);                      // give it time to get there
  }
  arm.detach();                      // stop pulses: no buzzing, less power
}

void loop() {}
```

SIMULATED RUN (pin changes and Serial output)

```
[     0 ms] servo attached to pin 9
[     0 ms] servo pin 9 -> 0 deg
angle 0
[   400 ms] servo pin 9 -> 45 deg
angle 45
[   800 ms] servo pin 9 -> 90 deg
angle 90
[  1200 ms] servo pin 9 -> 135 deg
angle 135
[  1600 ms] servo pin 9 -> 180 deg
angle 180
[  2000 ms] servo pin 9 -> 90 deg
angle 90
```

9.2.2 Standard vs continuous-rotation servos

A **standard** (positional) servo turns to angles within about 180°. A **continuous-rotation** servo has no position feedback: the same pulse sets **speed and direction** instead (1.5 ms = stop, 1 ms = full speed one way, 2 ms = full speed the other). They make easy, slow wheel drives, but you cannot command a position.

9.2.3 Torque ratings

Servo torque is quoted in **kg·cm** at a voltage: an SG90 micro servo ≈ 1.8 kg·cm at 4.8 V; an MG996R ≈ 9–11 kg·cm at 6 V. It means the servo can hold 1.8 kg at 1 cm from the shaft, or 0.18 kg at 10 cm: torque = mass

× distance. Each servo can draw 0.5–2.5 A when stalled, so power several servos from their own 5–6 V supply.

WORKED EXAMPLE

Can the servo lift it?

An arm link holds 150 g at 12 cm and the link itself weighs 40 g with its centre at 6 cm: torque = 0.15 × 12 + 0.04 × 6 = 1.8 + 0.24 = **2.04 kg·cm**. An SG90 (1.8 kg·cm) is too weak; an MG996R has ample margin.

9.3 Stepper motors

9.3.1 Steps, step angle and precise positioning

A **stepper motor** moves in fixed steps when its coils are energised in sequence, so a controller can position it accurately by counting steps, with no feedback sensor (open loop). Common **step angle**: 1.8° (200 steps per revolution) for NEMA 17 motors; the 28BYJ-48 has a gearbox giving 2048 full steps per output revolution. Steppers hold position strongly when powered but lose steps silently if overloaded or accelerated too fast.

STEPPERMOVE.INO

```
#include <Stepper.h>
const int STEPS_PER_REV = 200;              // 1.8 degree motor
Stepper motor(STEPS_PER_REV, 8, 9, 10, 11);

void setup() {
  Serial.begin(9600);
  motor.setSpeed(60);                       // rpm
  int degrees = 90;
  int steps = degrees * STEPS_PER_REV / 360;
  Serial.print("moving "); Serial.print(degrees); Serial.print(" deg = ");
  Serial.print(steps); Serial.println(" steps");
  motor.step(steps);
  motor.step(-steps);                       // and back
  Serial.println("done");
}

void loop() {}
```

SIMULATED RUN (pin changes and Serial output)

```
[     0 ms] stepper speed 60 rpm
moving 90 deg = 50 steps
[     0 ms] stepper moves 50 steps
[     0 ms] stepper moves -50 steps
done
```

9.3.2 Unipolar vs bipolar

	Unipolar	Bipolar
Wires	5 or 6 (centre-tapped coils)	4 (two coils)

	Unipolar	Bipolar
Driver	simple transistor array (ULN2003)	H-bridge per coil (A4988, DRV8825, TMC2209)
Torque	lower (half of each coil used)	higher for the same size
Example	28BYJ-48 (5 V)	NEMA 17 (12–24 V)

9.3.3 Microstepping

Microstepping drivers control coil currents in fine proportions, splitting each full step into 2, 4, 8, 16 or more microsteps. Motion is smoother and quieter, and resolution increases (200 × 16 = 3200 microsteps per revolution), though the extra positions are less accurate under load. 3D printers and CNC machines use 16× microstepping.

9.4 Brushless DC (BLDC) motors and ESCs

A **brushless motor** has the magnets on the rotating part and fixed coils, switched electronically by an **ESC** (electronic speed controller) instead of brushes: efficient, powerful, long-lived and fast. Drones, e-bikes and fast robots use them. A hobby ESC takes the same 1–2 ms 50 Hz signal as a servo (1 ms = stop, 2 ms = full throttle) and must be "armed" with a low signal at start-up. The **KV rating** is rpm per volt with no load: a 1000 KV motor on 11.1 V spins at about 11 100 rpm.

> **WATCH OUT**
> Brushless motors with propellers are dangerous: remove propellers while testing on the bench, and arm ESCs only when everyone is clear.

9.5 Other actuators

9.5.1 Solenoids

A **solenoid** is a coil that pulls (or pushes) an iron plunger when powered: door locks, valves, pinball flippers, kicking mechanisms. It draws high current and is inductive, so drive it with a MOSFET and a flyback diode, and do not leave it on longer than its duty rating.

9.5.2 Linear actuators

A **linear actuator** converts motor rotation into straight-line push and pull, typically with a leadscrew: strong, slow and self-locking. Many have built-in limit switches and some a position potentiometer for feedback. Used to lift lids, adjust tables, tilt solar panels and move robot parts.

9.5.3 Pneumatic and hydraulic actuators (overview)

Pneumatic cylinders use compressed air: fast, clean, simple on/off motion, used widely in factory pick-and-place and soft grippers. **Hydraulic** cylinders use pressurised oil: enormous forces in a small size (excavators, heavy industrial robots, some legged robots), but heavy and prone to leaks. Valves controlled by solenoids switch both.

9.6 Choosing motors

9.6.1 Torque, speed, voltage and current requirements

Need	Best actuator
continuous rotation, speed control (wheels)	geared DC motor (with encoder for precision)
move to an angle and hold it (arm joint, steering)	servo
precise position without feedback (3D printer, slider)	stepper
high speed and power efficiently (drone, fast robot)	brushless motor + ESC
short straight push or lock	solenoid
strong slow straight motion	linear actuator

1. Work out the **torque** at the wheel or joint (module 5) and add a 2× safety factor.
2. Work out the **speed** needed (rpm = v × 60 ÷ (π × D)).
3. Pick a motor and gear ratio whose rated (not stall) torque and speed meet both.
4. Check the **voltage** matches your battery and the **stall current** is within your driver's rating.
5. Check size, weight, mounting and cost.

CHOOSE_MOTOR.PY

```
import math
def requirement(mass_kg, wheel_d_m, speed_ms, slope_deg, motors=2, safety=2.0):
    force = mass_kg * 9.81 * math.sin(math.radians(slope_deg)) + 0.05 * mass_kg * 9.81   # slope +
rolling
    torque = force * wheel_d_m / 2 / motors * safety
    rpm = speed_ms * 60 / (math.pi * wheel_d_m)
    return torque / 0.0981, rpm                                  # kg*cm, rpm

MOTORS = [("BO motor 48:1", 0.8, 200), ("N20 100:1", 1.6, 300), ("JGA25-370 34:1", 3.5, 280),
("JGA25-370 78:1", 7.0, 130)]
need_t, need_rpm = requirement(mass_kg=2.0, wheel_d_m=0.065, speed_ms=0.5, slope_deg=10)
print(f"need {need_t:.2f} kg*cm rated torque and {need_rpm:.0f} rpm per motor")
for name, t, rpm in MOTORS:
    ok = t >= need_t and rpm >= need_rpm
    print(f"  {name:16} {t:4.1f} kg*cm  {rpm:3d} rpm  {'OK' if ok else '--'}")
```

OUTPUT

```
need 1.45 kg*cm rated torque and 147 rpm per motor
  BO motor 48:1     0.8 kg*cm  200 rpm  --
  N20 100:1         1.6 kg*cm  300 rpm  OK
  JGA25-370 34:1    3.5 kg*cm  280 rpm  OK
  JGA25-370 78:1    7.0 kg*cm  130 rpm  --
```

PRACTICE CHECKPOINT

Control the speed and direction of a DC motor, sweep a servo to precise angles, and move a stepper motor an exact number of steps.

9.7 Practice questions

9.7.1 Basic level

Q1 What part of a brushed DC motor reverses the coil current each half turn?

(a) the magnet **(b)** the gearbox **(c)** the encoder **(d)** the commutator

Q2 How do you reverse a DC motor?

(a) increase PWM
(b) add a capacitor
(c) lower the voltage
(d) reverse the polarity of its supply (with an H-bridge)

Q3 What pulse width puts a standard servo at 90°?

(a) 0.5 ms **(b)** 20 ms **(c)** 1.5 ms **(d)** 3 ms

Q4 How often is a servo control pulse repeated?

(a) every 20 ms (50 Hz) **(b)** every 1 s
(c) every 1 ms **(d)** once only

Q5 How many full steps per revolution does a 1.8° stepper have?

(a) 180 **(b)** 360 **(c)** 100 **(d)** 200

Q6 What does an ESC control?

(a) a servo's angle **(b)** a brushless motor
(c) a solenoid **(d)** a stepper

Q7 Which actuator holds a fixed angle against a load?

(a) continuous-rotation servo **(b)** servo
(c) brushed DC motor alone **(d)** solenoid

Q8 Which actuator pushes a plunger in a straight line when powered?

(a) stepper **(b)** BLDC **(c)** servo **(d)** solenoid

9.7.2 Intermediate level

Q9 What does PWM do to a DC motor?

(a) changes the average voltage, controlling speed
(b) changes direction
(c) reverses polarity
(d) measures speed

Q10 What does a 1.5 ms pulse do to a continuous-rotation servo?

(a) full speed forward **(b)** turns to 90°
(c) stops it **(d)** reverses at full speed

Q11 Which stepper type needs an H-bridge for each of its two coils?

(a) unipolar **(b)** brushed **(c)** BLDC **(d)** bipolar

Q12 What does microstepping improve?

(a) maximum torque **(b)** smoothness and resolution
(c) top speed only **(d)** battery voltage

Q13 What happens if a stepper is overloaded?

(a) it silently loses steps, so its position becomes wrong
(b) it speeds up
(c) it reports an error
(d) it reverses

Q14 Why use a geared motor for robot wheels?

(a) gears add speed and torque together
(b) gears reduce weight
(c) bare motors are too fast with too little torque
(d) motors cannot run without gears

Q15 Which actuator gives huge forces using oil pressure?

(a) hydraulic cylinder **(b)** pneumatic cylinder
(c) servo **(d)** BO motor

Q16 What do the KV ratings of brushless motors describe?

(a) kilovolts **(b)** rpm per volt with no load
(c) torque per amp **(d)** weight

9.7.3 Advanced level

Q17 A 920 KV motor runs on a 3S LiPo (11.1 V). What is its approximate no-load speed?

(a) about 920 rpm **(b)** about 3 300 rpm
(c) about 101 000 rpm **(d)** about 10 200 rpm

Q18 A servo must hold 200 g at 8 cm. What torque is needed?

(a) 25 kg·cm **(b)** 0.16 kg·cm **(c)** 1.6 kg·cm **(d)** 16 kg·cm

Q19 How many microsteps per revolution does a 200-step motor have at 16× microstepping?

(a) 216 **(b)** 1600 **(c)** 3200 **(d)** 12 800

Q20 A wheel of 6.5 cm diameter must move the robot at 0.4 m/s. What rpm is needed?

(a) about 40 rpm **(b)** about 6.2 rpm **(c)** about 390 rpm **(d)** about 118 rpm

Q21 Write a sketch that ramps a motor on an L298N (ENA 9, IN1 7, IN2 8) from 0 to 255 in steps of 51, printing each PWM value, then stops.

Q22 Move a servo smoothly from 0° to 180° in 10° steps (20 ms per step) and print the time it took.

Q23 For a 28BYJ-48 (2048 steps per output revolution), compute the steps and time at 10 rpm for moves of 45°, 90°, 360° and 1.5 turns.

Q24 Which motor suits a camera slider that must stop at exact positions without a sensor?

(a) BO motor **(b)** stepper motor

(c) solenoid
(d) brushless motor on an ESC

Q25 A gripper servo buzzes and gets hot while holding an object. What is happening?

(a) the load needs more torque than the servo can hold comfortably, so it strains at near-stall current
(b) the servo is broken
(c) the PWM frequency is too low
(d) the pot is unplugged

9.8 Solutions

Q1 **Answer: (d) the commutator**
Commutator and brushes.

Q2 **Answer: (d) reverse the polarity of its supply (with an H-bridge)**
Swap + and -.

Q3 **Answer: (c) 1.5 ms**
Centre = 1.5 ms.

Q4 **Answer: (a) every 20 ms (50 Hz)**
50 Hz.

Q5 **Answer: (d) 200**
360 ÷ 1.8.

Q6 **Answer: (b) a brushless motor**
Electronic speed controller.

Q7 **Answer: (b) servo**
Built-in feedback.

Q8 **Answer: (d) solenoid**
Solenoid.

Q9 **Answer: (a) changes the average voltage, controlling speed**
Duty cycle.

Q10 **Answer: (c) stops it**
Neutral.

Q11 **Answer: (d) bipolar**

A4988/DRV8825.

Q12 **Answer: (b) smoothness and resolution**

Finer steps.

Q13 **Answer: (a) it silently loses steps, so its position becomes wrong**

Open-loop limitation.

Q14 **Answer: (c) bare motors are too fast with too little torque**

Trade speed for torque.

Q15 **Answer: (a) hydraulic cylinder**

Hydraulics.

Q16 **Answer: (b) rpm per volt with no load**

KV × voltage ≈ no-load rpm.

Q17 **Answer: (d) about 10 200 rpm**

920 × 11.1 = **10 212 rpm**.

Q18 **Answer: (c) 1.6 kg·cm**

0.2 × 8 = **1.6 kg·cm**.

Q19 **Answer: (c) 3200**

200 × 16.

Q20 **Answer: (d) about 118 rpm**

0.4 × 60 ÷ (π × 0.065) = **117.5 rpm**.

Q21 **Answer: Solution program**

Gradual acceleration avoids current spikes and wheel spin.

```
void setup() {
  Serial.begin(9600);
  pinMode(9, OUTPUT); pinMode(7, OUTPUT); pinMode(8, OUTPUT);
  digitalWrite(7, HIGH); digitalWrite(8, LOW);    // forward
  for (int pwm = 0; pwm <= 255; pwm += 51) {
    analogWrite(9, pwm);
    Serial.print("pwm "); Serial.print(pwm);
    Serial.print(" = "); Serial.print(pwm * 100 / 255); Serial.println(" %");
    delay(300);
  }
  analogWrite(9, 0);
}
void loop() {}
```

SERIAL MONITOR

```
pwm 0 = 0 %
pwm 51 = 20 %
pwm 102 = 40 %
pwm 153 = 60 %
pwm 204 = 80 %
pwm 255 = 100 %
```

Q22 **Answer: Solution program**

Small steps with short delays make a smooth, controllable movement: 19 steps × 20 ms = 380 ms.

```
#include <Servo.h>
Servo s;
void setup() {
  Serial.begin(9600);
  s.attach(9);
  unsigned long t0 = millis();
  for (int a = 0; a <= 180; a += 10) { s.write(a); delay(20); }
  Serial.print("sweep took "); Serial.print(millis() - t0); Serial.println(" ms");
}
void loop() {}
```

SERIAL MONITOR

```
sweep took 380 ms
```

Q23 **Answer: Solution program**

steps = fraction of a turn × steps per revolution; time = turns ÷ rpm × 60.

```
STEPS, RPM = 2048, 10
for label, deg in (("45 deg", 45), ("90 deg", 90), ("1 turn", 360), ("1.5 turns", 540)):
    steps = round(deg / 360 * STEPS)
    secs = deg / 360 / RPM * 60
    print(f"{label:9} -> {steps:5d} steps, {secs:.2f} s")
```

OUTPUT

```
45 deg    ->   256 steps, 0.75 s
90 deg    ->   512 steps, 1.50 s
1 turn    ->  2048 steps, 6.00 s
1.5 turns ->  3072 steps, 9.00 s
```

Q24 **Answer: (b) stepper motor**

Counted steps.

Q25 **Answer: (a) the load needs more torque than the servo can hold comfortably, so it strains at near-stall current**

Use a stronger servo or reduce the load.

DC motors: speed by PWM, direction by H-bridge, torque from gearing (BO, N20, JGA25). Servos: 1–2 ms pulses at 50 Hz set the angle; continuous servos set speed; torque in kg·cm = mass × distance. Steppers: counted steps (1.8° = 200/rev), unipolar (ULN2003) or bipolar (A4988), microstepping for smoothness. BLDC + ESC for power (KV × V = rpm). Choose from torque, speed, voltage and stall current with a safety margin.

MODULE 10 · INTERMEDIATE

Motor drivers and power systems

Minimum duration **2 days** | Level **Intermediate**

Motors need more power than a microcontroller can supply, so drivers and power design are essential. This module explains the current limits of microcontroller pins, how an H-bridge controls direction, the L298N, L293D and TB6612FNG drivers, A4988 and DRV8825 stepper drivers, AA, 18650 and LiPo batteries with capacity, voltage and C-rating, battery management and charging safety, separate logic and motor power with a common ground, regulators and buck converters, power and battery-life calculations, and protection with fuses, flyback diodes and decoupling capacitors, followed by 25 solved questions and programs.

IN THIS MODULE

10.1 Why motor drivers
10.2 H-bridge
10.3 Stepper drivers
10.4 Power sources
10.5 Power distribution
10.6 Protection
10.7 Practice questions
10.8 Solutions

10.1 Why motor drivers

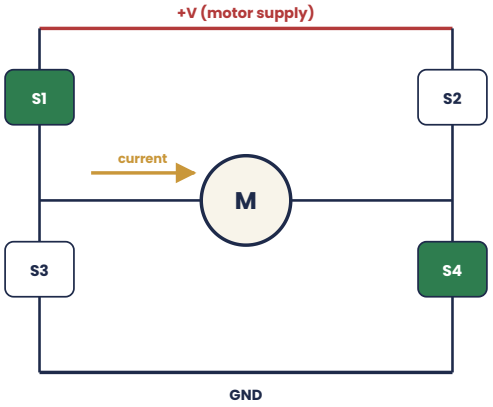
10.1.1 Current limits of microcontroller pins

Board	Per pin (recommended / absolute max)	Whole chip
Arduino Uno (ATmega328P)	20 mA / 40 mA	200 mA total
ESP32	about 20 mA (40 mA max)	a few hundred mA
Raspberry Pi Pico / Pi	about 12–16 mA	50 mA total recommended

A small BO motor draws 100–250 mA running and up to 1 A when it starts or stalls; it also needs 3–12 V and creates voltage spikes. Connected to a pin directly, it would not turn properly and could destroy the microcontroller. A **motor driver** uses the pin's tiny signal to switch a separate, powerful supply through the motor.

10.2 H-bridge

10.2.1 How an H-bridge controls direction



H-BRIDGE: S1 AND S4 CLOSED DRIVE CURRENT LEFT→RIGHT THROUGH THE MOTOR (FORWARD); S2 AND S3 CLOSED REVERSE IT

Four switches (transistors) form an "H" with the motor as the crossbar. Closing **S1 + S4** sends current one way; **S2 + S3** the other way. Closing S1 + S3 (or S2 + S4) would short the supply ("shoot-through"): driver chips prevent this. PWM on the switches controls speed.

IN1	IN2	Motor
HIGH	LOW	forward
LOW	HIGH	reverse
LOW	LOW	coast (free-wheel)
HIGH	HIGH	brake (short the motor terminals)

10.2.2 L298N, L293D, TB6612FNG drivers

Driver	Motor V	Current / channel	Voltage drop	Notes
L293D	4.5–36 V	0.6 A (1.2 A peak)	≈ 1.5–2.5 V	DIP chip, built-in diodes, small motors
L298N module	5–35 V	2 A	≈ 2–3 V	cheap and common; gets hot; has a 5 V regulator
TB6612FNG	2.5–13.5 V	1.2 A (3.2 A peak)	≈ 0.5 V	efficient MOSFET driver, small
DRV8833	2.7–10.8 V	1.5 A	low	tiny, good for N20 and BO motors
BTS7960	6–27 V	43 A	very low	large robots

WORKED EXAMPLE

Why the voltage drop matters

An L298N drops about 2 V: from a 6 V battery a 6 V motor gets only 4 V and runs about a third slower, and the driver wastes 2 V × 1 A = 2 W per motor as heat. A TB6612FNG drops about 0.5 V: the motor gets 5.5 V and only 0.5 W is wasted.

TB6612.INO

```
// TB6612FNG: two motors, standby pin
const int AIN1 = 7, AIN2 = 8, PWMA = 5, BIN1 = 12, BIN2 = 13, PWMB = 6, STBY = 4;

void motor(int in1, int in2, int pwm, int speed) {     // speed -255..255
  digitalWrite(in1, speed > 0); digitalWrite(in2, speed < 0);
  analogWrite(pwm, constrain(abs(speed), 0, 255));
}
void drive(int left, int right) {
  motor(AIN1, AIN2, PWMA, left);
  motor(BIN1, BIN2, PWMB, right);
  Serial.print("left "); Serial.print(left); Serial.print("  right "); Serial.println(right);
}

void setup() {
  Serial.begin(9600);
  int pins[] = {AIN1, AIN2, PWMA, BIN1, BIN2, PWMB, STBY};
  for (int p : pins) pinMode(p, OUTPUT);
  digitalWrite(STBY, HIGH);          // enable the driver
  drive(200, 200);  delay(1000);     // forward
  drive(150, -150); delay(400);      // spin right on the spot
  drive(-180, -180); delay(500);     // reverse
  drive(0, 0);
}
void loop() {}
```

SERIAL MONITOR

```
left 200  right 200
left 150  right -150
left -180  right -180
left 0  right 0
```

10.3 Stepper drivers

10.3.1 A4988 and DRV8825

These small boards drive one bipolar stepper. The microcontroller sends only **STEP** (one pulse = one step or microstep) and **DIR**; MS1–MS3 (M0–M2) pins set microstepping (A4988 up to 1/16, DRV8825 up to 1/32). Each has a potentiometer to set the coil **current limit**: set it before connecting the motor, using V_{ref} (A4988 with 0.068 Ω sense resistors: $I_{max} = V_{ref} \div (8 \times R_s)$; DRV8825: $I = 2 \times V_{ref}$). Put a 100 µF capacitor across the motor supply and never disconnect a motor while powered.

A4988.INO

```
const int STEP = 3, DIR = 4;
const int STEPS_PER_REV = 200 * 16;          // 1.8 deg motor at 1/16 microstepping

void move(long steps, bool cw, unsigned int us) {
  digitalWrite(DIR, cw);
  for (long i = 0; i < steps; i++) {
    digitalWrite(STEP, HIGH); delayMicroseconds(us);
    digitalWrite(STEP, LOW);  delayMicroseconds(us);
  }
}

void setup() {
  Serial.begin(9600);
  pinMode(STEP, OUTPUT); pinMode(DIR, OUTPUT);
  unsigned long t0 = millis();
  move(STEPS_PER_REV / 4, true, 300);        // quarter turn
  Serial.print("quarter turn: "); Serial.print(STEPS_PER_REV / 4);
  Serial.print(" microsteps in "); Serial.print(millis() - t0); Serial.println(" ms");
}
void loop() {}
```

SERIAL MONITOR

```
quarter turn: 800 microsteps in 480 ms
```

10.4 Power sources

10.4.1 Battery types: AA, Li-ion 18650, LiPo

Battery	Cell voltage	Typical capacity	Pros	Cons
AA alkaline	1.5 V (falls with use)	2000–2800 mAh at low current	everywhere, safe	not rechargeable; voltage sags under motor load
AA NiMH	1.2 V	2000–2500 mAh	rechargeable, safe, handles current	lower voltage per cell
Li-ion 18650	3.7 V nominal (4.2 full, ~3.0 empty)	2500–3500 mAh	high energy, rechargeable	needs protection and a proper charger
LiPo pack	3.7 V per cell (1S, 2S = 7.4 V, 3S = 11.1 V)	500–5000 mAh	light, very high current (C rating)	fragile, fire risk if abused

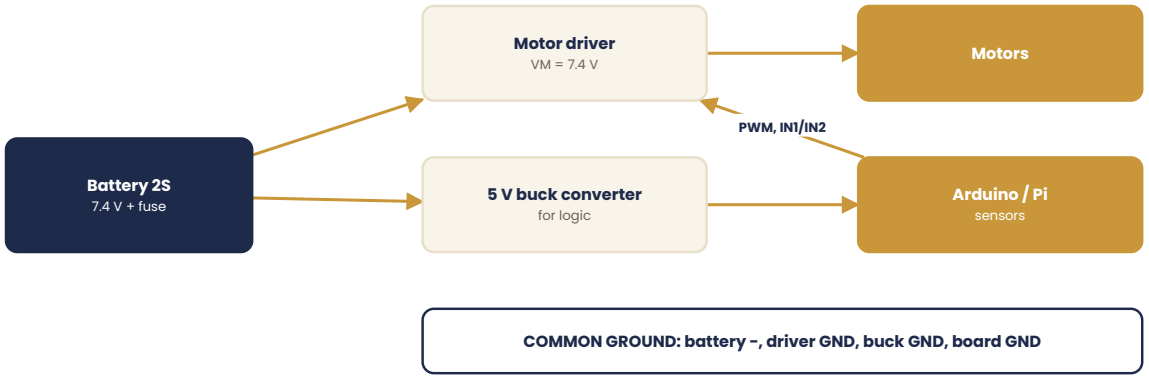
10.4.2 Capacity (mAh), voltage and C-rating

Capacity (mAh) is how much charge the battery holds: 2000 mAh can supply 2 A for 1 hour or 0.5 A for 4 hours (less at high currents). **Energy** (Wh) = V × Ah. Cells in **series** add voltage (2 × 3.7 V = 7.4 V, same mAh); in **parallel** add capacity (same voltage, double mAh). The **C-rating** is the safe maximum discharge current as a multiple of capacity: a 2200 mAh 20C pack supplies up to 44 A.

10.4.3 Battery management and charging safety

A **BMS** (battery management system) protects lithium packs against over-charge (above 4.2 V/cell), over-discharge (below ~2.8–3.0 V/cell), over-current and short circuit, and balances cells in series. Charge LiPos with a balance charger at 1C or less, on a non-flammable surface, never unattended; store them at about 3.8 V/cell. A 3S pack is full at 12.6 V and should be recharged before 9.9 V.

10.5 Power distribution



SEPARATE MOTOR AND LOGIC POWER FROM ONE BATTERY, WITH A SHARED GROUND

10.5.1 Separate logic and motor power

Motors cause current surges and electrical noise. If they share the logic supply, the voltage dips when they start and the microcontroller resets or behaves strangely ("brown-out"). Feed motors directly from the battery through the driver, and the electronics through their own regulator (or even a separate battery).

10.5.2 Common ground

All grounds must be connected: battery −, driver GND, regulator GND and microcontroller GND. Signals are voltages measured against ground, so without a common ground the driver cannot understand the control signals. (Only isolated designs with optocouplers deliberately separate grounds.)

10.5.3 Voltage regulators and buck converters

Use a **buck converter** (85–95 % efficient) to make 5 V for an Arduino or Raspberry Pi from a 7.4–12 V battery; a linear regulator (7805, or the Arduino's own regulator through VIN) wastes the extra voltage as heat and suits only small currents. A Raspberry Pi needs a solid 5 V at up to 3–5 A.

10.5.4 Calculating power consumption and battery life

Battery life (h) ≈ usable capacity (Ah) ÷ average current (A). Use about 80 % of rated capacity for lithium cells.

BATTERY_LIFE.PY

```
battery_mah, volts, usable = 2200, 7.4, 0.8
loads_ma = {                       # average current from the battery (motors at 7.4 V; logic via 90
% buck)
    "2 drive motors (avg)": 2 * 300,
    "Arduino + sensors (5 V 120 mA)": 120 * 5 / volts / 0.9,
    "servo (avg)": 150,
    "ultrasonic + LEDs": 30,
}
total = sum(loads_ma.values())
for name, ma in loads_ma.items():
    print(f"{name:32} {ma:6.0f} mA")
hours = battery_mah * usable / total
print(f"{'TOTAL':32} {total:6.0f} mA")
print(f"battery {battery_mah} mAh x {usable:.0%} usable -> about {hours:.2f} h = {hours * 60:.0f}
min")
print(f"energy {battery_mah / 1000 * volts:.1f} Wh, average power {total / 1000 * volts:.1f} W")
```

OUTPUT

```
2 drive motors (avg)               600 mA
Arduino + sensors (5 V 120 mA)      90 mA
servo (avg)                        150 mA
ultrasonic + LEDs                   30 mA
TOTAL                              870 mA
battery 2200 mAh x 80% usable -> about 2.02 h = 121 min
energy 16.3 Wh, average power 6.4 W
```

10.6 Protection

10.6.1 Fuses, flyback diodes and decoupling capacitors

Protection	Where	Why
Fuse (blade or polyfuse)	in the battery + lead, close to the battery	stops a short circuit from starting a fire
Main switch	after the fuse	turn everything off quickly
Flyback diodes	across motors, relay coils and solenoids (often inside drivers)	absorb inductive voltage spikes
Reverse-polarity protection	diode or P-MOSFET at the input	survive a battery plugged in backwards
Bulk capacitor (100–1000 µF)	across the motor driver supply	absorbs current surges, reduces dips
Decoupling capacitors (0.1 µF)	next to each chip's power pins	filter fast noise
Ceramic capacitor across motor terminals (0.1 µF)	soldered on the motor	reduces brush noise that disturbs sensors and radios

PRACTICE CHECKPOINT

Wire two DC motors through a motor driver with separate motor and logic power, and calculate the expected battery life of the robot.

10.7 Practice questions

10.7.1 Basic level

Q1 Why can't an Arduino pin drive a motor directly?

(a) motors need AC
(b) a pin supplies only about 20 mA, far less than a motor needs
(c) pins are inputs only
(d) motors need I2C

Q2 What circuit lets a DC motor run in both directions?

(a) an H-bridge **(b)** a voltage divider
(c) a low-pass filter **(d)** a Zener diode

Q3 L298N: IN1 HIGH, IN2 LOW. What does the motor do?

(a) runs in reverse **(b)** brakes
(c) runs forward **(d)** coasts

Q4 Which driver is more efficient: L298N or TB6612FNG?

(a) L298N **(b)** they are identical
(c) neither drives motors **(d)** TB6612FNG

Q5 Which signals does an A4988 need from the microcontroller?

(a) SDA and SCL **(b)** TX and RX **(c)** IN1–IN4 only **(d)** STEP and DIR

Q6 What is the nominal voltage of one Li-ion/LiPo cell?

(a) 1.5 V **(b)** 1.2 V **(c)** 3.7 V **(d)** 5 V

Q7 What voltage is a 2S LiPo pack?

(a) 7.4 V nominal **(b)** 3.7 V **(c)** 11.1 V **(d)** 14.8 V

Q8 Why must all grounds be connected?

(a) to charge the battery
(b) to make motors faster
(c) it is optional
(d) signals are measured relative to ground; without a common ground the driver cannot read them

10.7.2 Intermediate level

Q9 What does a fuse protect against?

(a) low voltage
(b) excessive current from a short circuit, which could cause a fire
(c) static electricity
(d) noise on I2C

Q10 What does "shoot-through" in an H-bridge mean?

(a) the motor spins backwards

(b) both switches on one side turn on, shorting the supply
(c) PWM is too slow
(d) the motor coasts

Q11 Why separate motor power from logic power?

(a) logic needs AC
(b) motors need 3.3 V
(c) it halves the battery voltage
(d) motor surges and noise can make the microcontroller reset or misbehave

Q12 Which regulator is best to make 5 V for a Raspberry Pi from a 3S LiPo?

(a) a buck converter rated for at least 3 A
(b) a 7805 with no heatsink
(c) the Arduino 5 V pin
(d) a resistor divider

Q13 What does the C-rating of a LiPo describe?

(a) its charging temperature
(b) its colour code
(c) its maximum safe discharge current as a multiple of capacity
(d) its cell count

Q14 What do cells in series do?

(a) add their capacities
(b) halve the voltage
(c) nothing
(d) add their voltages

Q15 Why place a 100–1000 µF capacitor across the motor supply?

(a) to store the program
(b) to absorb current surges and reduce voltage dips
(c) to make PWM
(d) to set the I2C address

Q16 Which device protects lithium packs from over-charge and over-discharge?

(a) a BMS (b) an ESC (c) an H-bridge (d) a servo

10.7.3 Advanced level

Q17 A 2200 mAh 25C LiPo can supply what maximum current?

(a) 2.2 A (b) 25 A (c) 55 A (d) 88 A

Q18 Two 3.7 V 3000 mAh cells in parallel make what pack?

(a) 3.7 V, 6000 mAh
(b) 7.4 V, 3000 mAh
(c) 7.4 V, 6000 mAh
(d) 3.7 V, 3000 mAh

Q19 An L298N drops 2 V while a motor draws 1.2 A. How much power does it waste?

(a) 1.2 W (b) 2.4 W (c) 0.6 W (d) 24 W

Q20 A robot draws 650 mA on average from a 2000 mAh pack (80 % usable). Roughly how long does it

run?

(a) about 3.1 hours
(b) about 1.3 hours
(c) about 30 minutes
(d) about 2.5 hours

Q21 A4988 current limit: compute V_ref to set 1.0 A and 1.5 A with 0.068 Ω sense resistors (I = V_ref ÷ (8 × R_s)), and the DRV8825 V_ref for the same currents (I = 2 × V_ref).

Q22 Battery voltage monitor: a 3S LiPo is read through a 10 kΩ/3.3 kΩ divider on a 5 V 10-bit ADC. Convert raw readings to pack voltage and print OK, LOW (< 10.5 V) or CRITICAL (< 9.9 V).

Q23 Write a sketch that drives two motors through an L298N to make a robot go forward 1 s, turn left 0.5 s (left motor reverse, right forward) and stop, printing each action.

Q24 A robot's Arduino resets every time the motors start. What is the best fix?

(a) remove the ground wire
(b) use a smaller battery
(c) power the logic from a separate regulator (or battery), add a bulk capacitor and keep a common ground
(d) add a pull-up resistor to the motor

Q25 Which set of protections belongs in every battery-powered robot?

(a) only a power LED
(b) a heatsink on the battery
(c) fuse and switch at the battery, flyback diodes on inductive loads, bulk and decoupling capacitors
(d) nothing if the battery is new

10.8 Solutions

Q1 **Answer: (b) a pin supplies only about 20 mA, far less than a motor needs**
Use a driver.

Q2 **Answer: (a) an H-bridge**
Four switches.

Q3 **Answer: (c) runs forward**
HIGH/LOW = forward.

Q4 **Answer: (d) TB6612FNG**
MOSFETs, ~0.5 V drop.

Q5 **Answer: (d) STEP and DIR**
Pulse + direction.

Q6 **Answer: (c) 3.7 V**

4.2 V full, about 3.0 V empty.

Q7 **Answer: (a) 7.4 V nominal**

2 × 3.7.

Q8 **Answer: (d) signals are measured relative to ground; without a common ground the driver cannot read them**

Common ground.

Q9 **Answer: (b) excessive current from a short circuit, which could cause a fire**

Fuse near the battery.

Q10 **Answer: (b) both switches on one side turn on, shorting the supply**

Driver chips prevent it.

Q11 **Answer: (d) motor surges and noise can make the microcontroller reset or misbehave**

Avoid brown-outs.

Q12 **Answer: (a) a buck converter rated for at least 3 A**

Efficient switching.

Q13 **Answer: (c) its maximum safe discharge current as a multiple of capacity**

2200 mAh 20C = 44 A.

Q14 **Answer: (d) add their voltages**

Series = more volts.

Q15 **Answer: (b) to absorb current surges and reduce voltage dips**

Bulk capacitance.

Q16 **Answer: (a) a BMS**

Battery management system.

Q17 **Answer: (c) 55 A**

2.2 × 25.

Q18 **Answer: (a) 3.7 V, 6000 mAh**

Parallel adds capacity.

Q19 **Answer: (b) 2.4 W**

2 × 1.2.

Q20 **Answer: (d) about 2.5 hours**

1600 ÷ 650 = **2.46 h**.

Q21 **Answer: Solution program**

Measure V_ref with a multimeter between the potentiometer and GND while turning it; set it below the motor's rated current.

```
Rs = 0.068
for amps in (1.0, 1.5):
    a4988 = amps * 8 * Rs
    drv = amps / 2
    print(f"{amps} A -> A4988 Vref {a4988:.3f} V, DRV8825 Vref {drv:.2f} V")
```

OUTPUT

```
1.0 A -> A4988 Vref 0.544 V, DRV8825 Vref 0.50 V
1.5 A -> A4988 Vref 0.816 V, DRV8825 Vref 0.75 V
```

Q22 **Answer: Solution program**

The divider scales 12.6 V down to 3.13 V (safe for the ADC); the program multiplies back up by (R1 + R2)/R2.

```
R1, R2, VREF = 10_000, 3_300, 5.0
def pack_volts(raw):
    return raw * VREF / 1023 * (R1 + R2) / R2
for raw in (620, 540, 505, 480):
    v = pack_volts(raw)
    state = "CRITICAL" if v < 9.9 else "LOW" if v < 10.5 else "OK"
    print(f"raw {raw} -> {v:5.2f} V  {state}")
```

OUTPUT

```
raw 620 -> 12.21 V  OK
raw 540 -> 10.64 V  OK
raw 505 ->  9.95 V  LOW
raw 480 ->  9.46 V  CRITICAL
```

Q23 **Answer: Solution program**

A helper function per motor and one per manoeuvre keep the code readable.

```
const int ENA = 9, IN1 = 7, IN2 = 8, ENB = 10, IN3 = 11, IN4 = 12;
void setMotor(int en, int a, int b, int s) {
  digitalWrite(a, s > 0); digitalWrite(b, s < 0); analogWrite(en, abs(s));
}
void go(int l, int r, const char *what, int ms) {
  setMotor(ENA, IN1, IN2, l); setMotor(ENB, IN3, IN4, r);
  Serial.println(what); delay(ms);
}
void setup() {
  Serial.begin(9600);
  int p[] = {ENA, IN1, IN2, ENB, IN3, IN4};
  for (int x : p) pinMode(x, OUTPUT);
  go(200, 200, "forward", 1000);
  go(-160, 160, "turn left", 500);
  go(0, 0, "stop", 0);
}
void loop() {}
```

SERIAL MONITOR

```
forward
turn left
stop
```

Q24 **Answer: (c) power the logic from a separate regulator (or battery), add a bulk capacitor and keep a common ground**

Separate supplies.

Q25 **Answer: (c) fuse and switch at the battery, flyback diodes on inductive loads, bulk and decoupling capacitors**

Basic protection.

Pins give signals (~20 mA); drivers switch motor power. H-bridge: IN1/IN2 set direction, PWM sets speed (L293D small, L298N cheap but lossy, TB6612FNG/DRV8833 efficient). Stepper drivers take STEP/DIR and need the current limit set. Batteries: NiMH 1.2 V, Li-ion/LiPo 3.7 V per cell; series adds volts, parallel adds mAh; C-rating = max current; use a BMS and safe charging. Separate motor and logic power, common ground, buck converters, life = usable Ah ÷ A. Protect with fuses, diodes and capacitors.

MODULE 11 · INTERMEDIATE

Robot chassis, locomotion and kinematics

Minimum duration **2 days** | Level **Intermediate**

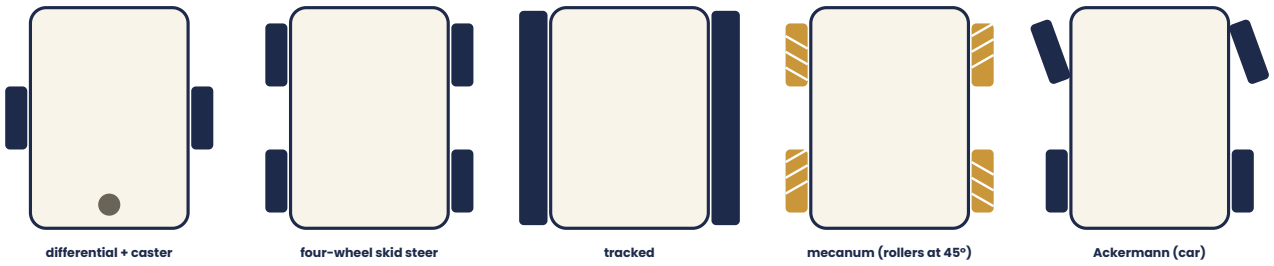
This module covers how mobile robots are built and how they move: two-wheel, four-wheel and tracked chassis, casters and balance; differential drive, skid steering, omni-wheel and mecanum drives and Ackermann steering; walking, balancing and snake robots; the kinematics of a differential-drive robot (wheel speeds to robot speed and turning, turning radius); and odometry with wheel encoders, followed by 25 solved questions and programs.

IN THIS MODULE

11.1 Chassis design
11.2 Drive systems
11.3 Legged and other locomotion (overview)
11.4 Kinematics of a differential-drive robot
11.5 Odometry with wheel encoders
11.6 Practice questions
11.7 Solutions

11.1 Chassis design

11.1.1 Two-wheel, four-wheel and tracked chassis



COMMON CHASSIS LAYOUTS SEEN FROM ABOVE

Chassis	Strengths	Weaknesses
2 driven wheels + caster	simple, cheap, turns on the spot, easy kinematics	caster struggles on rough ground
4WD skid steer	traction, carries more weight, rough ground	wheels scrub (slide) when turning: wastes power, poor odometry
Tracked	best grip on loose or uneven ground, climbs obstacles	heavy, inefficient turning, odometry very poor
Mecanum / omni	moves sideways and diagonally (holonomic)	complex, needs 4 motors, sensitive to floor

Chassis	Strengths	Weaknesses
Ackermann (car)	efficient at speed, stable	cannot turn on the spot; needs space to manoeuvre

11.1.2 Caster wheels and balance

A **caster** (ball caster or swivel wheel) supports a two-wheel robot so it does not tip. Place it so the centre of gravity sits between the drive axle and the caster, with most weight (60–80 %) on the driven wheels for grip. A caster that is too low lifts the drive wheels; too high lets the robot rock. Ball casters work best on smooth floors.

11.2 Drive systems

11.2.1 Differential drive

Two independently driven wheels on one axle. Equal speeds → straight; different speeds → curve; opposite speeds → spin on the spot. It is the most common layout for small robots (line followers, robot vacuums, TurtleBot) and has simple, well-understood kinematics (11.4).

11.2.2 Skid steering

Like differential drive but with four or six wheels (or tracks) where each side is driven together. Turning forces the wheels to slide sideways, so it needs more torque to turn and the real turning motion differs from the simple model. Use it when traction matters more than efficiency.

11.2.3 Omni-wheel and mecanum-wheel drives

Omni wheels have small free rollers around the rim at 90°; **mecanum wheels** have rollers at 45°. Combining the four wheel speeds lets the robot move in any direction and rotate at the same time (**holonomic** motion): ideal for warehouse robots and competition robots. For a mecanum robot (wheels FL, FR, RL, RR):

FL = vx − vy − ω·k FR = vx + vy + ω·k RL = vx + vy − ω·k RR = vx − vy + ω·k (vx forward, vy left, k = (L + W)/2)

MECANUM.PY

```
def mecanum(vx, vy, w, k=0.2):
    fl, fr = vx - vy - w * k, vx + vy + w * k
    rl, rr = vx + vy - w * k, vx - vy + w * k
    m = max(1.0, abs(fl), abs(fr), abs(rl), abs(rr))   # keep within -1..1
    return [round(v / m, 2) for v in (fl, fr, rl, rr)]

for name, cmd in [("forward", (1, 0, 0)), ("slide left", (0, 1, 0)), ("diagonal", (1, 1, 0)), ("spin
left", (0, 0, 5))]:
    print(f"{name:10} -> FL, FR, RL, RR = {mecanum(*cmd)}")
```

OUTPUT

```
forward    -> FL, FR, RL, RR = [1.0, 1.0, 1.0, 1.0]
slide left -> FL, FR, RL, RR = [-1.0, 1.0, 1.0, -1.0]
```

```
diagonal   -> FL, FR, RL, RR = [0.0, 1.0, 1.0, 0.0]
spin left  -> FL, FR, RL, RR = [-1.0, 1.0, -1.0, 1.0]
```

11.2.4 Ackermann (car-like) steering

Car steering turns the front wheels by different angles so all wheels roll around a common centre (no scrubbing). The robot cannot rotate on the spot; its minimum turning radius is $R \approx L \div \tan(\delta)$, with wheelbase L and steering angle δ. Self-driving car research and racing robots (F1TENTH) use it.

11.3 Legged and other locomotion (overview)

11.3.1 Walking robots, balancing robots, snake robots

Type	How it moves	Notes
Hexapod (6 legs)	tripod gait: three legs always on the ground	statically stable, easy for beginners (18 servos)
Quadruped (4 legs)	trot and walk gaits, dynamic balance	Boston Dynamics Spot, Unitree; needs fast control
Biped / humanoid	walks on two legs, constantly balancing	hardest; uses IMU and force sensors
Self-balancing robot	two wheels, an inverted pendulum kept upright by PID on the IMU angle	classic control project (module 12)
Snake robot	many joints making waves	pipes, rubble, search and rescue

11.4 Kinematics of a differential-drive robot

11.4.1 Wheel speeds to robot speed and turning

Kinematics relates wheel motion to robot motion (ignoring forces). With left and right wheel speeds v_L, v_R (m/s) and track width L (distance between the wheels):

$v = (v_R + v_L) \div 2$ $\omega = (v_R - v_L) \div L$ and in reverse: $v_L = v - \omega \cdot L/2$, $v_R = v + \omega \cdot L/2$



THE CENTRE MOVES AT THE AVERAGE SPEED; THE DIFFERENCE TURNS THE ROBOT

11.4.2 Turning radius

The robot turns around a point on the axle line at radius $\mathbf{R = v \div \omega = (L/2) \times (v_R + v_L) \div (v_R - v_L)}$. Equal speeds: R = ∞ (straight). One wheel stopped: R = L/2 (pivot about that wheel). Opposite equal speeds: R = 0 (spin on the spot).

DIFF_KINEMATICS.PY

```
L = 0.15                                  # track width (m)
def kinematics(vl, vr):
    v, w = (vr + vl) / 2, (vr - vl) / L
    R = float("inf") if w == 0 else v / w
    return v, w, R
for vl, vr in [(0.3, 0.3), (0.2, 0.3), (0.0, 0.3), (-0.2, 0.2), (0.3, 0.1)]:
    v, w, R = kinematics(vl, vr)
    rtxt = "straight" if R == float("inf") else f"R = {R:+.3f} m"
    print(f"vL {vl:+.2f}  vR {vr:+.2f} -> v {v:.2f} m/s, w {w:+.2f} rad/s, {rtxt}")

def wheel_speeds(v, w):                    # inverse kinematics
    return v - w * L / 2, v + w * L / 2
print("arc of radius 0.5 m at 0.25 m/s ->", tuple(round(x, 3) for x in wheel_speeds(0.25, 0.25 /
0.5)))
```

OUTPUT

```
vL +0.30  vR +0.30 -> v 0.30 m/s, w +0.00 rad/s, straight
vL +0.20  vR +0.30 -> v 0.25 m/s, w +0.67 rad/s, R = +0.375 m
vL +0.00  vR +0.30 -> v 0.15 m/s, w +2.00 rad/s, R = +0.075 m
vL -0.20  vR +0.20 -> v 0.00 m/s, w +2.67 rad/s, R = +0.000 m
vL +0.30  vR +0.10 -> v 0.20 m/s, w -1.33 rad/s, R = -0.150 m
arc of radius 0.5 m at 0.25 m/s -> (0.212, 0.287)
```

WORKED EXAMPLE

Turning 90° on the spot

Wheels at ±0.1 m/s with L = 0.15 m give $\omega = 0.2 \div 0.15 = 1.33$ rad/s. A 90° turn is $\pi/2 = 1.571$ rad, taking $1.571 \div 1.33 =$ **1.18 s**. Each wheel travels $(L/2) \times \pi/2 = 0.118$ m: with encoders, turn until each wheel has moved that far instead of trusting time.

11.5 Odometry with wheel encoders

11.5.1 Estimating distance travelled and position

Odometry estimates the robot's position (x, y) and heading θ by adding up small wheel movements measured by encoders. Each update: d_L, d_R = ticks × distance per tick; $d = (d_L + d_R)/2$; $\Delta\theta = (d_R - d_L)/L$; $x \mathrel{+}= d\cdot\cos(\theta + \Delta\theta/2)$; $y \mathrel{+}= d\cdot\sin(\theta + \Delta\theta/2)$; $\theta \mathrel{+}= \Delta\theta$.

ODOMETRY_SQUARE.PY

```
import math
D, TPR, L = 0.065, 40, 0.15            # wheel diameter, ticks per rev, track width
per_tick = math.pi * D / TPR
x = y = th = 0.0
def update(tl, tr):
    global x, y, th
    dl, dr = tl * per_tick, tr * per_tick
    d, dth = (dl + dr) / 2, (dr - dl) / L
    x += d * math.cos(th + dth / 2); y += d * math.sin(th + dth / 2); th += dth

turn_ticks = round((L / 2) * (math.pi / 2) / per_tick)     # ticks per wheel for 90 deg
side_ticks = round(0.5 / per_tick)                          # 0.5 m
print(f"per tick {per_tick * 1000:.2f} mm; 0.5 m = {side_ticks} ticks; 90 deg turn = {turn_ticks}
ticks/wheel")
for side in range(4):                                       # drive a 0.5 m square
    update(side_ticks, side_ticks)
    update(-turn_ticks, turn_ticks)
    print(f"after side {side + 1}: x {x:+.3f} m  y {y:+.3f} m  heading {math.degrees(th) % 360:6.1f}
deg")
```

OUTPUT

```
per tick 5.11 mm; 0.5 m = 98 ticks; 90 deg turn = 23 ticks/wheel
after side 1: x +0.500 m  y +0.000 m  heading   89.7 deg
after side 2: x +0.503 m  y +0.500 m  heading  179.4 deg
after side 3: x +0.003 m  y +0.506 m  heading  269.1 deg
after side 4: x -0.005 m  y +0.005 m  heading  358.8 deg
```

Even with perfect simulated encoders the square does not close exactly: tick counts are rounded, so each turn is slightly more or less than 90°, and the error grows with every side. Real robots add wheel slip and unequal wheel sizes, so odometry drifts; correct it with a gyro, landmarks or mapping (module 18).

PRACTICE CHECKPOINT

Assemble a differential-drive robot chassis, program it to drive straight, turn 90° and drive in a square, and measure its odometry error.

11.6 Practice questions

11.6.1 Basic level

Q1 What is differential drive?

(a) front wheels steer like a car
(b) only one motor
(c) legs instead of wheels
(d) two independently driven wheels; speed differences steer the robot

Q2 What does a caster wheel do on a two-wheel robot?

(a) drives the robot
(b) supports the robot so it does not tip
(c) measures distance
(d) steers like a car

Q3 Which chassis can move sideways without turning?

(a) mecanum (or omni) wheels
(b) differential drive
(c) Ackermann
(d) tracked

Q4 Which steering cannot turn on the spot?

(a) differential drive
(b) Ackermann (car-like)
(c) skid steer
(d) mecanum

Q5 What happens when both wheels of a differential robot turn at equal speed in opposite directions?

(a) it spins on the spot
(b) it goes straight
(c) it stops
(d) it drives in a large circle

Q6 What is odometry?

(a) measuring battery level
(b) reading colour
(c) estimating position from wheel movement
(d) mapping with a camera

Q7 Which chassis gives the best grip on loose ground?

(a) ball caster two-wheel
(b) mecanum
(c) omni
(d) tracked

Q8 What gait keeps three legs of a hexapod on the ground?

(a) trot
(b) gallop
(c) hop
(d) tripod gait

11.6.2 Intermediate level

Q9 Why does odometry drift?

(a) encoders reset every second
(b) batteries change direction
(c) wheel slip, unequal wheel sizes and rounding errors accumulate
(d) it does not drift

Q10 Why is skid steering inefficient when turning?

(a) wheels must slide sideways **(b)** it has no motors
(c) it uses servos **(d)** it cannot go straight

Q11 What is a self-balancing robot's control problem called?

(a) a lever **(b)** a pulley
(c) Ohm's law **(d)** an inverted pendulum

Q12 What does kinematics study?

(a) battery chemistry
(b) how wheel or joint motions relate to the robot's motion, ignoring forces
(c) radio communication
(d) forces and masses only

Q13 One wheel stopped, the other moving: the robot pivots about

(a) its centre
(b) the stopped wheel (R = L/2 from the centre)
(c) a point far away
(d) the caster only

Q14 Where should a two-wheel robot's centre of gravity be?

(a) behind the caster
(b) outside the wheels
(c) high above the robot
(d) between the drive axle and the caster, mostly over the drive wheels

Q15 Which drive does a warehouse robot that slides sideways between shelves likely use?

(a) mecanum or omni wheels **(b)** Ackermann
(c) single wheel **(d)** tracks

Q16 What is the formula for robot speed v in differential drive?

(a) $v = v_R - v_L$ **(b)** $v = v_R \times v_L$
(c) $v = (v_R + v_L) \div 2$ **(d)** $v = L \div 2$

11.6.3 Advanced level

Q17 $v_L = 0.2$ m/s, $v_R = 0.4$ m/s, $L = 0.2$ m. What is ω?

(a) 0.3 rad/s **(b)** 2.0 rad/s **(c)** 0.1 rad/s **(d)** 1.0 rad/s

Q18 For the same robot, what is the turning radius?

(a) 0.1 m **(b)** 0.3 m **(c)** 1.0 m **(d)** 3 m

Q19 An Ackermann robot has wheelbase 0.3 m and maximum steering angle 30°. What is its minimum turning radius?

(a) about 0.52 m **(b)** about 0.17 m **(c)** about 0.3 m **(d)** about 1.0 m

Q20 Wheels at ±0.15 m/s, L = 0.15 m. How long does a 180° spin take?

(a) about 3.14 s **(b)** about 0.79 s **(c)** about 1.57 s **(d)** about 1 s

Q21 Write inverse kinematics for a differential robot (L = 0.18 m): print wheel speeds for driving straight at 0.3 m/s, an arc of radius 0.6 m at 0.3 m/s, and spinning at 2 rad/s.

Q22 Compute the encoder ticks each wheel needs for a 90° on-the-spot turn and for 1 m forward: wheel diameter 6.7 cm, 360 ticks per revolution, track width 14 cm.

Q23 Simulate odometry error: a robot drives 1 m straight 5 times, but its right wheel is 1 % larger than assumed. Print the final position and heading.

Q24 A line follower must be fast and simple on a smooth floor. Which chassis fits best?

(a) two-wheel differential drive with a ball caster

(b) tracked chassis

(c) hexapod

(d) Ackermann car with a large turning radius

Q25 Why should a square-driving robot turn by encoder ticks rather than by time?

(a) ticks are faster to type

(b) time-based turns change with battery voltage, load and floor, while ticks measure the actual wheel movement

(c) time cannot be measured on a robot

(d) encoders make turns slower

11.7 Solutions

Q1 **Answer: (d) two independently driven wheels; speed differences steer the robot**

Most common small-robot drive.

Q2 **Answer: (b) supports the robot so it does not tip**

Passive support.

Q3 **Answer: (a) mecanum (or omni) wheels**

Holonomic.

Q4 **Answer: (b) Ackermann (car-like)**

Needs a turning radius.

Q5 **Answer: (a) it spins on the spot**

R = 0.

Q6 **Answer: (c) estimating position from wheel movement**

Dead reckoning with encoders.

Q7 **Answer: (d) tracked**

Large contact area.

Q8 **Answer: (d) tripod gait**

Statically stable.

Q9 **Answer: (c) wheel slip, unequal wheel sizes and rounding errors accumulate**

Errors add up.

Q10 **Answer: (a) wheels must slide sideways**

Scrubbing.

Q11 **Answer: (d) an inverted pendulum**

Keep it upright with PID.

Q12 **Answer: (b) how wheel or joint motions relate to the robot's motion, ignoring forces**

Geometry of motion.

Q13 **Answer: (b) the stopped wheel (R = L/2 from the centre)**

Pivot turn.

Q14 **Answer: (d) between the drive axle and the caster, mostly over the drive wheels**

Grip and stability.

Q15 **Answer: (a) mecanum or omni wheels**

Holonomic motion.

Q16 **Answer: (c) v = (v_R + v_L) ÷ 2**

Average.

Q17 **Answer: (d) 1.0 rad/s**

(0.4 - 0.2) ÷ 0.2.

Q18 **Answer: (b) 0.3 m**

v = 0.3, ω = 1 → R = 0.3 m.

Q19 **Answer: (a) about 0.52 m**

0.3 ÷ tan 30° = **0.52 m**.

Q20 **Answer: (c) about 1.57 s**

ω = 2 rad/s; π ÷ 2 = **1.57 s**.

Q21 **Answer: Solution program**

v_L = v - ωL/2 and v_R = v + ωL/2; for an arc, ω = v ÷ R.

```
L = 0.18
def wheels(v, w):
    return round(v - w * L / 2, 3), round(v + w * L / 2, 3)
print("straight:", wheels(0.3, 0))
print("arc R=0.6:", wheels(0.3, 0.3 / 0.6))
print("spin 2 rad/s:", wheels(0, 2))
```

OUTPUT

```
straight: (0.3, 0.3)
arc R=0.6: (0.255, 0.345)
spin 2 rad/s: (-0.18, 0.18)
```

Q22 **Answer: Solution program**

Distance per tick = πD ÷ ticks per revolution; for a spin each wheel travels an arc of radius L/2.

```
import math
D, TPR, L = 0.067, 360, 0.14
per_tick = math.pi * D / TPR
print(f"1 m forward: {1 / per_tick:.0f} ticks per wheel")
print(f"90 deg spin: {(L / 2) * (math.pi / 2) / per_tick:.0f} ticks per wheel (opposite
directions)")
```

OUTPUT

```
1 m forward: 1710 ticks per wheel
90 deg spin: 188 ticks per wheel (opposite directions)
```

Q23 **Answer: Solution program**

A 1 % wheel-size error turns the robot by (0.01 × 5) ÷ 0.15 rad ≈ 19° over 5 m and moves it about 0.8 m sideways: calibrate wheel diameters carefully.

```
import math
L, step = 0.15, 0.01                  # update every 1 cm
x = y = th = 0.0
for _ in range(500):                  # 5 m in 1 cm updates
    dl, dr = step, step * 1.01        # right wheel actually travels 1 % further
    d, dth = (dl + dr) / 2, (dr - dl) / L
    x += d * math.cos(th + dth / 2); y += d * math.sin(th + dth / 2); th += dth
print(f"believed: x 5.000 m, y 0.000 m, heading 0 deg")
print(f"actual:   x {x:.3f} m, y {y:.3f} m, heading {math.degrees(th):.1f} deg")
```

OUTPUT

```
believed: x 5.000 m, y 0.000 m, heading 0 deg
actual:   x 4.932 m, y 0.830 m, heading 19.1 deg
```

Q24 **Answer: (a) two-wheel differential drive with a ball caster**

Light, simple, turns sharply.

Q25 **Answer: (b) time-based turns change with battery voltage, load and floor, while ticks measure the actual wheel movement**

Measure, don't assume.

Choose the chassis for the terrain: differential + caster (simple), skid steer/tracks (grip), mecanum/omni (holonomic), Ackermann (car). Differential kinematics: $v = (v_R + v_L)/2$, $\omega = (v_R - v_L)/L$, $R = v/\omega$; inverse: $v_{L,R} = v \mp \omega L/2$. Odometry integrates encoder distances into x, y, θ but drifts: calibrate wheel size and track width and correct with other sensors.

MODULE 12 · INTERMEDIATE

Control systems and PID

Minimum duration **2 days** | Level **Intermediate**

Control systems keep robots doing what they are told, accurately and smoothly. This module explains open-loop and closed-loop control, setpoint, error and feedback, and block diagrams; on-off (bang-bang) control; the proportional, integral and derivative terms of PID, manual and Ziegler–Nichols tuning, overshoot, oscillation and steady-state error; and applies PID to line following, motor speed control with encoders, balancing robots and wall following, followed by 25 solved questions and programs. Every controller in this module was run in simulation.

IN THIS MODULE

12.1 Control basics
12.2 On-off (bang-bang) control
12.3 PID control
12.4 Applications
12.5 Practice questions
12.6 Solutions

12.1 Control basics

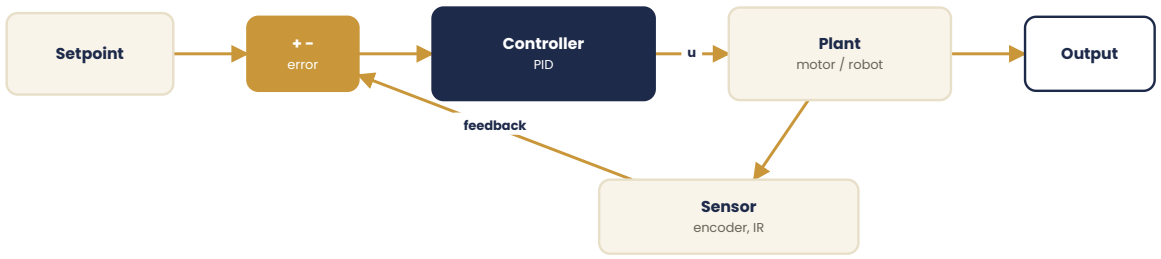
12.1.1 Open-loop vs closed-loop control

	Open loop	Closed loop (feedback)
How	send a command and hope: "motor at 60 % for 2 s"	measure the result and correct: "adjust power until speed = 120 rpm"
Sensor needed?	no	yes
Copes with load, battery and floor changes?	no	yes
Examples	toaster timer, timed turns, stepper without feedback	cruise control, thermostat, servo, line follower

12.1.2 Setpoint, error, feedback

The **setpoint** (SP) is the desired value (120 rpm, line in the centre, 25 cm from the wall). The **process variable** (PV) is the measured value. The **error** e = SP - PV. The controller turns the error into a corrective **output** (motor power); the sensor **feeds back** the new PV, closing the loop.

12.1.3 Block diagrams



A CLOSED-LOOP CONTROL SYSTEM

12.2 On-off (bang-bang) control

The simplest feedback: output fully ON when below the setpoint, fully OFF (or fully reversed) when above. Room thermostats work like this. For robots it causes constant **oscillation**: a bang-bang line follower zig-zags because it always over-corrects. A small **hysteresis** band (switch on below SP - h, off above SP + h) reduces rapid switching.

BANG_BANG.PY

```
class Motor:
    """simple DC motor + gearbox model: speed responds to drive (0..1) with lag and load"""
    def __init__(self, max_rpm=200, tau=0.15, load=0.0):
        self.rpm, self.max, self.tau, self.load = 0.0, max_rpm, tau, load
    def step(self, u, dt):
        u = max(-1.0, min(1.0, u))
        target = self.max * u - self.load * self.max
        self.rpm += (target - self.rpm) * dt / self.tau
        return self.rpm

m = Motor()
dt, target = 0.02, 120
for k in range(1, 76):
    u = 1.0 if m.rpm < target else 0.0          # bang-bang
    m.step(u, dt)
    if k % 15 == 0:
        print(f"t={k * dt:.2f} s  drive {u:.0f}  speed {m.rpm:6.1f} rpm")
```

OUTPUT

```
t=0.30 s  drive 0  speed  107.1 rpm
t=0.60 s  drive 0  speed  112.4 rpm
t=0.90 s  drive 1  speed  119.0 rpm
t=1.20 s  drive 1  speed  122.9 rpm
t=1.50 s  drive 1  speed  128.1 rpm
```

The speed keeps crossing 120 rpm because the drive is either full or zero; on a real robot this shows up as jerky motion and wasted energy.

12.3 PID control

output = Kp·e + Ki·∫e dt + Kd·de/dt (P: present error · I: accumulated past error · D: predicted future error)

12.3.1 Proportional term

P pushes harder the bigger the error: output = K_p × e. It responds immediately, but a P-only controller usually settles with a remaining **steady-state error** (a motor needs some drive just to keep turning, and that drive only exists while there is error). Too large a K_p causes overshoot and oscillation.

12.3.2 Integral term

I adds up the error over time: output += K_i × Σ e·dt. Any lasting error keeps growing the integral until it is eliminated, so the I term removes steady-state error. Too much I causes slow oscillation and overshoot; **integral windup** (the sum growing huge while the output is saturated) is prevented by clamping the integral.

12.3.3 Derivative term

D reacts to how fast the error changes: output += K_d × (e - $e_{previous}$) ÷ dt. It acts like a brake, damping overshoot and oscillation as the value approaches the setpoint. It amplifies sensor noise, so use small values and filter noisy measurements.

PID_COMPARE.PY

```
class Motor:
    """simple DC motor + gearbox model: speed responds to drive (0..1) with lag and load"""
    def __init__(self, max_rpm=200, tau=0.15, load=0.0):
        self.rpm, self.max, self.tau, self.load = 0.0, max_rpm, tau, load
    def step(self, u, dt):
        u = max(-1.0, min(1.0, u))
        target = self.max * u - self.load * self.max
        self.rpm += (target - self.rpm) * dt / self.tau
        return self.rpm

class PID:
    def __init__(self, kp, ki, kd, lo=-1.0, hi=1.0):
        self.kp, self.ki, self.kd, self.lo, self.hi = kp, ki, kd, lo, hi
        self.i, self.prev = 0.0, None
    def update(self, sp, pv, dt):
        e = sp - pv
        self.i = max(self.lo, min(self.hi, self.i + self.ki * e * dt))   # anti-windup
        d = 0.0 if self.prev is None else (e - self.prev) / dt
        self.prev = e
        return max(self.lo, min(self.hi, self.kp * e + self.i + self.kd * d))

def run(kp, ki, kd, load=0.1, t_end=1.5, dt=0.01):
    m, pid, sp = Motor(load=load), PID(kp, ki, kd), 120
    peak = 0
    for _ in range(int(t_end / dt)):
        m.step(pid.update(sp, m.rpm, dt), dt)
        peak = max(peak, m.rpm)
    return m.rpm, max(0, peak - sp)

print("controller            final rpm   overshoot")
for name, g in [("P  (Kp=0.01)", (0.01, 0, 0)), ("P  (Kp=0.05)", (0.05, 0, 0)),
                ("PI (Kp=0.01, Ki=0.05)", (0.01, 0.05, 0)), ("PI (Kp=0.01, Ki=0.15)", (0.01, 0.15,
0)),
                ("PID (Kp=0.01, Ki=0.15, Kd=0.0004)", (0.01, 0.15, 0.0004))]:
    final, over = run(*g)
    print(f"{name:34} {final:7.1f}   {over:6.1f}")
```

OUTPUT

```
controller            final rpm   overshoot
P  (Kp=0.01)                          73.3      0.0
P  (Kp=0.05)                         107.3      0.0
PI (Kp=0.01, Ki=0.05)                119.9      0.0
PI (Kp=0.01, Ki=0.15)                120.0     13.2
PID (Kp=0.01, Ki=0.15, Kd=0.0004)    120.0     11.5
```

With P only the motor settles well below 120 rpm (more gain helps but never removes the error); the integral term brings it to 120 rpm; a large integral overshoots, and a small derivative term trims that overshoot. These are exactly the effects you see when tuning a real robot.

12.3.4 Tuning PID gains (manual and Ziegler–Nichols overview)

1. Set $K_i = K_d = 0$. Increase K_p until the response is fast and just starts to oscillate, then back off to about half.

2. Increase K_d to damp overshoot and oscillation.
3. Increase K_i slowly until the steady-state error disappears in a reasonable time without big overshoot.
4. Test at different speeds, loads and battery levels; adjust.

Ziegler–Nichols: with I and D off, raise K_p until the system oscillates steadily; note that "ultimate gain" K_u and the oscillation period T_u. Then:

Controller	Kp	Ki	Kd
P	0.5 Ku	—	—
PI	0.45 Ku	0.54 Ku / Tu	—
PID	0.6 Ku	1.2 Ku / Tu	0.075 Ku·Tu

Ziegler–Nichols gives an aggressive starting point (about 25 % overshoot); fine-tune by hand from there.

12.3.5 Overshoot, oscillation and steady-state error

Symptom	Likely cause	Fix
slow to reach the target	Kp too small	raise Kp
oscillates around the target	Kp too big or Kd too small	lower Kp, raise Kd
overshoots then settles	too much Ki or not enough Kd	lower Ki, raise Kd
settles short of the target	no or too little Ki	add Ki
jittery, noisy output	Kd amplifying noise	lower Kd, filter the sensor
huge overshoot after being blocked	integral windup	clamp the integral

12.4 Applications

12.4.1 Line following with PID

The line sensor array gives a position error (module 8: 0 when centred, ±2000 at the edges). The PID output becomes a steering correction added to one motor and subtracted from the other: left = base + correction, right = base - correction. Compared with bang-bang it follows curves smoothly at higher speed.

PIDLINEFOLLOWER.INO

```
const int N = 5; const int pins[N] = {A0, A1, A2, A3, A4};
const float KP = 0.06, KI = 0.0, KD = 0.4;
const int BASE = 150;
float lastError = 0, integral = 0;

long linePos() {
  long w = 0, s = 0;
  for (int i = 0; i < N; i++) { int v = 1023 - analogRead(pins[i]); w += (long)v * i * 1000; s += v;
}
  return w / s;                                        // 0..4000
}

void setup() { Serial.begin(9600); }

void loop() {
  float error = linePos() - 2000;                      // + = line to the right
  integral += error;
  float corr = KP * error + KI * integral + KD * (error - lastError);
  lastError = error;
  int left = constrain(BASE + corr, 0, 255);
  int right = constrain(BASE - corr, 0, 255);
  Serial.print("error "); Serial.print(error, 0);
  Serial.print("  left "); Serial.print(left);
  Serial.print("  right "); Serial.println(right);
}
```

SERIAL MONITOR

```
error 0  left 150  right 150
error 246  left 255  right 36
error 781  left 255  right 0
error 386  left 15  right 255
```

12.4.2 Motor speed control with encoders

Measure speed from encoder ticks per interval, compare with the target rpm and let a PI controller adjust PWM. Matching both wheels to the same target speed is the proper way to make a robot drive straight, regardless of small motor differences or battery level.

12.4.3 Balancing robots (overview)

A two-wheel balancing robot is an inverted pendulum: the IMU (complementary filter) gives the tilt angle, and a fast PID loop (100–200 Hz) drives the wheels under the falling body to keep the angle at the balance point. A second, slower loop controls position or speed by shifting the balance setpoint slightly. It needs a fast, low-noise angle and careful tuning: a classic advanced project.

12.4.4 Wall following

WALL_FOLLOW.PY

```
class PID:
    def __init__(s, kp, ki, kd): s.kp, s.ki, s.kd, s.i, s.prev = kp, ki, kd, 0.0, None
    def update(s, e, dt):
        s.i += e * dt; d = 0 if s.prev is None else (e - s.prev) / dt; s.prev = e
        return s.kp * e + s.ki * s.i + s.kd * d

import math
dist, heading, v, dt = 0.45, 0.0, 0.25, 0.05       # start 45 cm from the wall, parallel to it
pid, target = PID(6.0, 0.0, 6.0), 0.25
for k in range(1, 121):
    w = pid.update(dist - target, dt)              # too far -> turn towards the wall
    w = max(-1.5, min(1.5, w))                     # limit the turn rate
    heading = max(-0.785, min(0.785, heading - w * dt))   # keep within +/-45 deg of the wall
    dist += v * math.sin(heading) * dt             # positive heading = away from the wall
    if k % 15 == 0:
        print(f"t={k * dt:.2f} s  distance {dist * 100:5.1f} cm  heading
{math.degrees(heading):6.1f} deg")
```

OUTPUT

```
t=0.75 s  distance  39.1 cm  heading  -27.7 deg
t=1.50 s  distance  30.5 cm  heading  -23.7 deg
t=2.25 s  distance  25.1 cm  heading  -10.9 deg
t=3.00 s  distance  23.3 cm  heading   -1.5 deg
t=3.75 s  distance  23.6 cm  heading    2.1 deg
t=4.50 s  distance  24.3 cm  heading    2.2 deg
t=5.25 s  distance  24.9 cm  heading    1.2 deg
t=6.00 s  distance  25.1 cm  heading    0.3 deg
```

The robot steers towards the wall, the derivative term (which reacts to the distance changing) straightens it up before it overshoots far, and it settles close to 25 cm, running nearly parallel. The same structure keeps a robot centred in a corridor (error = left distance - right distance).

PRACTICE CHECKPOINT

Build a PID-controlled line follower, tune its gains, and compare its speed and smoothness against an on-off controlled version.

12.5 Practice questions

12.5.1 Basic level

Q1 What is the difference between open-loop and closed-loop control?

(a) open-loop uses more sensors
(b) closed-loop has no controller
(c) there is no difference
(d) closed-loop measures the result and corrects using feedback

Q2 What is the error in a control system?

(a) the sensor's price
(b) the output power
(c) setpoint minus measured value
(d) a programming bug

Q3 What does bang-bang control do?

(a) adds the error over time
(b) predicts the future error
(c) switches the output fully on or fully off
(d) never changes the output

Q4 Which PID term reacts to the present error?

(a) integral **(b)** derivative **(c)** none **(d)** proportional

Q5 Which PID term removes steady-state error?

(a) proportional **(b)** integral **(c)** derivative **(d)** setpoint

Q6 Which PID term damps overshoot?

(a) derivative
(b) integral
(c) proportional
(d) feed-forward only

Q7 What is a setpoint?

(a) the measured value
(b) the desired value the controller aims for
(c) the output
(d) the sensor offset

Q8 Which is an example of closed-loop control?

(a) a servo holding an angle using its potentiometer
(b) a timed toast cycle
(c) a robot turning by time only
(d) a light on a timer

12.5.2 Intermediate level

Q9 Why does a bang-bang line follower zig-zag?

(a) its sensors are broken
(b) PID is too smooth

(c) it always over-corrects with full steering
(d) the battery is full

Q10 What is integral windup?

(a) the robot winding its cable
(b) a gear problem
(c) the derivative term becoming zero
(d) the integral grows huge while the output is saturated, causing overshoot later

Q11 Why must the derivative term be used carefully?

(a) it removes all error
(b) it slows the loop down
(c) it needs a GPS
(d) it amplifies sensor noise

Q12 A system oscillates around the target. What should you try first?

(a) increase Kp a lot
(b) remove the sensor
(c) reduce Kp or increase Kd
(d) increase Ki a lot

Q13 A motor settles at 110 rpm instead of 120 rpm with P control. What fixes it?

(a) add an integral term
(b) raise the setpoint to 120
(c) remove P
(d) add more noise

Q14 In PID line following, how is the correction applied?

(a) to both motors equally
(b) to the sensor
(c) to the battery voltage
(d) added to one motor's speed and subtracted from the other

Q15 What do Ziegler–Nichols tuning rules start from?

(a) the battery capacity
(b) the ultimate gain Ku and oscillation period Tu
(c) the motor colour
(d) random numbers

Q16 Which robot is an inverted pendulum?

(a) a line follower with a caster
(b) a two-wheel self-balancing robot
(c) a robot arm on a table
(d) a mecanum cart

12.5.3 Advanced level

Q17 SP = 30 cm, measured = 42 cm, Kp = 0.05. What is the P output?

(a) 0.6
(b) 12
(c) -12
(d) -0.6

Q18 Ziegler–Nichols PID: Ku = 2.0, Tu = 0.5 s. What is Kp?

(a) 1.2
(b) 1.0
(c) 0.9
(d) 2.4

Q19 For the same values, what is Ki (= 1.2 Ku/Tu)?

(a) 1.2
(b) 0.6
(c) 4.8
(d) 9.6

Q20 And Kd (= 0.075 Ku·Tu)?

(a) 0.15 **(b)** 0.3 **(c)** 0.0375 **(d)** 0.075

Q21 Implement a PID class with output limits and anti-windup, and print the outputs for errors [10, 8, 5, 2, 0, -1] at dt = 0.1 s with Kp = 0.5, Ki = 0.2, Kd = 0.1.

Q22 Compare bang-bang and proportional control of a heater: room at 18 °C, target 22 °C, heater up to 3 °C/min, losses 0.1 × (T - 15) °C/min, 30 minutes, 0.5-minute steps. Print the temperature every 5 minutes for both.

Q23 Write an Arduino speed controller: a P controller sets PWM from the error between target 100 ticks/s and measured speed. Use measured values 0, 40, 75, 95, 105 and print error and PWM (Kp = 1.5, base 120).

Q24 Which is the best order for manual PID tuning?

(a) set all three high at once

(b) tune Kp first, then add Kd to damp, then a little Ki to remove offset

(c) tune Ki first with Kp = 0

(d) only change Kd

Q25 A PID line follower works at slow speed but oscillates when the base speed is raised. What should you do?

(a) retune: lower Kp and raise Kd for the higher speed (or schedule gains by speed)

(b) remove the sensors

(c) switch to bang-bang

(d) increase Ki a lot

12.6 Solutions

Q1 **Answer: (d) closed-loop measures the result and corrects using feedback**

Feedback.

Q2 **Answer: (c) setpoint minus measured value**

e = SP - PV.

Q3 **Answer: (c) switches the output fully on or fully off**

On-off control.

Q4 **Answer: (d) proportional**

P.

Q5 **Answer: (b) integral**

I accumulates error.

Q6 **Answer: (a) derivative**

D acts as a brake.

Q7 **Answer: (b) the desired value the controller aims for**

Target.

Q8 **Answer: (a) a servo holding an angle using its potentiometer**

Feedback inside the servo.

Q9 **Answer: (c) it always over-corrects with full steering**

All or nothing.

Q10 **Answer: (d) the integral grows huge while the output is saturated, causing overshoot later**

Clamp the integral.

Q11 **Answer: (d) it amplifies sensor noise**

Filter and keep Kd small.

Q12 **Answer: (c) reduce Kp or increase Kd**

Damp it.

Q13 **Answer: (a) add an integral term**

Steady-state error → I.

Q14 **Answer: (d) added to one motor's speed and subtracted from the other**

Differential steering.

Q15 **Answer: (b) the ultimate gain Ku and oscillation period Tu**

Measured at steady oscillation.

Q16 **Answer: (b) a two-wheel self-balancing robot**

Must be actively balanced.

Q17 **Answer: (d) -0.6**

e = -12; -12 × 0.05 = **-0.6**.

Q18 **Answer: (a) 1.2**

0.6 × 2.0.

Q19 **Answer: (c) 4.8**

1.2 × 2 ÷ 0.5.

Q20 **Answer: (d) 0.075**

0.075 × 2 × 0.5.

Q21 **Answer: Solution program**

P follows the error, I accumulates slowly, D (negative here because the error is shrinking) softens the response.

```
class PID:
    def __init__(self, kp, ki, kd, lo=-10, hi=10):
        self.kp, self.ki, self.kd, self.lo, self.hi = kp, ki, kd, lo, hi
        self.i, self.prev = 0.0, None
    def step(self, e, dt):
        self.i = max(self.lo, min(self.hi, self.i + self.ki * e * dt))
        d = 0.0 if self.prev is None else (e - self.prev) / dt
        self.prev = e
        return max(self.lo, min(self.hi, self.kp * e + self.i + self.kd * d))
pid = PID(0.5, 0.2, 0.1)
for e in [10, 8, 5, 2, 0, -1]:
    print(f"error {e:3d} -> output {pid.step(e, 0.1):6.2f}")
```

OUTPUT

```
error  10 -> output   5.20
error   8 -> output   2.36
error   5 -> output  -0.04
error   2 -> output  -1.50
error   0 -> output  -1.50
error  -1 -> output  -1.02
```

Q22 **Answer: Solution program**

Bang-bang overshoots and chatters around 22 °C; P control approaches smoothly but settles slightly below the target (steady-state error), which an integral term would remove.

```
def simulate(controller):
    T, out = 18.0, []
    for k in range(61):
        if k % 10 == 0: out.append(round(T, 2))
        power = controller(22 - T)
        T += (3 * power - 0.1 * (T - 15)) * 0.5
    return out
bang = lambda e: 1.0 if e > 0 else 0.0
prop = lambda e: max(0.0, min(1.0, 0.8 * e))
print("minute: ", [5 * i for i in range(7)])
print("bang:   ", simulate(bang))
print("P only: ", simulate(prop))
```

OUTPUT

```
minute:  [0, 5, 10, 15, 20, 25, 30]
bang:    [18.0, 22.24, 23.05, 22.28, 23.07, 22.29, 23.08]
P only:  [18.0, 21.72, 21.72, 21.72, 21.72, 21.72, 21.72]
```

Q23 **Answer: Solution program**

A base (feed-forward) PWM plus a proportional correction: the output falls as the speed approaches the target and drops below base when it overshoots.

```
const float KP = 1.5;
const int BASE = 120, TARGET = 100;
int measured[] = {0, 40, 75, 95, 105};
void setup() {
  Serial.begin(9600);
  for (int i = 0; i < 5; i++) {
    int err = TARGET - measured[i];
    int pwm = constrain(BASE + KP * err, 0, 255);
    Serial.print("speed "); Serial.print(measured[i]);
    Serial.print("  error "); Serial.print(err);
    Serial.print("  pwm "); Serial.println(pwm);
  }
}
void loop() {}
```

SERIAL MONITOR

```
speed 0  error 100  pwm 255
speed 40  error 60  pwm 210
speed 75  error 25  pwm 157
speed 95  error 5  pwm 127
speed 105  error -5  pwm 112
```

Q24 **Answer: (b) tune Kp first, then add Kd to damp, then a little Ki to remove offset**

P → D → I.

Q25 **Answer: (a) retune: lower Kp and raise Kd for the higher speed (or schedule gains by speed)**

Higher speed needs more damping.

Closed loop: measure, compute error = setpoint - measured, correct. Bang-bang oscillates. PID: P reacts to the present error (leaves steady-state error), I removes lasting error (clamp against windup), D damps overshoot (amplifies noise). Tune P → D → I, or start from Ziegler–Nichols (0.6Ku, 1.2Ku/Tu, 0.075KuTu). Uses: PID line following, encoder speed control, balancing robots, wall following.

MODULE 13 · INTERMEDIATE

Building mobile robots

Minimum duration **3 days** | Level **Intermediate**

This module combines sensors, motors and control into complete autonomous robots: a line follower with calibrated sensors and junction handling, an obstacle avoider that scans with a servo-mounted ultrasonic sensor, an edge-avoiding table-top robot, a light follower, a smartphone-controlled Bluetooth robot, and a maze solver using wall following, the left-hand rule and flood fill, plus a systematic method for testing and debugging robots, followed by 25 solved questions and programs.

IN THIS MODULE

13.1 Line follower robot
13.2 Obstacle-avoiding robot
13.3 Edge-avoiding (table-top) robot
13.4 Light-following robot
13.5 Bluetooth-controlled robot
13.6 Maze-solving robot
13.7 Testing and debugging robots
13.8 Practice questions
13.9 Solutions

All the robots in this module share one platform: a two-wheel differential-drive chassis with a caster, an Arduino Uno, a TB6612FNG or L298N driver and a 2S battery with a 5 V regulator (modules 9–11). Only the sensors and the program change. Each sketch uses this helper for the motors:

SHARED MOTOR HELPER (PINS AS IN MODULE 10)

```
void drive(int left, int right) {          // -255..255 per side
  digitalWrite(AIN1, left > 0);  digitalWrite(AIN2, left < 0);  analogWrite(PWMA, abs(left));
  digitalWrite(BIN1, right > 0); digitalWrite(BIN2, right < 0); analogWrite(PWMB, abs(right));
}
```

13.1 Line follower robot

13.1.1 Sensor placement and calibration

Mount the line sensors 3–10 mm above the floor (check the module's ideal height) and a few centimetres **ahead** of the wheels: the further ahead, the earlier the robot sees curves, but the harder it is to control. Space sensors so that at least one always sees a line of the expected width (usually 18–25 mm). Floors and lighting differ, so **calibrate** at start-up: sweep the sensors over the line and background, record each sensor's minimum and maximum, then scale every reading to 0–1000.

LINECALIBRATE.INO

```
const int N = 5; const int pins[N] = {A0, A1, A2, A3, A4};
int lo[N], hi[N];

void calibrate(int samples) {
  for (int i = 0; i < N; i++) { lo[i] = 1023; hi[i] = 0; }
  for (int s = 0; s < samples; s++)                 // robot sweeps left/right over the line
    for (int i = 0; i < N; i++) { int v = analogRead(pins[i]); lo[i] = min(lo[i], v); hi[i] =
max(hi[i], v); }
}
int scaled(int i) {                                  // 0 = white .. 1000 = black
  long v = analogRead(pins[i]);
  return constrain(1000L * (hi[i] - v) / (hi[i] - lo[i]), 0, 1000);
}

void setup() {
  Serial.begin(9600);
  calibrate(4);
  for (int i = 0; i < N; i++) { Serial.print("S"); Serial.print(i); Serial.print(" white ");
Serial.print(hi[i]); Serial.print(" black "); Serial.println(lo[i]); }
  Serial.print("scaled: ");
  for (int i = 0; i < N; i++) { Serial.print(scaled(i)); Serial.print(" "); }
  Serial.println();
}
void loop() {}
```

SERIAL MONITOR

```
S0 white 880 black 250
S1 white 900 black 300
S2 white 910 black 270
S3 white 890 black 260
S4 white 870 black 240
scaled: 0 433 953 15 15
```

13.1.2 Handling junctions and sharp turns

A plain PID follower (module 12) handles curves. Special cases need extra rules: all sensors black = **junction or crossing** (go straight, or count junctions for a route); all white = **line lost** (turn towards the side where the line was last seen); outer sensor only = **sharp turn** (slow down and pivot). These rules sit on top of the PID loop as a small decision layer.

LINEDECISIONS.INO

```
const int N = 5; const int pins[N] = {A0, A1, A2, A3, A4};
int lastSide = 0;                                  // -1 left, +1 right

String decide() {
  int b[N]; int count = 0;
  for (int i = 0; i < N; i++) { b[i] = analogRead(pins[i]) < 500; count += b[i]; }
  if (count == N) return "junction: go straight and count";
  if (count == 0) return lastSide < 0 ? "line lost: pivot left" : "line lost: pivot right";
  if (b[0] && !b[2]) { lastSide = -1; return "sharp left: slow + pivot"; }
  if (b[4] && !b[2]) { lastSide = 1; return "sharp right: slow + pivot"; }
  lastSide = (b[0] || b[1]) ? -1 : (b[3] || b[4]) ? 1 : lastSide;
  return "PID follow";
}

void setup() { Serial.begin(9600); }
void loop() { Serial.println(decide()); }
```

SERIAL MONITOR

```
PID follow
junction: go straight and count
line lost: pivot right
sharp left: slow + pivot
line lost: pivot left
```

13.2 Obstacle-avoiding robot

13.2.1 Ultrasonic scanning with a servo

Mounting the ultrasonic sensor on a servo lets one sensor look in several directions. When the path ahead is blocked, the robot stops, looks left and right, and turns towards the more open side. Allow the servo time to move (about 200–300 ms) before each measurement.



ONE SENSOR, THREE VIEWING ANGLES

13.2.2 Decision logic

SCANAVOID.INO

```
#include <Servo.h>
Servo neck;
const int TRIG = 9, ECHO = 10;
long cm() {
  digitalWrite(TRIG, HIGH); delayMicroseconds(10); digitalWrite(TRIG, LOW);
  long us = pulseIn(ECHO, HIGH, 30000);
  return us == 0 ? 400 : us / 58;                  // no echo = nothing within range
}
long lookAt(int angle) { neck.write(angle); delay(250); return cm(); }

void setup() {
  Serial.begin(9600);
  pinMode(TRIG, OUTPUT); pinMode(ECHO, INPUT);
  neck.attach(6);
}

void loop() {
  long ahead = lookAt(90);
  if (ahead > 25) { Serial.print(ahead); Serial.println(" cm ahead: drive forward"); return; }
  Serial.print(ahead); Serial.println(" cm ahead: blocked, scanning");
  long left = lookAt(150), right = lookAt(30);
  neck.write(90);
  Serial.print("  left "); Serial.print(left); Serial.print(" cm, right "); Serial.print(right);
Serial.println(" cm");
  if (left < 20 && right < 20) Serial.println("  both blocked: reverse and turn around");
  else if (left > right) Serial.println("  turn left");
  else Serial.println("  turn right");
}
```

SERIAL MONITOR

```
50 cm ahead: drive forward
15 cm ahead: blocked, scanning
  left 60 cm, right 20 cm
  turn left
25 cm ahead: blocked, scanning
  left 10 cm, right 11 cm
  both blocked: reverse and turn around
400 cm ahead: drive forward
```

13.3 Edge-avoiding (table-top) robot

Downward-facing IR sensors at the front corners see the table surface (reflection) or empty space (no reflection) at the edge. The moment either sensor loses the surface, stop, reverse and turn away from that side. Mount the sensors well ahead of the wheels and keep speed modest so the robot can stop in time.

EDGEAVOID.INO

```
const int LEFT_EDGE = 2, RIGHT_EDGE = 3;     // IR modules: LOW = surface seen, HIGH = no surface
void setup() { Serial.begin(9600); pinMode(LEFT_EDGE, INPUT); pinMode(RIGHT_EDGE, INPUT); }
void loop() {
  bool l = digitalRead(LEFT_EDGE), r = digitalRead(RIGHT_EDGE);
  if (l && r)      Serial.println("edge ahead: reverse, turn 180");
  else if (l)      Serial.println("left edge: reverse, turn right");
  else if (r)      Serial.println("right edge: reverse, turn left");
  else             Serial.println("table ok: forward");
  delay(100);
}
```

SERIAL MONITOR

```
table ok: forward
table ok: forward
edge ahead: reverse, turn 180
edge ahead: reverse, turn 180
```

13.4 Light-following robot

Two LDRs angled left and right (in voltage dividers) compare light levels. Steering proportional to the difference makes the robot turn towards the brighter side (a **Braitenberg vehicle**: crossing the connections makes it seek light; straight connections make it avoid light). Shade the LDRs with short tubes so they "look" in a direction.

LIGHTFOLLOW.INO

```
const int BASE = 140; const float K = 0.4;
void setup() { Serial.begin(9600); }
void loop() {
  int left = analogRead(A0), right = analogRead(A1);   // higher = brighter
  int diff = left - right;
  int lm = constrain(BASE - K * diff, 0, 255);          // brighter on the left -> slow left wheel
  int rm = constrain(BASE + K * diff, 0, 255);
  Serial.print("L "); Serial.print(left); Serial.print(" R "); Serial.print(right);
  Serial.print(" -> motors "); Serial.print(lm); Serial.print(", "); Serial.println(rm);
}
```

SERIAL MONITOR

```
L 600 R 600 -> motors 140, 140
L 850 R 500 -> motors 0, 255
L 400 R 820 -> motors 255, 0
```

13.5 Bluetooth-controlled robot

13.5.1 Control from a smartphone app

An **HC-05** Bluetooth module (or an ESP32's built-in Bluetooth) appears to the Arduino as a serial port. A phone app such as "Bluetooth RC Controller" or "Arduino Bluetooth Control" sends single letters: F, B, L, R, S.

Wire HC-05 TX to Arduino RX (through SoftwareSerial on pins 10/11) and Arduino TX to HC-05 RX through a 1 kΩ/2 kΩ divider (the HC-05 is 3.3 V logic). Add a **timeout**: stop if no command arrives for half a second.

BLUETOOTHCAR.INO

```
#include <SoftwareSerial.h>
SoftwareSerial bt(10, 11);                // RX, TX
unsigned long lastCmd = 0;

void setup() { Serial.begin(9600); bt.begin(9600); }

void loop() {
  if (bt.available()) {
    char c = bt.read();
    lastCmd = millis();
    switch (c) {
      case 'F': Serial.println("F -> forward");  break;
      case 'B': Serial.println("B -> backward"); break;
      case 'L': Serial.println("L -> left");     break;
      case 'R': Serial.println("R -> right");    break;
      case 'S': Serial.println("S -> stop");     break;
      default:  break;                          // ignore unknown characters
    }
  }
}
```

SERIAL MONITOR

```
F -> forward
F -> forward
L -> left
R -> right
B -> backward
S -> stop
```

13.6 Maze-solving robot

13.6.1 Wall-following and left-hand rule

In a maze whose walls are all connected (no islands), keeping one hand on a wall always leads to the exit. The **left-hand rule**: at each step, turn left if possible; otherwise go straight; otherwise turn right; otherwise turn back. A robot needs left, front and right distance sensors (IR or ToF) and accurate 90° turns.

LEFT_HAND_RULE.PY

```
MAZE = ["#########",
        "#S..#...#",
        "#.#.#.#.#",
        "#.#...#.#",
        "#.#####.#",
        "#...#..E#",
        "#########"]
DIRS = [(-1, 0), (0, 1), (1, 0), (0, -1)]           # N, E, S, W (row, col)
open_ = lambda r, c: MAZE[r][c] != "#"
r, c = 1, 1; d = 1                                  # start facing east
steps = 0
while MAZE[r][c] != "E" and steps < 200:
    for turn in (-1, 0, 1, 2):                      # left, straight, right, back
        nd = (d + turn) % 4
        nr, nc = r + DIRS[nd][0], c + DIRS[nd][1]
        if open_(nr, nc):
            d, r, c = nd, nr, nc
            break
    steps += 1
print(f"reached the exit at row {r}, col {c} in {steps} moves (left-hand rule)")
```

OUTPUT

```
reached the exit at row 5, col 7 in 14 moves (left-hand rule)
```

13.6.2 Flood-fill algorithm (overview)

Micromouse robots use **flood fill**: give every cell its distance (in moves) to the goal, starting from 0 at the goal and spreading outwards like water. The robot always moves to the neighbouring cell with a smaller number. As it discovers walls, it updates the numbers and re-floods. After exploring, it can run the shortest path at full speed.

FLOOD_FILL.PY

```
from collections import deque
MAZE = ["#########",
        "#S..#...#",
        "#.#.#.#.#",
        "#.#...#.#",
        "#.#####.#",
        "#...#..E#",
        "#########"]
R, C = len(MAZE), len(MAZE[0])
goal = next((r, c) for r in range(R) for c in range(C) if MAZE[r][c] == "E")
dist = [[None] * C for _ in range(R)]
dist[goal[0]][goal[1]] = 0
q = deque([goal])
while q:                                          # flood outwards from the goal
    r, c = q.popleft()
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        nr, nc = r + dr, c + dc
        if MAZE[nr][nc] != "#" and dist[nr][nc] is None:
            dist[nr][nc] = dist[r][c] + 1
            q.append((nr, nc))
for r in range(R):
    print(" ".join("##" if MAZE[r][c] == "#" else f"{dist[r][c]:2d}" for c in range(C)))
print("shortest path from S:", dist[1][1], "moves")
```

OUTPUT

```
## ## ## ## ## ## ## ## ##
## 14 13 12 ##  6  5  4 ##
## 15 ## 11 ##  7 ##  3 ##
## 16 ## 10  9  8 ##  2 ##
## 17 ## ## ## ## ##  1 ##
## 18 19 20 ##  2  1  0 ##
## ## ## ## ## ## ## ## ##
shortest path from S: 14 moves
```

In this maze the left-hand rule happened to find the shortest route (14 moves), because no dead end branches off to its left. In general it explores every left-hand dead end first, while flood fill always knows the shortest distance from every cell, and works even when walls are not connected.

13.7 Testing and debugging robots

13.7.1 Isolating mechanical, electrical and software problems

Symptom	Check first	Typical cause
robot does nothing	power LED, battery voltage, fuse, switch	flat battery, loose power wire
Arduino resets when motors start	battery voltage under load	shared supply, no bulk capacitor
one motor dead or weak	swap motor connections at the driver	wiring, driver channel, standby pin, motor

Symptom	Check first	Typical cause
robot curves when it should go straight	wheel diameters, friction, motor speed	motors differ: use encoders/PID
sensor values random	print raw readings; check GND and pull-ups	floating input, loose wire, noise
works on the bench, fails on the floor	lighting, surface, battery under load	calibration, sunlight on IR sensors
code seems ignored	serial prints at each decision	logic error, wrong pin number, blocking delay()

1. **Reproduce** the problem reliably.
2. **Isolate**: test each subsystem alone: motors with a simple test sketch, each sensor with a print sketch, power with a multimeter under load.
3. **Change one thing at a time** and note the result.
4. **Instrument**: serial prints, LEDs for states, a logic analyser for buses.
5. **Fix the cause**, not the symptom, then retest the whole robot.

SHORTCUT TRICK

Keep tiny test sketches for each part (motor_test, sensor_test, servo_test). When the full robot misbehaves, running them takes two minutes and immediately shows whether the fault is mechanical, electrical or in the main program.

PRACTICE CHECKPOINT

Build and demonstrate a line follower, an obstacle avoider and a Bluetooth-controlled robot, and document the circuit, code and lessons learned for each.

13.8 Practice questions

13.8.1 Basic level

Q1 Where should line sensors be mounted?

(a) on top of the robot
(b) behind the wheels, high up
(c) close to the floor and a little ahead of the wheels
(d) on the battery

Q2 Why calibrate line sensors at start-up?

(a) floors, line colours and lighting differ
(b) to charge them
(c) they need a password
(d) calibration is not needed

Q3 What does "all sensors black" usually mean for a line follower?

(a) the line is lost
(b) a junction or crossing
(c) a sharp right turn
(d) the battery is low

Q4 Why mount the ultrasonic sensor on a servo?

(a) to make it louder
(b) to cool it
(c) to power it
(d) one sensor can look in several directions

Q5 Which sensors detect a table edge?

(a) an upward ultrasonic sensor
(b) a DHT22
(c) downward-facing IR sensors
(d) a magnetometer

Q6 Which module adds Bluetooth to an Arduino?

(a) L298N **(b)** MPU6050 **(c)** HC-05 **(d)** A4988

Q7 What does the left-hand rule do in a maze?

(a) always turns right
(b) finds the shortest path immediately
(c) needs GPS
(d) keeps following the left wall until the exit

Q8 Which algorithm do micromouse robots use to find the shortest path?

(a) bubble sort **(b)** flood fill **(c)** bang-bang **(d)** PWM

Q9 What does "all line sensors white" usually mean?

(a) the line has been lost
(b) a junction
(c) the robot is centred
(d) the sensors are calibrated

13.8.2 Intermediate level

Q10 Why is a timeout important for a Bluetooth robot?

(a) it stops the robot if the connection drops
(b) it speeds up the robot
(c) it saves Bluetooth power only
(d) it is required by the phone

Q11 How does a light-following robot steer?

(a) by GPS
(b) compares two light sensors and turns towards the brighter side
(c) by sound
(d) by random turns

Q12 Why must servo-mounted sensors wait before measuring?

(a) the servo needs time to reach the angle
(b) the sensor needs to charge
(c) sound needs to warm up
(d) to save memory

Q13 When does the left-hand rule fail?

(a) when the maze has right turns
(b) when the robot is fast
(c) when the exit is on an island not connected to the outer walls
(d) it never fails

Q14 Why connect the Arduino TX to the HC-05 RX through a divider?

(a) to slow the data
(b) to add encryption
(c) it is not needed
(d) the HC-05 uses 3.3 V logic

Q15 What is the first step in debugging a robot?

(a) rewrite all the code
(b) buy new parts
(c) change several things at once
(d) reproduce the problem reliably

Q16 A robot's Arduino resets when the motors start. Which subsystem is at fault?

(a) software logic
(b) sensor calibration
(c) power (supply voltage dips)
(d) the maze algorithm

Q17 A line follower works indoors but fails near a sunny window. Why?

(a) sunlight contains infrared that swamps the IR sensors
(b) the motors get hot
(c) the battery charges
(d) Bluetooth interference

13.8.3 Advanced level

Q18 A line sensor reads 300 on black and 900 on white. What is the scaled value (0 white – 1000 black) of a reading of 600?

(a) 600 **(b)** 300 **(c)** 667 **(d)** 500

Q19 A robot must stop within 5 cm at an edge. It travels 0.4 m/s and needs 0.05 s to react plus braking 0.04 m. Is 5 cm enough?

(a) yes: it needs 4 cm
(b) no: it needs 6 cm
(c) yes: it needs 2 cm
(d) no: it needs 20 cm

Q20 Write the flood-fill distances for a 5 × 5 open grid (no walls) with the goal in the centre and print the grid.

Q21 Simulate a junction counter: a line follower passes junctions and must turn right at the 3rd one. Given sensor patterns per step, print the action.

Q22 Write an obstacle-avoider decision sketch: if the front distance is below 20 cm, print "turn"; between 20 and 40 cm "slow"; otherwise "cruise". Use echo times for 50, 30 and 12 cm.

Q23 Write a Bluetooth command parser that accepts speed commands like "V150" (set speed) and letters F/S, printing the robot state.

Q24 Your robot curves left when told to go straight. Which order of checks is sensible?

(a) rewrite the whole program first
(b) check wheel/motor mechanics and wiring, test each motor alone, then add encoder-based speed matching
(c) replace the battery with mains power
(d) add more sensors immediately

Q25 A maze robot explores and then must race to the goal by the shortest route. Which approach fits?

(a) left-hand rule at full speed
(b) random turns
(c) bang-bang line following
(d) map walls while exploring, then flood-fill and follow decreasing distances

13.9 Solutions

Q1 **Answer: (c) close to the floor and a little ahead of the wheels**

Low and forward.

Q2 **Answer: (a) floors, line colours and lighting differ**

Scale min/max to 0–1000.

Q3 **Answer: (b) a junction or crossing**

Junction.

Q4 **Answer: (d) one sensor can look in several directions**

Scanning.

Q5 **Answer: (c) downward-facing IR sensors**

No reflection = edge.

Q6 **Answer: (c) HC-05**

Serial Bluetooth.

Q7 **Answer: (d) keeps following the left wall until the exit**

Wall following.

Q8 **Answer: (b) flood fill**

Distance numbers from the goal.

Q9 **Answer: (a) the line has been lost**

Recover by turning towards the last seen side.

Q10 **Answer: (a) it stops the robot if the connection drops**

Fail-safe.

Q11 **Answer: (b) compares two light sensors and turns towards the brighter side**

Braitenberg vehicle.

Q12 **Answer: (a) the servo needs time to reach the angle**

200–300 ms.

Q13 **Answer: (c) when the exit is on an island not connected to the outer walls**

Disconnected walls.

Q14 **Answer: (d) the HC-05 uses 3.3 V logic**

5 V → 3.3 V.

Q15 **Answer: (d) reproduce the problem reliably**

Reproduce, isolate, change one thing.

Q16 **Answer: (c) power (supply voltage dips)**

Electrical.

Q17 **Answer: (a) sunlight contains infrared that swamps the IR sensors**

Shade or recalibrate.

Q18 **Answer: (d) 500**

1000 × (900 - 600) ÷ (900 - 300) = **500**.

Q19 **Answer: (b) no: it needs 6 cm**

0.4 × 0.05 = 0.02 m + 0.04 m = **6 cm**.

Q20 **Answer: Solution program**

With no walls the values are the Manhattan distance to the centre; walls make the numbers bend around them.

```
from collections import deque
N = 5; goal = (2, 2)
dist = [[None] * N for _ in range(N)]
dist[2][2] = 0; q = deque([goal])
while q:
    r, c = q.popleft()
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        nr, nc = r + dr, c + dc
        if 0 <= nr < N and 0 <= nc < N and dist[nr][nc] is None:
            dist[nr][nc] = dist[r][c] + 1; q.append((nr, nc))
for row in dist:
    print(" ".join(str(v) for v in row))
```

OUTPUT

```
4 3 2 3 4
3 2 1 2 3
2 1 0 1 2
3 2 1 2 3
4 3 2 3 4
```

Q21 **Answer: Solution program**

Counting junction events (all sensors on the line) lets a line follower follow a planned route.

```
patterns = ["00100", "11111", "00100", "01100", "11111", "00110", "11111", "00100"]
junctions = 0
for p in patterns:
    if p == "11111":
        junctions += 1
        action = "turn right" if junctions == 3 else "go straight"
        print(f"{p}: junction {junctions} -> {action}")
    else:
        print(f"{p}: follow line")
```

OUTPUT

```
00100: follow line
11111: junction 1 -> go straight
00100: follow line
01100: follow line
11111: junction 2 -> go straight
00110: follow line
11111: junction 3 -> turn right
00100: follow line
```

Q22 **Answer: Solution program**

Graded responses (cruise, slow, turn) make motion smoother than a single stop/go threshold.

```
void setup() {
  Serial.begin(9600);
  pinMode(9, OUTPUT); pinMode(10, INPUT);
}
void loop() {
  digitalWrite(9, HIGH); delayMicroseconds(10); digitalWrite(9, LOW);
  long d = pulseIn(10, HIGH, 30000) / 58;
  Serial.print(d); Serial.print(" cm: ");
  if (d < 20) Serial.println("turn");
  else if (d < 40) Serial.println("slow");
  else Serial.println("cruise");
}
```

SERIAL MONITOR

```
50 cm: cruise
30 cm: slow
12 cm: turn
```

Q23 **Answer: Solution program**

Digits after V are accumulated into a number and constrained to the PWM range.

```
#include <SoftwareSerial.h>
SoftwareSerial bt(10, 11);
int speed = 100;
void setup() { Serial.begin(9600); bt.begin(9600); }
void loop() {
  if (!bt.available()) return;
  char c = bt.read();
  if (c == 'V') {
    int v = 0;
    while (bt.available()) { char d = bt.peek(); if (d < '0' || d > '9') break; v = v * 10 +
(bt.read() - '0'); }
    speed = constrain(v, 0, 255);
    Serial.print("speed set to "); Serial.println(speed);
  } else if (c == 'F') { Serial.print("forward at "); Serial.println(speed); }
  else if (c == 'S') Serial.println("stop");
}
```

SERIAL MONITOR

```
forward at 100
speed set to 200
forward at 200
speed set to 255
stop
```

Q24 **Answer: (b) check wheel/motor mechanics and wiring, test each motor alone, then add encoder-based speed matching**

Isolate mechanical, electrical, then software.

Q25 **Answer: (d) map walls while exploring, then flood-fill and follow decreasing distances**

Micromouse strategy.

Line follower: sensors low and ahead, calibrate to 0–1000, PID plus rules for junctions, lost lines and sharp turns. Obstacle avoider: servo-scan and turn to the open side. Edge avoider: downward IR, stop and back away. Light follower: steer by LDR difference. Bluetooth: HC-05 as serial, single-letter commands, timeout. Mazes: left-hand rule (connected walls) or flood fill (shortest path). Debug by reproducing, isolating subsystems and changing one thing at a time.

MODULE 14 · INTERMEDIATE

Communication between devices

Minimum duration **2 days** | Level **Intermediate**

Robots must communicate between sensors, controllers, other robots and users. This module covers wired protocols (UART serial, I2C, SPI and an overview of CAN bus), wireless links (Bluetooth Classic with the HC-05 and Bluetooth Low Energy, Wi-Fi with the ESP32, nRF24L01 and LoRa radios, IR remote control), robot-to-robot messaging with ESP-NOW, and data formats for sending commands and sensor data reliably, followed by 25 solved questions and programs.

IN THIS MODULE

14.1 Wired communication protocols
14.2 Wireless communication
14.3 Communication between robots
14.4 Data formats
14.5 Practice questions
14.6 Solutions

14.1 Wired communication protocols

Protocol	Wires	Speed	Devices	Robot use
UART	TX, RX, GND	9600–1 000 000 baud	2 (point to point)	Arduino ↔ Pi, GPS, Bluetooth, LiDAR
I2C	SDA, SCL, GND	100–400 kHz	up to ~112 by address	IMU, OLED, ToF, servo driver
SPI	MOSI, MISO, SCK, CS per device	1–50 MHz	several (one CS each)	SD card, radio, fast displays
CAN	CAN-H, CAN-L (twisted pair)	125 kbit/s – 1 Mbit/s	many nodes on a bus	cars, industrial robots, smart motors

14.1.1 UART (serial)

Two devices send bytes over TX → RX (crossed) at an agreed **baud rate**, with a shared ground. No clock wire: both sides time the bits themselves. Format 8N1 = 1 start bit, 8 data bits, no parity, 1 stop bit (10 bits per byte). The Uno's hardware UART (pins 0/1) is shared with USB; use SoftwareSerial (or a Mega/ESP32 with extra UARTs) for a second device.

SERIALCOMMANDS.INO

```
// Receive text commands like "SPEED 150" or "LED ON" from another board or the PC
String line = "";

void handle(String cmd) {
  cmd.trim();
  if (cmd.startsWith("SPEED ")) {
    int v = constrain(cmd.substring(6).toInt(), 0, 255);
    Serial.print("ACK speed="); Serial.println(v);
  } else if (cmd == "LED ON" || cmd == "LED OFF") {
    Serial.print("ACK "); Serial.println(cmd);
  } else {
    Serial.print("ERR unknown: "); Serial.println(cmd);
  }
}

void setup() { Serial.begin(115200); }

void loop() {
  while (Serial.available()) {
    char c = Serial.read();
    if (c == '\n') { handle(line); line = ""; }    // one command per line
    else line += c;
  }
}
```

SERIAL MONITOR

```
ACK speed=150
ACK LED ON
ERR unknown: JUMP
ACK speed=255
```

14.1.2 I2C

Two shared wires (SDA data, SCL clock) with pull-up resistors connect many devices; the controller addresses each by its 7-bit address. A scanner sketch finds which addresses respond: the first thing to run when an I2C device "does not work".

I2CSCANNER.INO

```
#include <Wire.h>
void setup() {
  Serial.begin(9600);
  Wire.begin();
  int found = 0;
  for (int addr = 1; addr < 127; addr++) {
    Wire.beginTransmission(addr);
    if (Wire.endTransmission() == 0) {             // 0 = device acknowledged
      Serial.print("device at 0x"); Serial.println(addr, HEX);
      found++;
    }
  }
  Serial.print(found); Serial.println(" device(s) found");
}
void loop() {}
```

SERIAL MONITOR

```
device at 0x29
device at 0x3C
device at 0x68
3 device(s) found
```

Here the scanner found a VL53L0X (0x29), an OLED (0x3C) and an MPU6050 (0x68). Two identical devices with the same fixed address need an address jumper, a multiplexer (TCA9548A) or a second bus.

14.1.3 SPI

SPI uses a clock (SCK), data out (MOSI), data in (MISO) and one chip-select (CS) line per device, pulled LOW to talk to it. It is full-duplex and much faster than I2C, so it suits SD cards, nRF24L01 radios and colour displays. More devices cost one extra CS pin each.

14.1.4 CAN bus (overview)

CAN (Controller Area Network), designed for cars, connects many nodes on two twisted wires with differential signalling that resists electrical noise over long cables. Messages have IDs that set priority, and the hardware detects errors and retransmits automatically. Industrial robot joints, smart servo motors (ODrive, some robot arms) and vehicles use it; an MCP2515 module or the ESP32's built-in TWAI controller adds it to hobby boards.

14.2 Wireless communication

14.2.1 Bluetooth Classic (HC-05) and Bluetooth Low Energy

	Bluetooth Classic (HC-05, ESP32)	Bluetooth Low Energy (BLE)
Model	a wireless serial cable (SPP)	small "characteristics" read, written or notified
Power	higher	very low (coin cells)
Range	≈ 10 m	≈ 10–50 m
Phones	Android apps; iPhones do not support SPP	Android and iPhone
Robot use	simple RC control from an Android app	phone apps, wearables, sensor tags

14.2.2 Wi-Fi with ESP32

An ESP32 joins the home Wi-Fi (station mode) or creates its own network (access-point mode) and can serve a web page with control buttons, open to any phone or laptop with a browser: no app needed. For telemetry it can publish MQTT messages (IoT, module 23).

WIFIROBOT.INO

```
#include <WiFi.h>
#include <WebServer.h>
WebServer server(80);
String state = "stopped";

void page() {
  server.send(200, "text/html",
    "<h1>Robot</h1><p>State: " + state + "</p>"
    "<a href='/go?d=F'>Forward</a> <a href='/go?d=L'>Left</a> "
    "<a href='/go?d=R'>Right</a> <a href='/go?d=S'>Stop</a>");
}
void go() {
  String d = server.arg("d");
  state = d == "F" ? "forward" : d == "L" ? "turning left" : d == "R" ? "turning right" : "stopped";
  Serial.print("command "); Serial.print(d); Serial.print(" -> "); Serial.println(state);
  page();
}

void setup() {
  Serial.begin(115200);
  WiFi.softAP("RobotCar", "robot1234");          // the robot makes its own network
  Serial.print("open http://"); Serial.println(WiFi.softAPIP());
  server.on("/", page);
  server.on("/go", go);
  server.begin();
}

void loop() { server.handleClient(); }
```

SERIAL MONITOR

```
open http://192.168.4.1
--> browser requests GET /
<-- 200 text/html: <h1>Robot</h1><p>State: stopped</p><a href='/go?d=F'>Forward</a> <a href='/go?
d=L'>Left</a> <a href='/go?d=R'>Right</a> <a href='/go?d=S'>Stop</a>
--> browser requests GET /go?d=F
command F -> forward
<-- 200 text/html: <h1>Robot</h1><p>State: forward</p><a href='/go?d=F'>Forward</a> <a href='/go?
d=L'>Left</a> <a href='/go?d=R'>Right</a> <a href='/go?d=S'>Stop</a>
--> browser requests GET /go?d=L
command L -> turning left
<-- 200 text/html: <h1>Robot</h1><p>State: turning left</p><a href='/go?d=F'>Forward</a> <a
href='/go?d=L'>Left</a> <a href='/go?d=R'>Right</a> <a href='/go?d=S'>Stop</a>
--> browser requests GET /go?d=S
command S -> stopped
<-- 200 text/html: <h1>Robot</h1><p>State: stopped</p><a href='/go?d=F'>Forward</a> <a href='/go?
d=L'>Left</a> <a href='/go?d=R'>Right</a> <a href='/go?d=S'>Stop</a>
```

14.2.3 RF modules (nRF24L01) and LoRa

Radio	Range	Data rate	Use
nRF24L01 (2.4 GHz)	≈ 30–100 m (PA+LNA versions ~1 km)	250 kbit/s – 2 Mbit/s	custom RC transmitters, robot swarms; SPI, 32-byte packets

Radio	Range	Data rate	Use
433 MHz ASK modules	≈ 20–100 m	slow	very cheap one-way remotes
LoRa (SX1276, 865–867 MHz band in India)	2–15 km	0.3–37 kbit/s	farm robots, remote sensors: tiny messages, very long range

14.2.4 IR remote control

A TV remote flashes an IR LED in coded bursts at 38 kHz; a receiver module (TSOP38238, VS1838B) demodulates them. The common **NEC** code sends an address and a command byte (and their inverses for checking). The IRremote library decodes it, so any spare remote can steer a robot indoors, as long as there is line of sight.

14.3 Communication between robots

14.3.1 ESP-NOW and multi-robot messaging

ESP-NOW lets ESP32/ESP8266 boards send short packets (up to 250 bytes) directly to each other by MAC address, with no router, very low latency (a few milliseconds) and low power: ideal for a controller and several robots, or robots sharing positions. For larger systems, robots exchange messages through an MQTT broker or ROS 2 topics over Wi-Fi.

ESP-NOW SENDER (REFERENCE)

```
// sender (ESP32): broadcast a small struct to robot 2
#include <esp_now.h>
#include <WiFi.h>
typedef struct { int16_t speed; int16_t turn; uint8_t id; } Command;
uint8_t robot2[] = {0x24, 0x6F, 0x28, 0xAA, 0xBB, 0xCC};       // receiver MAC address
void setup() {
  WiFi.mode(WIFI_STA);
  esp_now_init();
  esp_now_peer_info_t peer = {}; memcpy(peer.peer_addr, robot2, 6); esp_now_add_peer(&peer);
}
void loop() {
  Command c = {150, -20, 1};
  esp_now_send(robot2, (uint8_t *)&c, sizeof(c));              // 5 bytes, a few ms
  delay(50);
}
```

14.4 Data formats

14.4.1 Sending commands and sensor data reliably

Format	Example	Pros / cons
Single characters	F, B, L, R, S	simplest; limited
Text lines	SPEED 150\n, T=24.5,H=60\n	human-readable, easy to debug; slower to parse

Format	Example	Pros / cons
CSV	120,95,1\n	compact text; fields must stay in order
JSON	{"l":120,"r":95}	self-describing; larger; needs a library on small boards
Binary packet	header + bytes + checksum	fast and compact; harder to debug

Reliable messages need: a clear **start/end** (newline, or a header byte such as 0xAA), a **length** or fixed size, a **checksum** to detect corruption, and **timeouts** (stop the robot if commands stop). Acknowledgements ("ACK") let the sender retry.

PACKETS.PY

```
import struct

def make_packet(left, right, flags):
    payload = struct.pack("<hhB", left, right, flags)        # 2 signed shorts + 1 byte
    checksum = 0
    for b in payload:
        checksum ^= b                                         # XOR checksum
    return bytes([0xAA, len(payload)]) + payload + bytes([checksum])

def parse_packet(data):
    if len(data) < 3 or data[0] != 0xAA:
        return "bad header"
    n = data[1]; payload = data[2:2 + n]
    if len(payload) != n or len(data) < 3 + n:
        return "incomplete"
    check = 0
    for b in payload:
        check ^= b
    if check != data[2 + n]:
        return "checksum error"
    return struct.unpack("<hhB", payload)

pkt = make_packet(180, -60, 1)
print("packet:", pkt.hex(" "))
print("parsed:", parse_packet(pkt))
damaged = bytearray(pkt); damaged[3] ^= 0x10                  # one bit flipped in transit
print("damaged:", parse_packet(bytes(damaged)))
print("cut short:", parse_packet(pkt[:4]))
```

OUTPUT

```
packet: aa 05 b4 00 c4 ff 01 8e
parsed: (180, -60, 1)
damaged: checksum error
cut short: incomplete
```

PRACTICE CHECKPOINT

Connect two sensors over I2C on one bus, send sensor data between two boards over serial, and control a robot over Wi-Fi from a web page.

14.5 Practice questions

14.5.1 Basic level

Q1 How are TX and RX connected between two UART devices?

(a) TX to TX, RX to RX
(b) only TX
(c) TX to RX and RX to TX, with a common ground
(d) through I2C pull-ups

Q2 Which protocol uses SDA and SCL?

(a) SPI **(b)** UART **(c)** CAN **(d)** I2C

Q3 Which protocol needs one chip-select line per device?

(a) I2C **(b)** SPI **(c)** UART **(d)** ESP-NOW

Q4 Which Bluetooth type uses very little power and works with iPhones?

(a) Bluetooth Low Energy **(b)** Bluetooth Classic SPP
(c) IR **(d)** LoRa

Q5 Which radio gives kilometres of range for tiny messages?

(a) Bluetooth Classic **(b)** IR remote
(c) LoRa **(d)** I2C

Q6 Which ESP32 feature sends packets directly between boards without a router?

(a) ESP-NOW **(b)** BLE beacons only
(c) I2C **(d)** CAN

Q7 What frequency do IR remote receivers usually demodulate?

(a) 2.4 GHz **(b)** 38 kHz **(c)** 50 Hz **(d)** 433 MHz

Q8 Which bus was designed for cars and resists electrical noise over long wires?

(a) I2C **(b)** SPI **(c)** UART TTL **(d)** CAN

14.5.2 Intermediate level

Q9 What does a checksum do?

(a) encrypts data
(b) speeds up transmission
(c) detects whether data was corrupted in transit
(d) compresses data

Q10 How many bits are sent per byte in 8N1 serial?

(a) 8 **(b)** 9 **(c)** 10 **(d)** 16

Q11 What is the first thing to run when an I2C device does not respond?

(a) a motor test **(b)** a Wi-Fi test **(c)** nothing **(d)** an I2C scanner

Q12 Why add a timeout to remote control?

(a) to make it faster
(b) so the robot stops if commands stop arriving
(c) to save flash
(d) to change the baud rate

Q13 Which ESP32 mode creates its own Wi-Fi network?

(a) access point (softAP) (b) station
(c) deep sleep (d) BLE only

Q14 Two identical I2C sensors share address 0x68. What is a fix?

(a) connect both to SDA only
(b) use the address jumper (0x69) or a TCA9548A multiplexer
(c) raise the baud rate
(d) use pulseIn()

Q15 Why is text (e.g. "SPEED 150\n") easier to debug than binary packets?

(a) you can read it directly in a serial monitor
(b) it is smaller
(c) it is faster
(d) it needs no newline

Q16 Which nRF24L01 interface connects to the microcontroller?

(a) I2C (b) UART (c) SPI (d) analog

14.5.3 Advanced level

Q17 At 9600 baud (8N1), how many bytes per second can be sent?

(a) 9600 (b) 1200 (c) 96 (d) 960

Q18 How long does a 20-byte message take at 115 200 baud (8N1)?

(a) about 0.17 ms (b) about 17.4 ms (c) about 20 ms (d) about 1.74 ms

Q19 What is the XOR checksum of bytes 0x12, 0x34, 0x56?

(a) 0x9C (b) 0x00 (c) 0x70 (d) 0x56

Q20 Write an Arduino sketch that sends three sensor readings as one CSV line every 500 ms ("T,H,D").

Q21 Parse CSV lines arriving from an Arduino ("T,H,D"), skipping corrupted lines, and print each reading.

Q22 Compute an 8-bit sum checksum (sum of bytes mod 256) and an XOR checksum for the message "MOVE 120" and show that swapping two bytes is caught only by neither, one or both.

Q23 Write an IR-style command decoder: given NEC command codes 0x18, 0x52, 0x08, 0x5A, 0x1C, print FORWARD, BACK, LEFT, RIGHT, STOP.

Q24 A farm robot 3 km from the base must send soil readings every 10 minutes. Which link fits?

(a) LoRa (b) Bluetooth Classic
(c) I2C (d) IR remote

Q25 A team of five ESP32 robots must exchange positions 20 times per second with no router. Which

method fits best?

(a) LoRa **(b)** HC-05 pairs **(c)** IR remote **(d)** ESP-NOW

14.6 Solutions

Q1 **Answer: (c) TX to RX and RX to TX, with a common ground**

Crossed.

Q2 **Answer: (d) I2C**

I2C.

Q3 **Answer: (b) SPI**

SPI.

Q4 **Answer: (a) Bluetooth Low Energy**

BLE.

Q5 **Answer: (c) LoRa**

Long range, low data rate.

Q6 **Answer: (a) ESP-NOW**

Peer to peer.

Q7 **Answer: (b) 38 kHz**

TSOP38238.

Q8 **Answer: (d) CAN**

Differential pair.

Q9 **Answer: (c) detects whether data was corrupted in transit**

Error detection.

Q10 **Answer: (c) 10**

Start + 8 data + stop.

Q11 **Answer: (d) an I2C scanner**

Find its address.

Q12 **Answer: (b) so the robot stops if commands stop arriving**

Fail-safe.

Q13 **Answer: (a) access point (softAP)**

Phone connects to the robot.

Q14 **Answer: (b) use the address jumper (0x69) or a TCA9548A multiplexer**

Unique addresses.

Q15 **Answer: (a) you can read it directly in a serial monitor**

Human-readable.

Q16 **Answer: (c) SPI**

SPI, 32-byte payloads.

Q17 **Answer: (d) 960**

9600 ÷ 10.

Q18 **Answer: (d) about 1.74 ms**

200 bits ÷ 115 200 = **1.74 ms**.

Q19 **Answer: (c) 0x70**

0x12 ^ 0x34 = 0x26; 0x26 ^ 0x56 = **0x70**.

Q20 **Answer: Solution program**

One line per sample with comma-separated fields is easy to parse on a Raspberry Pi or in a spreadsheet.

```
float t[] = {24.5, 24.6, 24.8}; int h[] = {60, 61, 61}; int d[] = {42, 38, 35};
int i = 0;
void setup() { Serial.begin(115200); }
void loop() {
  if (i >= 3) return;
  Serial.print(t[i], 1); Serial.print(',');
  Serial.print(h[i]); Serial.print(',');
  Serial.println(d[i]);
  i++;
  delay(500);
}
```

SERIAL MONITOR

```
24.5,60,42
24.6,61,38
24.8,61,35
```

Q21 **Answer: Solution program**

Validate the field count and types; never trust a serial line to be complete.

```
lines = ["24.5,60,42", "24.6,61,38", "24.7,6", "garbage", "24.8,61,35"]
for ln in lines:
    parts = ln.split(",")
    if len(parts) != 3:
        print(f"skip {ln!r}"); continue
    try:
        t, h, d = float(parts[0]), int(parts[1]), int(parts[2])
    except ValueError:
        print(f"skip {ln!r}"); continue
    print(f"T={t} C  H={h} %  D={d} cm")
```

OUTPUT

```
T=24.5 C  H=60 %  D=42 cm
T=24.6 C  H=61 %  D=38 cm
skip '24.7,6'
skip 'garbage'
T=24.8 C  H=61 %  D=35 cm
```

Q22 **Answer: Solution program**

Neither simple checksum notices reordered bytes, because addition and XOR ignore order. CRC checksums (used by CAN and many protocols) do detect them.

```
msg = b"MOVE 120"
swapped = b"MOVE 210"
def s8(b): return sum(b) % 256
def x8(b):
    v = 0
    for c in b: v ^= c
    return v
print(f"original: sum {s8(msg):3d}  xor 0x{x8(msg):02X}")
print(f"swapped:  sum {s8(swapped):3d}  xor 0x{x8(swapped):02X}")
print("caught by sum?", s8(msg) != s8(swapped), "| caught by xor?", x8(msg) != x8(swapped))
```

OUTPUT

```
original: sum 234  xor 0x02
swapped:  sum 234  xor 0x02
caught by sum? False | caught by xor? False
```

Q23 **Answer: Solution program**

A switch maps each remote button's code to an action; read the codes for your own remote with the IRremote library's example sketch.

```
const byte codes[] = {0x18, 0x52, 0x08, 0x5A, 0x1C};
String name(byte c) {
  switch (c) {
    case 0x18: return "FORWARD";
    case 0x52: return "BACK";
    case 0x08: return "LEFT";
    case 0x5A: return "RIGHT";
    case 0x1C: return "STOP";
    default:   return "unknown";
  }
}
void setup() {
  Serial.begin(9600);
  for (byte c : codes) { Serial.print("0x"); Serial.print(c, HEX); Serial.print(" -> ");
Serial.println(name(c)); }
}
void loop() {}
```

SERIAL MONITOR

```
0x18 -> FORWARD
0x52 -> BACK
0x8 -> LEFT
0x5A -> RIGHT
0x1C -> STOP
```

Q24 **Answer: (a) LoRa**

Long range, small data.

Q25 **Answer: (d) ESP-NOW**

Fast, direct, many peers.

Wired: UART (TX→RX crossed, baud, point to point), I2C (SDA/SCL, addresses, scanner), SPI (fast, CS per device), CAN (robust bus). Wireless: Bluetooth Classic (HC-05 serial) and BLE, ESP32 Wi-Fi web control, nRF24L01, LoRa (km range), IR remotes, ESP-NOW between robots. Reliable messages: framing, length, checksum (CRC best), acknowledgements and timeouts.

MODULE 15 · INTERMEDIATE

Robotic arms and manipulators

Minimum duration **2 days** | Level **Intermediate**

A robotic arm places a tool exactly where it is needed. This module covers arm anatomy (links, joints, degrees of freedom and workspace), the main configurations (Cartesian, cylindrical, spherical, SCARA, articulated and delta), end effectors and grippers, forward and inverse kinematics of a two-link arm with worked Python code, an overview of Denavit–Hartenberg parameters, trajectory planning, and building a servo-driven arm with Arduino, followed by 25 solved questions and programs.

IN THIS MODULE

15.1 Arm anatomy
15.2 Arm configurations
15.3 End effectors
15.4 Kinematics
15.5 Trajectory planning
15.6 Building a servo arm
15.7 Practice questions
15.8 Solutions

15.1 Arm anatomy

A manipulator is a chain of rigid **links** connected by **joints**, starting at a fixed **base** and ending at the **end effector** (gripper or tool). Each joint is driven by an actuator (servo, stepper or geared motor with encoder) and adds one **degree of freedom** (DOF).

Part	What it is	Hobby example
Base	fixed mount; often a rotating "waist" joint	servo turntable
Shoulder, elbow	joints that lift and extend the arm	MG996R servos
Wrist	pitch, roll (and yaw) joints that orient the tool	SG90 / MG90S servos
End effector	gripper, suction cup, pen, welding torch	claw gripper
Links	rigid segments between joints	aluminium brackets, 3D-printed parts

Joint type	Motion	Symbol
Revolute (R)	rotation about an axis (like an elbow)	θ (angle)
Prismatic (P)	straight-line sliding (like a drawer)	d (distance)

NOTE

Degrees of freedom. 3 DOF can place a point anywhere in a 3-D workspace; 6 DOF are needed to reach any position *and* any orientation. A human arm has 7, so it can move the elbow while keeping the hand still (redundancy). The **workspace** is the set of all points the end effector can reach.

SHORTCUT TRICK
Count the motors to count the DOF: a hobby "4-DOF arm" has base, shoulder, elbow and wrist servos; the gripper servo opens and closes the claw and is usually not counted.

15.2 Arm configurations

Type	Joints	Workspace	Typical use
Cartesian (gantry)	P P P	box	3D printers, CNC, pick-and-place gantries
Cylindrical	R P P	cylinder	simple loading machines
Spherical (polar)	R R P	part of a sphere	older industrial arms
SCARA	R R P (+ R)	flat cylinder	fast assembly on a table, electronics
Articulated	R R R (+ wrist R R R)	roughly a sphere	welding, painting, most industrial and hobby arms
Delta (parallel)	3 parallel arms	small dome	very fast picking of light items (food packing)

Serial arms (one chain) have a large workspace but errors add up along the chain; **parallel** arms (delta, Stewart platform) are stiff and fast because several actuators share the load, but reach less.

15.3 End effectors

Gripper	How it holds	Good for
Parallel jaw	two fingers slide together	boxes, blocks, machined parts
Angular (claw)	fingers pivot closed	hobby arms, round objects
Vacuum (suction)	suction cup and pump or venturi	flat smooth items: boxes, glass, sheets
Magnetic	electromagnet	steel parts
Soft gripper	flexible fingers inflated by air	fruit, eggs, delicate objects
Tools	pen, laser, welding torch, screwdriver	drawing, cutting, assembly

WATCH OUT
Grip force must hold the part against gravity *and* acceleration. Too much force crushes it: a hobby claw servo should stop pushing once closed (write a slightly-open angle after closing, or use a force sensor).

15.4 Kinematics

15.4.1 Forward kinematics of a two-link arm

Forward kinematics (FK): given the joint angles, find where the tip is. For a planar arm with link lengths L1, L2, shoulder angle θ1 (from the x-axis) and elbow angle θ2 (relative to link 1):

> **NOTE**
> $x = L1 \cdot \cos\theta1 + L2 \cdot \cos(\theta1 + \theta2)$
> $y = L1 \cdot \sin\theta1 + L2 \cdot \sin(\theta1 + \theta2)$

15.4.2 Inverse kinematics

Inverse kinematics (IK): given a target (x, y), find the angles. The law of cosines gives the elbow angle, then the shoulder angle follows:

> **NOTE**
> $\cos\theta2 = (x^2 + y^2 - L1^2 - L2^2) / (2 \cdot L1 \cdot L2)$
> $\theta1 = \text{atan2}(y, x) - \text{atan2}(L2 \cdot \sin\theta2, L1 + L2 \cdot \cos\theta2)$

If $|\cos\theta2| > 1$ the point is out of reach. Otherwise there are usually **two** solutions, elbow-down ($+\theta2$) and elbow-up ($-\theta2$); the program picks one that respects the servo limits. Reachable distances run from $|L1 - L2|$ to $L1 + L2$.

ARM2.PY

```
import math

L1, L2 = 10.0, 8.0                      # link lengths in cm

def fk(t1, t2):
    a, b = math.radians(t1), math.radians(t1 + t2)
    return L1 * math.cos(a) + L2 * math.cos(b), L1 * math.sin(a) + L2 * math.sin(b)

def ik(x, y, elbow=+1):
    c2 = (x * x + y * y - L1 * L1 - L2 * L2) / (2 * L1 * L2)
    if abs(c2) > 1:
        return None                      # out of reach
    t2 = elbow * math.acos(c2)
    t1 = math.atan2(y, x) - math.atan2(L2 * math.sin(t2), L1 + L2 * math.cos(t2))
    return math.degrees(t1), math.degrees(t2)

x, y = fk(30, 45)
print(f"FK(30, 45)  -> x={x:.2f}  y={y:.2f}")
for target in [(12, 6), (0, 15), (20, 5)]:
    for elbow in (+1, -1):
        sol = ik(*target, elbow)
        if sol is None:
            print(f"IK{target}: out of reach (max {L1 + L2:.0f} cm)"); break
        t1, t2 = sol
        cx, cy = fk(t1, t2)
        print(f"IK{target} elbow {'down' if elbow > 0 else 'up  '}: t1={t1:7.2f}  t2={t2:7.2f}
check ({cx:.2f}, {cy:.2f})")
```

OUTPUT

```
FK(30, 45)  -> x=10.73  y=12.73
IK(12, 6) elbow down: t1=  -9.83  t2=  84.26  check (12.00, 6.00)
IK(12, 6) elbow up  : t1=  62.96  t2= -84.26  check (12.00, 6.00)
IK(0, 15) elbow down: t1=  60.46  t2=  67.59  check (0.00, 15.00)
IK(0, 15) elbow up  : t1= 119.54  t2= -67.59  check (0.00, 15.00)
IK(20, 5): out of reach (max 18 cm)
```

Every IK answer is checked by running FK on it: the tip lands back on the target. (20, 5) is 20.6 cm away, beyond the 18 cm reach.

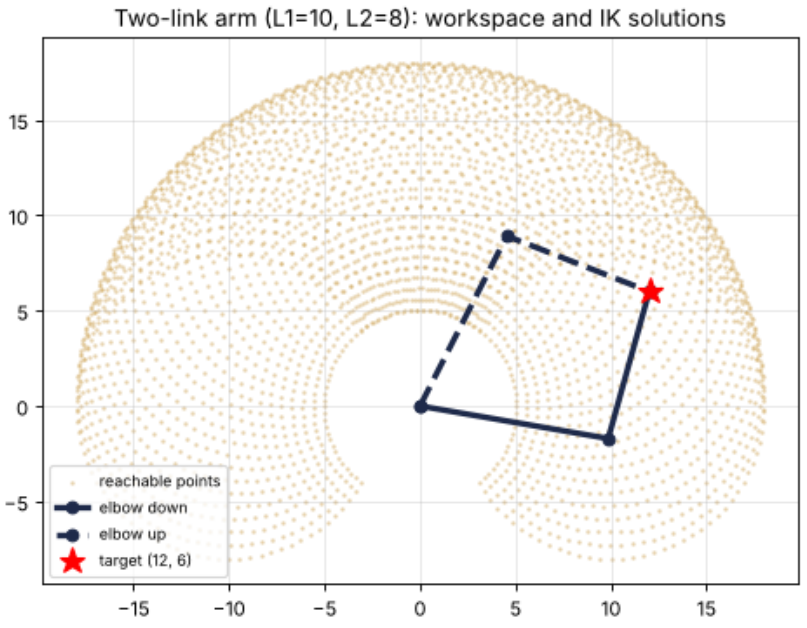
PLOT_ARM.PY

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import math

L1, L2 = 10, 8
fig, ax = plt.subplots(figsize=(6, 5))
# workspace: sweep both joints over 0..180 and -150..150 degrees
xs, ys = [], []
for t1 in range(0, 181, 4):
    for t2 in range(-150, 151, 4):
        a, b = math.radians(t1), math.radians(t1 + t2)
        xs.append(L1 * math.cos(a) + L2 * math.cos(b)); ys.append(L1 * math.sin(a) + L2 *
math.sin(b))
ax.scatter(xs, ys, s=2, color="#C9973A", alpha=0.25, label="reachable points")
# two IK solutions for the target (12, 6)
for t1, t2, style, name in [(-9.83, 84.26, "-", "elbow down"), (63.0, -84.26, "--", "elbow up")]:
    a, b = math.radians(t1), math.radians(t1 + t2)
    ex, ey = L1 * math.cos(a), L1 * math.sin(a)
    ax.plot([0, ex, 12], [0, ey, 6], style, color="#1E2A4A", lw=3, marker="o", label=name)
ax.plot(12, 6, "r*", ms=14, label="target (12, 6)")
ax.set_aspect("equal"); ax.grid(alpha=0.3); ax.legend(loc="lower left", fontsize=8)
ax.set_title("Two-link arm (L1=10, L2=8): workspace and IK solutions")
fig.tight_layout(); fig.savefig("arm.png", dpi=80)
print("points plotted:", len(xs))
```

OUTPUT

```
points plotted: 3496
```



WORKSPACE OF THE TWO-LINK ARM AND THE TWO WAYS TO REACH (12, 6)

15.4.3 Denavit–Hartenberg (DH) parameters: overview

For arms with many joints, each link gets a coordinate frame and is described by four **DH parameters**: θ (joint angle about z), d (offset along z), a (link length along x) and α (twist about x). Each row becomes a 4 × 4 transformation matrix; multiplying the matrices from base to tip gives the tool position and orientation. Libraries (Robotics Toolbox for Python, MoveIt 2, ikpy) build these chains from a table or a URDF file.

Link	θ	d	a	α
1	θ1	0	L1 = 10	0
2	θ2	0	L2 = 8	0

DH_TWO_LINK.PY

```
import numpy as np

def dh(theta, d, a, alpha):
    t, al = np.radians(theta), np.radians(alpha)
    return np.array([[np.cos(t), -np.sin(t) * np.cos(al),  np.sin(t) * np.sin(al), a * np.cos(t)],
                     [np.sin(t),  np.cos(t) * np.cos(al), -np.cos(t) * np.sin(al), a * np.sin(t)],
                     [0,          np.sin(al),              np.cos(al),             d],
                     [0,          0,                       0,                      1]])

T = dh(30, 0, 10, 0) @ dh(45, 0, 8, 0)          # base -> link 1 -> link 2
print("tool position:", np.round(T[:3, 3], 2))
print("tool angle   :", round(np.degrees(np.arctan2(T[1, 0], T[0, 0])), 1), "deg")
```

OUTPUT

```
tool position: [10.73 12.73  0.  ]
tool angle   : 75.0 deg
```

The DH chain gives the same tip (10.73, 12.73) as the FK formula, plus the tool orientation (30° + 45° = 75°).

15.4.4 Singularities

At a **singularity** the arm loses a direction of motion: a fully stretched two-link arm ($\theta 2 = 0$) cannot move straight outwards, and a 6-axis wrist with two axes lined up cannot rotate one way. Near singularities IK asks for huge joint speeds, so planners steer around them.

15.5 Trajectory planning

Jumping a servo straight to a new angle jerks the arm, shakes the load and draws current spikes. A **trajectory** moves smoothly over time.

Method	Idea	Result
Joint space	interpolate each joint angle from start to goal	simple; the tip follows a curve
Cartesian space	interpolate the tip position in a straight line, run IK at each step	straight tool path (welding, drawing); needs IK every step
Linear interpolation	equal steps	sudden start and stop
Cubic / S-curve	start and end with zero speed	smooth acceleration, less vibration

NOTE
Cubic profile with zero start and end speed over time T: $s(t) = 3(t/T)^2 - 2(t/T)^3$, angle = start + (goal - start)·s(t).

TRAJECTORY.PY

```
start, goal, T = 20.0, 110.0, 2.0              # degrees, seconds

def cubic(t):
    u = t / T
    return start + (goal - start) * (3 * u * u - 2 * u ** 3)

prev = start
print(" t(s)  linear   cubic   cubic step")
for i in range(11):
    t = i * T / 10
    lin = start + (goal - start) * t / T
    cub = cubic(t)
    print(f"{t:4.1f}  {lin:6.1f}  {cub:6.1f}   {cub - prev:+5.1f}")
    prev = cub
```

OUTPUT

```
 t(s)  linear   cubic   cubic step
 0.0    20.0    20.0    +0.0
 0.2    29.0    22.5    +2.5
 0.4    38.0    29.4    +6.8
 0.6    47.0    39.4   +10.1
 0.8    56.0    51.7   +12.2
 1.0    65.0    65.0   +13.3
```

```
1.2    74.0    78.3   +13.3
1.4    83.0    90.6   +12.2
1.6    92.0   100.6   +10.1
1.8   101.0   107.5    +6.8
2.0   110.0   110.0    +2.5
```

The linear plan moves 9° every step from the very first instant; the cubic plan starts and ends with tiny steps and moves fastest in the middle, so the arm accelerates gently.

15.6 Building a servo arm

15.6.1 Hardware and power

Part	Choice	Note
Kit	4-DOF acrylic (SG90) or 6-DOF aluminium (MG996R)	metal-gear servos for shoulder and elbow
Controller	Arduino Uno / Nano or ESP32	PCA9685 board for 6+ servos (I2C, 16 channels)
Power	5–6 V supply, 2–5 A	never from the Arduino 5 V pin; common ground
Input	potentiometers, joystick, serial or a phone app	teach-and-replay is a classic project

WATCH OUT
Each MG996R can draw over 1 A when stalled. Four of them on the Arduino's 5 V pin will brown it out and reset the board. Use a separate supply with a big capacitor (≥ 1000 µF) and connect the grounds.

SHORTCUT TRICK
Torque needed at the shoulder ≈ weight × horizontal distance. A 200 g load 25 cm out needs 0.2 × 9.81 × 0.25 ≈ 0.49 N·m = 5 kg·cm, before adding the arm's own weight, so choose servos with at least twice that.

15.6.2 Smooth servo moves and pick-and-place

PICKPLACE.INO

```
#include <Servo.h>
Servo base, shoulder, elbow, grip;
int pos[4] = {90, 90, 90, 60};                // current angles: base, shoulder, elbow, grip
Servo* joint[4] = {&base, &shoulder, &elbow, &grip};

void moveTo(int b, int s, int e, int g, int ms) {   // move all joints together, smoothly
  int goal[4] = {b, s, e, g}, from[4];
  for (int j = 0; j < 4; j++) from[j] = pos[j];
  const int steps = 20;
  for (int i = 1; i <= steps; i++) {
    float u = (float)i / steps;
    float k = 3 * u * u - 2 * u * u * u;          // cubic ease in/out
    for (int j = 0; j < 4; j++) {
      pos[j] = from[j] + round((goal[j] - from[j]) * k);
      joint[j]->write(pos[j]);
    }
    delay(ms / steps);
  }
  Serial.print("at "); for (int j = 0; j < 4; j++) { Serial.print(pos[j]); Serial.print(j < 3 ? ","
: "\n"); }
}

void setup() {
  Serial.begin(9600);
  base.attach(3); shoulder.attach(5); elbow.attach(6); grip.attach(9);
  Serial.println("home");     moveTo(90, 90, 90, 60, 400);
  Serial.println("reach");    moveTo(40, 60, 120, 60, 800);
  Serial.println("grip");     moveTo(40, 60, 120, 15, 300);    // close claw
  Serial.println("lift");     moveTo(40, 95, 90, 15, 600);
  Serial.println("swing");    moveTo(140, 95, 90, 15, 800);
  Serial.println("lower");    moveTo(140, 65, 115, 15, 600);
  Serial.println("release");  moveTo(140, 65, 115, 60, 300);
  Serial.println("home");     moveTo(90, 90, 90, 60, 800);
}
void loop() {}
```

SERIAL MONITOR

```
home
at 90,90,90,60
reach
at 40,60,120,60
grip
at 40,60,120,15
lift
at 40,95,90,15
swing
at 140,95,90,15
lower
at 140,65,115,15
release
at 140,65,115,60
home
at 90,90,90,60
```

Each named pose is a list of four angles; `moveTo()` moves every joint together with the same cubic easing as section 15.5, so all joints arrive at once. Record poses with potentiometers (below), then play them back.

15.6.3 Potentiometer control (teach mode)

POTARM.INO

```
#include <Servo.h>
Servo shoulder;
int last = -1;
void setup() { Serial.begin(9600); shoulder.attach(5); }
void loop() {
  int raw = analogRead(A0);                  // 0..1023 from the potentiometer
  int angle = map(raw, 0, 1023, 15, 165);    // stay inside safe mechanical limits
  if (abs(angle - last) > 2) {               // ignore small jitter
    shoulder.write(angle);
    Serial.print("pot "); Serial.print(raw); Serial.print(" -> "); Serial.print(angle);
Serial.println(" deg");
    last = angle;
  }
  delay(50);
}
```

SERIAL MONITOR

```
pot 512 -> 90 deg
pot 700 -> 117 deg
pot 1023 -> 165 deg
pot 0 -> 15 deg
```

The dead band (only move when the angle changes by more than 2°) stops the servo from twitching on noisy readings; mapping to 15–165° keeps the joint off its end stops.

PRACTICE CHECKPOINT

Describe the joints and DOF of an arm, solve forward and inverse kinematics for a two-link arm, plan a smooth trajectory, and program a servo arm to pick and place an object.

15.7 Practice questions

15.7.1 Basic level

Q1 What is the end effector of a robotic arm?

(a) the base motor
(b) the gripper or tool at the end of the arm
(c) the power supply
(d) the controller board

Q2 Which joint type slides in a straight line?

(a) revolute **(b)** prismatic **(c)** spherical **(d)** universal

Q3 How many degrees of freedom are needed to reach any position and any orientation in 3-D?

(a) 3 **(b)** 4 **(c)** 2 **(d)** 6

Q4 A 3D printer moving its nozzle along X, Y and Z is which configuration?

(a) Cartesian **(b)** SCARA **(c)** articulated **(d)** delta only

Q5 Which gripper suits flat, smooth cardboard boxes?

(a) magnetic **(b)** angular claw only
(c) vacuum suction **(d)** soft gripper only

Q6 What is the workspace of an arm?

(a) the table it is mounted on
(b) its control software
(c) its power rating
(d) the set of points the end effector can reach

Q7 Which arm type is used for very fast picking of light items such as biscuits?

(a) Cartesian gantry **(b)** delta (parallel)
(c) cylindrical **(d)** spherical

Q8 Which configuration is common for fast table-top electronics assembly?

(a) SCARA **(b)** delta **(c)** Cartesian **(d)** spherical

15.7.2 Intermediate level

Q9 What does forward kinematics calculate?

(a) the joint angles from a target position
(b) the motor current
(c) the tool position from the joint angles
(d) the battery life

Q10 What does inverse kinematics calculate?

(a) the joint angles needed to reach a target position
(b) the tool position from angles

(c) the servo PWM frequency
(d) the torque only

Q11 A two-link arm usually reaches a point in how many ways?

(a) one
(b) two (elbow up and elbow down)
(c) four
(d) infinite

Q12 What is the maximum reach of a two-link arm with L1 = 12 cm and L2 = 9 cm?

(a) 3 cm
(b) 12 cm
(c) 108 cm
(d) 21 cm

Q13 What is the closest reachable distance for L1 = 12 cm, L2 = 9 cm (full elbow range)?

(a) 0 cm
(b) 9 cm
(c) 3 cm
(d) 21 cm

Q14 How many parameters describe each link in the DH convention?

(a) 2
(b) 3
(c) 4 (θ, d, a, α)
(d) 6

Q15 How should the servos of a 4-DOF MG996R arm be powered?

(a) from the Arduino 5 V pin
(b) from a 9 V PP3 battery
(c) from a digital pin
(d) from a separate 5–6 V, several-amp supply with a common ground

Q16 Why use a cubic (S-curve) trajectory instead of jumping to the goal?

(a) it is shorter
(b) smooth acceleration with less shaking and lower current spikes
(c) it needs no motors
(d) it removes the need for IK

Q17 What happens near a singularity?

(a) the arm loses a direction of motion and IK demands very large joint speeds
(b) the arm moves faster safely
(c) the gripper opens
(d) the servos turn off

15.7.3 Advanced level

Q18 L1 = L2 = 10 cm, $\theta 1 = 0°$, $\theta 2 = 90°$. Where is the tip?

(a) (20, 0)
(b) (10, 10)
(c) (0, 20)
(d) (10, 0)

Q19 L1 = 10, L2 = 8 cm, target (12, 6). What is the elbow angle $\theta 2$ (elbow down)?

(a) about 84.3°
(b) about 45°
(c) about 30°
(d) about 120°

Q20 Approximately what shoulder torque holds 0.2 kg at 25 cm horizontal distance?

(a) about 0.5 kg·cm
(b) about 50 kg·cm
(c) about 5 kg·cm (0.49 N·m)
(d) about 1 kg·cm

Q21 Write a function that tells whether a point is reachable by a two-link arm (L1 = 10, L2 = 8) and test several points.

Q22 Plan a straight-line (Cartesian) move of the tip from (14, 2) to (6, 12) in 4 steps and print the joint angles for each point using IK.

Q23 Write a sketch that moves a servo from 0° to 120° in 1° steps every 10 ms and prints the time taken.

Q24 Record three poses ("teach") into arrays and replay them in order ("play"), printing each pose.

Q25 An arm must draw a straight line with a pen. Which planning method fits?

(a) joint-space interpolation only
(b) random angles
(c) moving one joint at a time
(d) Cartesian-space interpolation with IK at each step

15.8 Solutions

Q1 **Answer: (b) the gripper or tool at the end of the arm**
The part that does the work.

Q2 **Answer: (b) prismatic**
P joint, variable d.

Q3 **Answer: (d) 6**
3 for position + 3 for orientation.

Q4 **Answer: (a) Cartesian**
Three prismatic axes.

Q5 **Answer: (c) vacuum suction**
Suction holds flat faces.

Q6 **Answer: (d) the set of points the end effector can reach**
Reachable volume.

Q7 **Answer: (b) delta (parallel)**
Light, stiff and fast.

Q8 **Answer: (a) SCARA**
R R P, rigid vertically.

Q9 **Answer: (c) the tool position from the joint angles**
Angles → position.

Q10 **Answer: (a) the joint angles needed to reach a target position**

Position → angles.

Q11 **Answer: (b) two (elbow up and elbow down)**

Mirror solutions.

Q12 **Answer: (d) 21 cm**

L1 + L2 (fully stretched).

Q13 **Answer: (c) 3 cm**

|L1 − L2|.

Q14 **Answer: (c) 4 (θ, d, a, α)**

Two rotations, two offsets.

Q15 **Answer: (d) from a separate 5–6 V, several-amp supply with a common ground**

Stall current over 1 A each.

Q16 **Answer: (b) smooth acceleration with less shaking and lower current spikes**

Zero speed at both ends.

Q17 **Answer: (a) the arm loses a direction of motion and IK demands very large joint speeds**

Avoid in planning.

Q18 **Answer: (b) (10, 10)**

x = 10 + 10 cos 90° = **10**, y = 0 + 10 sin 90° = **10**.

Q19 **Answer: (a) about 84.3°**

cos θ2 = (144 + 36 − 100 − 64) / 160 = 0.1, θ2 = acos 0.1 = **84.3°**.

Q20 **Answer: (c) about 5 kg·cm (0.49 N·m)**

0.2 × 9.81 × 0.25 = **0.49 N·m** ≈ 5 kg·cm.

Q21 **Answer: Solution program**

A point is reachable when its distance from the shoulder lies between |L1 − L2| and L1 + L2 (assuming the joints can turn far enough).

```
import math
L1, L2 = 10, 8
def reachable(x, y):
    r = math.hypot(x, y)
    return abs(L1 - L2) <= r <= L1 + L2
for p in [(12, 6), (1, 1), (15, 10), (0, 18), (18, 1)]:
    print(p, "distance", round(math.hypot(*p), 2), "->", "reachable" if reachable(*p) else "out of
reach")
```

OUTPUT

```
(12, 6) distance 13.42 -> reachable
(1, 1) distance 1.41 -> out of reach
(15, 10) distance 18.03 -> out of reach
(0, 18) distance 18.0 -> reachable
(18, 1) distance 18.03 -> out of reach
```

Q22 **Answer: Solution program**

Interpolating the tip and solving IK at each step keeps the tool on a straight line, while the joint angles change unevenly.

```
import math
L1, L2 = 10, 8
def ik(x, y):
    c2 = (x * x + y * y - L1 ** 2 - L2 ** 2) / (2 * L1 * L2)
    t2 = math.acos(c2)
    t1 = math.atan2(y, x) - math.atan2(L2 * math.sin(t2), L1 + L2 * math.cos(t2))
    return math.degrees(t1), math.degrees(t2)
(x0, y0), (x1, y1) = (14, 2), (6, 12)
for i in range(5):
    u = i / 4
    x, y = x0 + (x1 - x0) * u, y0 + (y1 - y0) * u
    t1, t2 = ik(x, y)
    print(f"step {i}: tip ({x:4.1f}, {y:4.1f})  t1={t1:6.1f}  t2={t2:5.1f}")
```

OUTPUT

```
step 0: tip (14.0,  2.0)  t1= -25.3  t2= 77.0
step 1: tip (12.0,  4.5)  t1= -18.1  t2= 89.9
step 2: tip (10.0,  7.0)  t1=  -5.7  t2= 95.4
step 3: tip ( 8.0,  9.5)  t1=   9.9  t2= 93.5
step 4: tip ( 6.0, 12.0)  t1=  27.0  t2= 84.3
```

Q23 **Answer: Solution program**

121 writes × 10 ms ≈ 1.21 s. Small steps with a delay slow the move down; a real arm would ease in and out (section 15.6.2).

```
#include <Servo.h>
Servo s;
void setup() {
  Serial.begin(9600);
  s.attach(9);
  unsigned long t0 = millis();
  for (int a = 0; a <= 120; a++) { s.write(a); delay(10); }
  Serial.print("reached 120 deg in ");
  Serial.print(millis() - t0); Serial.println(" ms");
}
void loop() {}
```

SERIAL MONITOR

```
reached 120 deg in 1210 ms
```

Q24 **Answer: Solution program**

Teach-and-replay is how many industrial arms are programmed with a teach pendant: move to a pose, save it, then play the list back. Store poses in EEPROM to keep them after power-off.

```
#include <Servo.h>
Servo base, arm;
int poses[3][2];
int n = 0;
void teach(int b, int a) { poses[n][0] = b; poses[n][1] = a; n++; Serial.print("saved pose ");
Serial.println(n); }
void setup() {
  Serial.begin(9600);
  base.attach(3); arm.attach(5);
  teach(90, 90); teach(30, 60); teach(150, 120);      // in a real arm: read pots on a button press
  for (int i = 0; i < n; i++) {
    base.write(poses[i][0]); arm.write(poses[i][1]);
    Serial.print("play "); Serial.print(i + 1); Serial.print(": base ");
    Serial.print(poses[i][0]); Serial.print(" arm "); Serial.println(poses[i][1]);
    delay(500);
  }
}
void loop() {}
```

SERIAL MONITOR

```
saved pose 1
saved pose 2
saved pose 3
play 1: base 90 arm 90
play 2: base 30 arm 60
play 3: base 150 arm 120
```

Q25 **Answer: (d) Cartesian-space interpolation with IK at each step**

Straight tool path.

Arm = base + links + joints (revolute/prismatic) + end effector; DOF = number of joints (6 for full pose). Configurations: Cartesian, cylindrical, spherical, SCARA, articulated, delta. FK: $x = L1\cos\theta1 + L2\cos(\theta1+\theta2)$; IK by the law of cosines (two solutions, reach $|L1-L2|$ to $L1+L2$). DH: 4 parameters per link, 4×4 matrices. Plan smooth (cubic) trajectories, power servos separately, ease all joints together.

MODULE 16 · INTERMEDIATE

Embedded programming patterns for robots

Minimum duration **2 days** | Level **Intermediate**

A robot must read sensors, drive motors, talk to the user and blink a status light at the same time, on a processor that runs one instruction after another. This module shows why delay() freezes a robot and how millis() timing fixes it, uses external and timer interrupts for encoders and buttons with interrupt-safe variables, designs behaviour as a state machine, organises code into functions, classes and configuration constants, introduces FreeRTOS tasks on the ESP32 and covers practical debugging, followed by 25 solved questions and programs.

IN THIS MODULE

16.1 Non-blocking code
16.2 Interrupts
16.3 State machines
16.4 Code organisation
16.5 Real-time operating systems (overview)
16.6 Debugging embedded systems
16.7 Practice questions
16.8 Solutions

16.1 Non-blocking code

16.1.1 Problems with delay()

`delay(500)` stops the whole program for half a second. During that time the robot cannot read its bumper, check the ultrasonic sensor, receive a Bluetooth command or count encoder pulses with polling. One blinking LED with delay() is fine; a robot that must react within 20 ms is not.

BLOCKING: THE BUMPER IS CHECKED ONCE A SECOND

```
void loop() {
  digitalWrite(13, HIGH); delay(500);    // blind for 500 ms
  digitalWrite(13, LOW);  delay(500);    // blind again
  if (digitalRead(4) == LOW) stopMotors();   // checked only once per second!
}
```

16.1.2 Using millis() timing

`millis()` returns the milliseconds since power-up. Instead of waiting, each job remembers when it last ran and runs again when enough time has passed. The loop spins thousands of times per second, so every job gets its turn quickly.

THREEJOBS.INO

```
const int LED = 13;
unsigned long lastBlink = 0, lastSensor = 0, lastReport = 0;
bool ledOn = false;
int readings = 0, blinks = 0;

void setup() { Serial.begin(115200); pinMode(LED, OUTPUT); }

void loop() {
  unsigned long now = millis();
  if (now - lastBlink >= 250) {           // job 1: status LED every 250 ms
    lastBlink = now; ledOn = !ledOn; digitalWrite(LED, ledOn); blinks++;
  }
  if (now - lastSensor >= 50) {           // job 2: read sensor every 50 ms
    lastSensor = now; readings++;
  }
  if (now - lastReport >= 400) {          // job 3: report every 400 ms
    lastReport = now;
    Serial.print(now); Serial.print(" ms: "); Serial.print(readings);
    Serial.print(" sensor reads, "); Serial.print(blinks); Serial.println(" LED toggles");
  }
}
```

SERIAL MONITOR

```
400 ms: 8 sensor reads, 1 LED toggles
800 ms: 16 sensor reads, 3 LED toggles
1200 ms: 24 sensor reads, 4 LED toggles
```

Three jobs at three different rates share one loop, and none of them blocks the others. The 50 ms sensor job runs 8 times per 400 ms report and the LED toggles every 250 ms.

SHORTCUT TRICK

Always write `now - last >= interval`, never `now >= last + interval`. The subtraction stays correct when millis() wraps back to 0 after about 49.7 days, because unsigned arithmetic wraps too.

16.2 Interrupts

16.2.1 External interrupts for encoders and buttons

An **interrupt** pauses the main program the instant a pin changes, runs a short function (the **ISR**, interrupt service routine) and returns. No pulse is missed even while loop() is busy. On the Uno only pins 2 and 3 have external interrupts; on the ESP32 almost every pin does.

ENCODERISR.INO

```
const int ENC_A = 2;
volatile long ticks = 0;                  // changed inside the ISR -> volatile

void onTick() { ticks++; }                // ISR: keep it tiny

void setup() {
  Serial.begin(115200);
  pinMode(ENC_A, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(ENC_A), onTick, RISING);
}

unsigned long last = 0; long lastTicks = 0;
void loop() {
  if (millis() - last >= 100) {
    last = millis();
    noInterrupts(); long t = ticks; interrupts();     // copy safely
    long dt = t - lastTicks; lastTicks = t;
    Serial.print(last); Serial.print(" ms  ticks="); Serial.print(t);
    Serial.print("  speed="); Serial.print(dt * 10); Serial.println(" ticks/s");
  }
}
```

SERIAL MONITOR

```
100 ms  ticks=10  speed=100 ticks/s
200 ms  ticks=20  speed=100 ticks/s
300 ms  ticks=30  speed=100 ticks/s
400 ms  ticks=40  speed=100 ticks/s
500 ms  ticks=45  speed=50 ticks/s
600 ms  ticks=50  speed=50 ticks/s
700 ms  ticks=55  speed=50 ticks/s
800 ms  ticks=60  speed=50 ticks/s
```

The ISR counts every rising edge while loop() reports speed every 100 ms. Here the wheel gives 100 ticks/s, then slows to half speed (50 ticks/s) after 400 ms; the count in each window is exact because no edge depends on loop() timing.

16.2.2 Timer interrupts

A **timer interrupt** runs an ISR at a fixed rate set by a hardware timer, independent of what loop() is doing: ideal for a steady PID update, stepper pulses or sampling a sensor at exactly 100 Hz. On AVR the TimerOne library sets this up; on the ESP32 use `timerBegin()` / `timerAttachInterrupt()`.

TIMER1PID.INO

```
#include <TimerOne.h>
volatile bool pidDue = false;
void tick() { pidDue = true; }            // 100 times per second
void setup() {
  Timer1.initialize(10000);               // period in microseconds = 10 ms
  Timer1.attachInterrupt(tick);
}
void loop() {
  if (pidDue) { pidDue = false; /* read encoders, update PID, set PWM */ }
}
```

The ISR only raises a flag; the slow work (maths, Serial) happens in loop(). This keeps the interrupt short and the control rate steady.

16.2.3 Interrupt-safe variables

Rule	Why
Declare shared variables `volatile`	stops the compiler keeping a stale copy in a register
Copy multi-byte values with interrupts off	on an 8-bit Uno a long is read in 4 steps; an ISR in the middle gives a corrupted value
Keep ISRs short: no delay(), no Serial.print()	they block other interrupts; delay() and Serial depend on interrupts themselves
Set flags or counters in the ISR, do the work in loop()	predictable timing
ESP32: mark ISRs IRAM_ATTR	the ISR must run from RAM, not flash

16.3 State machines

16.3.1 Designing robot behaviour as states and transitions

A **finite state machine** (FSM) describes behaviour as a few named **states** and the **events** that move between them. Each state knows what to do and when to leave. Combined with millis() timing, the robot never blocks: a "back up for 400 ms" step is a state with a timer, not a delay.



OBSTACLE AVOIDER AS A STATE MACHINE (TURN RETURNS TO FORWARD)

STATEMACHINEAVOIDER.INO

```
enum State { FORWARD, BACKUP, TURN };
State state = FORWARD;
unsigned long entered = 0, lastSonar = 0, lastBlink = 0;
int toggles = 0;
const char* names[] = {"FORWARD", "BACKUP", "TURN"};

void go(State s) {
  state = s; entered = millis();
  Serial.print(entered); Serial.print(" ms -> "); Serial.print(names[s]);
  Serial.print("   (LED toggles so far: "); Serial.print(toggles); Serial.println(")");
}

long distanceCm() { return pulseIn(8, HIGH, 30000) / 58; }

void setup() { Serial.begin(115200); pinMode(13, OUTPUT); go(FORWARD); }

void loop() {
  unsigned long now = millis();
  if (now - lastBlink >= 100) { lastBlink = now; digitalWrite(13, !digitalRead(13)); toggles++; }
// always blinking

  switch (state) {
    case FORWARD:                          // motors forward
      if (now - lastSonar >= 100) {
        lastSonar = now;
        long d = distanceCm();
        Serial.print("   sonar "); Serial.print(d); Serial.println(" cm");
        if (d < 20) go(BACKUP);
      }
      break;
    case BACKUP:                           // motors reverse
      if (now - entered >= 400) go(TURN);
      break;
    case TURN:                             // spin right
      if (now - entered >= 300) go(FORWARD);
      break;
  }
}
```

SERIAL MONITOR

```
0 ms -> FORWARD   (LED toggles so far: 0)
   sonar 40 cm
   sonar 30 cm
   sonar 20 cm
   sonar 15 cm
400 ms -> BACKUP   (LED toggles so far: 4)
800 ms -> TURN   (LED toggles so far: 8)
1100 ms -> FORWARD   (LED toggles so far: 11)
   sonar 50 cm
   sonar 45 cm
   sonar 45 cm
```

The sonar is read only in FORWARD; at 15 cm the robot switches to BACKUP, then TURN, then FORWARD again. The LED counter keeps rising through every state, proving nothing blocks. Adding a new behaviour

(for example SEARCH or STUCK) means adding one state and its transitions, not rewriting the loop.

16.4 Code organisation

16.4.1 Functions, libraries and modular code

Split the program by job: a **function** for each action (drive(), readSonar()), a **class** for each kind of hardware (Motor, Encoder), and separate files (`motor.h`, `sensors.h`) once the sketch grows. The main loop then reads like the robot's plan.

MOTORCLASS.INO

```
class Motor {
  int pwm, in1, in2;
public:
  Motor(int p, int a, int b) : pwm(p), in1(a), in2(b) {}
  void begin() { pinMode(pwm, OUTPUT); pinMode(in1, OUTPUT); pinMode(in2, OUTPUT); }
  void set(int speed) {                          // -255..255
    speed = constrain(speed, -255, 255);
    digitalWrite(in1, speed > 0); digitalWrite(in2, speed < 0);
    analogWrite(pwm, abs(speed));
  }
};

Motor left(5, 7, 8), right(6, 9, 10);

void drive(int l, int r) {
  left.set(l); right.set(r);
  Serial.print("drive "); Serial.print(l); Serial.print(" "); Serial.println(r);
}

void setup() {
  Serial.begin(9600);
  left.begin(); right.begin();
  drive(200, 200);       // forward
  drive(-150, 150);      // spin left
  drive(300, -400);      // clamped inside set()
  drive(0, 0);
}
void loop() {}
```

SERIAL MONITOR

```
drive 200 200
drive -150 150
drive 300 -400
drive 0 0
```

Both motors share one tested class, so a fix in `set()` fixes both wheels. The printed values are the requested speeds; set() clamps 300/-400 to ±255 before writing the PWM.

16.4.2 Configuration constants

CONFIG.H

```
// config.h - every pin and tuning number in one place
#pragma once
const int  PIN_LEFT_PWM   = 5;
const int  PIN_RIGHT_PWM  = 6;
const int  PIN_SONAR_TRIG = 7;
const int  PIN_SONAR_ECHO = 8;
const int  OBSTACLE_CM    = 20;     // tune here, not inside the logic
const int  CRUISE_SPEED   = 180;
const float KP = 1.2, KI = 0.0, KD = 0.4;
#define DEBUG 1                     // 0 removes debug prints
```

SHORTCUT TRICK

Name every magic number. "if (d < OBSTACLE_CM)" explains itself and changes in one place; "if (d < 20)" scattered across files does neither.

16.5 Real-time operating systems (overview)

16.5.1 FreeRTOS tasks on ESP32

An **RTOS** runs several **tasks**, each an endless loop with its own priority, and switches between them every millisecond. A task that waits with `vTaskDelay()` gives the CPU to others instead of blocking. The ESP32 Arduino core already runs FreeRTOS on its two cores, so tasks are available with no extra library.

ESP32 FREERTOS TASKS (REFERENCE)

```
void sensorTask(void *arg) {
  for (;;) { readSonar(); readIMU(); vTaskDelay(pdMS_TO_TICKS(20)); }    // 50 Hz
}
void controlTask(void *arg) {
  for (;;) { updatePID(); vTaskDelay(pdMS_TO_TICKS(10)); }              // 100 Hz
}
void commsTask(void *arg) {
  for (;;) { sendTelemetry(); vTaskDelay(pdMS_TO_TICKS(200)); }         // 5 Hz
}
void setup() {
  xTaskCreatePinnedToCore(controlTask, "control", 4096, NULL, 3, NULL, 1);  // highest priority
  xTaskCreatePinnedToCore(sensorTask,  "sensors", 4096, NULL, 2, NULL, 1);
  xTaskCreatePinnedToCore(commsTask,   "comms",   8192, NULL, 1, NULL, 0);  // Wi-Fi core
}
void loop() { vTaskDelay(portMAX_DELAY); }
```

RTOS tool	Use
Task priorities	control loop pre-empts slow telemetry
Queues	pass sensor readings from one task to another safely
Mutex / semaphore	stop two tasks using Serial or I2C at the same time
Software timers	run a callback periodically

16.6 Debugging embedded systems

16.6.1 Serial logging, LEDs as indicators, logic analysers

Tool	What it shows	Tip
Serial logging	values and the path through the code	timestamp lines with millis(); switch off with a DEBUG flag
Serial Plotter	live graphs of numbers	print "a b c" per line: great for PID tuning
Status LED / buzzer	state without a computer attached	blink codes: 1 = OK, 2 = low battery, fast = error
Multimeter	voltages, continuity, current	check supply under load: brown-outs cause random resets
Logic analyser (~₹800, 8 channels)	exact timing of I2C, SPI, UART, PWM	decodes the bytes on the wire with PulseView/Sigrok
Oscilloscope	analog signals, noise, ripple	motor noise on the 5 V rail

DEBUGMACRO.INO

```
#define DEBUG 1
#if DEBUG
  #define LOG(label, value) do { Serial.print(millis()); Serial.print(" "); \
                                 Serial.print(label); Serial.print("="); Serial.println(value); }
while (0)
#else
  #define LOG(label, value)
#endif

int error = 40;
void setup() {
  Serial.begin(115200);
  for (int i = 0; i < 4; i++) {
    int correction = error / 2;
    LOG("error", error);
    LOG("correction", correction);
    error -= correction;
    delay(20);
  }
}
void loop() {}
```

SERIAL MONITOR

```
0 error=40
0 correction=20
20 error=20
20 correction=10
40 error=10
40 correction=5
60 error=5
60 correction=2
```

Setting `DEBUG` to 0 removes every LOG line at compile time, so the final robot has no Serial delays. Timestamps reveal both values and timing.

WATCH OUT

Serial.print() at 9600 baud takes about 1 ms per character. A few long debug lines in a fast control loop can slow it badly: use 115200 baud and print every N loops.

PRACTICE CHECKPOINT

Rewrite the obstacle-avoiding robot as a non-blocking state machine with encoder interrupts, so it can read sensors, drive and blink a status LED at the same time.

16.7 Practice questions

16.7.1 Basic level

Q1 What is the main problem with delay() in a robot program?

(a) it uses too much flash
(b) it only works on the ESP32
(c) it changes the PWM frequency
(d) the processor can do nothing else while it waits

Q2 What does millis() return?

(a) microseconds since reset
(b) the current time of day
(c) milliseconds since the board started
(d) the CPU temperature

Q3 Which Uno pins support external interrupts?

(a) 2 and 3 **(b)** 0 and 1 **(c)** A4 and A5 **(d)** 11 and 12

Q4 What is an ISR?

(a) a type of sensor
(b) a serial protocol
(c) a motor driver
(d) the function that runs when an interrupt happens

Q5 Which keyword marks a variable changed inside an ISR?

(a) static **(b)** volatile **(c)** const **(d)** unsigned

Q6 Which function should never be used inside an ISR?

(a) a simple counter increment
(b) delay()
(c) setting a bool flag
(d) reading a pin

Q7 What is a finite state machine?

(a) a type of motor
(b) an RTOS
(c) a memory chip
(d) behaviour described as states and the events that switch between them

Q8 Where should pin numbers and tuning values be kept?

(a) in named constants, ideally a config header
(b) typed as numbers wherever they are used
(c) in comments only
(d) in EEPROM only

16.7.2 Intermediate level

Q9 Which comparison handles millis() overflow correctly?

(a) now >= last + interval
(b) now == last + interval
(c) now - last >= interval
(d) now > interval

Q10 After how long does millis() overflow on an Arduino?

(a) about 70 minutes **(b)** about 1 year
(c) never **(d)** about 49.7 days

Q11 Why copy a long counter with interrupts disabled on an Uno?

(a) to save power
(b) an 8-bit CPU reads it in several steps and an ISR could change it halfway
(c) to make it faster
(d) because longs are read-only

Q12 What is the best way to run a PID update exactly 100 times per second?

(a) a timer interrupt that sets a flag, with the update done in loop()
(b) delay(10) inside the PID
(c) Serial.print timing
(d) a for loop

Q13 What does vTaskDelay() do in FreeRTOS?

(a) stops the whole ESP32
(b) resets the board
(c) pauses this task and lets other tasks run
(d) deletes the task

Q14 Which tool shows the exact bytes and timing on an I2C bus?

(a) a logic analyser **(b)** a multimeter
(c) a soldering iron **(d)** a voltage regulator

Q15 Which ESP32 attribute should an ISR carry?

(a) PROGMEM **(b)** IRAM_ATTR **(c)** volatile **(d)** inline

Q16 In the state machine, how is "back up for 400 ms" implemented without blocking?

(a) delay(400)
(b) a while loop until 400
(c) a 400-step for loop
(d) store the time the state was entered and leave when millis() - entered ≥ 400

Q17 Which FreeRTOS feature passes sensor readings between tasks safely?

(a) a global variable without protection **(b)** delay()
(c) a queue **(d)** an LED

16.7.3 Advanced level

Q18 An encoder gives 20 ticks per wheel revolution. 50 ticks are counted in 100 ms. How many revolutions per second?

(a) 2.5 rev/s **(b)** 5 rev/s **(c)** 25 rev/s **(d)** 50 rev/s

Q19 How long does printing a 30-character line take at 9600 baud?

(a) about 3 ms **(b)** about 0.3 ms **(c)** about 300 ms **(d)** about 31 ms

Q20 Blink two LEDs at different rates (13 every 300 ms, 12 every 700 ms) without delay(), printing each toggle with its time.

Q21 Use a button on pin 2 with an interrupt and a 50 ms software debounce to count presses; the button bounces on each press.

Q22 Write a traffic-light state machine (GREEN 600 ms → YELLOW 200 ms → RED 400 ms → GREEN) using millis().

Q23 Simulate a robot state machine in Python: it drives forward until a bump, backs up for 2 ticks, turns for 1 tick, then continues. Bumps happen at ticks 3 and 8.

Q24 A sketch prints debug lines inside a 10 ms control loop and the robot wobbles. What is the most likely fix?

(a) add delay(100)

(b) print less often (every N loops) and at 115200 baud, or disable debug output

(c) remove the volatile keyword

(d) use a bigger battery

Q25 An ESP32 robot must run PID at 100 Hz, read sensors at 50 Hz and send Wi-Fi telemetry at 5 Hz. What structure fits best?

(a) three FreeRTOS tasks with priorities control > sensors > telemetry

(b) one loop with delay(10)

(c) only interrupts

(d) three separate ESP32 boards

16.8 Solutions

Q1 **Answer: (d) the processor can do nothing else while it waits**

Blocking.

Q2 **Answer: (c) milliseconds since the board started**

Unsigned long counter.

Q3 **Answer: (a) 2 and 3**

INT0 and INT1.

Q4 **Answer: (d) the function that runs when an interrupt happens**

Interrupt service routine.

Q5 **Answer: (b) volatile**

Always reread from memory.

Q6 **Answer: (b) delay()**

delay() depends on interrupts.

Q7 **Answer: (d) behaviour described as states and the events that switch between them**

FSM.

Q8 **Answer: (a) in named constants, ideally a config header**

One place to change.

Q9 **Answer: (c) now - last >= interval**

Unsigned subtraction wraps.

Q10 **Answer: (d) about 49.7 days**

2^{32} ms.

Q11 **Answer: (b) an 8-bit CPU reads it in several steps and an ISR could change it halfway**

Atomic read.

Q12 **Answer: (a) a timer interrupt that sets a flag, with the update done in loop()**

Steady rate, short ISR.

Q13 **Answer: (c) pauses this task and lets other tasks run**

Non-blocking for the system.

Q14 **Answer: (a) a logic analyser**

Decodes protocols.

Q15 **Answer: (b) IRAM_ATTR**

Run from RAM.

Q16 **Answer: (d) store the time the state was entered and leave when millis() - entered ≥ 400**

Timed state.

Q17 **Answer: (c) a queue**

Thread-safe FIFO.

Q18 **Answer: (c) 25 rev/s**

50 ticks/0.1 s = 500 ticks/s; 500 ÷ 20 = **25 rev/s**.

Q19 **Answer: (d) about 31 ms**

30 characters × 10 bits = 300 bits; 300 ÷ 9600 s ≈ **31 ms**.

Q20 **Answer: Solution program**

Each LED has its own timer variable, so their rates are independent. With delay() this would need careful interleaving.

```
unsigned long t13 = 0, t12 = 0;
bool s13 = false, s12 = false;
void setup() { Serial.begin(9600); pinMode(13, OUTPUT); pinMode(12, OUTPUT); }
void loop() {
  unsigned long now = millis();
  if (now - t13 >= 300) { t13 = now; s13 = !s13; digitalWrite(13, s13); Serial.print(now);
Serial.println(" LED13"); }
  if (now - t12 >= 700) { t12 = now; s12 = !s12; digitalWrite(12, s12); Serial.print(now);
Serial.println(" LED12"); }
}
```

SERIAL MONITOR

```
300 LED13
600 LED13
700 LED12
900 LED13
1200 LED13
1400 LED12
```

Q21 **Answer: Solution program**

Each press makes several falling edges a few milliseconds apart; the 50 ms window counts only the first. (millis() is readable in an ISR, it just does not advance there.)

```
volatile unsigned long lastPress = 0;
volatile int presses = 0;
void onPress() {
  unsigned long now = millis();
  if (now - lastPress > 50) { presses++; lastPress = now; }    // ignore bounces within 50 ms
}
void setup() {
  Serial.begin(9600);
  pinMode(2, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(2), onPress, FALLING);
}
int shown = 0;
void loop() {
  noInterrupts(); int p = presses; interrupts();
  if (p != shown) { shown = p; Serial.print(millis()); Serial.print(" ms  presses=");
Serial.println(p); }
}
```

SERIAL MONITOR

```
100 ms  presses=1
500 ms  presses=2
800 ms  presses=3
```

Q22 **Answer: Solution program**

A table of durations indexed by the state keeps the machine short; changing a timing is a one-number edit.

```
enum Light { GREEN, YELLOW, RED };
Light s = GREEN;
unsigned long since = 0;
const unsigned long dur[] = {600, 200, 400};
const char* name[] = {"GREEN", "YELLOW", "RED"};
void setup() { Serial.begin(9600); Serial.println("0 GREEN"); }
void loop() {
  if (millis() - since >= dur[s]) {
    since = millis();
    s = (Light)((s + 1) % 3);
    Serial.print(since); Serial.print(" "); Serial.println(name[s]);
  }
}
```

SERIAL MONITOR

```
0 GREEN
600 YELLOW
800 RED
1200 GREEN
1800 YELLOW
2000 RED
2400 GREEN
```

Q23 **Answer: Solution program**

The same structure (current state, timer, events) transfers directly to C++ with an enum and a switch. Prototyping behaviour in Python first is quick.

```
state, timer = "FORWARD", 0
bumps = {3, 8}
for tick in range(12):
    if state == "FORWARD" and tick in bumps:
        state, timer = "BACKUP", 2
    elif state == "BACKUP":
        timer -= 1
        if timer == 0: state, timer = "TURN", 1
    elif state == "TURN":
        timer -= 1
        if timer == 0: state = "FORWARD"
    print(tick, state)
```

OUTPUT

```
0 FORWARD
1 FORWARD
2 FORWARD
3 BACKUP
4 BACKUP
5 TURN
6 FORWARD
```

```
7 FORWARD
8 BACKUP
9 BACKUP
10 TURN
11 FORWARD
```

Q24 **Answer: (b) print less often (every N loops) and at 115200 baud, or disable debug output**

Serial printing takes time.

Q25 **Answer: (a) three FreeRTOS tasks with priorities control > sensors > telemetry**

Priorities keep control steady.

delay() blocks; millis() timing with "now - last ≥ interval" runs many jobs at once. Interrupts catch every edge (Uno pins 2, 3): tiny ISRs, volatile variables, atomic copies, no delay/Serial. Timer interrupts give fixed-rate control. State machines (enum + switch + entry time) structure behaviour. Organise with functions, classes and config constants. FreeRTOS tasks, queues and mutexes on the ESP32. Debug with timestamped logs, plotter, LEDs and a logic analyser.

LEVEL 3

Advanced level

Add computer vision, navigation and path planning, use ROS 2 and simulators, apply AI, understand drones and IoT robots, design parts in CAD, learn industrial automation and complete a capstone robot

10 MODULES · 23 DAYS MINIMUM

Module 17 · Robot vision and perception	**2 days**
Module 18 · Localisation, mapping and navigation	**3 days**
Module 19 · Robot Operating System (ROS 2)	**3 days**
Module 20 · Robot simulation	**3 days**
Module 21 · Artificial intelligence and machine learning in robotics	**2 days**
Module 22 · Drones and aerial robotics	**2 days**
Module 23 · IoT robotics and cloud connectivity	**2 days**
Module 24 · Mechanical design, CAD and 3D printing	**2 days**
Module 25 · Industrial robotics, automation, safety and ethics	**2 days**
Module 26 · Competitions, capstone projects and careers	**2 days**

MODULE 17 · ADVANCED

Robot vision and perception

Minimum duration **3 days** | Level **Advanced**

Cameras let robots recognise objects, colours, people and paths. This module compares robot cameras (USB, Pi Camera, ESP32-CAM and depth cameras), explains pixels, resolution and colour spaces, and uses OpenCV for video capture, colour detection and tracking, thresholding and contours, edge detection, face detection, QR codes and ArUco markers. It then turns vision into behaviour with a ball-following robot and camera-based line following, and closes with pretrained object detectors such as YOLO, followed by 25 solved questions and programs.

IN THIS MODULE

17.1 Cameras for robots
17.2 Image basics
17.3 Computer vision with OpenCV
17.4 Vision-based robot behaviours
17.5 AI vision models (overview)
17.6 Practice questions
17.7 Solutions

17.1 Cameras for robots

17.1.1 USB cameras, Pi Camera, ESP32-CAM

Camera	Connects to	Typical spec	Best for
USB webcam	Pi, Jetson, laptop (UVC, no driver)	720p–1080p, 30 fps	quick start, any computer
Raspberry Pi Camera Module 3	Pi CSI ribbon	12 MP, autofocus, wide and NoIR versions	compact Pi robots, low CPU use
ESP32-CAM (OV2640)	built-in ESP32 with Wi-Fi	2 MP, ~10–20 fps at 320×240	cheap streaming (≈ ₹600); the image is processed on a PC or Pi
Global-shutter camera	Pi / USB	no "jelly" distortion at speed	fast drones, line followers
Pixy2 / HuskyLens	Arduino via UART/I2C	on-board colour/face/tag detection	vision for Arduino robots without a Pi

17.1.2 Depth cameras (overview)

A normal camera loses distance. **Depth cameras** give a distance for every pixel, so a robot can see the floor, obstacles and the size of objects.

Type	How it measures	Examples	Notes
Stereo	two cameras; distance from the shift between views (like eyes)	Intel RealSense D435, OAK-D	works outdoors; needs texture
Structured light	projects an IR dot pattern and watches it distort	original Kinect, phone face unlock	indoors, short range
Time of flight	times reflected IR light for each pixel	Azure Kinect, ToF modules	fast, accurate indoors
LiDAR	spinning or solid-state laser ranging	RPLIDAR A1 (2-D), Livox (3-D)	mapping and navigation (module 18)

17.2 Image basics

17.2.1 Pixels, resolution, colour spaces (RGB, HSV, grayscale)

An image is a grid of **pixels**: a NumPy array of **height × width × channels**, values 0–255. OpenCV stores colour as **BGR**. **Grayscale** has one channel and is enough for edges, lines and markers. **HSV** (hue, saturation, value) separates the colour itself from its brightness, so a ball keeps nearly the same hue in shade and in sunlight: the best space for colour tracking.

Resolution	Pixels	Robot use
160 × 120	19 200	very fast line following on a small CPU
320 × 240	76 800	colour tracking on a Pi at 30 fps
640 × 480	307 200	markers, faces, general vision
1280 × 720	921 600	detail, recording; slow to process on a Pi

> **SHORTCUT TRICK**
> Halving the width and height quarters the pixels, roughly quartering the processing time. Robots almost always process small frames.

The examples use a test frame drawn by this helper, so every result can be checked:

ARENA.PY

```
import cv2, numpy as np
def arena(ball_x=230, ball_y=150, r=28):
    """a test camera frame: floor, wall, orange ball, blue box, ArUco marker id 7"""
    img = np.full((240, 320, 3), (170, 175, 180), np.uint8)           # grey floor (BGR)
    img[:70] = (215, 210, 200)                                         # wall
    cv2.rectangle(img, (30, 110), (100, 190), (190, 90, 20), -1)      # blue box
    cv2.circle(img, (ball_x, ball_y), r, (0, 130, 250), -1)            # orange ball
    d = cv2.aruco.getPredefinedDictionary(cv2.aruco.DICT_4X4_50)
    m = cv2.aruco.generateImageMarker(d, 7, 50)
    img[10:60, 135:185] = cv2.cvtColor(np.pad(m, 0), cv2.COLOR_GRAY2BGR)  # marker on the wall
    img[5:10, 130:190] = img[60:65, 130:190] = 255; img[5:65, 130:135] = img[5:65, 185:190] = 255  #
white border
    noise = np.random.default_rng(1).integers(-10, 11, img.shape)
    return np.clip(img.astype(int) + noise, 0, 255).astype(np.uint8)
```

PIXELS.PY

```
import cv2
from arena import arena
img = arena()
print("shape (h, w, ch):", img.shape)
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
for name, (x, y) in {"floor": (250, 220), "ball": (230, 150), "blue box": (65, 150)}.items():
    b, g, r = img[y, x]; h, s, v = hsv[y, x]
    print(f"{name:9} BGR=({int(b):3d},{int(g):3d},{int(r):3d})  HSV=({int(h):3d},{int(s):3d},
{int(v):3d})")
```

OUTPUT

```
shape (h, w, ch): (240, 320, 3)
floor     BGR=(178,176,172)  HSV=(100,  9,178)
ball      BGR=(  0,127,250)  HSV=( 15,255,250)
blue box  BGR=(193, 96, 10)  HSV=(106,242,193)
```

The grey floor has almost no saturation, while the ball has hue ≈ 15 (orange) and high saturation: an HSV range picks the ball out cleanly.

17.3 Computer vision with OpenCV

17.3.1 Reading and displaying video

VIDEO_LOOP.PY (REFERENCE)

```
import cv2
cap = cv2.VideoCapture(0)                        # 0 = first USB camera (or a file / stream URL)
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 320)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 240)
while True:
    ok, frame = cap.read()
    if not ok:
        break                                    # camera unplugged or stream ended
    # ... process frame ...
    cv2.imshow("robot view", frame)              # needs a desktop; on a headless robot stream or
save
    if cv2.waitKey(1) & 0xFF == ord("q"):
        break
cap.release(); cv2.destroyAllWindows()
```

NOTE

Pi Camera: use Picamera2 and `capture_array()` each loop. ESP32-CAM: open its MJPEG stream URL, e.g. `cv2.VideoCapture("http://192.168.1.50:81/stream")`.

17.3.2 Colour detection and tracking

17.3.3 Thresholding and contours

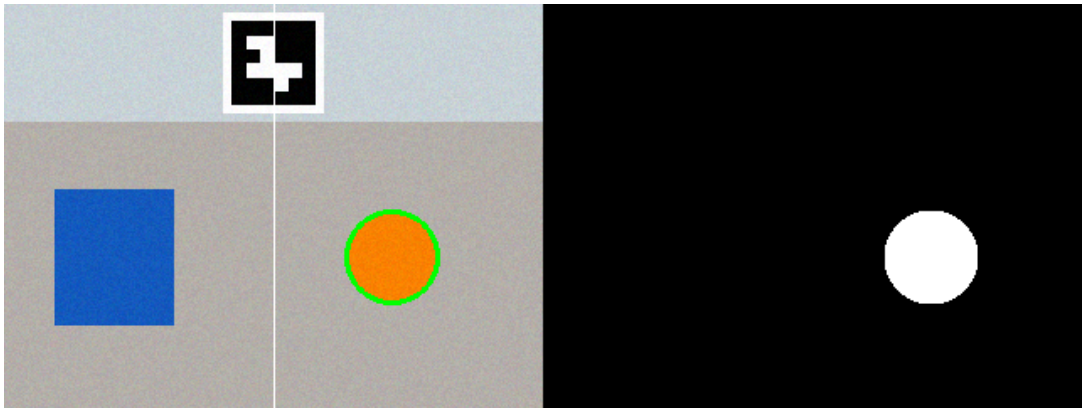
The classic colour-tracking pipeline: convert to HSV → `inRange` mask → clean with morphology → find **contours** → take the largest → its centre and size. The centre's left/right offset steers the robot; its size tells how near the object is.

TRACK_BALL.PY

```
import cv2, numpy as np
from arena import arena
img = arena()
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
mask = cv2.inRange(hsv, (8, 120, 120), (25, 255, 255))                # orange
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, np.ones((5, 5), np.uint8))
contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
ball = max(contours, key=cv2.contourArea)
(x, y), r = cv2.minEnclosingCircle(ball)
offset = x - img.shape[1] / 2
print(f"ball centre ({x:.0f}, {y:.0f})  radius {r:.0f}px  area {cv2.contourArea(ball):.0f}")
print(f"offset from image centre: {offset:+.0f} px ->", "turn right" if offset > 20 else "turn left"
if offset < -20 else "straight")
out = img.copy()
cv2.circle(out, (int(x), int(y)), int(r), (0, 255, 0), 2)
cv2.line(out, (160, 0), (160, 240), (255, 255, 255), 1)
cv2.imwrite("track.png", cv2.hconcat([out, cv2.cvtColor(mask, cv2.COLOR_GRAY2BGR)]))
```

OUTPUT

```
ball centre (230, 150)  radius 28px  area 2370
offset from image centre: +70 px -> turn right
```



DETECTED BALL (LEFT) AND THE HSV MASK (RIGHT)

17.3.4 Edge detection

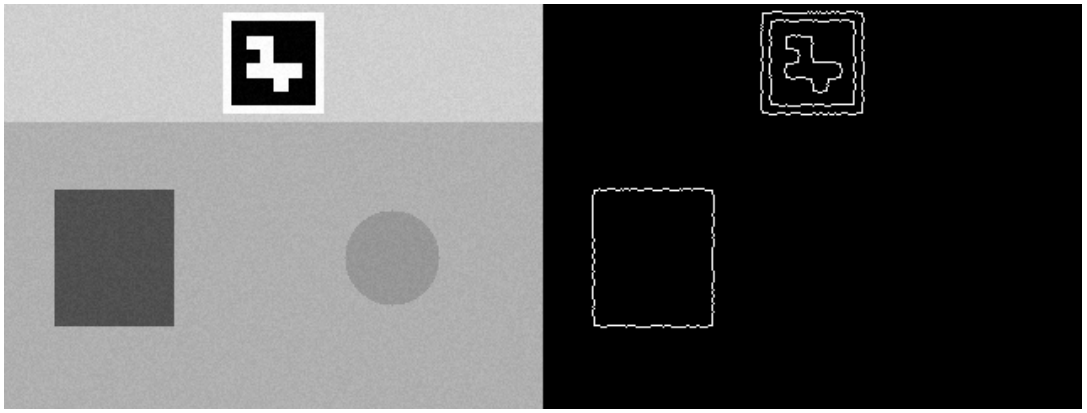
Canny marks pixels where brightness changes sharply (two thresholds, ratio about 1:2–1:3) after a Gaussian blur. Edges find object outlines, lane borders and the straight lines of walls and doors (with the Hough line transform).

EDGES.PY

```
import cv2
from arena import arena
gray = cv2.cvtColor(arena(), cv2.COLOR_BGR2GRAY)
edges = cv2.Canny(cv2.GaussianBlur(gray, (5, 5), 0), 40, 120)
print("edge pixels:", int((edges > 0).sum()), "of", edges.size)
cv2.imwrite("edges.png", cv2.hconcat([gray, edges]))
```

OUTPUT

```
edge pixels: 866 of 76800
```



GRAYSCALE FRAME AND ITS CANNY EDGES

17.3.5 Face detection

OpenCV ships pretrained **Haar cascades**: fast, CPU-friendly detectors for frontal faces, eyes and more. They need good, even light and a face looking at the camera; a DNN face detector (YuNet in OpenCV) is more robust. A robot can turn to face a person or greet them.

FACE DETECTION (REFERENCE)

```
face = cv2.CascadeClassifier(cv2.data.haarcascades + "haarcascade_frontalface_default.xml")
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
faces = face.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5, minSize=(40, 40))
for (x, y, w, h) in faces:
    cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)
    error = (x + w / 2) - frame.shape[1] / 2      # steer a pan servo or the robot toward the face
```

17.3.6 QR codes and ArUco markers

QR codes carry text (a station name, a command). **ArUco markers** are simpler black-and-white squares with an ID, detected fast and reliably even when small, and their four corners give the marker's position and angle: robots use them as landmarks, docking targets and position references.

QR_ARUCO.PY

```
import cv2, numpy as np
from arena import arena

qr = cv2.QRCodeEncoder.create().encode("STATION-3")              # make a QR code
qr = cv2.resize(qr, None, fx=6, fy=6, interpolation=cv2.INTER_NEAREST)
qr = cv2.copyMakeBorder(qr, 20, 20, 20, 20, cv2.BORDER_CONSTANT, value=255)
text, _, _ = cv2.QRCodeDetector().detectAndDecode(qr)
print("QR says:", text)

img = arena()
d = cv2.aruco.getPredefinedDictionary(cv2.aruco.DICT_4X4_50)
corners, ids, _ = cv2.aruco.ArucoDetector(d).detectMarkers(cv2.cvtColor(img, cv2.COLOR_BGR2GRAY))
c = corners[0][0]
side_px = np.linalg.norm(c[0] - c[1])
cx = c[:, 0].mean()
F_PX, SIDE_CM, HFOV = 300, 10.0, 62.2                            # focal length (px), real marker
size, camera field of view
print("ArUco id:", int(ids.flatten()[0]), "| side", round(float(side_px)), "px")
print("distance ~", round(F_PX * SIDE_CM / side_px), "cm")
print("bearing  ~", round((cx - 160) / 320 * HFOV, 1), "deg")
```

OUTPUT

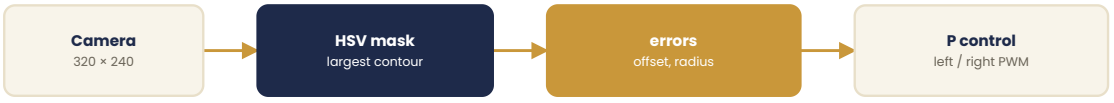
```
QR says: STATION-3
ArUco id: 7 | side 49 px
distance ~ 61 cm
bearing  ~ -0.1 deg
```

Distance comes from similar triangles: **distance = focal length (px) × real size ÷ size in pixels**. Calibrate the focal length once by measuring a marker at a known distance. For full 3-D pose use camera calibration and `cv2.solvePnP`.

17.4 Vision-based robot behaviours

17.4.1 Ball-following robot

Each frame gives the ball's horizontal offset (steering) and radius (distance). A proportional controller turns those errors into wheel speeds; if no ball is seen, the robot spins slowly to search.



BALL-FOLLOWING PIPELINE, REPEATED 15–30 TIMES PER SECOND

FOLLOW_BALL.PY

```
import cv2, numpy as np
from arena import arena

def find_ball(img):
    hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
    mask = cv2.inRange(hsv, (8, 120, 120), (25, 255, 255))
    cs, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    cs = [c for c in cs if cv2.contourArea(c) > 80]
    if not cs:
        return None
    (x, y), r = cv2.minEnclosingCircle(max(cs, key=cv2.contourArea))
    return x, r

BASE, KT, KD, TARGET_R = 120, 0.8, 4.0, 40          # cruise speed, turn gain, distance gain, wanted
radius
frames = [arena(260, 150, 18), arena(200, 150, 24), arena(165, 150, 34), arena(160, 150, 40),
arena(160, 150, 52), arena(400, 150, 20)]
for i, f in enumerate(frames):
    b = find_ball(f)
    if b is None:
        print(f"frame {i}: no ball -> search spin  L= 80 R=-80"); continue
    x, r = b
    turn = KT * (x - 160)                            # + means ball is to the right
    fwd = max(-BASE, min(BASE, KD * (TARGET_R - r)))  # slow down as the ball gets big
    L, R = int(np.clip(fwd + turn, -255, 255)), int(np.clip(fwd - turn, -255, 255))
    print(f"frame {i}: x={x:5.1f} r={r:4.1f}  ->  L={L:4d} R={R:4d}")
```

OUTPUT

```
frame 0: x=260.0 r=18.0  ->  L= 167 R=   7
frame 1: x=200.0 r=24.0  ->  L=  95 R=  31
frame 2: x=165.0 r=34.0  ->  L=  27 R=  19
frame 3: x=160.0 r=40.0  ->  L=   0 R=   0
frame 4: x=160.0 r=52.0  ->  L= -48 R= -48
frame 5: no ball -> search spin  L= 80 R=-80
```

Far and to the right: both wheels forward, left faster (turn right). Centred at the target size: stop. Too close: reverse. Ball out of view (frame 5): search. On the robot, send L and R to the motor driver with gpiozero or

over serial to an Arduino.

17.4.2 Vision-based line following

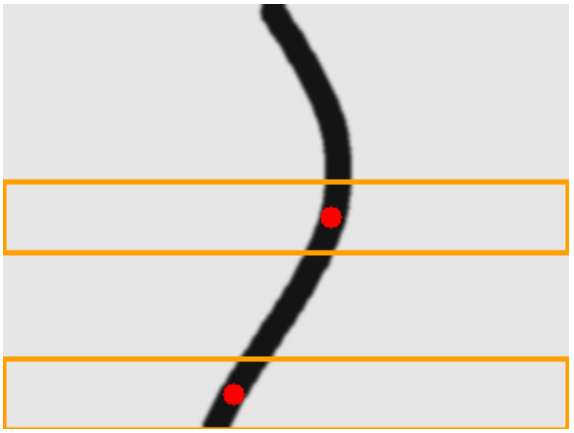
A camera sees the line *ahead*, not just under the robot. Threshold the bottom strip of the frame (the region of interest, ROI), find the line's centroid with image moments, and steer by its offset. Looking at a higher strip as well lets the robot slow down before curves.

VISION_LINE.PY

```
import cv2, numpy as np
img = np.full((240, 320), 230, np.uint8)                        # white floor
pts = np.array([[150 + 40 * np.sin(y / 60), y] for y in range(240, 0, -4)], np.int32)
cv2.polylines(img, [pts], False, 20, 14)                         # curving black tape
img = cv2.GaussianBlur(img, (5, 5), 0)
_, bw = cv2.threshold(img, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)
out = cv2.cvtColor(img, cv2.COLOR_GRAY2BGR)
for name, (y1, y2) in {"near": (200, 240), "far": (100, 140)}.items():
    roi = bw[y1:y2]
    m = cv2.moments(roi)
    cx = m["m10"] / m["m00"]
    err = cx - 160
    print(f"{name:4} strip y={y1}-{y2}: line at x={cx:5.1f}  error {err:+5.1f}px  steer {0.5 *
err:+5.1f}")
    cv2.rectangle(out, (0, y1), (319, y2), (0, 160, 255), 2)
    cv2.circle(out, (int(cx), (y1 + y2) // 2), 6, (0, 0, 255), -1)
cv2.imwrite("line.png", out)
```

OUTPUT

```
near strip y=200-240: line at x=130.3  error -29.7px  steer -14.8
far  strip y=100-140: line at x=185.2  error +25.2px  steer +12.6
```



TWO REGIONS OF INTEREST AND THE LINE CENTROID IN EACH

The near strip steers now; the far strip shows the line bending the other way, so the robot can slow down early. This look-ahead is why camera line followers beat IR arrays on fast, curvy tracks.

17.5 AI vision models (overview)

17.5.1 Object detection with pretrained models (YOLO-style)

Colour rules break when lighting or objects change. A **neural-network detector** trained on thousands of images outputs boxes, class names ("person", "bottle", "dog") and confidences for 80+ classes in one pass. **YOLO** ("you only look once") models are fast enough for robots.

YOLO DETECTION (REFERENCE)

```
from ultralytics import YOLO                    # pip install ultralytics
model = YOLO("yolov8n.pt")                      # "n" = nano: smallest and fastest
results = model(frame, imgsz=320, conf=0.5)
for box in results[0].boxes:
    name = model.names[int(box.cls)]
    x1, y1, x2, y2 = box.xyxy[0].tolist()
    print(name, round(float(box.conf), 2), (x1 + x2) / 2)   # steer toward the centre
```

Hardware	YOLO nano at 320 px (approx.)	Notes
Raspberry Pi 4	2–5 fps	use NCNN/TFLite exports, small input
Raspberry Pi 5	5–12 fps	or the AI Kit / AI HAT+ (Hailo) for 30+ fps
Jetson Orin Nano	30–60 fps	GPU with TensorRT
ESP32-S3	tiny models only	Edge Impulse FOMO for simple object spotting

WATCH OUT

Detectors make mistakes: always add a confidence threshold, require a detection in several consecutive frames before acting, and never rely on vision alone for safety stops.

PRACTICE CHECKPOINT

Build a robot that detects and follows a coloured ball using a camera and OpenCV, and detect ArUco markers for position reference.

17.6 Practice questions

17.6.1 Basic level

Q1 Which colour space separates colour from brightness, making colour tracking robust?

(a) BGR (b) HSV (c) grayscale (d) binary

Q2 In what channel order does OpenCV store colour images?

(a) BGR (b) RGB (c) HSV (d) CMYK

Q3 Which cheap board combines a camera with Wi-Fi for streaming?

(a) Arduino Uno (b) L298N (c) ESP32-CAM (d) HC-05

Q4 What extra information does a depth camera provide?

(a) higher colour accuracy only (b) sound
(c) GPS position (d) the distance to every pixel

Q5 Which OpenCV function makes a mask of pixels within a colour range?

(a) cv2.Canny (b) cv2.resize (c) cv2.imshow (d) cv2.inRange

Q6 Which algorithm finds edges in an image?

(a) inRange (b) Otsu only (c) Canny (d) A*

Q7 What are ArUco markers used for in robotics?

(a) landmarks with IDs that give position and angle
(b) storing long text
(c) colour calibration only
(d) charging robots

Q8 Which pretrained detector family is known for fast real-time object detection?

(a) Haar line (b) PID (c) Dijkstra (d) YOLO

17.6.2 Intermediate level

Q9 Why do robots usually process 320 × 240 frames rather than full HD?

(a) cameras cannot do more
(b) fewer pixels means much faster processing
(c) colours are better
(d) OpenCV requires it

Q10 What do contours give you after thresholding?

(a) the image histogram
(b) outlines of white regions, with area, centre and bounding box
(c) the camera focal length
(d) audio data

Q11 Why clean a mask with morphological opening?

(a) it changes the colour space (b) it detects faces

(c) it increases resolution (d) it removes small specks of noise

Q12 In a ball follower, what does the ball's horizontal offset control?

(a) the turning (difference between wheel speeds)
(b) the camera exposure
(c) the battery voltage
(d) the frame rate

Q13 In a ball follower, what does the ball's apparent radius indicate?

(a) its colour (b) its weight
(c) how close the ball is (d) the motor temperature

Q14 Why does camera line following use a region of interest near the bottom of the frame?

(a) the top is always black
(b) OpenCV cannot read the top
(c) it increases brightness
(d) it shows the line just ahead of the robot and processes fewer pixels

Q15 Which method detects faces quickly on a CPU using OpenCV's bundled files?

(a) Canny (b) Haar cascade classifier
(c) Otsu threshold (d) inRange

Q16 Which ArUco output gives the marker's position in the image?

(a) its four corner points (b) its hue
(c) the frame rate (d) the dictionary name

Q17 Which device runs YOLO at 30+ fps on a robot?

(a) Arduino Uno
(b) ESP-01
(c) Jetson Orin Nano (or a Pi with a Hailo AI accelerator)
(d) HC-SR04

17.6.3 Advanced level

Q18 A 10 cm marker appears 50 px wide with a focal length of 300 px. How far away is it?

(a) 60 cm (b) 15 cm (c) 150 cm (d) 600 cm

Q19 Halving the width and height of a frame reduces the pixel count to what fraction?

(a) one half (b) one quarter (c) one eighth (d) unchanged

Q20 Find every blue object in the test frame with an HSV mask and print its centre and area.

Q21 Convert a horizontal pixel offset into a bearing angle for a camera with a 62.2° field of view and a 320-pixel-wide image, for x = 0, 80, 160, 240, 320.

Q22 Smooth a noisy ball x-position with an exponential moving average (alpha = 0.4) before steering.

Q23 Decide whether a detection is trustworthy: act only if the object was detected in at least 3 of the last 5 frames with confidence ≥ 0.5.

Q24 A ball tracker works in the lab but fails in sunlight. What is the best first fix?

(a) switch to BGR thresholds
(b) increase the frame size to 4K
(c) remove the morphology step
(d) re-sample the ball's HSV values under the new light and widen the range

Q25 A delivery robot must dock precisely at a charging station. Which vision aid is most reliable?

(a) a red wall
(b) a face detector
(c) an ArUco marker on the station
(d) Canny on the floor

17.7 Solutions

Q1 **Answer: (b) HSV**
Hue is the colour.

Q2 **Answer: (a) BGR**
Blue, green, red.

Q3 **Answer: (c) ESP32-CAM**
OV2640 + ESP32.

Q4 **Answer: (d) the distance to every pixel**
Depth map.

Q5 **Answer: (d) cv2.inRange**
Lower/upper bounds.

Q6 **Answer: (c) Canny**
Gradient + hysteresis.

Q7 **Answer: (a) landmarks with IDs that give position and angle**
Fast fiducials.

Q8 **Answer: (d) YOLO**
You only look once.

Q9 **Answer: (b) fewer pixels means much faster processing**
Speed.

Q10 **Answer: (b) outlines of white regions, with area, centre and bounding box**
Shape analysis.

Q11 **Answer: (d) it removes small specks of noise**

Erode then dilate.

Q12 **Answer: (a) the turning (difference between wheel speeds)**

Steering error.

Q13 **Answer: (c) how close the ball is**

Bigger = nearer.

Q14 **Answer: (d) it shows the line just ahead of the robot and processes fewer pixels**

ROI.

Q15 **Answer: (b) Haar cascade classifier**

Pretrained cascades.

Q16 **Answer: (a) its four corner points**

Corners → centre and size.

Q17 **Answer: (c) Jetson Orin Nano (or a Pi with a Hailo AI accelerator)**

Accelerated inference.

Q18 **Answer: (a) 60 cm**

300 × 10 ÷ 50 = **60 cm**.

Q19 **Answer: (b) one quarter**

½ × ½ = **¼**.

Q20 **Answer: Solution program**

Only the colour range changes from the ball tracker: OpenCV blue sits around hue 100–130. The box spans x 30–100, y 110–190, so its centre is (65, 150).

```
import cv2, numpy as np
from arena import arena
hsv = cv2.cvtColor(arena(), cv2.COLOR_BGR2HSV)
mask = cv2.inRange(hsv, (100, 120, 80), (130, 255, 255))          # blue hues
cs, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
for c in cs:
    if cv2.contourArea(c) < 100: continue
    m = cv2.moments(c)
    print("blue object at", (int(m["m10"] / m["m00"]), int(m["m01"] / m["m00"])), "area",
int(cv2.contourArea(c)))
```

OUTPUT

```
blue object at (65, 150) area 5600
```

Q21 **Answer: Solution program**

A linear estimate is close enough for steering (exact: atan of the offset over the focal length). Positive means the target is to the right.

```
W, FOV = 320, 62.2
for x in [0, 80, 160, 240, 320]:
    bearing = (x - W / 2) / W * FOV
    print(f"x={x:3d}  bearing {bearing:+6.2f} deg")
```

OUTPUT

```
x=  0  bearing -31.10 deg
x= 80  bearing -15.55 deg
x=160  bearing  +0.00 deg
x=240  bearing +15.55 deg
x=320  bearing +31.10 deg
```

Q22 **Answer: Solution program**

Filtering removes frame-to-frame jitter so the robot does not wiggle; a smaller alpha is smoother but reacts more slowly.

```
raw = [160, 175, 150, 210, 205, 140, 215, 220]
alpha, smooth = 0.4, raw[0]
for x in raw:
    smooth = alpha * x + (1 - alpha) * smooth
    print(f"raw {x:3d}  smoothed {smooth:6.1f}")
```

OUTPUT

```
raw 160  smoothed  160.0
raw 175  smoothed  166.0
raw 150  smoothed  159.6
raw 210  smoothed  179.8
raw 205  smoothed  189.9
raw 140  smoothed  169.9
raw 215  smoothed  187.9
raw 220  smoothed  200.8
```

Q23 **Answer: Solution program**

Voting over a short window ignores one-frame false positives and short dropouts.

```
from collections import deque
hist = deque(maxlen=5)
frames = [0.62, 0.30, 0.71, 0.55, 0.0, 0.80, 0.90, 0.0, 0.0, 0.2]
for i, conf in enumerate(frames):
    hist.append(conf >= 0.5)
    act = sum(hist) >= 3
    print(f"frame {i}: conf {conf:.2f}  recent hits {sum(hist)}/5  ->", "ACT" if act else "wait")
```

OUTPUT

```
frame 0: conf 0.62  recent hits 1/5  -> wait
frame 1: conf 0.30  recent hits 1/5  -> wait
frame 2: conf 0.71  recent hits 2/5  -> wait
frame 3: conf 0.55  recent hits 3/5  -> ACT
frame 4: conf 0.00  recent hits 3/5  -> ACT
frame 5: conf 0.80  recent hits 3/5  -> ACT
frame 6: conf 0.90  recent hits 4/5  -> ACT
frame 7: conf 0.00  recent hits 3/5  -> ACT
frame 8: conf 0.00  recent hits 2/5  -> wait
frame 9: conf 0.20  recent hits 2/5  -> wait
```

Q24 **Answer: (d) re-sample the ball's HSV values under the new light and widen the range**

Lighting shifts S and V.

Q25 **Answer: (c) an ArUco marker on the station**

Known ID, size and corners.

Cameras: USB, Pi Camera, ESP32-CAM, global shutter, depth (stereo, structured light, ToF, LiDAR). Images are h × w × ch arrays (BGR); HSV for colour. Pipeline: HSV → inRange → morphology → contours → centre/size. Canny for edges, Haar for faces, QR for text, ArUco for landmarks (distance = f × size ÷ pixels). Ball following: offset → turn, radius → speed. Line following: ROI centroid. YOLO for general objects; filter and vote before acting.

MODULE 18 · ADVANCED

Localisation, mapping and navigation

Minimum duration **3 days** | Level **Advanced**

Autonomous robots must know where they are and how to reach their goal. This module estimates position with wheel odometry, orientation with an IMU and a complementary filter, fuses sensors with a one-dimensional Kalman filter and reads GPS for outdoor robots. It builds occupancy grid maps from range scans, explains SLAM, plans paths on grids with BFS, Dijkstra and A*, replans around new obstacles, and drives with go-to-goal, wall-following and combined behaviours, followed by 25 solved questions and programs.

IN THIS MODULE

18.1 Localisation
18.2 Mapping
18.3 Path planning
18.4 Navigation behaviours
18.5 Practice questions
18.6 Solutions

18.1 Localisation

Localisation answers "where am I?" as a **pose**: position (x, y) and heading θ. No single sensor is perfect: wheels slip, gyros drift, GPS jumps by metres. Good robots combine several.

Source	Gives	Strength	Weakness
Wheel encoders (odometry)	distance and turning	smooth, high rate	drifts without limit (slip, wheel size errors)
IMU gyro	turn rate → heading	fast, no slip	heading drifts slowly
IMU accelerometer	tilt (gravity direction)	no drift in tilt	noisy, disturbed by motion
Magnetometer	compass heading	absolute	disturbed by motors and steel
GPS	latitude / longitude	absolute outdoors	±2–5 m, none indoors
Landmarks (ArUco, LiDAR features)	position relative to a map	corrects drift	needs a map or markers

18.1.1 Dead reckoning and odometry

Dead reckoning adds up small movements from a known start. For a differential-drive robot, each update converts encoder ticks to wheel distances d_L, d_R, then:

NOTE

$d = (d_L + d_R) / 2 \quad \Delta\theta = (d_R - d_L) / L \quad (L = \text{wheel separation})$

$x \mathrel{+}= d\cdot\cos(\theta + \Delta\theta/2) \quad y \mathrel{+}= d\cdot\sin(\theta + \Delta\theta/2) \quad \theta \mathrel{+}= \Delta\theta$

ODOMETRY.PY

```
import math

TICKS_PER_REV, WHEEL_D, TRACK = 20, 6.5, 13.0          # encoder, wheel diameter (cm), wheel
separation (cm)
CM_PER_TICK = math.pi * WHEEL_D / TICKS_PER_REV

x = y = th = 0.0
# (left ticks, right ticks) counted in each 100 ms update
updates = [(10, 10)] * 5 + [(-5, 5)] * 2 + [(10, 10)] * 5
for i, (tl, tr) in enumerate(updates, 1):
    dl, dr = tl * CM_PER_TICK, tr * CM_PER_TICK
    d, dth = (dl + dr) / 2, (dr - dl) / TRACK
    x += d * math.cos(th + dth / 2)
    y += d * math.sin(th + dth / 2)
    th += dth
    if i in (5, 7, 12):
        print(f"after update {i:2d}: x={x:6.1f} cm  y={y:6.1f} cm  heading={math.degrees(th):6.1f}
deg")
```

OUTPUT

```
after update  5: x=  51.1 cm  y=   0.0 cm  heading=   0.0 deg
after update  7: x=  51.1 cm  y=   0.0 cm  heading=  90.0 deg
after update 12: x=  51.1 cm  y=  51.1 cm  heading=  90.0 deg
```

The robot drives 51 cm, spins on the spot (≈ 90°), then drives 51 cm along +y. In reality each step carries a small error: a 1 % wheel-size error is 1 cm per metre, and heading errors grow into large position errors over distance. Odometry is excellent over short distances and must be corrected over long ones.

18.1.2 IMU-based orientation

Integrating the gyro's turn rate gives heading or tilt quickly, but a tiny bias (say 0.5°/s) adds up to 30° in a minute. The accelerometer measures tilt from gravity with no drift, but every bump disturbs it. A **complementary filter** trusts the gyro in the short term and the accelerometer in the long term:

NOTE

angle = α·(angle + gyro_rate·dt) + (1 − α)·accel_angle (α ≈ 0.95–0.98)

COMPLEMENTARY.PY

```
import random
random.seed(4)
dt, alpha = 0.01, 0.98
true_angle = 10.0                       # robot is tilted 10 degrees and still
gyro_only = comp = 0.0
for step in range(1, 501):              # 5 seconds at 100 Hz
    gyro = 0.0 + 0.5 + random.gauss(0, 0.3)          # true rate 0 + bias 0.5 deg/s + noise
    accel = true_angle + random.gauss(0, 3.0)         # correct on average but noisy
    gyro_only += gyro * dt
    comp = alpha * (comp + gyro * dt) + (1 - alpha) * accel
    if step in (1, 50, 100, 300, 500):
        print(f"t={step * dt:4.2f}s  accel={accel:6.2f}  gyro-only={gyro_only:5.2f}  complementary=
{comp:5.2f}")
```

OUTPUT

```
t=0.01s  accel= 11.39  gyro-only= 0.01  complementary= 0.23
t=0.50s  accel=  7.22  gyro-only= 0.26  complementary= 6.45
t=1.00s  accel=  6.91  gyro-only= 0.57  complementary= 8.50
t=3.00s  accel=  9.36  gyro-only= 1.59  complementary=10.74
t=5.00s  accel= 12.94  gyro-only= 2.63  complementary=10.13
```

The gyro-only estimate never finds the 10° tilt and keeps drifting with the bias; the raw accelerometer jumps around by several degrees; the complementary filter settles near 10° and stays smooth. Balancing robots and drones use exactly this (or a Kalman filter).

18.1.3 Sensor fusion and the Kalman filter (overview)

A **Kalman filter** keeps an estimate *and* its uncertainty. Each cycle it **predicts** (move the estimate using the motion model; uncertainty grows) and **updates** (blend in a measurement; uncertainty shrinks). The **Kalman gain** K decides how much to trust the measurement: near 1 if the sensor is precise, near 0 if it is noisy.

KALMAN_1D.PY

```
import random
random.seed(2)
x, P = 0.0, 100.0          # estimate (cm) and its variance: we start very unsure
Q, R = 1.0, 25.0           # motion noise variance, sensor noise variance (std 5 cm)
true = 0.0
for step in range(1, 9):
    true += 10                                  # robot really moves 10 cm per step
    x, P = x + 10, P + Q                        # predict with odometry
    z = true + random.gauss(0, 5)               # noisy range-sensor position
    K = P / (P + R)                             # Kalman gain
    x, P = x + K * (z - x), (1 - K) * P         # update
    print(f"step {step}: true {true:5.1f}  sensor {z:6.1f}  estimate {x:6.1f}  gain {K:.2f}  std {P
** 0.5:4.1f}")
```

OUTPUT

```
step 1: true  10.0  sensor   21.7  estimate   19.4  gain 0.80  std  4.5
step 2: true  20.0  sensor   16.7  estimate   23.6  gain 0.46  std  3.4
step 3: true  30.0  sensor   32.0  estimate   33.0  gain 0.33  std  2.9
step 4: true  40.0  sensor   40.7  estimate   42.4  gain 0.27  std  2.6
step 5: true  50.0  sensor   54.2  estimate   52.8  gain 0.24  std  2.4
step 6: true  60.0  sensor   53.0  estimate   60.7  gain 0.22  std  2.3
step 7: true  70.0  sensor   67.9  estimate   70.1  gain 0.20  std  2.3
step 8: true  80.0  sensor   76.2  estimate   79.4  gain 0.20  std  2.2
```

At first the filter trusts the sensor (gain 0.80); as its own estimate becomes confident, the gain drops and the estimate is smoother than the sensor. The extended Kalman filter (EKF) does the same for full robot poses and is used in ROS 2's `robot_localization` package to fuse wheels, IMU and GPS.

18.1.4 GPS for outdoor robots

A GPS module (NEO-6M, NEO-M8N) sends NMEA text lines over serial at 9600 baud. **$GPGGA** gives the fix: time, latitude, longitude, fix quality, satellites and altitude. Typical accuracy is 2–5 m; RTK GPS reaches about 2 cm using a base station.

GPS_NAV.PY

```
import math

def nmea_deg(value, hemi):                  # ddmm.mmmm -> decimal degrees
    v = float(value); d = int(v // 100)
    deg = d + (v - d * 100) / 60
    return -deg if hemi in "SW" else deg

line = "$GPGGA,063015.00,1258.3721,N,07735.6552,E,1,09,0.9,920.4,M,-86.5,M,,*5C"
f = line.split(",")
lat, lon = nmea_deg(f[2], f[3]), nmea_deg(f[4], f[5])
print(f"fix={f[6]} sats={int(f[7])} lat={lat:.6f} lon={lon:.6f} alt={f[9]} m")

def dist_bearing(lat1, lon1, lat2, lon2):
    R = 6371000
    p1, p2, dl = math.radians(lat1), math.radians(lat2), math.radians(lon2 - lon1)
    a = math.sin((p2 - p1) / 2) ** 2 + math.cos(p1) * math.cos(p2) * math.sin(dl / 2) ** 2
    d = 2 * R * math.asin(math.sqrt(a))                                  # haversine
    b = math.degrees(math.atan2(math.sin(dl) * math.cos(p2), math.cos(p1) * math.sin(p2) -
math.sin(p1) * math.cos(p2) * math.cos(dl)))
    return d, (b + 360) % 360

d, b = dist_bearing(lat, lon, 12.9740, 77.5960)
print(f"waypoint: {d:.0f} m away, bearing {b:.0f} deg (0 = north)")
```

OUTPUT

```
fix=1 sats=9 lat=12.972868 lon=77.594253 alt=920.4 m
waypoint: 227 m away, bearing 56 deg (0 = north)
```

An outdoor rover compares the bearing to the next waypoint with its compass heading and steers to reduce the difference, switching to the next waypoint when the distance drops below a few metres.

18.2 Mapping

18.2.1 Occupancy grid maps

An **occupancy grid** divides the world into cells (say 5 cm) marked **free**, **occupied** or **unknown**. For each range reading, the cells along the beam are free and the cell where it hit is occupied. Real systems store a probability (log-odds) per cell, so repeated readings build confidence and moving people fade away.

OCCUPANCY_GRID.PY

```
import math
W, H, CELL = 24, 12, 10                      # 24 x 12 cells of 10 cm
grid = [["?"] * W for _ in range(H)]

def wall(x, y):                              # the real room (in cm): walls and a box
    return x < 5 or x > 235 or y < 5 or y > 115 or (150 <= x <= 180 and 30 <= y <= 60)

rx, ry = 50, 55                              # robot position in cm
for deg in range(0, 360, 3):                 # simulated 2-D LiDAR scan
    a = math.radians(deg)
    for r in range(0, 300, 2):
        x, y = rx + r * math.cos(a), ry + r * math.sin(a)
        cx, cy = int(x // CELL), int(y // CELL)
        if not (0 <= cx < W and 0 <= cy < H): break
        if wall(x, y):
            grid[cy][cx] = "#"; break        # beam stopped: occupied
        if grid[cy][cx] != "#": grid[cy][cx] = "."   # beam passed: free
grid[ry // CELL][rx // CELL] = "R"
for row in reversed(grid):
    print("".join(row))
```

OUTPUT

```
################.#.#..#
#.....................#
#.....................#
#.....................#
#.....................#
#.....................#
#....R.........#????????
#.............#????????
#.............#????????
#................??????
#....................??
###############.#.#.#..#
```

From one position the robot maps the free floor (.) and walls (#), but the area behind the box stays unknown (?). Driving to other spots and merging scans fills the shadow: this is how a LiDAR robot maps a room.

18.2.2 SLAM concepts (simultaneous localisation and mapping)

Mapping needs an accurate pose, and localisation needs a map. **SLAM** solves both at once: the robot matches each new scan against the map built so far to correct its odometry, then adds the scan to the map. When it returns to a known place (**loop closure**) it corrects the accumulated drift around the whole loop.

SLAM system	Sensor	Notes
slam_toolbox (ROS 2)	2-D LiDAR	default for Nav2; online and offline mapping
Cartographer	2-D/3-D LiDAR + IMU	Google; very accurate
Hector SLAM	2-D LiDAR, no odometry	works hand-held

SLAM system	Sensor	Notes
ORB-SLAM3	camera (mono, stereo, RGB-D)	visual SLAM
RTAB-Map	RGB-D camera / LiDAR	3-D maps, good loop closure

18.3 Path planning

18.3.1 Grid-based planning

On a grid map, each free cell is a node connected to its neighbours (4 or 8 directions). Path planning searches this graph from start to goal. Before planning, **inflate** obstacles by the robot's radius so the path keeps a safe distance.

18.3.2 BFS and Dijkstra

Algorithm	Explores	Finds	Use
BFS (breadth-first)	in rings outward from the start	fewest steps (equal costs)	simple grids
Dijkstra	cheapest-first using a priority queue	lowest total cost (varied costs: mud, slopes)	weighted maps
A*	cheapest f = g + h first (cost so far + estimate to goal)	lowest cost, exploring far fewer cells	most robot planners
Greedy best-first	smallest h only	fast but not always shortest	quick rough paths

18.3.3 A* algorithm

A* scores each cell by **f = g + h**: g is the cost from the start, h a heuristic estimate to the goal (Manhattan distance on a 4-connected grid). If h never overestimates, A* is guaranteed to find the shortest path, while the heuristic pulls the search toward the goal.

ASTAR_GRID.PY

```
import heapq
from collections import deque

MAP = ["S.........",
       ".####.###.",
       ".#......#.",
       ".#.####.#.",
       "...#..#...",
       "##.#.G#.##",
       "...#..#...",
       ".###.##.#.",
       ".........",]
grid = [list(r) for r in MAP]
H, W = len(grid), len(grid[0])
start = next((r, c) for r in range(H) for c in range(W) if grid[r][c] == "S")
goal = next((r, c) for r in range(H) for c in range(W) if grid[r][c] == "G")

def neighbours(r, c):
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        nr, nc = r + dr, c + dc
        if 0 <= nr < H and 0 <= nc < W and grid[nr][nc] != "#":
            yield nr, nc

def bfs():
    came, q, seen = {start: None}, deque([start]), 0
    while q:
        cur = q.popleft(); seen += 1
        if cur == goal: break
        for n in neighbours(*cur):
            if n not in came: came[n] = cur; q.append(n)
    return came, seen

def astar():
    h = lambda p: abs(p[0] - goal[0]) + abs(p[1] - goal[1])        # Manhattan heuristic
    came, g, pq, seen = {start: None}, {start: 0}, [(h(start), 0, start)], 0
    while pq:
        f, cost, cur = heapq.heappop(pq)
        if cost > g[cur]: continue                                  # stale queue entry
        seen += 1
        if cur == goal: break
        for n in neighbours(*cur):
            ng = g[cur] + 1
            if ng < g.get(n, 1e9):
                g[n], came[n] = ng, cur
                heapq.heappush(pq, (ng + h(n), ng, n))
    return came, seen

def path_of(came):
    p, cur = [], goal
    while cur: p.append(cur); cur = came[cur]
    return p[::-1]

for name, fn in (("BFS", bfs), ("A*", astar)):
    came, seen = fn()
    print(f"{name:3}: path {len(path_of(came)) - 1} steps, cells expanded {seen}")
for r, c in path_of(astar()[0])[1:-1]:
```

```
    grid[r][c] = "*"
print("\n".join("".join(row) for row in grid))
```

OUTPUT

```
BFS: path 20 steps, cells expanded 58
A* : path 20 steps, cells expanded 54
S.........
*####.###.
*#......#.
*#.####.#.
***#..#...
##*#*G#.##
***#*.#...
*###*##.#.
*****.....
```

Both find a shortest path (the same length), but A* expands fewer cells because the heuristic steers it toward G. On large maps with thousands of cells the saving is huge. Dijkstra behaves like BFS here because every step costs 1.

18.3.4 Obstacle avoidance while following a path

The global plan uses the map; a **local planner** reacts to what the sensors see right now (a person, a chair). If the path is blocked, the robot stops or steers round locally and, if needed, **replans** from its current position.

REPLAN.PY

```
import heapq
def plan(grid, s, g):
    H, W = len(grid), len(grid[0]); h = lambda p: abs(p[0] - g[0]) + abs(p[1] - g[1])
    pq, came, cost = [(h(s), 0, s)], {s: None}, {s: 0}
    while pq:
        _, c, cur = heapq.heappop(pq)
        if cur == g:
            p = []
            while cur: p.append(cur); cur = came[cur]
            return p[::-1]
        for d in ((0, 1), (1, 0), (0, -1), (-1, 0)):
            n = (cur[0] + d[0], cur[1] + d[1])
            if 0 <= n[0] < H and 0 <= n[1] < W and grid[n[0]][n[1]] == "." and c + 1 < cost.get(n,
1e9):
                cost[n], came[n] = c + 1, cur; heapq.heappush(pq, (c + 1 + h(n), c + 1, n))
    return None

grid = [list(".........") for _ in range(5)]
path = plan(grid, (2, 0), (2, 7))
print("first plan :", len(path) - 1, "steps", path)
grid[2][4] = "#"                                    # a box appears on the path
pos = path[3]                                       # robot has already moved 3 cells
blocked = any(grid[r][c] == "#" for r, c in path[3:])
print("blocked ahead?", blocked, "| robot at", pos)
new = plan(grid, pos, (2, 7))
print("replanned  :", len(new) - 1, "steps", new)
```

OUTPUT

```
first plan : 7 steps [(2, 0), (2, 1), (2, 2), (2, 3), (2, 4), (2, 5), (2, 6), (2, 7)]
blocked ahead? True | robot at (2, 3)
replanned  : 6 steps [(2, 3), (1, 3), (1, 4), (1, 5), (1, 6), (1, 7), (2, 7)]
```

18.4 Navigation behaviours

18.4.1 Go-to-goal, wall following, combining behaviours

Go-to-goal steers the heading toward the target: angle error = atan2(dy, dx) - θ, turn rate = K·error, forward speed reduced while the error is large. **Wall following** keeps a side distance constant with PID (module 13). **Combining**: a priority scheme (avoid obstacle > follow wall > go to goal) or a weighted sum of each behaviour's steering.

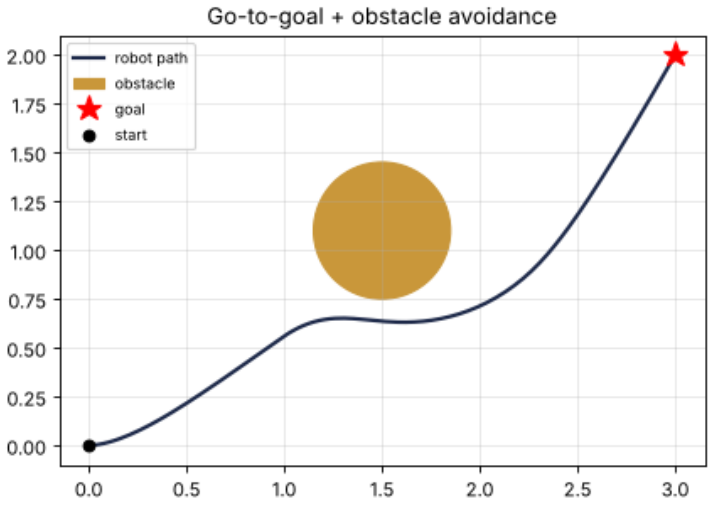
GO_TO_GOAL.PY

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import math

goal, obstacle, R_OBS = (3.0, 2.0), (1.5, 1.1), 0.35
x, y, th, dt = 0.0, 0.0, 0.0, 0.05
xs, ys = [x], [y]
for step in range(400):
    dx, dy = goal[0] - x, goal[1] - y
    dist = math.hypot(dx, dy)
    if dist < 0.05: break
    err = math.atan2(math.sin(math.atan2(dy, dx) - th), math.cos(math.atan2(dy, dx) - th))
    turn = 2.5 * err                                        # go-to-goal
    ox, oy = x - obstacle[0], y - obstacle[1]
    d_obs = math.hypot(ox, oy)
    if d_obs < 0.8:                                         # avoid: push heading away, stronger when
close
        away = math.atan2(oy, ox)
        e2 = math.atan2(math.sin(away - th), math.cos(away - th))
        turn += 3.0 * (0.8 - d_obs) / 0.8 * e2
    v = min(0.6, dist) * max(0.2, math.cos(err))
    th += turn * dt; x += v * math.cos(th) * dt; y += v * math.sin(th) * dt
    xs.append(x); ys.append(y)
print(f"reached ({x:.2f}, {y:.2f}) in {step * dt:.1f} s, closest approach to obstacle "
      f"{min(math.hypot(a - obstacle[0], b - obstacle[1]) for a, b in zip(xs, ys)) - R_OBS:.2f} m
from its edge")
fig, ax = plt.subplots(figsize=(6, 4))
ax.plot(xs, ys, color="#1E2A4A", lw=2, label="robot path")
ax.add_patch(plt.Circle(obstacle, R_OBS, color="#C9973A", label="obstacle"))
ax.plot(*goal, "r*", ms=14, label="goal"); ax.plot(0, 0, "ko", label="start")
ax.set_aspect("equal"); ax.grid(alpha=0.3); ax.legend(fontsize=8, loc="upper left")
ax.set_title("Go-to-goal + obstacle avoidance"); fig.tight_layout(); fig.savefig("nav.png", dpi=80)
```

OUTPUT

```
reached (2.98, 1.96) in 8.4 s, closest approach to obstacle 0.11 m from its edge
```



TWO BEHAVIOURS BLENDED: GO-TO-GOAL STEERING PLUS A PUSH AWAY FROM THE OBSTACLE

Blending lets the robot curve smoothly round the obstacle and continue to the goal. Purely reactive blends can get stuck in U-shaped traps, which is why real systems combine them with a global planner (A*) that sees the whole map.

18.4.2 Following a planned path with odometry

PATH_TO_COMMANDS.PY

```
import math
path = [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (2, 3)]        # A* cells, 1 cell = 25 cm
CELL, heading = 25, 90                                          # start facing +y (north)
pts = [path[0]] + [p for i, p in enumerate(path[1:-1], 1)
                   if (p[0] - path[i - 1][0], p[1] - path[i - 1][1]) != (path[i + 1][0] - p[0],
path[i + 1][1] - p[1])] + [path[-1]]
print("waypoints (corners only):", pts)
for (x0, y0), (x1, y1) in zip(pts, pts[1:]):
    want = math.degrees(math.atan2(y1 - y0, x1 - x0))
    turn = (want - heading + 180) % 360 - 180
    dist = math.hypot(x1 - x0, y1 - y0) * CELL
    if turn: print(f"TURN {turn:+.0f} deg")
    print(f"DRIVE {dist:.0f} cm")
    heading = want
```

OUTPUT

```
waypoints (corners only): [(0, 0), (0, 2), (2, 2), (2, 3)]
DRIVE 50 cm
TURN -90 deg
DRIVE 50 cm
TURN +90 deg
DRIVE 25 cm
```

The path is reduced to its corners, then each leg becomes a turn and a drive. The robot executes them with encoder odometry (module 13): count ticks to drive the distance, and turn by the wheel-difference formula.

PRACTICE CHECKPOINT

Implement A* path planning on a grid map in Python, and make a robot follow the planned path using odometry.

18.5 Practice questions

18.5.1 Basic level

Q1 What does localisation estimate?

(a) the battery level
(b) the map only
(c) the robot's pose: position and heading
(d) the motor temperature

Q2 What is dead reckoning?

(a) using GPS only
(b) reading ArUco markers
(c) random guessing
(d) adding up measured movements from a known start

Q3 Why does odometry drift?

(a) encoders are random
(b) small errors from wheel slip and size add up without limit
(c) GPS interferes
(d) the battery charges

Q4 What is the weakness of integrating a gyroscope for heading?

(a) the bias makes the heading drift over time
(b) it is too slow
(c) it needs GPS
(d) it cannot measure rotation

Q5 What does an occupancy grid store for each cell?

(a) the colour of the floor
(b) whether it is free, occupied or unknown
(c) the robot speed
(d) the GPS time

Q6 What does SLAM stand for?

(a) simultaneous localisation and mapping
(b) sensor logic and motor
(c) serial link access module
(d) smart LiDAR and mapping

Q7 Which search algorithm uses f = g + h?

(a) BFS **(b)** DFS **(c)** A* **(d)** bubble sort

Q8 Which NMEA sentence gives the GPS fix with satellites and altitude?

(a) $GPGSV **(b)** $GPTXT **(c)** $GPVTG **(d)** $GPGGA

18.5.2 Intermediate level

Q9 What is a typical accuracy of a basic GPS module?

(a) 2 mm **(b)** 2 km **(c)** 20 m always **(d)** 2–5 m

Q10 Why combine gyro and accelerometer in a complementary filter?

(a) to increase the sample rate only
(b) to save power
(c) gyro is smooth short-term, accelerometer is drift-free long-term
(d) because the gyro measures tilt directly

Q11 In a Kalman filter, what does a high Kalman gain mean?

(a) the measurement is trusted more than the prediction
(b) the prediction is trusted more
(c) the filter has failed
(d) the robot is stopped

Q12 What is loop closure in SLAM?

(a) closing the robot lid
(b) a for loop in code
(c) stopping the motors
(d) recognising a place visited before to correct accumulated drift

Q13 Why inflate obstacles before planning?

(a) to make the map smaller
(b) so the path keeps the robot's body clear of obstacles
(c) to speed up the motors
(d) to remove unknown cells

Q14 Which algorithm handles cells with different costs (mud, slopes) and finds the cheapest path without a heuristic?

(a) BFS
(b) Dijkstra
(c) greedy best-first
(d) Canny

Q15 When is A* guaranteed to find the shortest path?

(a) always, whatever the heuristic
(b) only on 8-connected grids
(c) never
(d) when the heuristic never overestimates the true cost

Q16 What does a local planner do?

(a) reacts to nearby obstacles seen by sensors while following the global path
(b) builds the whole map
(c) charges the battery
(d) decodes GPS

Q17 Which ROS 2 package fuses wheel odometry, IMU and GPS with an EKF?

(a) rviz2
(b) turtlesim
(c) robot_localization
(d) rosbag2

18.5.3 Advanced level

Q18 A wheel of diameter 6.5 cm with 20 ticks per revolution: how far does it move per tick?

(a) about 0.33 cm **(b)** about 6.5 cm **(c)** about 20 cm **(d)** about 1.02 cm

Q19 A gyro has a bias of 0.5°/s. How much heading error builds up in 2 minutes?

(a) 1° **(b)** 60° **(c)** 6° **(d)** 120°

Q20 Compute the Manhattan and straight-line (Euclidean) distances from (2, 3) to (7, 11) and say which heuristic suits a 4-connected grid.

Q21 Odometry: the left wheel travels 30 cm and the right wheel 40 cm with 13 cm wheel separation. Find the distance moved and the change in heading.

Q22 Run Dijkstra on a small weighted graph of rooms (edge costs in metres) and print the cheapest route from A to E.

Q23 Parse a $GPRMC sentence and print the position and ground speed in km/h (speed is given in knots).

Q24 A warehouse robot must find its way with a known floor map and a 2-D LiDAR. Which combination fits?

(a) localise against the map (e.g. AMCL), plan with A* and avoid obstacles with a local planner
(b) GPS waypoints only
(c) odometry only, forever
(d) a colour sensor only

Q25 A reactive go-to-goal robot gets stuck in a U-shaped obstacle. What fixes this?

(a) higher speed
(b) a bigger battery
(c) a global planner such as A* over the map
(d) removing the obstacle-avoidance behaviour

18.6 Solutions

Q1 **Answer: (c) the robot's pose: position and heading**

Where am I?

Q2 **Answer: (d) adding up measured movements from a known start**

Odometry.

Q3 **Answer: (b) small errors from wheel slip and size add up without limit**

Unbounded error.

Q4 **Answer: (a) the bias makes the heading drift over time**

Drift.

Q5 **Answer: (b) whether it is free, occupied or unknown**

Map cells.

Q6 **Answer: (a) simultaneous localisation and mapping**

Both at once.

Q7 **Answer: (c) A***

Cost + heuristic.

Q8 **Answer: (d) $GPGGA**

Global positioning fix data.

Q9 **Answer: (d) 2–5 m**

RTK reaches cm level.

Q10 **Answer: (c) gyro is smooth short-term, accelerometer is drift-free long-term**

Best of both.

Q11 **Answer: (a) the measurement is trusted more than the prediction**

K near 1.

Q12 **Answer: (d) recognising a place visited before to correct accumulated drift**

Drift correction.

Q13 **Answer: (b) so the path keeps the robot's body clear of obstacles**

Robot radius.

Q14 **Answer: (b) Dijkstra**

Priority queue on cost.

Q15 **Answer: (d) when the heuristic never overestimates the true cost**

Admissible heuristic.

Q16 **Answer: (a) reacts to nearby obstacles seen by sensors while following the global path**

Short horizon.

Q17 **Answer: (c) robot_localization**

Extended Kalman filter.

Q18 **Answer: (d) about 1.02 cm**

π × 6.5 ÷ 20 = **1.02 cm**.

Q19 **Answer: (b) 60°**

0.5 × 120 s = **60°**.

Q20 **Answer: Solution program**

With only up/down/left/right moves, Manhattan distance equals the obstacle-free path length, so it is the tightest admissible heuristic. Euclidean is also admissible but weaker.

```
import math
a, b = (2, 3), (7, 11)
man = abs(a[0] - b[0]) + abs(a[1] - b[1])
euc = math.dist(a, b)
print("Manhattan:", man)
print("Euclidean:", round(euc, 2))
print("4-connected grid -> Manhattan (exact when no obstacles, never overestimates)")
```

OUTPUT

```
Manhattan: 13
Euclidean: 9.43
4-connected grid -> Manhattan (exact when no obstacles, never overestimates)
```

Q21 **Answer: Solution program**

The centre moves the average distance; the difference divided by the track width is the turn in radians. The right wheel went further, so the robot turned left.

```
import math
dl, dr, L = 30, 40, 13
d = (dl + dr) / 2
dth = (dr - dl) / L
print(f"distance {d} cm, heading change {dth:.3f} rad = {math.degrees(dth):.1f} deg (left turn)")
```

OUTPUT

```
distance 35.0 cm, heading change 0.769 rad = 44.1 deg (left turn)
```

Q22 **Answer: Solution program**

Dijkstra always expands the cheapest known node first, so A → C → B → D → E (2 + 1 + 5 + 2 = 10 m) beats the direct-looking A → C → E (12 m).

```
import heapq
G = {"A": {"B": 4, "C": 2}, "B": {"D": 5}, "C": {"B": 1, "D": 8, "E": 10}, "D": {"E": 2}, "E": {}}
dist, prev, pq = {"A": 0}, {}, [(0, "A")]
while pq:
    d, u = heapq.heappop(pq)
    if d > dist[u]: continue
    for v, w in G[u].items():
        if d + w < dist.get(v, 1e9):
            dist[v], prev[v] = d + w, u
            heapq.heappush(pq, (d + w, v))
path, n = ["E"], "E"
while n != "A": n = prev[n]; path.append(n)
print("route:", " -> ".join(reversed(path)), "| cost", dist["E"], "m")
```

OUTPUT

```
route: A -> C -> B -> D -> E | cost 10 m
```

Q23 **Answer: Solution program**

RMC adds speed over ground (knots × 1.852 = km/h) and course. Check the status field before trusting the position.

```
line = "$GPRMC,063015.00,A,1258.3721,N,07735.6552,E,3.20,45.0,151026,,,A*6B"
f = line.split(",")
def deg(v, h):
    v = float(v); d = int(v // 100); x = d + (v - d * 100) / 60
    return -x if h in "SW" else x
print("status:", "valid" if f[2] == "A" else "no fix")
print(f"lat {deg(f[3], f[4]):.5f}, lon {deg(f[5], f[6]):.5f}")
print(f"speed {float(f[7]) * 1.852:.2f} km/h, course {f[8]} deg")
```

OUTPUT

```
status: valid
lat 12.97287, lon 77.59425
speed 5.93 km/h, course 45.0 deg
```

Q24 **Answer: (a) localise against the map (e.g. AMCL), plan with A* and avoid obstacles with a local planner**

Indoors: map + LiDAR.

Q25 **Answer: (c) a global planner such as A* over the map**

Reactive behaviours have local minima.

Pose = (x, y, θ). Odometry: d = (dL + dR)/2, Δθ = (dR − dL)/L; drifts. Gyro drifts, accelerometer is noisy: complementary filter α ≈ 0.98. Kalman: predict then update, gain K = P/(P + R). GPS: NMEA GGA/RMC, haversine distance and bearing. Occupancy grid: free/occupied/unknown; SLAM with loop closure. Planning: BFS, Dijkstra (costs), A* (f = g + h, admissible h). Local planner + replanning. Behaviours: go-to-goal, wall following, blending and priorities.

MODULE 19 · ADVANCED

Robot Operating System (ROS 2)

Minimum duration **3 days** | Level **Advanced**

ROS 2 is the standard software framework for research and industrial robots. This module explains why ROS exists, how ROS 2 differs from ROS 1 and how to install it, then covers the core concepts (nodes, topics, messages, services, actions, parameters and launch files), the tools (ros2 command line, RViz, rqt and bag files), writing nodes in Python (rclpy) and C++ (rclcpp), describing robots with URDF and TF, the Nav2 and MoveIt 2 stacks and micro-ROS on microcontrollers, followed by 25 solved questions and programs.

IN THIS MODULE

19.1 Introduction to ROS 2
19.2 Core concepts
19.3 Tools
19.4 Writing ROS 2 programs
19.5 Robot description
19.6 Navigation and manipulation stacks (overview)
19.7 Microcontrollers with ROS 2
19.8 Practice questions
19.9 Solutions

19.1 Introduction to ROS 2

19.1.1 Why ROS

Every robot needs the same plumbing: drivers for sensors and motors, a way for programs to exchange data, coordinate frames, logging, visualisation, simulation, mapping and navigation. **ROS** (Robot Operating System) provides all of this as open-source libraries and tools. It is not an operating system; it runs on Linux (mainly Ubuntu), with Windows and macOS support. Thousands of ready packages mean you build on others' work: a LiDAR driver, SLAM and Nav2 can have a robot mapping a room on day one.

19.1.2 ROS 1 vs ROS 2

	ROS 1	ROS 2
Communication	custom TCP/UDP with a central roscore master	DDS middleware, no master: nodes discover each other
Real-time, reliability	limited	Quality-of-Service settings, real-time capable
Platforms	Ubuntu	Ubuntu, Windows, macOS, RTOS via micro-ROS
Security	none	SROS2 encryption and access control
Multi-robot	awkward	designed in (domains, namespaces)

	ROS 1	ROS 2
Status	last release Noetic, end of life May 2025	actively developed; use for all new work

19.1.3 Installing ROS 2 (Ubuntu or Docker)

Each ROS 2 release is tied to an Ubuntu version. **Jazzy Jalisco** (a long-term support release from May 2024, supported to 2029) runs on **Ubuntu 24.04** and is used in this module; check docs.ros.org for newer releases. Install from the official apt repository, or run it in Docker on any OS.

INSTALLING AND TESTING ROS 2

```
# Ubuntu 24.04: add the ROS 2 apt repository (see docs.ros.org for the current one-line setup),
then:
sudo apt update && sudo apt install ros-jazzy-desktop         # ROS, RViz, demos, turtlesim
echo "source /opt/ros/jazzy/setup.bash" >> ~/.bashrc
source ~/.bashrc
ros2 run demo_nodes_cpp talker          # terminal 1
ros2 run demo_nodes_py listener         # terminal 2

# Docker alternative (any OS with Docker):
docker run -it --rm --net=host osrf/ros:jazzy-desktop
```

19.2 Core concepts

19.2.1 Nodes

A **node** is a program with one job: a camera driver, a motor controller, a planner. A robot runs many nodes, possibly on different computers, which find each other automatically over the network when they share the same `ROS_DOMAIN_ID`.



NODES CONNECTED BY TOPICS (RQT_GRAPH SHOWS THE SAME PICTURE FOR A RUNNING SYSTEM)

19.2.2 Topics, publishers and subscribers

A **topic** is a named stream of messages. Any number of nodes can **publish** to it and any number can **subscribe**; they never call each other directly. Publishers and subscribers must agree on the message type and compatible **QoS** (quality of service): *reliable* for commands, *best effort* for high-rate sensor data where an old reading is useless anyway.

TALKER_LISTENER.PY

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import String

class Talker(Node):
    def __init__(self):
        super().__init__("talker")
        self.pub = self.create_publisher(String, "chatter", 10)     # topic name, queue depth
        self.n = 0
        self.create_timer(0.5, self.tick)                            # every 0.5 s
    def tick(self):
        self.n += 1
        msg = String(data=f"hello robot {self.n}")
        self.pub.publish(msg)
        self.get_logger().info(f"published: {msg.data}")

class Listener(Node):
    def __init__(self):
        super().__init__("listener")
        self.create_subscription(String, "chatter", self.on_msg, 10)
    def on_msg(self, msg):
        self.get_logger().info(f"heard: {msg.data}")

rclpy.init()
talker, listener = Talker(), Listener()
rclpy.spin(talker)          # normally each node runs in its own process with rclpy.spin(node)
```

OUTPUT

```
[INFO] [1773565200.500] [talker]: published: hello robot 1
[INFO] [1773565200.501] [listener]: heard: hello robot 1
[INFO] [1773565201.000] [talker]: published: hello robot 2
[INFO] [1773565201.001] [listener]: heard: hello robot 2
[INFO] [1773565201.500] [talker]: published: hello robot 3
[INFO] [1773565201.501] [listener]: heard: hello robot 3
```

19.2.3 Messages

Messages are typed structures defined in `.msg` files. Standard packages cover most needs; your own package can define custom ones.

Message type	Fields (main)	Typical topic
std_msgs/String, Float32, Bool	data	simple values
geometry_msgs/Twist	linear.x (m/s), angular.z (rad/s)	/cmd_vel
sensor_msgs/LaserScan	angle_min, angle_increment, ranges[]	/scan
sensor_msgs/Image	height, width, encoding, data[]	/image_raw
sensor_msgs/Imu	orientation, angular_velocity, linear_acceleration	/imu
nav_msgs/Odometry	pose, twist (with covariance)	/odom

Message type	Fields (main)	Typical topic
geometry_msgs/PoseStamped	header (frame, time), pose	goal poses

SHORTCUT TRICK
ROS uses SI units and right-handed frames everywhere: metres, radians, seconds; x forward, y left, z up. A Twist with angular.z = +0.5 turns the robot **left** at 0.5 rad/s.

19.2.4 Services and actions

	Topic	Service	Action
Pattern	stream, publish/subscribe	request → response	goal → feedback … → result
Duration	continuous	short (milliseconds)	long (seconds to minutes), cancellable
Example	/scan, /cmd_vel	/reset_odometry, /enable_motors	navigate_to_pose, follow_joint_trajectory

ENABLE_SERVICE.PY

```
import rclpy
from rclpy.node import Node
from std_srvs.srv import SetBool

class MotorBase(Node):
    def __init__(self):
        super().__init__("motor_base")
        self.enabled = False
        self.create_service(SetBool, "enable_motors", self.on_enable)
    def on_enable(self, req, res):
        self.enabled = req.data
        res.success, res.message = True, "motors ON" if req.data else "motors OFF"
        self.get_logger().info(f"request {req.data} -> {res.message}")
        return res

class Dashboard(Node):
    def __init__(self):
        super().__init__("dashboard")
        self.cli = self.create_client(SetBool, "enable_motors")
    def set_motors(self, on):
        if not self.cli.wait_for_service(timeout_sec=1.0):
            self.get_logger().error("service not available"); return
        fut = self.cli.call_async(SetBool.Request(data=on))
        rclpy.spin_until_future_complete(self, fut)
        r = fut.result()
        self.get_logger().info(f"reply: success={r.success}, message='{r.message}'")

rclpy.init()
base, dash = MotorBase(), Dashboard()
dash.set_motors(True)
dash.set_motors(False)
```

OUTPUT

```
[INFO] [1773565200.002] [motor_base]: request True -> motors ON
[INFO] [1773565200.002] [dashboard]: reply: success=True, message='motors ON'
[INFO] [1773565200.004] [motor_base]: request False -> motors OFF
[INFO] [1773565200.004] [dashboard]: reply: success=True, message='motors OFF'
```

A service waits for its reply, so it suits quick commands. For "drive to the kitchen", which takes a minute and should report progress and allow cancelling, ROS 2 uses an **action**:

SENDING AN ACTION GOAL (REFERENCE)

```
from rclpy.action import ActionClient
from nav2_msgs.action import NavigateToPose

client = ActionClient(node, NavigateToPose, "navigate_to_pose")
goal = NavigateToPose.Goal()
goal.pose.header.frame_id = "map"
goal.pose.pose.position.x, goal.pose.pose.position.y = 2.0, 1.5
send = client.send_goal_async(goal, feedback_callback=lambda fb:
        print("distance remaining:", fb.feedback.distance_remaining))
```

19.2.5 Parameters

Parameters are named settings inside a node (max speed, PID gains, frame names) that can be set at start-up from a YAML file or changed while running, so the same code fits different robots without recompiling.

CONFIG/DRIVER.YAML

```
turtle_driver:
  ros__parameters:
    max_speed: 1.5          # m/s
    kp_turn: 4.0
    goal_tolerance: 0.1
```

19.2.6 Launch files

A **launch file** starts a whole robot with one command: several nodes, their parameters, remappings and namespaces.

LAUNCH/TURTLE.LAUNCH.PY (RUN: ROS2 LAUNCH MY_ROBOT TURTLE.LAUNCH.PY)

```
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        Node(package="turtlesim", executable="turtlesim_node", name="sim"),
        Node(package="my_robot", executable="turtle_driver",
             parameters=["config/driver.yaml"],
             remappings=[("cmd_vel", "/turtle1/cmd_vel")]),
    ])
```

19.3 Tools

19.3.1 ros2 command-line tools

INSPECTING A RUNNING SYSTEM

```
$ ros2 node list
/sim
/turtle_driver
$ ros2 topic list
/parameter_events
/rosout
/turtle1/cmd_vel
/turtle1/pose
$ ros2 topic hz /turtle1/pose
average rate: 62.502
$ ros2 topic echo /turtle1/pose --once
x: 5.544444561004639
y: 5.544444561004639
theta: 0.0
linear_velocity: 0.0
angular_velocity: 0.0
---
$ ros2 topic pub --once /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 2.0}, angular: {z:
1.8}}"
$ ros2 param get /turtle_driver max_speed
Double value is: 1.5
$ ros2 service call /enable_motors std_srvs/srv/SetBool "{data: true}"
```

Command	Purpose
ros2 run pkg exe	start one node
ros2 launch pkg file	start a launch file
ros2 node list / info	running nodes and their topics
ros2 topic list / echo / hz / pub	see and send messages
ros2 service list / call	call services
ros2 param list / get / set	read and change parameters
ros2 interface show	show a message definition
ros2 doctor	check the installation and network

19.3.2 RViz for visualisation

RViz 2 shows what the robot knows in 3-D: its model, coordinate frames, laser scans, point clouds, camera images, maps, planned paths and goals. Add displays (RobotModel, TF, LaserScan, Map, Path), set the fixed frame (usually `map` or `odom`), and click "2D Goal Pose" to send navigation goals.

19.3.3 rqt tools

Plug-in	Shows
rqt_graph	nodes and topics as a live diagram
rqt_plot	live graphs of message fields (great for PID tuning)
rqt_image_view	camera topics
rqt_console	filter and search log messages
rqt_reconfigure	sliders for parameters
rqt_robot_steering	a joystick-like /cmd_vel publisher

19.3.4 Bag files for recording data

RECORDING AND REPLAYING WITH ROS2 BAG

```
ros2 bag record /scan /odom /imu -o corridor_run     # record chosen topics (or -a for all)
ros2 bag info corridor_run                           # duration, topics, message counts
ros2 bag play corridor_run --rate 0.5                # replay at half speed
```

Bags let you drive a corridor once and then test SLAM or detection code on the same data again and again at your desk.

19.4 Writing ROS 2 programs

19.4.1 Python (rclpy) and C++ (rclcpp) nodes

The example below drives a turtle to a goal: it subscribes to the pose, computes the heading error and publishes velocity commands, using a parameter for the top speed. A small helper node, `sim_turtle.py`, stands in for `turtlesim_node` so the output can be shown here; with ROS 2 installed, run the real turtlesim instead.

TURTLE_DRIVER.PY

```
import math
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist
from turtlesim.msg import Pose
from sim_turtle import SimTurtle

class TurtleDriver(Node):
    def __init__(self, goal):
        super().__init__("turtle_driver")
        self.max_speed = self.declare_parameter("max_speed", 1.5).value
        self.goal, self.pose, self.done = goal, None, False
        self.pub = self.create_publisher(Twist, "/turtle1/cmd_vel", 10)
        self.create_subscription(Pose, "/turtle1/pose", self.on_pose, 10)
        self.create_timer(0.1, self.control)
        self.ticks = 0
    def on_pose(self, msg):
        self.pose = msg
    def control(self):
        if self.pose is None or self.done:
            return
        dx, dy = self.goal[0] - self.pose.x, self.goal[1] - self.pose.y
        dist = math.hypot(dx, dy)
        err = math.atan2(dy, dx) - self.pose.theta
        err = math.atan2(math.sin(err), math.cos(err))           # wrap to -pi..pi
        cmd = Twist()
        if dist < 0.1:
            self.done = True
            self.get_logger().info(f"goal reached at ({self.pose.x:.2f}, {self.pose.y:.2f})")
        else:
            cmd.angular.z = 4.0 * err
            cmd.linear.x = min(self.max_speed, 1.0 * dist) * max(0.0, math.cos(err))
        self.pub.publish(cmd)
        self.ticks += 1
        if self.ticks % 10 == 0 and not self.done:
            self.get_logger().info(f"pose ({self.pose.x:.2f}, {self.pose.y:.2f})  dist {dist:.2f}  v
{cmd.linear.x:.2f}  w {cmd.angular.z:+.2f}")

rclpy.init()
sim, driver = SimTurtle(), TurtleDriver(goal=(8.0, 9.0))
rclpy.spin(driver)
```

OUTPUT

```
[INFO] [1773565201.000] [turtle_driver]: pose (6.27, 6.41)  dist 3.12  v 1.50  w -0.01
[INFO] [1773565202.000] [turtle_driver]: pose (7.10, 7.66)  dist 1.62  v 1.50  w +0.00
[INFO] [1773565203.000] [turtle_driver]: pose (7.70, 8.55)  dist 0.54  v 0.54  w -0.00
[INFO] [1773565204.000] [turtle_driver]: pose (7.90, 8.85)  dist 0.18  v 0.18  w +0.00
[INFO] [1773565204.600] [turtle_driver]: goal reached at (7.95, 8.93)
```

The turtle first turns toward the goal (forward speed is scaled by cos(error), so it barely moves while facing away), then drives in, slowing as the distance shrinks. Exactly the same node drives a real TurtleBot by remapping the topics to `/cmd_vel` and `/odom`.

SPINNER.CPP: THE SAME PATTERN IN C++ (RCLCPP)

```
#include <memory>
#include "rclcpp/rclcpp.hpp"
#include "geometry_msgs/msg/twist.hpp"
using namespace std::chrono_literals;

class Spinner : public rclcpp::Node {
public:
  Spinner() : Node("spinner") {
    pub_ = create_publisher<geometry_msgs::msg::Twist>("/turtle1/cmd_vel", 10);
    timer_ = create_wall_timer(100ms, [this]() {
      geometry_msgs::msg::Twist cmd;
      cmd.linear.x = 1.0; cmd.angular.z = 1.0;          // drive in a circle
      pub_->publish(cmd);
    });
  }
private:
  rclcpp::Publisher<geometry_msgs::msg::Twist>::SharedPtr pub_;
  rclcpp::TimerBase::SharedPtr timer_;
};

int main(int argc, char **argv) {
  rclcpp::init(argc, argv);
  rclcpp::spin(std::make_shared<Spinner>());
  rclcpp::shutdown();
}
```

Python is quicker to write; C++ is faster and used for drivers and heavy processing. Both live in **packages** inside a workspace, built with **colcon**:

CREATING, BUILDING AND RUNNING A PACKAGE

```
mkdir -p ~/ros2_ws/src && cd ~/ros2_ws/src
ros2 pkg create my_robot --build-type ament_python --dependencies rclpy geometry_msgs turtlesim
#   put turtle_driver.py in my_robot/my_robot/ and add an entry point in setup.py:
#   'turtle_driver = my_robot.turtle_driver:main'
cd ~/ros2_ws && colcon build --symlink-install
source install/setup.bash
ros2 run my_robot turtle_driver
```

19.5 Robot description

19.5.1 URDF robot models

URDF (Unified Robot Description Format) is XML describing the robot's **links** (rigid parts with shape, colour, mass) and **joints** (how links connect: fixed, continuous, revolute, prismatic). RViz draws the model, Gazebo simulates it and TF uses it for coordinate frames. Xacro adds variables and macros to keep URDF short.

URDF/MINI_BOT.URDF

```
<?xml version="1.0"?>
<robot name="mini_bot">
  <link name="base_link">
    <visual><geometry><box size="0.20 0.15 0.05"/></geometry></visual>
  </link>
  <link name="left_wheel">
    <visual><geometry><cylinder radius="0.033" length="0.025"/></geometry></visual>
  </link>
  <link name="right_wheel">
    <visual><geometry><cylinder radius="0.033" length="0.025"/></geometry></visual>
  </link>
  <link name="laser">
    <visual><geometry><cylinder radius="0.03" length="0.04"/></geometry></visual>
  </link>
  <joint name="left_wheel_joint" type="continuous">
    <parent link="base_link"/><child link="left_wheel"/>
    <origin xyz="0 0.09 -0.01" rpy="-1.5708 0 0"/><axis xyz="0 0 1"/>
  </joint>
  <joint name="right_wheel_joint" type="continuous">
    <parent link="base_link"/><child link="right_wheel"/>
    <origin xyz="0 -0.09 -0.01" rpy="-1.5708 0 0"/><axis xyz="0 0 1"/>
  </joint>
  <joint name="laser_joint" type="fixed">
    <parent link="base_link"/><child link="laser"/>
    <origin xyz="0.06 0 0.05" rpy="0 0 0"/>
  </joint>
</robot>
```

INSPECT_URDF.PY

```
import xml.etree.ElementTree as ET
robot = ET.parse("mini_bot.urdf").getroot()
print("robot:", robot.get("name"))
print("links:", [l.get("name") for l in robot.findall("link")])
for j in robot.findall("joint"):
    o = j.find("origin").get("xyz")
    print(f"  {j.get('name'):18} {j.get('type'):10} {j.find('parent').get('link')} ->
{j.find('child').get('link'):11} at xyz=({o})")
```

OUTPUT

```
robot: mini_bot
links: ['base_link', 'left_wheel', 'right_wheel', 'laser']
  left_wheel_joint   continuous base_link -> left_wheel  at xyz=(0 0.09 -0.01)
  right_wheel_joint  continuous base_link -> right_wheel at xyz=(0 -0.09 -0.01)
  laser_joint        fixed      base_link -> laser       at xyz=(0.06 0 0.05)
```

VIEWING THE MODEL IN RVIZ

```
ros2 launch urdf_tutorial display.launch.py model:=$PWD/urdf/mini_bot.urdf     # model + joint
sliders in RViz
```

19.5.2 TF coordinate transforms

Every sensor measures in its own **frame**. **TF** keeps the tree of transforms between frames over time, so any node can ask "where is this laser point in the map?". Standard frames (REP 105): `map` → `odom` → `base_link` → sensor frames such as `laser`. Fixed joints in the URDF are published by `robot_state_publisher`; odometry publishes odom → base_link; SLAM or AMCL publishes map → odom.

TF_CHAIN.PY

```
import numpy as np

def tf(x, y, yaw_deg):                      # 2-D homogeneous transform
    c, s = np.cos(np.radians(yaw_deg)), np.sin(np.radians(yaw_deg))
    return np.array([[c, -s, x], [s, c, y], [0, 0, 1]])

odom_T_base = tf(2.0, 1.0, 90)              # robot at (2, 1) in odom, facing +y
base_T_laser = tf(0.06, 0.0, 0)             # laser 6 cm ahead of the centre (from the URDF)
p_laser = np.array([0.50, 0.0, 1])          # obstacle 0.5 m straight ahead of the laser

p_base = base_T_laser @ p_laser
p_odom = odom_T_base @ base_T_laser @ p_laser
print("in laser frame    :", p_laser[:2])
print("in base_link frame:", np.round(p_base[:2], 3))
print("in odom frame     :", np.round(p_odom[:2], 3))
```

OUTPUT

```
in laser frame    : [0.5 0. ]
in base_link frame: [0.56 0.  ]
in odom frame     : [2.   1.56]
```

Chaining transforms by matrix multiplication is exactly what TF does (in 3-D, with quaternions for rotation). In code you just call `tf_buffer.lookup_transform("odom", "laser", time)`; on the command line, `ros2 run tf2_ros tf2_echo odom laser`.

19.6 Navigation and manipulation stacks (overview)

19.6.1 Nav2 and MoveIt 2

	Nav2 (mobile robots)	MoveIt 2 (arms)
Goal	drive to a pose on a map	move the gripper to a pose without collisions
Inputs	map, /scan, /odom, TF	URDF + SRDF, joint states, 3-D scene (point clouds)
Main parts	AMCL localisation, global planner, local controller, costmaps, behaviour trees, recoveries	inverse kinematics, motion planners (OMPL), collision checking, trajectory execution
Output	/cmd_vel	joint trajectories to the arm controller
Try it	TurtleBot3/4 simulation + nav2_bringup	MoveIt Setup Assistant + the Panda arm demo

SENDING A NAV2 GOAL FROM PYTHON (REFERENCE)

```
from nav2_simple_commander.robot_navigator import BasicNavigator
from geometry_msgs.msg import PoseStamped

nav = BasicNavigator()
nav.waitUntilNav2Active()
goal = PoseStamped(); goal.header.frame_id = "map"
goal.pose.position.x, goal.pose.position.y = 3.0, 1.2
goal.pose.orientation.w = 1.0
nav.goToPose(goal)
while not nav.isTaskComplete():
    fb = nav.getFeedback()          # distance remaining, recoveries, time
print(nav.getResult())
```

19.7 Microcontrollers with ROS 2

19.7.1 micro-ROS (overview)

micro-ROS runs a small ROS 2 client on microcontrollers (ESP32, Raspberry Pi Pico, Teensy, STM32). The board publishes and subscribes to real ROS 2 topics through a **micro-ROS agent** running on the Pi or PC, over serial, Wi-Fi (UDP) or Ethernet. The microcontroller handles real-time motor and encoder control; the Pi runs SLAM and Nav2.

ESP32 SUBSCRIBING TO /CMD_VEL WITH MICRO-ROS (REFERENCE)

```
#include <micro_ros_arduino.h>
#include <rcl/rcl.h>
#include <rclc/rclc.h>
#include <rclc/executor.h>
#include <geometry_msgs/msg/twist.h>

rcl_subscription_t sub; geometry_msgs__msg__Twist cmd;
rclc_executor_t executor; rclc_support_t support; rcl_node_t node;

void on_cmd(const void *m) {
  const geometry_msgs__msg__Twist *t = (const geometry_msgs__msg__Twist *)m;
  float v = t->linear.x, w = t->angular.z;
  setWheels(v - w * 0.065, v + w * 0.065);      // left/right wheel speeds (m/s)
}
void setup() {
  set_microros_wifi_transports("ssid", "password", "192.168.1.20", 8888);   // agent address
  rcl_allocator_t a = rcl_get_default_allocator();
  rclc_support_init(&support, 0, NULL, &a);
  rclc_node_init_default(&node, "esp32_base", "", &support);
  rclc_subscription_init_default(&sub, &node, ROSIDL_GET_MSG_TYPE_SUPPORT(geometry_msgs, msg,
Twist), "cmd_vel");
  rclc_executor_init(&executor, &support.context, 1, &a);
  rclc_executor_add_subscription(&executor, &sub, &cmd, &on_cmd, ON_NEW_DATA);
}
void loop() { rclc_executor_spin_some(&executor, RCL_MS_TO_NS(10)); }
```

STARTING THE AGENT

```
ros2 run micro_ros_agent micro_ros_agent udp4 --port 8888      # on the Pi or PC
```

PRACTICE CHECKPOINT

Write publisher and subscriber nodes, control a simulated TurtleBot with velocity commands, and visualise a simple URDF robot model in RViz.

19.8 Practice questions

19.8.1 Basic level

Q1 What is ROS 2?

(a) a framework of libraries and tools for building robot software
(b) an operating system kernel
(c) a microcontroller
(d) a robot kit

Q2 What is a ROS 2 node?

(a) a cable
(b) a program that does one job and communicates with other nodes
(c) a sensor type
(d) a map cell

Q3 Which communication pattern does a topic use?

(a) request/response
(b) goal/feedback/result
(c) shared memory only
(d) publish/subscribe

Q4 Which message type is used on /cmd_vel?

(a) sensor_msgs/LaserScan
(b) std_msgs/String
(c) geometry_msgs/Twist
(d) nav_msgs/Path

Q5 Which tool visualises robot models, scans and maps in 3-D?

(a) colcon **(b)** rqt_console **(c)** RViz **(d)** ros2 bag

Q6 Which command records topics to a file?

(a) ros2 topic echo
(b) ros2 run
(c) ros2 doctor
(d) ros2 bag record

Q7 What does URDF describe?

(a) network settings
(b) a robot's links and joints
(c) a map
(d) Python packages

Q8 Which ROS 1 feature was removed in ROS 2?

(a) the central roscore master
(b) topics
(c) messages
(d) launch files

19.8.2 Intermediate level

Q9 Which pattern should "drive to the kitchen with progress updates" use?

(a) a topic **(b)** an action **(c)** a service **(d)** a parameter

Q10 Which pattern suits a quick "enable motors" command with a reply?

(a) a service **(b)** an action **(c)** a bag file **(d)** a TF frame

Q11 What are ROS 2 parameters used for?

(a) streaming sensor data
(b) drawing maps
(c) settings of a node that can be changed without recompiling
(d) building packages

Q12 What does a launch file do?

(a) compiles C++
(b) records data
(c) installs ROS
(d) starts several nodes with their parameters and remappings

Q13 Which tool builds a ROS 2 workspace?

(a) catkin_make only
(b) pip
(c) rviz2
(d) colcon

Q14 What does TF manage?

(a) Wi-Fi traffic
(b) battery charging
(c) transforms between coordinate frames over time
(d) message types

Q15 What is the standard frame chain for a mobile robot (REP 105)?

(a) map → odom → base_link → sensor frames
(b) base_link → map → odom
(c) laser → map → odom
(d) world → laser only

Q16 Which stack plans and executes collision-free arm motions?

(a) Nav2
(b) slam_toolbox
(c) turtlesim
(d) MoveIt 2

Q17 How does an ESP32 join a ROS 2 system as a node?

(a) it cannot
(b) micro-ROS, talking to a micro-ROS agent on a Pi or PC
(c) by running full ROS 2 Jazzy
(d) through rqt only

19.8.3 Advanced level

Q18 A Twist has linear.x = 0.2 and angular.z = 0.5. What does a differential robot do?

(a) turns right on the spot
(b) drives forward at 0.2 m/s while turning left at 0.5 rad/s
(c) reverses
(d) stops

Q19 For v = 0.2 m/s and ω = 0.5 rad/s with wheel separation 0.13 m, what is the right wheel speed?

(a) 0.1675 m/s
(b) 0.2 m/s
(c) 0.265 m/s
(d) 0.2325 m/s

Q20 Convert linear and angular velocity commands (v, ω) into left and right wheel speeds for a robot with 0.13 m wheel separation.

Q21 Write an rclpy node that publishes a Float32 battery voltage every second, falling by 0.05 V each time, and logs a warning below 11.0 V.

Q22 Write an AddTwoInts service and call it from a client node for 7 + 5.

Q23 A laser point is 1.0 m ahead of the laser. The laser is 0.1 m ahead of base_link, and the robot is at (3, 2) facing 180° in the map. Where is the point in the map frame?

Q24 Two robots on the same Wi-Fi see each other's topics and get mixed up. What is the simplest fix?

(a) give each robot a different ROS_DOMAIN_ID (or namespace)
(b) use ROS 1 instead
(c) turn off DDS
(d) rename the laptop

Q25 A subscriber to a camera topic receives nothing although the topic exists. The publisher uses best-effort QoS and the subscriber reliable. What is wrong?

(a) the message type is too big
(b) RViz is not open
(c) incompatible QoS settings
(d) colcon was not run

19.9 Solutions

Q1 **Answer: (a) a framework of libraries and tools for building robot software**
Middleware + tools.

Q2 **Answer: (b) a program that does one job and communicates with other nodes**
Building block.

Q3 **Answer: (d) publish/subscribe**
Many-to-many streams.

Q4 **Answer: (c) geometry_msgs/Twist**
linear + angular velocity.

Q5 **Answer: (c) RViz**
Visualisation.

Q6 **Answer: (d) ros2 bag record**
Bag files.

Q7 **Answer: (b) a robot's links and joints**
Robot model XML.

Q8 **Answer: (a) the central roscore master**
DDS discovery.

Q9 **Answer: (b) an action**

Long, with feedback and cancel.

Q10 **Answer: (a) a service**

Request/response.

Q11 **Answer: (c) settings of a node that can be changed without recompiling**

Configuration.

Q12 **Answer: (d) starts several nodes with their parameters and remappings**

One command.

Q13 **Answer: (d) colcon**

colcon build.

Q14 **Answer: (c) transforms between coordinate frames over time**

Frame tree.

Q15 **Answer: (a) map → odom → base_link → sensor frames**

Global to local.

Q16 **Answer: (d) MoveIt 2**

Manipulation.

Q17 **Answer: (b) micro-ROS, talking to a micro-ROS agent on a Pi or PC**

Agent bridge.

Q18 **Answer: (b) drives forward at 0.2 m/s while turning left at 0.5 rad/s**

Positive z = counter-clockwise = left; radius 0.2/0.5 = 0.4 m.

Q19 **Answer: (d) 0.2325 m/s**

$v_R = v + \omega \cdot L/2 = 0.2 + 0.5 \times 0.065 =$ **0.2325 m/s**.

Q20 **Answer: Solution program**

This is what every differential-drive base node does with each /cmd_vel message: the inverse of the odometry formulas from module 18.

```
L = 0.13
for v, w in [(0.2, 0.0), (0.2, 0.5), (0.0, 1.0), (-0.1, 0.0)]:
    vl, vr = v - w * L / 2, v + w * L / 2
    print(f"v={v:+.2f} m/s  w={w:+.2f} rad/s  ->  left {vl:+.4f}  right {vr:+.4f} m/s")
```

OUTPUT

```
v=+0.20 m/s  w=+0.00 rad/s  ->  left +0.2000  right +0.2000 m/s
v=+0.20 m/s  w=+0.50 rad/s  ->  left +0.1675  right +0.2325 m/s
v=+0.00 m/s  w=+1.00 rad/s  ->  left -0.0650  right +0.0650 m/s
v=-0.10 m/s  w=+0.00 rad/s  ->  left -0.1000  right -0.1000 m/s
```

Q21 **Answer: Solution program**

A timer callback publishes and logs; other nodes (a dashboard, a safety node that sends the robot home) can subscribe to battery_voltage.

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import Float32

class Battery(Node):
    def __init__(self):
        super().__init__("battery")
        self.v = 11.2
        self.pub = self.create_publisher(Float32, "battery_voltage", 10)
        self.create_timer(1.0, self.tick)
    def tick(self):
        self.pub.publish(Float32(data=self.v))
        if self.v < 11.0:
            self.get_logger().warn(f"low battery: {self.v:.2f} V")
        else:
            self.get_logger().info(f"battery {self.v:.2f} V")
        self.v = round(self.v - 0.05, 2)

rclpy.init()
rclpy.spin(Battery())
```

OUTPUT

```
[INFO] [1773565201.000] [battery]: battery 11.20 V
[INFO] [1773565202.000] [battery]: battery 11.15 V
[INFO] [1773565203.000] [battery]: battery 11.10 V
[INFO] [1773565204.000] [battery]: battery 11.05 V
[INFO] [1773565205.000] [battery]: battery 11.00 V
[WARN] [1773565206.000] [battery]: low battery: 10.95 V
```

Q22 **Answer: Solution program**

The standard first service example: the server fills the response object; the client waits on a future for the reply.

```
import rclpy
from rclpy.node import Node
from example_interfaces.srv import AddTwoInts

class Adder(Node):
    def __init__(self):
        super().__init__("adder")
        self.create_service(AddTwoInts, "add_two_ints", self.add)
    def add(self, req, res):
        res.sum = req.a + req.b
        self.get_logger().info(f"{req.a} + {req.b} = {res.sum}")
        return res

rclpy.init()
server = Adder()
client_node = Node("client")
cli = client_node.create_client(AddTwoInts, "add_two_ints")
cli.wait_for_service()
fut = cli.call_async(AddTwoInts.Request(a=7, b=5))
rclpy.spin_until_future_complete(client_node, fut)
client_node.get_logger().info(f"result: {fut.result().sum}")
```

OUTPUT

```
[INFO] [1773565200.002] [adder]: 7 + 5 = 12
[INFO] [1773565200.002] [client]: result: 12
```

Q23 **Answer: Solution program**

The point is 1.1 m in front of the robot centre; facing 180° means "in front" is -x, so it lies at (3 - 1.1, 2) = (1.9, 2).

```
import numpy as np
def tf(x, y, yaw):
    c, s = np.cos(np.radians(yaw)), np.sin(np.radians(yaw))
    return np.array([[c, -s, x], [s, c, y], [0, 0, 1]])
p = tf(3, 2, 180) @ tf(0.1, 0, 0) @ np.array([1.0, 0, 1])
print("point in map:", np.round(p[:2], 3))
```

OUTPUT

```
point in map: [1.9 2. ]
```

Q24 **Answer: (a) give each robot a different ROS_DOMAIN_ID (or namespace)**

Isolate discovery.

Q25 **Answer: (c) incompatible QoS settings**

QoS must be compatible.

ROS 2 = DDS-based framework (no master) on Ubuntu (Jazzy on 24.04). Nodes talk via topics (pub/sub, QoS), services (request/response) and actions (goal/feedback/result); parameters and launch files configure them. Tools: ros2 CLI, RViz, rqt, ros2 bag. Nodes in rclpy/rclcpp, packages built with colcon. URDF links/joints; TF frames map → odom → base_link → sensors. Nav2 for navigation, MoveIt 2 for arms, micro-ROS for microcontrollers.

MODULE 20 · ADVANCED

Robot simulation

Minimum duration **3 days** | Level **Advanced**

Simulation lets you test robots safely and cheaply before building them. This module explains why roboticists simulate, uses Tinkercad Circuits and Wokwi for microcontroller projects, compares the robot simulators Gazebo, Webots, CoppeliaSim and NVIDIA Isaac Sim, shows how worlds, robots and simulated sensors (cameras, LiDAR, IMU) are described and connected to ROS 2, builds a small Python simulator of a LiDAR robot avoiding obstacles, and examines the gap between simulation and reality, followed by 25 solved questions and programs.

IN THIS MODULE

20.1 Why simulate
20.2 Circuit simulators
20.3 Robot simulators
20.4 Simulating robots
20.5 Simulation vs reality gap
20.6 Practice questions
20.7 Solutions

20.1 Why simulate

Benefit	Example
Safety	test a 50 kg robot's emergency stop or a drone's failsafe without risk
Cost	no broken parts or burnt drivers while learning
Speed	run hundreds of trials faster than real time, overnight, in parallel
Repeatability	the same start, same obstacles, every run: fair comparison of two algorithms
Before hardware exists	choose motors, sensor positions and wheel size before buying
Data and AI	generate labelled images or train reinforcement-learning policies
Teamwork	software and hardware teams work in parallel

NOTE

A simulator is a model, not the world. Use it to find bugs early and to compare ideas, then confirm on the real robot (section 20.5).

20.2 Circuit simulators

20.2.1 Tinkercad Circuits and Wokwi for microcontroller projects

	Tinkercad Circuits	Wokwi
Runs	in the browser (free Autodesk account)	in the browser, or in VS Code
Boards	Arduino Uno, micro:bit	Uno, Mega, Nano, ESP32 (with Wi-Fi), Raspberry Pi Pico, STM32
Parts	LEDs, motors, servos, sensors, breadboard view, multimeter, oscilloscope	large library: OLEDs, NeoPixels, servos, sensors, logic analyser
Strength	beginner-friendly, block coding, circuit diagrams	fast, ESP32 Wi-Fi/MQTT, MicroPython, shareable links
Limits	few boards, no Wi-Fi	no real motor physics or mechanics

Wokwi stores the wiring in a `diagram.json` file next to the sketch, so a project can be shared or put under version control:

DIAGRAM.JSON (WOKWI): AN ULTRASONIC SENSOR AND A SERVO ON AN UNO

```
{
  "version": 1,
  "parts": [
    { "type": "wokwi-arduino-uno", "id": "uno" },
    { "type": "wokwi-hc-sr04", "id": "sonar", "left": 200, "top": -60 },
    { "type": "wokwi-servo", "id": "servo", "left": 200, "top": 80 }
  ],
  "connections": [
    [ "sonar:TRIG", "uno:9", "green", [] ],
    [ "sonar:ECHO", "uno:10", "blue", [] ],
    [ "sonar:VCC", "uno:5V", "red", [] ],
    [ "sonar:GND", "uno:GND.1", "black", [] ],
    [ "servo:PWM", "uno:6", "orange", [] ],
    [ "servo:V+", "uno:5V", "red", [] ],
    [ "servo:GND", "uno:GND.2", "black", [] ]
  ]
}
```

SHORTCUT TRICK
Simulate the electronics and code first: wiring mistakes, wrong pins and logic bugs show up in seconds, and the same sketch then runs on the real board unchanged.

20.3 Robot simulators

A robot simulator combines a **physics engine** (gravity, friction, collisions, motor torque), **sensor models** (cameras, LiDAR, IMU, GPS) and a **3-D renderer**, and lets your control code run as if it were on the real robot.

Simulator	Strengths	Typical use	Cost
Gazebo (gz-sim; Harmonic pairs with ROS 2 Jazzy)	native ROS 2 integration, many robot models, plug-ins	ROS 2 mobile robots, Nav2, TurtleBot, drones (PX4/ArduPilot)	free, open source
Webots	easy installation, many ready robots (e-puck, TurtleBot, Pioneer, NAO), Python/C/C++ controllers	teaching, competitions, quick experiments; ROS 2 interface available	free, open source
CoppeliaSim	very flexible scene editor, several physics engines, Lua/Python scripting	manipulators, research prototypes	free education licence
NVIDIA Isaac Sim	photorealistic rendering on the Omniverse platform, GPU physics, synthetic data, reinforcement learning (Isaac Lab)	AI training, perception, industrial and humanoid robots	free to use; needs a powerful NVIDIA RTX GPU

20.3.1 Gazebo

Gazebo is the usual partner of ROS 2. Worlds and models are written in **SDF** (Simulation Description Format) or converted from URDF; plug-ins provide sensors and a differential-drive controller; **ros_gz_bridge** passes messages between Gazebo topics and ROS 2 topics. (The older "Gazebo Classic" reached end of life in January 2025.)

20.3.2 Webots

Webots runs each robot's **controller** as a normal program that steps the simulation. A complete obstacle avoider for the e-puck robot is only a few lines:

WEBOTS E-PUCK CONTROLLER (PYTHON, REFERENCE)

```
from controller import Robot

robot = Robot()
step = int(robot.getBasicTimeStep())                    # e.g. 32 ms
ps = [robot.getDevice(f"ps{i}") for i in range(8)]      # 8 infrared proximity sensors
for s in ps: s.enable(step)
left, right = robot.getDevice("left wheel motor"), robot.getDevice("right wheel motor")
for m in (left, right):
    m.setPosition(float("inf"))                         # velocity control mode
    m.setVelocity(0.0)

while robot.step(step) != -1:                           # advance the simulation one step
    v = [s.getValue() for s in ps]
    right_obstacle = v[0] > 80 or v[1] > 80 or v[2] > 80
    left_obstacle = v[5] > 80 or v[6] > 80 or v[7] > 80
    l, r = 3.0, 3.0                                     # rad/s
    if left_obstacle:   l, r = 3.0, -3.0                # turn right
    elif right_obstacle: l, r = -3.0, 3.0               # turn left
    left.setVelocity(l); right.setVelocity(r)
```

20.3.3 CoppeliaSim (overview)

CoppeliaSim (formerly V-REP) builds scenes from objects with attached scripts, offers inverse kinematics, path planning and four physics engines (Bullet, ODE, Vortex, Newton), and can be driven from Python through its remote API. It is popular for arm and gripper research and teaching.

20.3.4 NVIDIA Isaac Sim (overview)

Isaac Sim renders physically accurate light and materials, so camera images look real enough to train neural networks on **synthetic data** with automatic labels. Isaac Lab trains reinforcement-learning policies for thousands of robots in parallel on one GPU, which is how many modern legged and humanoid robots learn to walk.

20.4 Simulating robots

20.4.1 Creating worlds and robots

A **world** contains the ground, lighting, walls and objects; a **model** is the robot (links with visual shape, collision shape and inertia, joints, sensors and plug-ins). Keep collision shapes simple (boxes, cylinders) for fast, stable physics, and give every link a realistic mass and inertia.

WORLDS/ROOM.SDF (GAZEBO): GROUND, ONE BOX AND THE ROBOT

```
<?xml version="1.0"?>
<sdf version="1.9">
  <world name="room">
    <physics name="1ms" type="ignored"><max_step_size>0.001</max_step_size></physics>
    <plugin filename="gz-sim-physics-system" name="gz::sim::systems::Physics"/>
    <plugin filename="gz-sim-sensors-system" name="gz::sim::systems::Sensors">
      <render_engine>ogre2</render_engine>
    </plugin>
    <light type="directional" name="sun"><direction>-0.5 0.1 -0.9</direction></light>
    <model name="ground"><static>true</static>
      <link name="l"><collision name="c"><geometry><plane><normal>0 0 1</normal></plane></geometry>
</collision>
        <visual name="v"><geometry><plane><normal>0 0 1</normal><size>10 10</size></plane>
</geometry></visual></link>
    </model>
    <model name="box_obstacle"><static>true</static><pose>1.5 0.3 0.25 0 0 0</pose>
      <link name="l"><collision name="c"><geometry><box><size>0.5 0.5 0.5</size></box></geometry>
</collision>
        <visual name="v"><geometry><box><size>0.5 0.5 0.5</size></box></geometry></visual></link>
    </model>
    <include><uri>model://mini_bot</uri><pose>0 0 0.05 0 0 0</pose></include>
  </world>
</sdf>
```

20.4.2 Simulated sensors (cameras, LiDAR, IMU)

Sensors are added to the robot model with an update rate, range and **noise**. The differential-drive plug-in turns /cmd_vel into wheel motion and publishes odometry.

LIDAR, IMU AND DIFFERENTIAL-DRIVE PLUG-INS (GAZEBO)

```
<!-- inside the robot model (URDF <gazebo> tags or SDF) -->
<sensor name="lidar" type="gpu_lidar">
  <pose>0.06 0 0.05 0 0 0</pose>
  <topic>scan</topic>
  <update_rate>10</update_rate>
  <lidar>
    <scan><horizontal><samples>360</samples><min_angle>-3.14159</min_angle>
<max_angle>3.14159</max_angle></horizontal></scan>
    <range><min>0.12</min><max>8.0</max></range>
    <noise><type>gaussian</type><mean>0</mean><stddev>0.01</stddev></noise>
  </lidar>
</sensor>
<sensor name="imu" type="imu"><topic>imu</topic><update_rate>100</update_rate></sensor>
<plugin filename="gz-sim-diff-drive-system" name="gz::sim::systems::DiffDrive">
  <left_joint>left_wheel_joint</left_joint>
  <right_joint>right_wheel_joint</right_joint>
  <wheel_separation>0.18</wheel_separation>
  <wheel_radius>0.033</wheel_radius>
  <topic>cmd_vel</topic>
  <odom_topic>odom</odom_topic>
</plugin>
```

20.4.3 Connecting simulation with ROS 2

BRIDGING GAZEBO TOPICS TO ROS 2

```
sudo apt install ros-jazzy-ros-gz                       # Gazebo + ROS 2 integration
gz sim worlds/room.sdf                                   # start the world
ros2 run ros_gz_bridge parameter_bridge \
  /cmd_vel@geometry_msgs/msg/Twist]gz.msgs.Twist \
  /scan@sensor_msgs/msg/LaserScan[gz.msgs.LaserScan \
  /odom@nav_msgs/msg/Odometry[gz.msgs.Odometry
ros2 topic echo /scan --once                             # simulated LiDAR data in ROS 2
ros2 run teleop_twist_keyboard teleop_twist_keyboard     # drive with the keyboard
```

In the bridge syntax, `]` sends ROS → Gazebo, `[` sends Gazebo → ROS and `@` both ways. Once bridged, your ROS 2 nodes (the obstacle avoider, SLAM, Nav2) cannot tell the simulated robot from the real one. Set `use_sim_time:=true` so nodes follow the simulator clock.

20.4.4 A tiny simulator in Python

The idea behind every simulator fits in one page: a world, a robot model updated in small time steps, and sensors computed from the world. This one simulates a differential-drive robot with a 9-beam LiDAR avoiding obstacles in a room.

MINI_SIM.PY

```
import math, random
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

random.seed(3)
ROOM = 4.0                                              # 4 m x 4 m room
BOXES = [(1.2, 1.0, 0.5, 0.5), (2.6, 2.2, 0.6, 0.4), (0.8, 2.8, 0.4, 0.6)]   # x, y, w, h

def hit(x, y):
    if not (0 < x < ROOM and 0 < y < ROOM): return True
    return any(bx <= x <= bx + w and by <= y <= by + h for bx, by, w, h in BOXES)

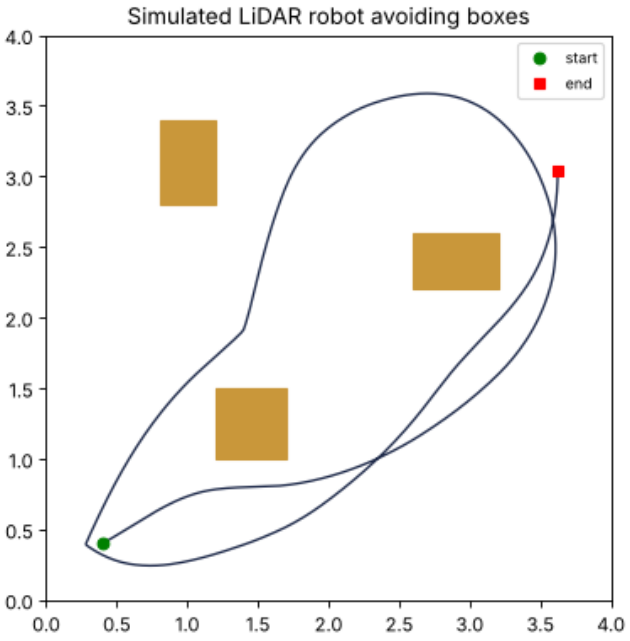
def lidar(x, y, th, noise=0.01):                        # 9 beams from -60 to +60 degrees
    out = []
    for k in range(9):
        a = th + math.radians(-60 + 15 * k)
        r = 0.0
        while r < 3.0 and not hit(x + r * math.cos(a), y + r * math.sin(a)):
            r += 0.02
        out.append(r + random.gauss(0, noise))
    return out

x, y, th, dt = 0.4, 0.4, math.radians(30), 0.1
path, collisions = [(x, y)], 0
for step in range(600):                                  # 60 simulated seconds
    s = lidar(x, y, th)
    front = min(s[3:6])
    if front < 0.45:                                     # obstacle ahead: turn toward the more open
side
        v, w = 0.02, (1.5 if sum(s[5:]) > sum(s[:4]) else -1.5)
    else:
        v, w = 0.25, 0.4 * (sum(s[5:]) - sum(s[:4])) / 9   # cruise, drift toward open space
    th += w * dt
    nx, ny = x + v * math.cos(th) * dt, y + v * math.sin(th) * dt
    if hit(nx, ny): collisions += 1
    else: x, y = nx, ny
    path.append((x, y))

dist = sum(math.dist(a, b) for a, b in zip(path, path[1:]))
print(f"simulated 60 s: travelled {dist:.1f} m, collisions {collisions}")
fig, ax = plt.subplots(figsize=(5, 5))
for bx, by, w, h in BOXES: ax.add_patch(plt.Rectangle((bx, by), w, h, color="#C9973A"))
ax.plot(*zip(*path), color="#1E2A4A", lw=1.2)
ax.plot(*path[0], "go", label="start"); ax.plot(*path[-1], "rs", label="end")
ax.set_xlim(0, ROOM); ax.set_ylim(0, ROOM); ax.set_aspect("equal"); ax.legend(fontsize=8)
ax.set_title("Simulated LiDAR robot avoiding boxes"); fig.tight_layout(); fig.savefig("sim.png",
dpi=80)
```

OUTPUT

```
simulated 60 s: travelled 14.7 m, collisions 0
```



THE ROBOT'S PATH OVER 60 SIMULATED SECONDS IN A 4 M × 4 M ROOM

Change one line (sensor noise, turn rate, safety distance) and rerun in a second: that is the power of simulation. Real simulators add proper physics (wheel slip, inertia, motor limits), 3-D sensors and rendering.

20.5 Simulation vs reality gap

Controllers that work perfectly in simulation often fail on the real robot. The difference is called the **reality gap** (or sim-to-real gap).

Simulation often assumes	Reality has
perfect, instant sensors	noise, dropouts, reflections (glass, black surfaces), latency
ideal motors	dead band, backlash, battery sag, different left/right motors
perfect friction	slipping wheels, carpets, ramps, dust
exact dimensions and mass	build tolerances, cables, loose screws
clean lighting	sunlight, shadows, flicker for cameras
no computing delay	CPU load, Wi-Fi lag, message queues

Ways to close the gap: add realistic **noise and delay** to simulated sensors, **measure** the real robot (mass, motor curves) and put the numbers in the model, use **domain randomisation** (vary friction, mass, lighting and textures in every run so the controller learns to cope), and always **test on hardware** step by step at low speed.

REALITY_GAP.PY

```
import random

def run(noise, delay_steps, seed):
    random.seed(seed)
    pos, v, queue, stopped_at = 0.0, 0.5, [], None   # drive toward a wall at 2.0 m, stop at 0.3 m
    for step in range(200):
        reading = (2.0 - pos) + random.gauss(0, noise)
        queue.append(reading)
        seen = queue[-1 - delay_steps] if len(queue) > delay_steps else 2.0
        if seen < 0.30 and stopped_at is None:
            v, stopped_at = 0.0, 2.0 - pos
        pos += v * 0.05
    return stopped_at

print("ideal simulation       : stops", round(run(0.00, 0, 1), 3), "m from the wall")
print("noise 3 cm             : stops", round(run(0.03, 0, 1), 3), "m")
print("noise 3 cm + 200 ms lag: stops", round(run(0.03, 4, 1), 3), "m")
worst = min(run(0.03, 4, s) for s in range(50))
print("worst of 50 noisy runs : stops", round(worst, 3), "m")
```

OUTPUT

```
ideal simulation       : stops 0.275 m from the wall
noise 3 cm             : stops 0.35 m
noise 3 cm + 200 ms lag: stops 0.25 m
worst of 50 noisy runs : stops 0.15 m
```

The ideal controller stops at 0.275 m, just inside 0.30 m, because the robot moves 2.5 cm per 50 ms step. Noise alone can trigger an early stop (0.35 m); add a 200 ms delay (four steps) and the robot keeps moving after the reading crosses the threshold. Over 50 randomised runs the worst case is 0.15 m, half the planned distance, so the safety margin must be chosen from many noisy runs, not one perfect one.

PRACTICE CHECKPOINT

Simulate a differential-drive robot with a LiDAR in Gazebo or Webots, and make it avoid obstacles in a simulated room.

20.6 Practice questions

20.6.1 Basic level

Q1 What is a main reason to simulate a robot before building it?

(a) simulation is always identical to reality
(b) it removes the need for hardware forever
(c) it makes motors stronger
(d) to test safely and cheaply, and find bugs early

Q2 Which browser tool simulates an ESP32 with Wi-Fi and a logic analyser?

(a) Tinkercad Circuits **(b)** Wokwi
(c) Isaac Sim **(d)** RViz

Q3 Which simulator is the standard partner of ROS 2?

(a) Gazebo **(b)** Tinkercad **(c)** Excel **(d)** Fritzing

Q4 Which simulator is known for photorealistic rendering and GPU-based AI training?

(a) Tinkercad Circuits **(b)** Wokwi
(c) NVIDIA Isaac Sim **(d)** turtlesim

Q5 What file format describes Gazebo worlds and models?

(a) SDF **(b)** CSV **(c)** MP4 **(d)** GPX

Q6 What does a physics engine simulate?

(a) only colours
(b) gravity, friction, collisions and forces
(c) only Wi-Fi
(d) only sound

Q7 What is the reality gap?

(a) the distance between two robots
(b) a gap in the chassis
(c) the Wi-Fi range
(d) the difference between simulated and real robot behaviour

Q8 Which Webots robot is a small educational robot with 8 IR sensors?

(a) TurtleBot 4 **(b)** Panda arm **(c)** e-puck **(d)** Spot

20.6.2 Intermediate level

Q9 Which package connects Gazebo topics to ROS 2 topics?

(a) rqt_graph **(b)** colcon
(c) ros_gz_bridge **(d)** micro_ros_agent

Q10 Why add noise to simulated sensors?

(a) to make simulation slower

(b) to save memory
(c) because simulators require it
(d) so controllers are tested against realistic imperfect data

Q11 What is domain randomisation?

(a) using one fixed world forever
(b) varying friction, mass, lighting and textures across simulation runs
(c) randomly deleting code
(d) changing the ROS domain ID

Q12 Why use simple collision shapes in simulation?

(a) faster and more stable physics
(b) better colours
(c) smaller URDF text only
(d) required by Wi-Fi

Q13 Which ROS 2 setting makes nodes follow the simulator clock?

(a) ROS_DOMAIN_ID (b) use_sim_time (c) max_speed (d) qos_depth

Q14 What does the Gazebo DiffDrive plug-in do?

(a) turns /cmd_vel commands into wheel motion and publishes odometry
(b) renders shadows
(c) records bags
(d) creates maps

Q15 Which simulator offers four physics engines and Lua/Python scripting, popular for arms?

(a) Wokwi (b) Tinkercad (c) CoppeliaSim (d) turtlesim

Q16 In the ros_gz_bridge argument "/cmd_vel@geometry_msgs/msg/Twist]gz.msgs.Twist", which way do messages go?

(a) from Gazebo to ROS 2
(b) both ways
(c) nowhere
(d) from ROS 2 to Gazebo

Q17 Which is a typical limitation of Tinkercad Circuits?

(a) no Arduino support
(b) it needs a GPU
(c) it only simulates drones
(d) few boards and no Wi-Fi simulation

20.6.3 Advanced level

Q18 A robot at 0.5 m/s has a 200 ms sensor delay. How far does it travel before it even sees an obstacle reading?

(a) 1 cm (b) 25 cm (c) 10 cm (d) 100 cm

Q19 A simulation runs 10× faster than real time. How long does a 2-hour test take?

(a) 12 minutes (b) 20 minutes (c) 1.2 minutes (d) 2 hours

Q20 Simulate a robot moving at 0.4 m/s for 3 s using time steps of 0.5 s and 0.05 s, with a constant turn rate of 0.5 rad/s, and compare final positions (Euler integration).

Q21 Add Gaussian noise (σ = 2 cm) to a simulated distance sensor and estimate how often a single reading falls below a 25 cm stop threshold when the true distance is 30 cm.

Q22 Generate a randomised list of 5 simulation worlds (domain randomisation): floor friction 0.4–1.0, robot mass 1.8–2.4 kg, light level 30–100 %, obstacle count 3–8.

Q23 Count how many of the 9 simulated LiDAR beams see an obstacle closer than 0.5 m and decide the turn direction.

Q24 A line follower works in Gazebo but oscillates on the real track. What is the most likely cause?

(a) the simulator was too accurate

(b) ROS 2 is not installed

(c) the track is too wide

(d) real motor delays, sensor noise or different friction not modelled in the simulation

Q25 A team wants to train a walking policy for a legged robot with millions of trials. Which tool fits best?

(a) Tinkercad Circuits

(b) GPU-parallel simulation such as Isaac Sim / Isaac Lab

(c) a real robot only

(d) a spreadsheet

20.7 Solutions

Q1 **Answer: (d) to test safely and cheaply, and find bugs early**

Safety, cost, speed.

Q2 **Answer: (b) Wokwi**

Wokwi supports ESP32.

Q3 **Answer: (a) Gazebo**

Native integration.

Q4 **Answer: (c) NVIDIA Isaac Sim**

Omniverse.

Q5 **Answer: (a) SDF**

Simulation Description Format.

Q6 **Answer: (b) gravity, friction, collisions and forces**

Dynamics.

Q7 **Answer: (d) the difference between simulated and real robot behaviour**

Sim-to-real.

Q8 **Answer: (c) e-puck**

e-puck.

Q9 **Answer: (c) ros_gz_bridge**

Bridge.

Q10 **Answer: (d) so controllers are tested against realistic imperfect data**

Narrow the gap.

Q11 **Answer: (b) varying friction, mass, lighting and textures across simulation runs**

Robust learning.

Q12 **Answer: (a) faster and more stable physics**

Boxes, cylinders.

Q13 **Answer: (b) use_sim_time**

Simulation time.

Q14 **Answer: (a) turns /cmd_vel commands into wheel motion and publishes odometry**

Drive system.

Q15 **Answer: (c) CoppeliaSim**

Formerly V-REP.

Q16 **Answer: (d) from ROS 2 to Gazebo**

] = ROS → Gazebo.

Q17 **Answer: (d) few boards and no Wi-Fi simulation**

Beginner tool.

Q18 **Answer: (c) 10 cm**

0.5 × 0.2 = **0.1 m**.

Q19 **Answer: (a) 12 minutes**

120 ÷ 10 = **12 min**.

Q20 **Answer: Solution program**

Large time steps accumulate integration error: the robot ends up several centimetres off. Simulators use small steps (1 ms for physics) for this reason.

```
import math
def sim(dt):
    x = y = th = 0.0
    for _ in range(round(3 / dt)):
        x += 0.4 * math.cos(th) * dt; y += 0.4 * math.sin(th) * dt; th += 0.5 * dt
    return x, y
exact = (0.4 / 0.5 * math.sin(1.5), 0.4 / 0.5 * (1 - math.cos(1.5)))
for dt in (0.5, 0.05):
    x, y = sim(dt); err = math.dist((x, y), exact)
    print(f"dt={dt:4}: ({x:.3f}, {y:.3f})  error {err * 100:.1f} cm")
print(f"exact   : ({exact[0]:.3f}, {exact[1]:.3f})")
```

OUTPUT

```
dt= 0.5: (0.887, 0.640)  error 13.6 cm
dt=0.05: (0.807, 0.733)  error 1.4 cm
exact   : (0.798, 0.743)
```

Q21 **Answer: Solution program**

A 5 cm margin is 2.5 σ, so about 0.6 % of single readings trigger a false stop; averaging three readings reduces σ by √3 and makes false stops very rare. Simulation with noise reveals such issues before the robot hesitates in a corridor.

```
import random
random.seed(7)
N = 10000
false_stops = sum(1 for _ in range(N) if 30 + random.gauss(0, 2) < 25)
print(f"{false_stops} of {N} readings below 25 cm ({100 * false_stops / N:.2f} %)")
avg3 = sum(1 for _ in range(N) if sum(30 + random.gauss(0, 2) for _ in range(3)) / 3 < 25)
print(f"averaging 3 readings: {avg3} of {N} ({100 * avg3 / N:.2f} %)")
```

OUTPUT

```
62 of 10000 readings below 25 cm (0.62 %)
averaging 3 readings: 0 of 10000 (0.00 %)
```

Q22 **Answer: Solution program**

Each training or test run uses a different world, so the controller does not overfit one perfect simulation.

```
import random
random.seed(11)
for i in range(5):
    world = dict(friction=round(random.uniform(0.4, 1.0), 2), mass=round(random.uniform(1.8, 2.4),
2),
                 light=random.randint(30, 100), obstacles=random.randint(3, 8))
    print(f"world {i}: {world}")
```

OUTPUT

```
world 0: {'friction': 0.67, 'mass': 2.14, 'light': 89, 'obstacles': 6}
world 1: {'friction': 0.7, 'mass': 2.15, 'light': 53, 'obstacles': 7}
world 2: {'friction': 0.69, 'mass': 2.17, 'light': 53, 'obstacles': 3}
```

```
world 3: {'friction': 0.67, 'mass': 1.89, 'light': 98, 'obstacles': 8}
world 4: {'friction': 0.78, 'mass': 2.16, 'light': 80, 'obstacles': 6}
```

Q23 **Answer: Solution program**

The three centre beams are blocked; the left side (positive angles) is more open, so the robot turns left, the same rule as mini_sim.py.

```
beams = [2.1, 1.8, 0.9, 0.42, 0.38, 0.47, 1.2, 2.5, 2.8]      # -60 .. +60 degrees
close = [i for i, r in enumerate(beams) if r < 0.5]
left_space, right_space = sum(beams[5:]), sum(beams[:4])
print("close beams:", close)
print("turn", "LEFT" if left_space > right_space else "RIGHT", f"(left space {left_space:.2f}, right
space {right_space:.2f})")
```

OUTPUT

```
close beams: [3, 4, 5]
turn LEFT (left space 6.97, right space 5.22)
```

Q24 **Answer: (d) real motor delays, sensor noise or different friction not modelled in the simulation**

Reality gap: retune gains on hardware.

Q25 **Answer: (b) GPU-parallel simulation such as Isaac Sim / Isaac Lab**

Massively parallel RL.

Simulate for safety, cost, speed and repeatability. Circuits: Tinkercad (beginner) and Wokwi (ESP32, Pico, diagram.json). Robots: Gazebo (ROS 2 partner, SDF, plug-ins, ros_gz_bridge), Webots (easy controllers), CoppeliaSim (arms, scripting), Isaac Sim (photorealism, GPU RL). Model worlds, robots, sensors with noise; use_sim_time. Reality gap: noise, delay, friction, motors. Close it with measured parameters, domain randomisation and careful hardware tests.

MODULE 21 · ADVANCED

Artificial intelligence and machine learning in robotics

Minimum duration **2 days** | Level **Advanced**

AI enables robots to recognise, decide and learn. This module compares rule-based and learning-based behaviour and the three kinds of machine learning, classifies robot sensor data and deploys a trained model to a microcontroller, introduces TinyML with Edge Impulse and TensorFlow Lite Micro and AI accelerators (Raspberry Pi AI Kit, NVIDIA Jetson, Google Coral), trains a reinforcement-learning agent by reward, and adds voice commands and large language models for task planning with a safety layer, followed by 25 solved questions and programs.

IN THIS MODULE

21.1 AI concepts for robotics
21.2 Machine learning on robots
21.3 Edge AI
21.4 Reinforcement learning (overview)
21.5 Voice and language interfaces
21.6 Practice questions
21.7 Solutions

21.1 AI concepts for robotics

21.1.1 Rule-based vs learning-based behaviour

	Rule-based	Learning-based
How	a programmer writes if/else rules	a model learns patterns from examples or rewards
Example	"if distance < 20 cm then turn"	recognise a cat, a gesture or a cracked part from images
Strengths	predictable, explainable, no data needed	handles messy, high-dimensional input (images, sound, vibration)
Weaknesses	breaks when the world is too complex to describe	needs data, can fail in surprising ways, harder to verify
Best use	safety logic, simple behaviours	perception and pattern recognition

NOTE

Real robots mix both: learning for **perception** ("that is a person"), rules for **safety** ("stop within 0.5 m of a person").

21.1.2 Supervised, unsupervised and reinforcement learning

Type	Learns from	Robot example
Supervised	labelled examples (input → correct answer)	classify terrain from vibration; detect objects in images
Unsupervised	unlabelled data; finds groups or anomalies	cluster sensor readings; spot unusual motor current (fault)
Reinforcement	rewards and penalties from trying actions	learn to walk, balance or grasp in simulation

21.2 Machine learning on robots

21.2.1 Classifying sensor data

A typical pipeline: collect windows of sensor data → compute **features** (mean, standard deviation, peak, frequency energy) → train a classifier → test on data it has never seen. Here an IMU on a robot classifies the floor it drives on from vibration. The data is generated to resemble real recordings (smooth floor: low vibration; carpet: damped; gravel: strong, spiky).

TERRAIN_CLASSIFIER.PY

```
import numpy as np
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix

rng = np.random.default_rng(0)
def window(surface):                                  # 50 vertical-acceleration samples (m/s^2
around gravity)
    base = {"tile": 0.22, "carpet": 0.30, "gravel": 0.9}[surface]
    w = rng.normal(0, base, 50)
    if surface == "gravel":
        w[rng.integers(0, 50, 4)] += rng.normal(0, 2.5, 4)    # occasional bumps
    return w

def features(w):
    return [w.std(), np.abs(w).max(), np.abs(np.diff(w)).mean()]

labels = ["tile", "carpet", "gravel"]
X, y = [], []
for s in labels:
    for _ in range(120):
        X.append(features(window(s))); y.append(s)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=1, stratify=y)
clf = DecisionTreeClassifier(max_depth=3, random_state=0).fit(Xtr, ytr)
pred = clf.predict(Xte)
print("test accuracy:", round(accuracy_score(yte, pred), 3))
print("confusion matrix (rows = true", labels, "):")
print(confusion_matrix(yte, pred, labels=labels))
print(export_text(clf, feature_names=["std", "peak", "jerk"], decimals=2))
```

OUTPUT

```
test accuracy: 0.972
confusion matrix (rows = true ['tile', 'carpet', 'gravel'] ):
[[36  0  0]
 [ 3 33  0]
 [ 0  0 36]]
|--- peak <= 1.53
|   |--- std <= 0.26
|   |   |--- std <= 0.24
|   |   |   |--- class: tile
|   |   |--- std >  0.24
|   |   |   |--- class: tile
|   |--- std >  0.26
|   |   |--- jerk <= 0.30
|   |   |   |--- class: carpet
|   |   |--- jerk >  0.30
|   |   |   |--- class: carpet
|--- peak >  1.53
|   |--- class: gravel
```

The confusion matrix shows the only mistakes: 3 carpet windows predicted as tile, the two most similar surfaces. The tree's useful splits are just peak > 1.53 (gravel) and std ≤ 0.26 (tile), a few comparisons that can be copied straight into microcontroller code. Always test on data recorded on a different day or a different robot before trusting the accuracy.

21.2.2 Training simple models and deploying them

Step	Tools
Collect and label data	robot logs to CSV, Edge Impulse data forwarder, phone apps
Train	scikit-learn, TensorFlow/Keras, PyTorch, Edge Impulse Studio
Evaluate	held-out test set, confusion matrix, test on the real robot
Convert	TFLite (int8 quantisation), ONNX, or hand-written rules for trees
Deploy	Arduino/ESP32 library, Raspberry Pi Python, Jetson TensorRT

The decision tree above, rewritten as C, runs in microseconds on an Arduino:

TERRAINCLASSIFY.INO

```
// Terrain classifier: thresholds copied from the trained decision tree
float stdOf(const float *w, int n) {
  float m = 0; for (int i = 0; i < n; i++) m += w[i]; m /= n;
  float s = 0; for (int i = 0; i < n; i++) s += (w[i] - m) * (w[i] - m);
  return sqrt(s / n);
}
const char* classify(float sd) {
  if (sd <= 0.26) return "tile";
  if (sd <= 0.60) return "carpet";
  return "gravel";
}
void setup() {
  Serial.begin(115200);
  float tile[8]   = {0.1, -0.2, 0.15, -0.1, 0.05, 0.2, -0.15, 0.0};
  float carpet[8] = {0.4, -0.3, 0.35, -0.45, 0.2, 0.3, -0.4, 0.1};
  float gravel[8] = {1.2, -0.8, 2.1, -1.5, 0.4, -0.9, 1.8, -0.3};
  const float *wins[] = {tile, carpet, gravel};
  for (int i = 0; i < 3; i++) {
    float sd = stdOf(wins[i], 8);
    Serial.print("std "); Serial.print(sd, 2); Serial.print(" -> "); Serial.println(classify(sd));
  }
}
void loop() {}
```

SERIAL MONITOR

```
std 0.14 -> tile
std 0.33 -> carpet
std 1.25 -> gravel
```

The sketch uses the tree's std ≤ 0.26 split for tile and, to keep the example short, a standard-deviation threshold for gravel instead of the peak test; use the exact splits your own trained tree prints. The robot can then slow down on gravel or switch PID gains on carpet.

21.3 Edge AI

Edge AI runs models on the robot itself rather than in the cloud: no network lag, works offline, keeps camera data private, and saves bandwidth.

21.3.1 TinyML on microcontrollers (Edge Impulse, TensorFlow Lite Micro)

TinyML squeezes small neural networks (tens of kilobytes) onto microcontrollers such as the ESP32-S3, Arduino Nano 33 BLE Sense, Raspberry Pi Pico and Arduino Nicla. **Edge Impulse** handles the whole workflow in a browser:

1. Connect the board (or a phone) and record labelled samples: gestures from an accelerometer, keywords from a microphone, images from a camera.
2. Design an "impulse": a processing block (spectral features, MFCC for audio, image resize) plus a learning block (small neural network, anomaly detection).
3. Train and check the accuracy and confusion matrix; test on held-out samples.
4. Deploy as an Arduino library (.zip), C++ source or firmware; quantised to int8 for speed and size.

5. Call `run_classifier()` in your sketch and act on the most probable label.

USING AN EDGE IMPULSE GESTURE MODEL TO DRIVE A ROBOT (REFERENCE)

```
#include <gesture_inferencing.h>           // the library exported by Edge Impulse

float buffer[EI_CLASSIFIER_DSP_INPUT_FRAME_SIZE];   // filled with accelerometer samples

void loop() {
  readAccelerometerInto(buffer);                    // e.g. 2 s at 62.5 Hz, x/y/z
  signal_t signal;
  numpy::signal_from_buffer(buffer, EI_CLASSIFIER_DSP_INPUT_FRAME_SIZE, &signal);
  ei_impulse_result_t result;
  run_classifier(&signal, &result, false);
  for (size_t i = 0; i < EI_CLASSIFIER_LABEL_COUNT; i++) {
    if (result.classification[i].value > 0.8) {      // confident prediction
      const char *label = result.classification[i].label;   // "circle", "swipe", "idle"
      if (strcmp(label, "circle") == 0) spinRobot();
      if (strcmp(label, "swipe") == 0)  driveForward();
    }
  }
}
```

TensorFlow Lite for Microcontrollers (now part of Google's LiteRT) is the lower-level engine underneath many such libraries: you convert a Keras model to an int8 `.tflite` file, embed it as a C array, and call the interpreter yourself.

21.3.2 AI accelerators: Raspberry Pi AI Kit, NVIDIA Jetson, Google Coral (overview)

Accelerator	Performance (approx.)	Pairs with	Good for
Raspberry Pi AI Kit / AI HAT+ (Hailo-8L / Hailo-8)	13–26 TOPS	Raspberry Pi 5 (PCIe)	real-time YOLO detection, pose estimation with the Pi camera
Raspberry Pi AI Camera (Sony IMX500)	runs the model inside the camera sensor	any Raspberry Pi	low-power detection without loading the CPU
NVIDIA Jetson Orin Nano	tens of TOPS, CUDA GPU	stand-alone computer	multiple cameras, ROS 2, TensorRT, larger models
Google Coral USB / M.2 (Edge TPU)	4 TOPS	Pi, PC	TensorFlow Lite models compiled for the Edge TPU
ESP32-S3 vector instructions	small	stand-alone	keyword spotting, tiny image models

TOPS (tera operations per second) is only a rough guide: real speed depends on the model, input size and software support. Check that your model and framework are supported before buying.

21.4 Reinforcement learning (overview)

21.4.1 Learning by reward in simulation

In **reinforcement learning** (RL) an **agent** in **state** s chooses an **action** a, receives a **reward** r and a new state. Over many episodes it learns a **policy** that maximises total reward. **Q-learning** keeps a table Q(s, a) of expected future reward and updates it after every step:

> **NOTE**
> $Q(s,a) \leftarrow Q(s,a) + \alpha\cdot[\, r + \gamma\cdot\max_{a'} Q(s',a') - Q(s,a)\,]$ (α learning rate, γ discount)

Q_LEARNING.PY

```
import random
random.seed(0)
GRID = ["S...#....",
        ".##.#.##.",
        ".#.......",
        ".#.###.#.",
        "...#...#G"]
H, W = len(GRID), len(GRID[0])
ACTIONS = {"^": (-1, 0), "v": (1, 0), "<": (0, -1), ">": (0, 1)}
Q = {(r, c): {a: 0.0 for a in ACTIONS} for r in range(H) for c in range(W)}

def step(s, a):
    r, c = s[0] + ACTIONS[a][0], s[1] + ACTIONS[a][1]
    if not (0 <= r < H and 0 <= c < W) or GRID[r][c] == "#":
        return s, -5, False                 # bumped a wall: penalty, stay put
    if GRID[r][c] == "G":
        return (r, c), 100, True            # reached the goal
    return (r, c), -1, False                # each step costs a little: prefer short paths

alpha, gamma, eps = 0.5, 0.95, 0.2
for episode in range(800):
    s = (0, 0)
    for t in range(200):
        a = random.choice(list(ACTIONS)) if random.random() < eps else max(Q[s], key=Q[s].get)
        s2, rwd, done = step(s, a)
        Q[s][a] += alpha * (rwd + gamma * max(Q[s2].values()) * (not done) - Q[s][a])
        s = s2
        if done: break
    if episode in (0, 50, 799):
        print(f"episode {episode:3d}: {t + 1} steps")

s, route = (0, 0), []
while GRID[s[0]][s[1]] != "G" and len(route) < 40:
    a = max(Q[s], key=Q[s].get); route.append(a); s, _, _ = step(s, a)
print("learned route:", "".join(route), f"({len(route)} moves)")
for r in range(H):
    print(" ".join("#" if GRID[r][c] == "#" else "G" if GRID[r][c] == "G" else max(Q[(r, c)],
key=Q[(r, c)].get) for c in range(W)))
```

> **OUTPUT**
> ```
> episode   0: 200 steps
> episode  50: 16 steps
> ```

```
episode 799: 18 steps
learned route: >>>vv>>>>>vv (12 moves)
> > > v # v < > v
^ # # v # v # # v
^ # > > > > > > v
v # ^ # # # ^ # v
^ > ^ # ^ > ^ # G
```

Nobody told the agent where the goal was. It explored randomly at first (hundreds of steps), and the rewards gradually shaped the table until the greedy policy (arrows) leads straight to G. Real robots learn far harder skills (walking, grasping) the same way, but in simulation with deep neural networks instead of a table, because millions of trial-and-error falls on hardware would be slow and destructive.

WATCH OUT
An RL agent maximises exactly the reward you write, not what you meant. A badly designed reward ("move fast") can teach a robot to spin in circles. Test learned policies in simulation and add rule-based safety limits on the real robot.

21.5 Voice and language interfaces

21.5.1 Speech recognition and voice commands

Option	Runs	Notes
Vosk	offline on a Pi	small models, many languages including Indian English and Hindi
Whisper (whisper.cpp, faster-whisper)	offline; Pi 5 for small models, better on Jetson/PC	very accurate, multilingual
Cloud APIs (Google, Azure)	online	accurate, but needs internet and sends audio away
Keyword spotting (Edge Impulse)	microcontroller	a few fixed words: "go", "stop", "left"

OFFLINE SPEECH RECOGNITION WITH VOSK (REFERENCE)

```
from vosk import Model, KaldiRecognizer
import pyaudio, json
rec = KaldiRecognizer(Model("vosk-model-small-en-in-0.4"), 16000)
mic = pyaudio.PyAudio().open(format=pyaudio.paInt16, channels=1, rate=16000, input=True,
frames_per_buffer=4000)
while True:
    if rec.AcceptWaveform(mic.read(4000)):
        text = json.loads(rec.Result())["text"]          # e.g. "move forward two metres"
        handle(text)
```

VOICE_COMMANDS.PY

```
import re
WORDS = {"one": 1, "two": 2, "three": 3, "half": 0.5}

def parse(text):
    t = text.lower()
    if re.search(r"\b(stop|halt|freeze)\b", t):
        return ("STOP", 0)
    num = re.search(r"(\d+(?:\.\d+)?|one|two|three|half)", t)
    val = float(WORDS.get(num.group(1), 0) or num.group(1)) if num else None
    if re.search(r"\b(forward|ahead)\b", t): return ("DRIVE", val or 0.5)
    if re.search(r"\b(back|reverse)\b", t):  return ("DRIVE", -(val or 0.5))
    if "left" in t:  return ("TURN", val or 90)
    if "right" in t: return ("TURN", -(val or 90))
    return ("UNKNOWN", text)

for said in ["move forward two metres", "please turn left 45 degrees", "go back", "STOP now!", "make
me a sandwich"]:
    print(f"{said!r:32} -> {parse(said)}")
```

OUTPUT

```
'move forward two metres'        -> ('DRIVE', 2.0)
'please turn left 45 degrees'    -> ('TURN', 45.0)
'go back'                        -> ('DRIVE', -0.5)
'STOP now!'                      -> ('STOP', 0)
'make me a sandwich'             -> ('UNKNOWN', 'make me a sandwich')
```

The recogniser gives text; a small parser turns it into robot commands, with "stop" always checked first. Unknown phrases are rejected rather than guessed.

21.5.2 Large language models for robot task planning (overview)

A **large language model** (LLM) can turn a request such as "bring the red cup from the kitchen to the desk" into a sequence of steps using skills the robot already has (navigate, find object, pick, place). The LLM plans; the robot's tested software executes. Vision-language-action models go further and output motions directly, an active research area.

LLM_PLAN_GUARD.PY

```
import json
SKILLS = {"navigate": {"kitchen", "desk", "dock"}, "find": None, "pick": None, "place": None, "say":
None}
MAX_STEPS = 8

# a plan as an LLM might return it when asked to answer only with JSON using the listed skills
llm_reply = """[{"skill": "navigate", "arg": "kitchen"}, {"skill": "find", "arg": "red cup"},
 {"skill": "pick", "arg": "red cup"}, {"skill": "navigate", "arg": "desk"},
 {"skill": "place", "arg": "desk"}, {"skill": "open_window", "arg": "now"},
 {"skill": "navigate", "arg": "roof"}]"""

def validate(reply):
    try:
        plan = json.loads(reply)
    except json.JSONDecodeError:
        return [], ["reply is not valid JSON"]
    ok, problems = [], []
    for i, step in enumerate(plan[:MAX_STEPS]):
        s, arg = step.get("skill"), step.get("arg")
        if s not in SKILLS:
            problems.append(f"step {i}: unknown skill '{s}' rejected")
        elif SKILLS[s] is not None and arg not in SKILLS[s]:
            problems.append(f"step {i}: '{arg}' is not a known place for {s}")
        else:
            ok.append((s, arg))
    return ok, problems

plan, problems = validate(llm_reply)
for s, a in plan: print("EXECUTE", s, a)
for p in problems: print("BLOCKED", p)
```

OUTPUT

```
EXECUTE navigate kitchen
EXECUTE find red cup
EXECUTE pick red cup
EXECUTE navigate desk
EXECUTE place desk
BLOCKED step 5: unknown skill 'open_window' rejected
BLOCKED step 6: 'roof' is not a known place for navigate
```

The key design rule: the LLM may only choose from a fixed list of skills with checked arguments, and every plan passes through ordinary code that rejects anything unknown or unsafe. LLMs can produce confident nonsense ("hallucinations"); the robot must never execute their output unchecked, and physical safety stays in rule-based, tested code.

PRACTICE CHECKPOINT

Train a gesture or sound classifier with Edge Impulse, deploy it to a microcontroller, and use it to control a robot.

21.6 Practice questions

21.6.1 Basic level

Q1 Which kind of machine learning learns from labelled examples?

(a) unsupervised learning
(b) supervised learning
(c) reinforcement learning
(d) rule-based programming

Q2 Which kind of learning uses rewards and penalties?

(a) supervised learning
(b) unsupervised learning
(c) compiling
(d) reinforcement learning

Q3 Which kind of learning finds groups or anomalies in unlabelled data?

(a) unsupervised learning
(b) supervised learning
(c) reinforcement learning
(d) PID control

Q4 What is TinyML?

(a) a small robot
(b) a programming language
(c) running small machine-learning models on microcontrollers
(d) a cloud service only

Q5 Which platform helps collect data, train and deploy TinyML models from a browser?

(a) Tinkercad (b) RViz (c) Fritzing (d) Edge Impulse

Q6 Which board is an AI computer with a CUDA GPU for robots?

(a) Arduino Uno
(b) NVIDIA Jetson Orin Nano
(c) ESP-01
(d) Raspberry Pi Pico

Q7 What is the main advantage of edge AI over cloud AI on a robot?

(a) works offline with low latency and keeps data private
(b) unlimited model size
(c) no power needed
(d) no training needed

Q8 Which tool recognises speech offline on a Raspberry Pi?

(a) MQTT (b) Nav2 (c) Vosk (d) Gazebo

21.6.2 Intermediate level

Q9 What are features in sensor classification?

(a) numbers computed from raw data, such as mean, standard deviation or peak
(b) the robot's wheels
(c) the battery voltage only
(d) the label names

Q10 Why test a model on data it has never seen?

(a) to make training faster
(b) to measure how well it generalises, not just memorises
(c) to reduce memory
(d) it is not necessary

Q11 What does a confusion matrix show?

(a) training time
(b) battery usage
(c) model size
(d) how often each true class was predicted as each class

Q12 Why are models quantised to int8 for microcontrollers?

(a) to increase accuracy greatly
(b) to make them readable
(c) smaller and faster with little accuracy loss
(d) Arduino cannot store text

Q13 In Q-learning, what does γ (gamma) control?

(a) the learning speed only
(b) the number of actions
(c) how much future rewards count compared with immediate ones
(d) the motor speed

Q14 Why is reinforcement learning usually done in simulation first?

(a) real robots cannot run neural networks
(b) simulation is always perfect
(c) it is required by law
(d) it needs huge numbers of trials, many of which fail

Q15 What is the role of ε (epsilon) in ε-greedy exploration?

(a) the reward size
(b) the chance of taking a random action to explore
(c) the learning rate
(d) the grid size

Q16 When an LLM plans robot tasks, what must the robot software do?

(a) validate every step against allowed skills and safety rules before executing
(b) execute the text directly
(c) ignore the plan
(d) send it to the motors raw

Q17 Which device runs YOLO-style detection with a Raspberry Pi 5 over PCIe?

(a) Arduino Uno
(b) Raspberry Pi AI Kit / AI HAT+ (Hailo)
(c) HC-05
(d) L298N

21.6.3 Advanced level

Q18 A test set has 90 samples and the model gets 81 right. What is the accuracy?

(a) 90 % **(b)** 81 % **(c)** 9 % **(d)** 99 %

Q19 Q(s,a) = 10, reward = -1, max Q(s',·) = 20, $\alpha = 0.5$, $\gamma = 0.9$. What is the new Q(s,a)?

(a) 10 **(b)** 19 **(c)** 13.5 **(d)** 17

Q20 Classify robot gestures (circle, shake, idle) from accelerometer features with k-nearest neighbours (k = 3) and report the accuracy.

Q21 Detect a failing motor by flagging current readings more than 3 standard deviations from the normal mean (simple anomaly detection).

Q22 Run one Q-learning update for each of three steps and print the Q value: start Q = 0, $\alpha = 0.5$, $\gamma = 0.9$, rewards -1, -1, +10 with next-state max Q values 0, 2, 0 (the last step ends the episode).

Q23 Count how many words a voice-command parser should accept: map synonyms (go/forward/ahead, back/reverse, halt/stop/freeze) to commands and parse four phrases.

Q24 A delivery robot must recognise people in camera images and always stop 1 m from them. Which design is best?

(a) an LLM controlling the motors directly
(b) only rules on raw pixels
(c) reinforcement learning on the real street
(d) a learned person detector for perception plus a rule-based stop distance

Q25 A gesture model scores 99 % in the lab but fails on a new user. What is the best fix?

(a) raise the confidence threshold to 100 %
(b) remove the test set
(c) use a bigger battery
(d) collect training data from more people and conditions, then retrain

21.7 Solutions

Q1 **Answer: (b) supervised learning**

Input → correct answer.

Q2 **Answer: (d) reinforcement learning**

Trial and error.

Q3 **Answer: (a) unsupervised learning**

Clustering.

Q4 **Answer: (c) running small machine-learning models on microcontrollers**

ML on MCUs.

Q5 **Answer: (d) Edge Impulse**

End-to-end workflow.

Q6 **Answer: (b) NVIDIA Jetson Orin Nano**

GPU + TensorRT.

Q7 **Answer: (a) works offline with low latency and keeps data private**

On-device.

Q8 **Answer: (c) Vosk**

Offline ASR.

Q9 **Answer: (a) numbers computed from raw data, such as mean, standard deviation or peak**

Feature engineering.

Q10 **Answer: (b) to measure how well it generalises, not just memorises**

Held-out test set.

Q11 **Answer: (d) how often each true class was predicted as each class**

Which classes get mixed up.

Q12 **Answer: (c) smaller and faster with little accuracy loss**

4× smaller than float32.

Q13 **Answer: (c) how much future rewards count compared with immediate ones**

Discount factor.

Q14 **Answer: (d) it needs huge numbers of trials, many of which fail**

Safe, fast, parallel.

Q15 **Answer: (b) the chance of taking a random action to explore**

Explore vs exploit.

Q16 **Answer: (a) validate every step against allowed skills and safety rules before executing**

Never trust unchecked output.

Q17 **Answer: (b) Raspberry Pi AI Kit / AI HAT+ (Hailo)**

Hailo accelerator.

Q18 **Answer: (a) 90 %**

81 ÷ 90 = **0.9**.

Q19 **Answer: (c) 13.5**

10 + 0.5 × (−1 + 0.9 × 20 − 10) = 10 + 0.5 × 7 = **13.5**.

Q20 **Answer: Solution program**

Two well-chosen features already separate the gestures. KNN needs no training beyond storing examples but must compare against all of them at prediction time, which is fine for small datasets.

```
import numpy as np
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
rng = np.random.default_rng(5)
centres = {"idle": (0.05, 0.1), "circle": (0.8, 1.5), "shake": (1.6, 4.0)}   # (std, peak frequency
Hz)
X, y = [], []
for label, (sd, f) in centres.items():
    for _ in range(60):
        X.append([rng.normal(sd, 0.15), rng.normal(f, 0.5)]); y.append(label)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, random_state=0)
knn = KNeighborsClassifier(n_neighbors=3).fit(Xtr, ytr)
print("accuracy:", round(knn.score(Xte, yte), 3))
print("new sample (0.9, 1.4) ->", knn.predict([[0.9, 1.4]])[0])
```

OUTPUT

```
accuracy: 1.0
new sample (0.9, 1.4) -> circle
```

Q21 **Answer: Solution program**

Learn what "normal" looks like, then flag readings far outside it: unsupervised, because no examples of failures were needed.

```
import statistics
normal = [1.02, 0.98, 1.05, 1.00, 0.97, 1.03, 0.99, 1.01, 1.04, 0.96]   # amps, healthy motor
mu, sd = statistics.mean(normal), statistics.stdev(normal)
for reading in [1.00, 1.06, 1.35, 0.94, 1.80]:
    z = (reading - mu) / sd
    print(f"{reading:.2f} A  z={z:+5.1f}  ->", "ANOMALY" if abs(z) > 3 else "ok")
```

OUTPUT

```
1.00 A  z= -0.2  -> ok
1.06 A  z= +1.8  -> ok
1.35 A  z=+11.4  -> ANOMALY
0.94 A  z= -2.1  -> ok
1.80 A  z=+26.3  -> ANOMALY
```

Q22 **Answer: Solution program**

Each update moves Q halfway ($\alpha = 0.5$) toward the target "reward now plus discounted best future value".

```
Q, alpha, gamma = 0.0, 0.5, 0.9
for r, next_max, done in [(-1, 0, False), (-1, 2, False), (10, 0, True)]:
    target = r + (0 if done else gamma * next_max)
    Q += alpha * (target - Q)
    print(f"reward {r:+3d} target {target:5.2f} -> Q = {Q:.3f}")
```

OUTPUT

```
reward  -1 target -1.00 -> Q = -0.500
reward  -1 target  0.80 -> Q = 0.150
reward +10 target 10.00 -> Q = 5.075
```

Q23 **Answer: Solution program**

Synonym sets make the interface forgiving; checking STOP first makes it safe ("go go stop" stops).

```
SYN = {"FORWARD": {"go", "forward", "ahead"}, "BACK": {"back", "reverse"}, "STOP": {"halt", "stop",
"freeze"}}
def command(text):
    words = set(text.lower().replace("!", "").split())
    if words & SYN["STOP"]: return "STOP"                    # stop has priority
    for cmd, syn in SYN.items():
        if words & syn: return cmd
    return "IGNORE"
for t in ["Go ahead", "reverse slowly", "freeze!", "go go stop"]:
    print(f"{t!r:16} -> {command(t)}")
print("accepted words:", sum(len(s) for s in SYN.values()))
```

OUTPUT

```
'Go ahead'       -> FORWARD
'reverse slowly' -> BACK
'freeze!'        -> STOP
'go go stop'     -> STOP
accepted words: 8
```

Q24 **Answer: (d) a learned person detector for perception plus a rule-based stop distance**

Learn perception, rule safety.

Q25 **Answer: (d) collect training data from more people and conditions, then retrain**

Data diversity.

Rules for safety, learning for perception. Supervised (labels), unsupervised (clusters, anomalies), reinforcement (rewards). Pipeline: data → features → train → test on unseen data (confusion matrix) → convert/quantise → deploy. TinyML with Edge Impulse/TFLite Micro; accelerators: Pi AI Kit/HAT+, AI Camera, Jetson, Coral. Q-learning: $Q \leftarrow Q + \alpha(r + \gamma \max Q' - Q)$, ε-greedy, train in simulation. Voice: Vosk/Whisper + safe parser. LLM planning only through validated skills.

MODULE 22 · ADVANCED

Drones and aerial robotics

Minimum duration **2 days** | Level **Advanced**

Drones are flying robots used in photography, agriculture, delivery and inspection. This module explains multirotor, fixed-wing and hybrid VTOL types and how a quadcopter controls thrust, roll, pitch and yaw; covers every component (frame, BLDC motors, propellers, ESCs, flight controller and IMU, LiPo batteries, radio, GPS and telemetry); explains flight modes, PID tuning and the Betaflight, ArduPilot and PX4 firmware; summarises Indian drone rules and the Digital Sky platform with safe flying practice; and surveys applications, followed by 25 solved questions and programs.

IN THIS MODULE

22.1 Drone basics
22.2 Drone components
22.3 Flight control
22.4 Safety and regulations
22.5 Applications
22.6 Practice questions
22.7 Solutions

22.1 Drone basics

22.1.1 Types: multirotor, fixed-wing, hybrid VTOL

Type	How it flies	Strengths	Weaknesses
Multirotor (quad, hexa, octo)	propellers give all the lift	hovers, takes off vertically, easy to fly	short flight time (15–40 min)
Fixed-wing	wings give lift; a propeller pushes forward	efficient: 1–3 h, long range, mapping large areas	needs a runway or hand launch; cannot hover
Hybrid VTOL	lifts like a multirotor, cruises like a plane	vertical take-off and long range	complex, expensive
Single-rotor (helicopter)	one main rotor + tail rotor	heavy payloads, efficient hover	mechanically complex, dangerous blades

22.1.2 How quadcopters fly: thrust, roll, pitch, yaw

A quadcopter has four fixed-pitch propellers: two spin clockwise (CW) and two counter-clockwise (CCW), diagonally opposite, so their reaction torques cancel. Everything is controlled by changing the four motor speeds:

Movement	Axis	How (X-frame: 1 front-right CCW, 2 rear-left CCW, 3 front-left CW, 4 rear-right CW)
Throttle	up/down	all four faster or slower together
Roll	tilt left/right (x-axis)	left motors faster than right → rolls right and drifts right
Pitch	nose up/down (y-axis)	rear motors faster than front → nose down, flies forward
Yaw	rotate (z-axis)	CW pair faster than CCW pair → body turns CCW (reaction torque)

QUAD_MIXER.PY

```
def mix(throttle, roll, pitch, yaw):
    """X-quad mixer (Betaflight/ArduPilot style). Returns motor outputs 0..1 for
    M1 front-right (CCW), M2 rear-left (CCW), M3 front-left (CW), M4 rear-right (CW)."""
    m1 = throttle - roll + pitch + yaw
    m2 = throttle + roll - pitch + yaw
    m3 = throttle + roll + pitch - yaw
    m4 = throttle - roll - pitch - yaw
    return [round(min(1, max(0, m)), 2) for m in (m1, m2, m3, m4)]

print("              M1-FR  M2-RL  M3-FL  M4-RR")
for name, cmd in [("hover", (0.5, 0, 0, 0)), ("roll right", (0.5, 0.1, 0, 0)),
                  ("pitch forward", (0.5, 0, -0.1, 0)), ("yaw", (0.5, 0, 0, 0.1)), ("climb", (0.7,
0, 0, 0))]:
    print(f"{name:13}", "  ".join(f"{m:5.2f}" for m in mix(*cmd)))
```

OUTPUT

```
              M1-FR  M2-RL  M3-FL  M4-RR
hover          0.50   0.50   0.50   0.50
roll right     0.40   0.60   0.60   0.40
pitch forward  0.40   0.60   0.40   0.60
yaw            0.60   0.60   0.40   0.40
climb          0.70   0.70   0.70   0.70
```

The flight controller runs this **mixer** hundreds to thousands of times per second: the pilot (or autopilot) asks for throttle, roll, pitch and yaw, and the mixer converts them into four motor speeds. "Roll right" speeds up the left motors (M2, M3); "pitch forward" speeds up the rear motors (M2, M4).

22.2 Drone components

22.2.1 Frame, BLDC motors, propellers, ESCs

Part	What to know
Frame	carbon fibre (stiff, light) or plastic; size = diagonal motor-to-motor distance (e.g. 250 mm racer, 450 mm camera quad)
BLDC motor	brushless outrunner; size code 2306 = 23 mm stator diameter × 6 mm height; KV = rpm per volt (2400 KV on 4S ≈ 35 500 rpm unloaded)

Part	What to know
Propellers	5045 = 5 inch diameter, 4.5 inch pitch; bigger or steeper props give more thrust but draw more current; CW and CCW versions
ESC	electronic speed controller: drives each BLDC motor from the flight controller signal (PWM, OneShot, DShot); rated in amps (e.g. 35 A); 4-in-1 boards common

SHORTCUT TRICK
Rule of thumb: total maximum thrust ≥ 2 × all-up weight (thrust-to-weight ratio 2:1) for stable camera drones; racing quads use 5:1 or more.

22.2.2 Flight controller and IMU

The **flight controller** (FC) is a fast microcontroller board (STM32 F4/F7/H7) with an **IMU** (3-axis gyro + accelerometer), often a barometer (altitude) and a magnetometer. It reads the IMU thousands of times per second, estimates attitude, runs the PID loops and outputs motor commands. Popular boards: SpeedyBee F405, Matek H743, Pixhawk 6C (ArduPilot/PX4).

22.2.3 LiPo batteries and power

Rating	Meaning	Example
Cell count (S)	3.7 V nominal per cell (4.2 V full, ~3.5 V empty under load)	4S = 14.8 V nominal, 16.8 V full
Capacity (mAh)	charge stored	1500 mAh = 1.5 A for 1 hour
C rating	max continuous current = C × capacity	100C × 1.5 Ah = 150 A (often optimistic)
Connector	XT30, XT60, XT90	XT60 for 5-inch quads

WATCH OUT
LiPos can catch fire if punctured, over-charged, over-discharged or charged unattended. Charge with a balance charger at 1C in a fire-safe bag, never below about 3.3 V per cell, store at about 3.8 V per cell, and dispose of puffed packs safely.

FLIGHT_TIME.PY

```
weight_g = 1200                    # all-up weight with battery and camera
motors, thrust_per_motor_g = 4, 750 # from the motor/prop thrust table
battery_mAh, cells = 4000, 4
hover_current_A = 14               # measured or from motor tables at ~40 % throttle

twr = motors * thrust_per_motor_g / weight_g
usable_Ah = battery_mAh / 1000 * 0.8                    # use only 80 % to protect the LiPo
flight_min = usable_Ah / hover_current_A * 60
print(f"thrust-to-weight ratio: {twr:.1f} : 1")
print(f"usable capacity: {usable_Ah:.1f} Ah -> hover time about {flight_min:.1f} minutes")
print(f"battery energy: {battery_mAh / 1000 * 3.7 * cells:.1f} Wh")
```

OUTPUT

```
thrust-to-weight ratio: 2.5 : 1
usable capacity: 3.2 Ah -> hover time about 13.7 minutes
battery energy: 59.2 Wh
```

22.2.4 Radio transmitter and receiver

The pilot's **transmitter** (RadioMaster, FrSky, FlySky) sends stick positions to a **receiver** on the drone, which passes them to the FC over a digital link (SBUS, CRSF/ExpressLRS, iBUS). Channels: 1–4 for roll, pitch, throttle, yaw (mode 2: throttle and yaw on the left stick), extra switches for arming, flight modes and return-to-home. ExpressLRS gives long range and low latency. **Failsafe** must be set so a lost signal makes the drone land or return home, never hold full throttle.

22.2.5 GPS and telemetry

A **GPS module** with compass (M8N, M10) enables position hold, return-to-home and waypoint missions. **Telemetry** radios (SiK 433 MHz, ELRS telemetry, or Wi-Fi/4G) send live data (battery, altitude, GPS position, attitude) to a ground station such as **Mission Planner** or **QGroundControl** using the **MAVLink** protocol.

22.3 Flight control

22.3.1 Flight modes

Mode	Pilot controls	Drone holds	Use
Acro / Rate	rotation rates	nothing: sticks centred keeps the current angle	racing, freestyle
Angle / Stabilize	tilt angle	levels itself when sticks are centred	beginners
Altitude hold	tilt; throttle = climb rate	altitude (barometer)	smooth video
Position hold / Loiter	velocity	position (GPS)	photography, inspection
Auto / Mission	nothing	follows waypoints	mapping, spraying
RTL (return to launch)	nothing	climbs, flies home, lands	failsafe, low battery

22.3.2 PID tuning for stability

Each axis has a **rate PID** loop (gyro rate vs desired rate), and in angle modes an outer angle loop. **P** gives sharpness (too high: fast oscillation), **I** holds the attitude against wind and an off-centre battery (too high: slow wobble), **D** damps overshoot after flips (too high: hot motors, noise). Tune P, then D, then I, with filtered gyro data; Betaflight's blackbox logs show the result.

ROLL_PID.PY

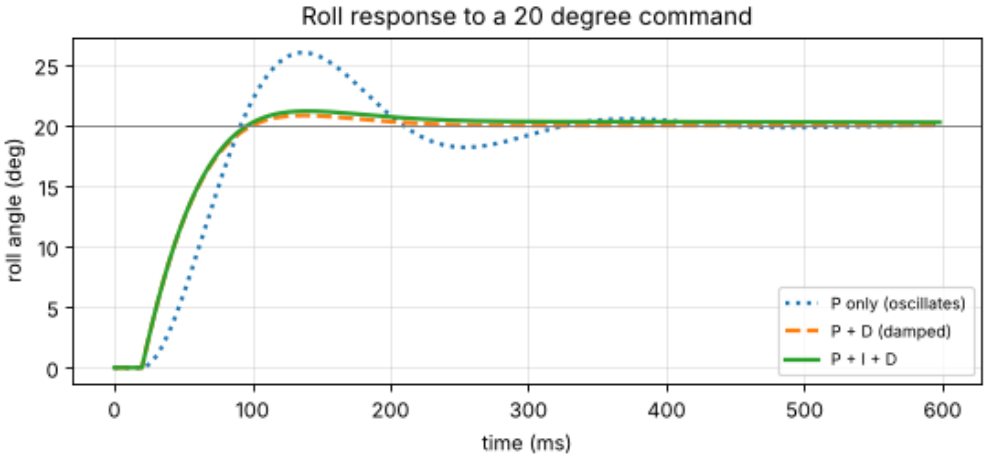
```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

def simulate(kp, ki, kd, steps=300, dt=0.002):
    angle, rate, integral, prev_err, out = 0.0, 0.0, 0.0, 0.0, []
    for k in range(steps):
        target = 20.0 if k > 10 else 0.0                    # pilot asks for 20 degrees roll
        err = target - angle
        integral += err * dt
        torque = kp * err + ki * integral + kd * (err - prev_err) / dt
        prev_err = err
        rate += (torque - 0.5 * rate) * dt * 40              # simple airframe: inertia + drag
        angle += rate * dt
        out.append(angle)
    return out

t = [k * 2 for k in range(300)]                              # milliseconds
fig, ax = plt.subplots(figsize=(7, 3.4))
for (kp, ki, kd), style, label in [((20, 0, 0.0), ":", "P only (oscillates)"),
                                   ((20, 0, 0.6), "--", "P + D (damped)"),
                                   ((20, 15, 0.6), "-", "P + I + D")]:
    y = simulate(kp, ki, kd)
    ax.plot(t, y, style, lw=2, label=label)
    print(f"{label:20} max {max(y):5.1f} deg, final {y[-1]:5.1f} deg")
ax.axhline(20, color="grey", lw=0.8); ax.set_xlabel("time (ms)"); ax.set_ylabel("roll angle (deg)")
ax.legend(fontsize=8); ax.grid(alpha=0.3); ax.set_title("Roll response to a 20 degree command")
fig.tight_layout(); fig.savefig("pid_roll.png", dpi=80)
```

OUTPUT

```
P only (oscillates)  max  26.0 deg, final  20.0 deg
P + D (damped)       max  20.8 deg, final  20.0 deg
P + I + D            max  21.2 deg, final  20.3 deg
```



ADDING D REMOVES THE OVERSHOOT AND OSCILLATION OF P ALONE (I MATTERS MOST AGAINST STEADY DISTURBANCES SUCH AS WIND)

22.3.3 Firmware: Betaflight, ArduPilot, PX4 (overview)

Firmware	Focus	Configurator	Notes
Betaflight	FPV racing and freestyle	Betaflight Configurator	very responsive manual flight; limited GPS autonomy
INAV	FPV with GPS	INAV Configurator	position hold, RTH, waypoints on Betaflight-style boards
ArduPilot	autonomous vehicles of every kind (copter, plane, rover, boat, sub)	Mission Planner / QGroundControl	mature, huge feature set, Lua scripting, SITL simulator
PX4	autonomous drones, research, industry	QGroundControl	ROS 2 integration (uXRCE-DDS), Gazebo simulation

Both ArduPilot and PX4 speak MAVLink and can be programmed from a companion computer (Raspberry Pi, Jetson) with **pymavlink**, **MAVSDK** or ROS 2. Before flying real hardware, run **SITL** (software in the loop): the real firmware flies a simulated drone on your PC.

RUNNING ARDUPILOT SITL (REFERENCE)

```
# ArduPilot SITL: the real copter firmware with a simulated drone
sim_vehicle.py -v ArduCopter --map --console
# in the MAVProxy console:
mode GUIDED
arm throttle
takeoff 10
mode AUTO                  # fly the loaded waypoint mission
```

MAKE_MISSION.PY

```
import math
home = (12.971600, 77.594600)                # latitude, longitude
alt = 20                                     # metres above home
square_m = 40                                # survey a 40 m square
dlat = square_m / 111320                     # metres -> degrees latitude
dlon = square_m / (111320 * math.cos(math.radians(home[0])))
wps = [home, (home[0] + dlat, home[1]), (home[0] + dlat, home[1] + dlon), (home[0], home[1] + dlon),
home]

lines = ["QGC WPL 110", f"0\t1\t0\t16\t0\t0\t0\t0\t{home[0]:.6f}\t{home[1]:.6f}\t0\t1"]
lines.append(f"1\t0\t3\t22\t0\t0\t0\t0\t0\t0\t{alt}\t1")                  # 22 = TAKEOFF
for i, (la, lo) in enumerate(wps[1:], 2):
    lines.append(f"{i}\t0\t3\t16\t0\t0\t0\t0\t{la:.6f}\t{lo:.6f}\t{alt}\t1")   # 16 = WAYPOINT
lines.append(f"{len(wps) + 1}\t0\t3\t20\t0\t0\t0\t0\t0\t0\t0\t1")            # 20 = RETURN TO LAUNCH
open("square.waypoints", "w").write("\n".join(lines) + "\n")
print("\n".join(lines))
dist = sum(math.dist(((a[0] - home[0]) * 111320, (a[1] - home[1]) * 111320 *
math.cos(math.radians(home[0]))),
                     ((b[0] - home[0]) * 111320, (b[1] - home[1]) * 111320 *
math.cos(math.radians(home[0]))))
           for a, b in zip(wps, wps[1:]))
print(f"route length {dist:.0f} m, about {dist / 5:.0f} s at 5 m/s")
```

OUTPUT

```
QGC WPL 110
0       1       0       16      0       0       0       0       12.971600       77.594600       0
1
1       0       3       22      0       0       0       0       0       0       20      1
2       0       3       16      0       0       0       0       12.971959       77.594600       20
1
3       0       3       16      0       0       0       0       12.971959       77.594969       20
1
4       0       3       16      0       0       0       0       12.971600       77.594969       20
1
5       0       3       16      0       0       0       0       12.971600       77.594600       20
1
6       0       3       20      0       0       0       0       0       0       0       1
route length 160 m, about 32 s at 5 m/s
```

This writes a mission file in the plain-text format that Mission Planner and QGroundControl load: take off to 20 m, fly a 40 m square, return to launch. Load it into SITL first and watch the simulated drone fly it before any real flight.

22.4 Safety and regulations

22.4.1 Indian drone rules and Digital Sky platform (overview)

Civil drones in India are governed by the **Drone Rules, 2021** (with later amendments) administered by the DGCA through the online **Digital Sky** platform. Rules change, so always check digitalsky.dgca.gov.in and the DGCA website before flying. The main points:

Topic	Summary
Weight categories	nano ≤ 250 g, micro 250 g–2 kg, small 2–25 kg, medium 25–150 kg, large > 150 kg
Registration	drones need a Unique Identification Number (UIN) on Digital Sky; nano drones and model aircraft have relaxations
Remote Pilot Certificate	needed to operate drones above 2 kg (and micro drones used commercially), from a DGCA-authorised training school
Airspace map	green zone: up to 400 ft (120 m), no permission needed; yellow zone: permission from the air traffic authority; red zone: flying only with central government permission
Other	third-party insurance for commercial use; no flying over crowds, near airports or sensitive installations without permission

22.4.2 Safe flying practices

- Check the airspace map on Digital Sky and local restrictions before every flight.
- Pre-flight check: props tight and undamaged, battery charged and secured, GPS lock and home point set, failsafe tested, compass calibrated away from metal.
- Keep the drone in visual line of sight, below 120 m, and away from people, roads and animals.
- Remove propellers whenever the drone is powered on the bench.
- Arm only when ready, land with at least 20–30 % battery, and never fly in rain or strong wind beyond the drone's rating.

- Respect privacy: do not photograph people or property without consent.

22.5 Applications

22.5.1 Agriculture spraying, mapping, inspection, delivery

Application	How drones help	Typical drone
Agriculture spraying	spray pesticide or fertiliser evenly over fields, far faster than manual spraying (supported in India by schemes such as Namo Drone Didi)	10–20 L tank multirotor
Mapping and surveying	overlapping photos stitched into maps and 3-D models (photogrammetry); land records (SVAMITVA scheme)	fixed-wing or quad with RTK GPS
Inspection	power lines, bridges, wind turbines, solar farms (thermal cameras find hot spots)	quad with zoom/thermal camera
Delivery	medicines and samples to remote areas, faster than road	VTOL or multirotor with drop mechanism
Disaster response	search and rescue, flood mapping, dropping supplies	long-endurance multirotor
Film and events	aerial cinematography, light shows	camera drones, swarms

PRACTICE CHECKPOINT

Identify every component of a quadcopter and its role, configure a flight controller in a simulator, and complete a simulated autonomous waypoint mission.

22.6 Practice questions

22.6.1 Basic level

Q1 How many propellers does a quadcopter have?

(a) 2 **(b)** 6 **(c)** 4 **(d)** 8

Q2 Why do two propellers spin clockwise and two counter-clockwise?

(a) so their reaction torques cancel and the drone does not spin
(b) to make more noise
(c) to save battery only
(d) for decoration

Q3 Which drone type flies longest and covers large areas but cannot hover?

(a) quadcopter **(b)** hexacopter **(c)** helicopter **(d)** fixed-wing

Q4 What does an ESC do?

(a) stores energy
(b) drives a brushless motor at the speed commanded by the flight controller
(c) receives radio signals
(d) measures altitude

Q5 What does KV mean for a BLDC motor?

(a) kilovolts **(b)** rpm per volt with no load
(c) kilowatts **(d)** torque per amp

Q6 What does a 4S LiPo mean?

(a) 4000 mAh **(b)** 4 C rating
(c) 4 hours **(d)** 4 cells in series, 14.8 V nominal

Q7 Which sensor in the flight controller measures rotation rates?

(a) gyroscope **(b)** GPS **(c)** barometer **(d)** camera

Q8 Which protocol do ArduPilot and PX4 use to talk to ground stations?

(a) MQTT **(b)** SBUS **(c)** MAVLink **(d)** HTTP

22.6.2 Intermediate level

Q9 How does a quadcopter yaw?

(a) by tilting the frame
(b) with a rudder
(c) by turning the propellers off
(d) by speeding up one diagonal pair (same direction) and slowing the other

Q10 To pitch forward, which motors speed up?

(a) the front motors **(b)** the rear motors
(c) the left motors **(d)** all motors equally

Q11 Which flight mode levels the drone automatically when the sticks are centred?

(a) Angle / Stabilize **(b)** Acro
(c) Manual passthrough **(d)** Turtle mode

Q12 Which flight mode holds position using GPS?

(a) Acro **(b)** Angle
(c) Loiter / Position hold **(d)** Turtle

Q13 What should a properly configured failsafe do when the radio signal is lost?

(a) land or return to launch **(b)** continue at full throttle
(c) flip over **(d)** turn off the camera only

Q14 Which firmware is designed for FPV racing and freestyle?

(a) PX4 **(b)** Betaflight
(c) ArduPilot Rover **(d)** Mission Planner

Q15 What is SITL?

(a) a type of propeller
(b) a LiPo charger
(c) a drone licence
(d) running real autopilot firmware with a simulated drone on a PC

Q16 What is the maximum height in India's green zone without permission?

(a) 50 ft **(b)** 1000 ft
(c) 400 ft (about 120 m) **(d)** no limit

Q17 Which drone category in India weighs 250 g or less?

(a) micro **(b)** small **(c)** nano **(d)** medium

22.6.3 Advanced level

Q18 A drone weighs 1.5 kg and each of its 4 motors gives 0.9 kg maximum thrust. What is its thrust-to-weight ratio?

(a) 1.7 : 1 **(b)** 3.6 : 1 **(c)** 0.6 : 1 **(d)** 2.4 : 1

Q19 A 1500 mAh 100C LiPo: what maximum continuous current is rated?

(a) 15 A **(b)** 150 A **(c)** 1.5 A **(d)** 1500 A

Q20 Estimate the hover time of a 5000 mAh battery at 20 A hover current using only 80 % of the capacity, and the time with 15 A.

Q21 Compute the unloaded motor speed and propeller tip speed of a 2400 KV motor on a 4S battery (16.8 V full) with 5-inch props.

Q22 Check a LiPo pack's cell voltages before flight: warn if any cell is below 3.7 V or the cells differ by more than 0.05 V.

Q23 Mix stick commands for an X-quad: hover at 0.5 throttle with a small left roll (-0.08) and print the four motor outputs.

Q24 A camera drone wobbles slowly in a gentle drift when hovering in wind. Which PID term is most

likely too low?

(a) I (integral) **(b)** P only

(c) D only **(d)** none: PID cannot help

Q25 A farmer wants to spray 10 acres of crops quickly and evenly. Which drone fits?

(a) a 250 g nano drone

(b) an agricultural multirotor with a spray tank, operated by a certified remote pilot

(c) an FPV racing quad

(d) a fixed-wing mapping drone without a tank

22.7 Solutions

Q1 **Answer: (c) 4**

Quad = four.

Q2 **Answer: (a) so their reaction torques cancel and the drone does not spin**

Torque balance.

Q3 **Answer: (d) fixed-wing**

Wings give lift.

Q4 **Answer: (b) drives a brushless motor at the speed commanded by the flight controller**

Electronic speed controller.

Q5 **Answer: (b) rpm per volt with no load**

2400 KV × 14.8 V ≈ 35 500 rpm.

Q6 **Answer: (d) 4 cells in series, 14.8 V nominal**

3.7 V per cell.

Q7 **Answer: (a) gyroscope**

IMU gyro.

Q8 **Answer: (c) MAVLink**

Micro air vehicle link.

Q9 **Answer: (d) by speeding up one diagonal pair (same direction) and slowing the other**

Reaction torque.

Q10 **Answer: (b) the rear motors**

Nose down.

Q11 **Answer: (a) Angle / Stabilize**

Self-levelling.

Q12 **Answer: (c) Loiter / Position hold**

GPS hold.

Q13 **Answer: (a) land or return to launch**

Safety.

Q14 **Answer: (b) Betaflight**

Manual acro flight.

Q15 **Answer: (d) running real autopilot firmware with a simulated drone on a PC**

Software in the loop.

Q16 **Answer: (c) 400 ft (about 120 m)**

Drone Rules 2021.

Q17 **Answer: (c) nano**

Nano ≤ 250 g.

Q18 **Answer: (d) 2.4 : 1**

4 × 0.9 ÷ 1.5 = **2.4**.

Q19 **Answer: (b) 150 A**

100 × 1.5 A = **150 A**.

Q20 **Answer: Solution program**

4 Ah usable ÷ 20 A = 0.2 h = 12 min; at 15 A, 16 min. Lighter drones and efficient props draw less current and fly longer.

```
for amps in (20, 15):
    minutes = 5.0 * 0.8 / amps * 60
    print(f"hover at {amps} A: {minutes:.1f} min")
```

OUTPUT

```
hover at 20 A: 12.0 min
hover at 15 A: 16.0 min
```

Q21 **Answer: Solution program**

Under load the speed drops by roughly a quarter, but tip speeds of about 200 m/s still explain why props are noisy and dangerous: always remove them for bench work.

```
import math
kv, volts, prop_in = 2400, 16.8, 5
rpm = kv * volts
tip = math.pi * prop_in * 0.0254 * rpm / 60
print(f"unloaded speed: {rpm:.0f} rpm")
print(f"tip speed: {tip:.0f} m/s ({tip / 343 * 100:.0f} % of the speed of sound)")
```

OUTPUT

```
unloaded speed: 40320 rpm
tip speed: 268 m/s (78 % of the speed of sound)
```

Q22 **Answer: Solution program**

A balance check catches a weak cell that a total-voltage reading would hide (pack B). Pack C must be charged first.

```
packs = {"pack A": [4.18, 4.17, 4.19, 4.18], "pack B": [3.95, 3.81, 3.96, 3.95], "pack C": [3.65,
3.66, 3.64, 3.65]}
for name, cells in packs.items():
    spread = max(cells) - min(cells)
    issues = []
    if min(cells) < 3.7: issues.append("too low to fly")
    if spread > 0.05: issues.append(f"imbalanced ({spread:.2f} V)")
    print(f"{name}: total {sum(cells):.2f} V ->", ", ".join(issues) if issues else "OK")
```

OUTPUT

```
pack A: total 16.72 V -> OK
pack B: total 15.67 V -> imbalanced (0.15 V)
pack C: total 14.60 V -> too low to fly
```

Q23 **Answer: Solution program**

Rolling left speeds up the right-side motors (M1, M4) and slows the left side (M2, M3).

```
def mix(t, r, p, y):
    return [round(v, 2) for v in (t - r + p + y, t + r - p + y, t + r + p - y, t - r - p - y)]
print("M1-FR, M2-RL, M3-FL, M4-RR =", mix(0.5, -0.08, 0, 0))
```

OUTPUT

```
M1-FR, M2-RL, M3-FL, M4-RR = [0.58, 0.42, 0.42, 0.58]
```

Q24 **Answer: (a) I (integral)**

I holds attitude against steady disturbances.

Q25 **Answer: (b) an agricultural multirotor with a spray tank, operated by a certified remote pilot**

Spraying drone + RPC.

Types: multirotor, fixed-wing, VTOL, helicopter. Quad: CW/CCW pairs; mixer turns throttle/roll/pitch/yaw into 4 motor speeds. Parts: frame, BLDC (KV), props (5045), ESC (DShot), FC + IMU, LiPo (S, mAh, C; handle safely), radio + failsafe, GPS + MAVLink telemetry. Modes: acro, angle, alt hold, loiter, auto, RTL. PID: P sharp, I holds, D damps. Betaflight, INAV, ArduPilot, PX4; test in SITL. India: Drone Rules 2021, Digital Sky, UIN, RPC, green/yellow/red zones, 120 m.

MODULE 23 · ADVANCED

IoT robotics and cloud connectivity

Minimum duration **2 days** | Level **Advanced**

Connecting robots to the internet enables remote monitoring and control. This module explains the device–gateway–cloud–app structure of the Internet of Things, the protocols robots use (HTTP and REST, MQTT publish-subscribe and WebSockets), IoT platforms (Blynk, ThingSpeak, Arduino Cloud, Firebase) and Node-RED dashboards, remote control with live dashboards and camera streaming, and security for connected robots (authentication, encryption and safe updates), followed by 25 solved questions and programs. MQTT programs ran against a simulated in-process broker and HTTP calls against a simulated server.

IN THIS MODULE

23.1 Internet of Things concepts
23.2 IoT protocols
23.3 IoT platforms
23.4 Remote robot control
23.5 Security for connected robots
23.6 Practice questions
23.7 Solutions

23.1 Internet of Things concepts

23.1.1 Devices, gateways, cloud and apps



THE FOUR LAYERS OF AN IOT SYSTEM

Layer	Role	Robot example
Device	senses and acts	ESP32 robot reads battery and distance, drives motors
Gateway	bridges local radios to the internet, filters data	Raspberry Pi running a Mosquitto broker and Node-RED
Cloud	stores data, routes messages, runs rules	MQTT broker, time-series database, alerts
App	shows data and sends commands	web dashboard or phone app with a joystick and camera view

NOTE

Keep **safety-critical control on the robot**. The cloud is for monitoring, logging and high-level commands; a robot must stop safely by itself if the internet drops.

23.2 IoT protocols

Protocol	Pattern	Strengths	Typical robot use
HTTP / REST	request → response	universal, simple, works through firewalls	upload readings, read settings, call cloud APIs
MQTT	publish / subscribe through a broker	tiny overhead, many devices, QoS, retained messages, last will	telemetry and commands for fleets of robots
WebSockets	two-way connection kept open	low latency, browser-native	live joystick control and telemetry in a web page
CoAP	REST over UDP	very light	battery sensors (less common in hobby robots)

23.2.1 HTTP and REST

REST APIs expose resources at URLs and use HTTP methods: GET (read), POST (create/send), PUT/PATCH (update), DELETE. Data is usually JSON. Status codes tell what happened: 200 OK, 201 Created, 400 bad request, 401 unauthorised, 404 not found, 500 server error.

THINGSPEAK_UPLOAD.PY

```
import requests, time

API = "https://api.thingspeak.com/update"
KEY = "WRITE_API_KEY"                                  # from the ThingSpeak channel settings

def upload(battery_v, distance_cm, retries=3):
    for attempt in range(1, retries + 1):
        try:
            r = requests.get(API, params={"api_key": KEY, "field1": battery_v, "field2":
distance_cm}, timeout=5)
            r.raise_for_status()
            print(f"uploaded, entry id {r.text}")
            return True
        except requests.exceptions.RequestException as e:
            print(f"attempt {attempt} failed: {type(e).__name__}")
            time.sleep(2 * attempt)                     # back off before retrying
    return False

for v, d in [(7.9, 42), (7.8, 35), (7.8, 18)]:
    upload(v, d)
    time.sleep(15)                                      # free ThingSpeak accounts accept one update
per 15 s
```

SIMULATED RUN (hardware events and printed output)

```
[  0.15 s] HTTP GET https://api.thingspeak.com/update {'api_key': 'WRITE_API_KEY', 'field1': 7.9,
'field2': 42}
```

```
uploaded, entry id 101
attempt 1 failed: ConnectionError
[ 17.45 s] HTTP GET https://api.thingspeak.com/update {'api_key': 'WRITE_API_KEY', 'field1': 7.8,
'field2': 35}
uploaded, entry id 103
[ 32.60 s] HTTP GET https://api.thingspeak.com/update {'api_key': 'WRITE_API_KEY', 'field1': 7.8,
'field2': 18}
uploaded, entry id 104
```

Network calls fail: always use a **timeout**, catch exceptions, and retry with a growing delay. Here the second request failed once and succeeded on the retry.

23.2.2 MQTT publish-subscribe

Devices **publish** messages to **topics** on a **broker** (Mosquitto, HiveMQ, EMQX); any client **subscribed** to the topic receives them. Topics form a hierarchy, e.g. `fleet/robot1/battery`; wildcards: `+` one level (`fleet/+/battery`), `#` everything below (`fleet/robot1/#`). **QoS** 0 = at most once, 1 = at least once, 2 = exactly once. A **retained** message is kept for new subscribers; a **last will** is published by the broker if a device disconnects unexpectedly.

Topic	Direction	Payload example
fleet/robot1/telemetry	robot → cloud	{"bat": 7.8, "dist": 35, "state": "FORWARD"}
fleet/robot1/cmd	cloud → robot	{"v": 0.2, "w": 0.0}
fleet/robot1/status	robot → cloud (retained, last will "offline")	online / offline

ROBOTMQTT.INO

```
#include <WiFi.h>
#include <PubSubClient.h>
WiFiClient net;
PubSubClient mqtt(net);
unsigned long lastSend = 0;
float battery = 7.9;

void onMessage(char *topic, byte *payload, unsigned int len) {
  String msg; for (unsigned int i = 0; i < len; i++) msg += (char)payload[i];
  Serial.print("command on "); Serial.print(topic); Serial.print(": "); Serial.println(msg);
}

void setup() {
  Serial.begin(115200);
  WiFi.begin("HomeWiFi", "password");
  while (WiFi.status() != WL_CONNECTED) delay(200);
  mqtt.setServer("192.168.1.20", 1883);
  mqtt.setCallback(onMessage);
  mqtt.connect("robot1");
  mqtt.subscribe("fleet/robot1/cmd");
}

void loop() {
  mqtt.loop();                                    // keep the connection alive, receive commands
  if (millis() - lastSend >= 1000) {
    lastSend = millis();
    char json[64];
    snprintf(json, sizeof(json), "{\"bat\":%.2f,\"dist\":%d}", battery, 40 - (int)(millis() / 1000)
* 5);
    mqtt.publish("fleet/robot1/telemetry", json);
    battery -= 0.01;
  }
}
```

SIMULATED RUN (pin changes and Serial output)

```
[     0 ms] WiFi connecting to HomeWiFi
[   600 ms] MQTT broker 192.168.1.20:1883
[   600 ms] MQTT connected as robot1
[   600 ms] MQTT subscribe fleet/robot1/cmd
command on fleet/robot1/cmd: {"v":0.2,"w":0.0}
[  1000 ms] MQTT publish fleet/robot1/telemetry = {"bat":7.90,"dist":35}
[  2000 ms] MQTT publish fleet/robot1/telemetry = {"bat":7.89,"dist":30}
[  3000 ms] MQTT publish fleet/robot1/telemetry = {"bat":7.88,"dist":25}
```

On the Raspberry Pi (or a cloud server) a Python program drives the robot from MQTT commands, publishes its state, and stops automatically if commands stop arriving:

MQTT_ROBOT.PY

```
import json, time
import paho.mqtt.client as mqtt
from gpiozero import Robot

robot = Robot(left=(4, 14), right=(17, 18))
last_cmd = time.time()

def on_connect(client, userdata, flags, rc, properties):
    client.subscribe("fleet/robot1/cmd", qos=1)
    client.publish("fleet/robot1/status", "online", retain=True)

def on_message(client, userdata, msg):
    global last_cmd
    try:
        c = json.loads(msg.payload)
        v, w = float(c["v"]), float(c.get("w", 0))
    except (ValueError, KeyError):
        print("ignored bad command:", msg.payload.decode()); return
    v, w = max(-1, min(1, v)), max(-1, min(1, w))          # clamp to safe limits
    left, right = max(-1, min(1, v - w)), max(-1, min(1, v + w))
    robot.value = (left, right)
    last_cmd = time.time()
    print(f"cmd v={v} w={w} -> motors ({left:.1f}, {right:.1f})")

client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2, client_id="robot1-pi")
client.will_set("fleet/robot1/status", "offline", retain=True)
client.on_connect, client.on_message = on_connect, on_message
client.connect("localhost", 1883)
client.loop_start()
while True:                                                # dead-man watchdog
    if time.time() - last_cmd > 1.0 and robot.value != (0, 0):
        robot.stop(); print("no command for 1 s -> STOP")
    time.sleep(0.1)
```

SIMULATED RUN (hardware events and printed output)

```
[  0.00 s] MQTT robot1-pi connect localhost:1883 -> accepted
[  0.00 s] MQTT robot1-pi subscribed fleet/robot1/cmd (QoS 1)
[  0.00 s] MQTT publish fleet/robot1/status online [retained]
[  1.00 s] Robot value (0.60, 0.60)
cmd v=0.6 w=0.0 -> motors (0.6, 0.6)
[  1.50 s] Robot value (0.20, 0.80)
cmd v=0.5 w=0.3 -> motors (0.2, 0.8)
ignored bad command: jump!
[  2.40 s] Robot value (1.00, 1.00)
cmd v=1 w=0.0 -> motors (1.0, 1.0)
[  3.50 s] Robot stop
no command for 1 s -> STOP
```

Bad commands are ignored, values are clamped, and when the operator's app stops sending (lost Wi-Fi, closed browser) the watchdog stops the robot after one second.

23.2.3 WebSockets

A **WebSocket** starts as an HTTP request and then stays open, so the browser and the robot can push messages to each other at any time with very little overhead: ideal for a joystick page that sends commands 20 times per second and receives telemetry back. Many brokers also accept MQTT over WebSockets, so a web page can be an MQTT client directly.

JOYSTICK CONTROL OVER WEBSOCKETS (REFERENCE)

```
# robot side (Python, pip install websockets)
import asyncio, json, websockets

async def handler(ws):
    async for text in ws:                       # each joystick update from the browser
        cmd = json.loads(text)
        drive(cmd["v"], cmd["w"])
        await ws.send(json.dumps({"bat": read_battery(), "dist": read_sonar()}))

async def main():
    async with websockets.serve(handler, "0.0.0.0", 8765):
        await asyncio.Future()                  # run forever

asyncio.run(main())

// browser side (JavaScript)
const ws = new WebSocket("ws://robot.local:8765");
ws.onmessage = e => showTelemetry(JSON.parse(e.data));
setInterval(() => ws.send(JSON.stringify({v: joyY, w: joyX})), 50);   // 20 Hz
```

23.3 IoT platforms

23.3.1 Blynk, ThingSpeak, Arduino Cloud, Firebase

Platform	What it offers	Best for
Blynk	drag-and-drop phone and web dashboards, ESP32/Arduino library, virtual pins	quick phone control of a robot
ThingSpeak (MathWorks)	store and chart sensor data over HTTP/MQTT, MATLAB analysis	logging and graphs of telemetry
Arduino Cloud	auto-generated sketches from "cloud variables", dashboards, OTA updates	Arduino/ESP32 projects with little code
Firebase (Google)	real-time database, authentication, hosting	custom web/mobile apps for robots
Self-hosted (Mosquitto + Node-RED + InfluxDB + Grafana)	full control, works on a local network without internet	labs, privacy, many robots

23.3.2 Node-RED dashboards

Node-RED (runs on a Raspberry Pi) builds programs by wiring nodes in a browser: an MQTT-in node receives telemetry, a function node processes it, and dashboard nodes show gauges, charts, buttons and sliders. Flows are saved as JSON and can be imported:

NODE-RED FLOW (SIMPLIFIED): BATTERY GAUGE AND A STOP BUTTON

```
[
  {"id": "in1", "type": "mqtt in", "topic": "fleet/robot1/telemetry", "broker": "local", "wires":
[["json1"]]},
  {"id": "json1", "type": "json", "wires": [["bat"]]},
  {"id": "bat", "type": "change", "rules": [{"t": "set", "p": "payload", "to": "payload.bat", "tot":
"msg"}], "wires": [["gauge1"]]},
  {"id": "gauge1", "type": "ui_gauge", "label": "Battery (V)", "min": 6.4, "max": 8.4},
  {"id": "btn1", "type": "ui_button", "label": "STOP", "payload": "{\"v\":0,\"w\":0}", "wires":
[["out1"]]},
  {"id": "out1", "type": "mqtt out", "topic": "fleet/robot1/cmd", "broker": "local"}
]
```

23.4 Remote robot control

23.4.1 Live dashboards and camera streaming

Method	Latency (typical)	Notes
MJPEG over HTTP	150–400 ms	simplest: an <img> tag shows the stream; ESP32-CAM and Picamera2 examples available
WebRTC	100–300 ms, even over the internet	browser-native video; used by many telepresence robots
RTSP / H.264	0.3–2 s	efficient for recording and CCTV-style viewing
ROS 2 web tools (rosbridge, web_video_server, Foxglove)	depends on the link	view ROS topics and video in a browser

MJPEG CAMERA STREAM FROM A PI ROBOT (REFERENCE)

```
from flask import Flask, Response
from picamera2 import Picamera2
import cv2

app = Flask(__name__)
cam = Picamera2(); cam.configure(cam.create_video_configuration(main={"size": (640, 480)}));
cam.start()

def frames():
    while True:
        ok, jpg = cv2.imencode(".jpg", cam.capture_array(), [cv2.IMWRITE_JPEG_QUALITY, 70])
        yield b"--frame\r\nContent-Type: image/jpeg\r\n\r\n" + jpg.tobytes() + b"\r\n"

@app.route("/stream")
def stream():
    return Response(frames(), mimetype="multipart/x-mixed-replace; boundary=frame")

app.run(host="0.0.0.0", port=8000)       # view at http://robot.local:8000/stream
```

For control over the internet (not just the local Wi-Fi), avoid opening router ports. Use a cloud MQTT broker with authentication, a VPN such as Tailscale or WireGuard, or a secure tunnel service, and keep the dead-man watchdog on the robot because internet latency is unpredictable.

23.5 Security for connected robots

23.5.1 Authentication, encryption and safe updates

Threat	Defence
Someone else drives your robot	broker usernames/passwords or client certificates; per-robot credentials; access-control lists per topic
Eavesdropping, stolen passwords	TLS encryption (MQTT on port 8883, HTTPS, WSS)
Replayed or forged commands	signed commands with a timestamp or counter (HMAC)
Default passwords, open ports	change defaults, close unused services, keep firmware updated
Malicious firmware update	signed OTA images, verify before flashing, roll back on failure
Unsafe commands	limits checked on the robot itself, physical emergency stop

SIGNED_COMMANDS.PY

```
import hmac, hashlib, json, time

SECRET = b"robot1-shared-secret"          # stored on the robot and the controller, never sent

def sign(cmd, counter):
    body = json.dumps({"cmd": cmd, "n": counter}, sort_keys=True)
    tag = hmac.new(SECRET, body.encode(), hashlib.sha256).hexdigest()[:16]
    return json.dumps({"body": body, "tag": tag})

last_seen = 0
def verify(message):
    global last_seen
    m = json.loads(message)
    good = hmac.new(SECRET, m["body"].encode(), hashlib.sha256).hexdigest()[:16]
    if not hmac.compare_digest(good, m["tag"]):
        return "REJECT: bad signature"
    body = json.loads(m["body"])
    if body["n"] <= last_seen:
        return "REJECT: replayed (old counter)"
    last_seen = body["n"]
    return f"ACCEPT {body['cmd']}"

m1 = sign({"v": 0.3, "w": 0}, 1)
m2 = sign({"v": 0.0, "w": 1}, 2)
forged = m2.replace('\\"w\\": 1', '\\"w\\": 9')     # attacker edits the command
print(verify(m1)); print(verify(m2)); print(verify(m1)); print(verify(forged))
```

OUTPUT

```
ACCEPT {'v': 0.3, 'w': 0}
ACCEPT {'v': 0.0, 'w': 1}
REJECT: replayed (old counter)
REJECT: bad signature
```

The robot accepts only commands signed with the shared secret and with a counter higher than the last one, so an attacker can neither forge a command nor record and replay an old one. TLS protects the link; signing protects each command end to end.

MQTT OVER TLS WITH A USERNAME AND PASSWORD (REFERENCE)

```
import ssl, paho.mqtt.client as mqtt
c = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2, client_id="robot1")
c.username_pw_set("robot1", "long-unique-password")
c.tls_set(ca_certs="ca.crt", cert_reqs=ssl.CERT_REQUIRED)     # verify the broker's certificate
c.connect("broker.example.com", 8883)
```

PRACTICE CHECKPOINT

Build a Wi-Fi robot that sends sensor data to a cloud dashboard over MQTT and can be driven remotely from a web or mobile interface.

23.6 Practice questions

23.6.1 Basic level

Q1 What does IoT stand for?

(a) Internet of Things **(b)** Input of Technology
(c) Internet of Tools **(d)** Integrated Online Terminal

Q2 Which device usually bridges local sensors to the internet?

(a) a servo **(b)** a motor driver **(c)** a gateway **(d)** an LED

Q3 Which protocol uses publish/subscribe through a broker?

(a) HTTP **(b)** I2C **(c)** SPI **(d)** MQTT

Q4 Which HTTP method normally sends new data to a server?

(a) GET **(b)** DELETE **(c)** HEAD **(d)** POST

Q5 What does HTTP status 404 mean?

(a) OK **(b)** server error **(c)** not found **(d)** unauthorised

Q6 Which tool builds flows and dashboards on a Raspberry Pi by wiring nodes?

(a) Node-RED **(b)** Gazebo **(c)** colcon **(d)** Fritzing

Q7 Which platform is known for drag-and-drop phone dashboards for ESP32 projects?

(a) Gazebo **(b)** RViz **(c)** Betaflight **(d)** Blynk

Q8 Which port is standard for MQTT over TLS?

(a) 1883 **(b)** 8883 **(c)** 80 **(d)** 22

23.6.2 Intermediate level

Q9 Which MQTT wildcard matches exactly one topic level?

(a) # **(b)** + **(c)** * **(d)** ?

Q10 What is a retained MQTT message?

(a) a deleted message
(b) an encrypted message
(c) a message larger than 1 MB
(d) the last message kept by the broker and sent to new subscribers

Q11 What is an MQTT last will used for?

(a) the broker announces a device went offline unexpectedly
(b) storing passwords
(c) firmware updates
(d) compressing data

Q12 Which QoS level guarantees delivery at least once (duplicates possible)?

(a) QoS 0 **(b)** QoS 2 **(c)** QoS 1 **(d)** QoS 3

Q13 Why are WebSockets good for browser joystick control?

(a) they need no network
(b) they compress video
(c) they replace Wi-Fi
(d) the connection stays open for low-latency messages in both directions

Q14 What should a remote-controlled robot do if commands stop arriving?

(a) keep the last speed forever
(b) stop automatically after a short timeout
(c) speed up
(d) reboot the router

Q15 Which streaming method shows video in a plain tag?

(a) MJPEG over HTTP **(b)** RTSP only
(c) MQTT QoS 2 **(d)** I2C

Q16 What does TLS provide?

(a) faster motors
(b) longer battery life
(c) encryption and server authentication for the connection
(d) compression only

Q17 Why sign commands with an HMAC and a counter?

(a) to detect forged and replayed commands **(b)** to make messages shorter
(c) to speed up Wi-Fi **(d)** to compress JSON

23.6.3 Advanced level

Q18 A robot publishes 200-byte telemetry 5 times per second. Roughly how much data is that per hour?

(a) about 36 KB **(b)** about 3.6 MB **(c)** about 360 MB **(d)** about 1 GB

Q19 A free ThingSpeak channel accepts one update per 15 s. How many updates per hour is that at most?

(a) 60 **(b)** 15 **(c)** 900 **(d)** 240

Q20 Build an MQTT telemetry message as JSON for a robot and check that it stays under 128 bytes.

Q21 Match MQTT topics against subscriptions with + and # wildcards.

Q22 Retry an upload with exponential back-off (1, 2, 4 s) when the first two attempts fail, and print the total waiting time.

Q23 Verify that an HMAC tag changes if even one character of a command changes.

Q24 Your robot must be driven from a phone over the internet. Which setup is safest?

(a) open port 1883 on the home router with no password
(b) share the Wi-Fi password publicly
(c) a cloud MQTT broker with TLS and per-robot passwords (or a VPN), plus a watchdog on the robot

(d) disable the watchdog to reduce lag

Q25 A fleet of 30 robots must report battery levels to one dashboard and receive individual commands. Which design fits?

(a) one HTTP server per robot polled every second

(b) Bluetooth from each robot to the phone

(c) MQTT with topics fleet/ID/battery and fleet/ID/cmd for each robot ID

(d) I2C cables to a central PC

23.7 Solutions

Q1 **Answer: (a) Internet of Things**

Connected devices.

Q2 **Answer: (c) a gateway**

Gateway layer.

Q3 **Answer: (d) MQTT**

Broker-based.

Q4 **Answer: (d) POST**

Create/send.

Q5 **Answer: (c) not found**

Resource missing.

Q6 **Answer: (a) Node-RED**

Flow-based.

Q7 **Answer: (d) Blynk**

Mobile dashboards.

Q8 **Answer: (b) 8883**

1883 plain, 8883 TLS.

Q9 **Answer: (b) +**

fleet/+/battery.

Q10 **Answer: (d) the last message kept by the broker and sent to new subscribers**

Latest state.

Q11 **Answer: (a) the broker announces a device went offline unexpectedly**

Presence.

Q12 **Answer: (c) QoS 1**

At least once.

Q13 **Answer: (d) the connection stays open for low-latency messages in both directions**

Full duplex.

Q14 **Answer: (b) stop automatically after a short timeout**

Dead-man watchdog.

Q15 **Answer: (a) MJPEG over HTTP**

Multipart JPEGs.

Q16 **Answer: (c) encryption and server authentication for the connection**

Secure transport.

Q17 **Answer: (a) to detect forged and replayed commands**

Integrity + freshness.

Q18 **Answer: (b) about 3.6 MB**

200 × 5 × 3600 = **3.6 MB** (plus protocol overhead).

Q19 **Answer: (d) 240**

3600 ÷ 15 = **240**.

Q20 **Answer: Solution program**

Compact separators and short keys keep messages small, which matters on slow links and for brokers with message limits.

```
import json, time
msg = {"id": "robot1", "bat": 7.82, "dist": 35, "state": "FORWARD", "x": 1.25, "y": -0.40, "ts":
1773565200}
payload = json.dumps(msg, separators=(",", ":"))            # compact: no spaces
print(payload)
print(len(payload), "bytes ->", "OK" if len(payload) < 128 else "too big")
```

OUTPUT

```
{"id":"robot1","bat":7.82,"dist":35,"state":"FORWARD","x":1.25,"y":-0.4,"ts":1773565200}
88 bytes -> OK
```

Q21 **Answer: Solution program**

+ replaces exactly one level, # everything below. This is how the broker decides which clients receive each message.

```
def matches(sub, topic):
    s, t = sub.split("/"), topic.split("/")
    for i, part in enumerate(s):
        if part == "#": return True
        if i >= len(t) or (part != "+" and part != t[i]): return False
    return len(s) == len(t)
subs = ["fleet/+/battery", "fleet/robot1/#", "fleet/robot2/cmd"]
for topic in ["fleet/robot1/battery", "fleet/robot2/battery", "fleet/robot1/arm/joint1",
"fleet/robot2/cmd"]:
    print(f"{topic:26}", [s for s in subs if matches(s, topic)])
```

OUTPUT

```
fleet/robot1/battery       ['fleet/+/battery', 'fleet/robot1/#']
fleet/robot2/battery       ['fleet/+/battery']
fleet/robot1/arm/joint1    ['fleet/robot1/#']
fleet/robot2/cmd           ['fleet/robot2/cmd']
```

Q22 **Answer: Solution program**

Doubling the delay avoids hammering a struggling server and saves battery; add a maximum delay and a little random jitter for fleets.

```
results = [False, False, True]          # simulated network: fails twice, then works
waited = 0
for attempt, ok in enumerate(results):
    if ok:
        print(f"attempt {attempt + 1}: success after waiting {waited} s"); break
    delay = 2 ** attempt
    print(f"attempt {attempt + 1}: failed, retry in {delay} s")
    waited += delay
```

OUTPUT

```
attempt 1: failed, retry in 1 s
attempt 2: failed, retry in 2 s
attempt 3: success after waiting 3 s
```

Q23 **Answer: Solution program**

Without the key an attacker cannot compute the right tag for a modified command, so the robot rejects it.

```
import hmac, hashlib
key = b"secret"
for cmd in ['{"v":0.3}', '{"v":0.4}']:
    print(cmd, hmac.new(key, cmd.encode(), hashlib.sha256).hexdigest()[:16])
```

OUTPUT

```
{"v":0.3} 995f0a4eafb8ddaa
{"v":0.4} 907c2e6471b7f8ca
```

Q24 **Answer: (c) a cloud MQTT broker with TLS and per-robot passwords (or a VPN), plus a watchdog on the robot**

Encrypt, authenticate, fail safe.

Q25 **Answer: (c) MQTT with topics fleet/ID/battery and fleet/ID/cmd for each robot ID**

Scales with topics.

IoT layers: device → gateway → cloud → app; safety stays on the robot. HTTP/REST (GET/POST, JSON, status codes, timeouts and retries), MQTT (broker, topics, + and #, QoS 0/1/2, retained, last will), WebSockets for live control. Platforms: Blynk, ThingSpeak, Arduino Cloud, Firebase, self-hosted Node-RED. Video: MJPEG, WebRTC. Security: authentication, TLS (8883), signed commands with counters, signed OTA, dead-man watchdog.

MODULE 24 · ADVANCED

Mechanical design, CAD and 3D printing

Minimum duration **2 days** | Level **Advanced**

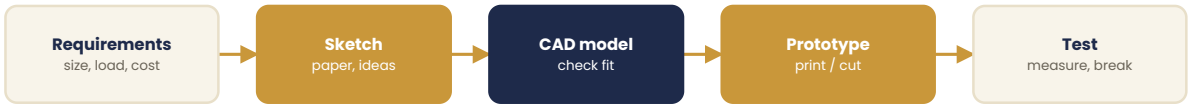
Custom parts make robots stronger, lighter and better suited to their tasks. This module follows the design process from requirements to testing, introduces CAD with Tinkercad, Fusion 360, Onshape and FreeCAD (sketches, extrusions, assemblies and joints), explains FDM 3D printing, materials, slicer settings and design for printability, covers laser cutting and CNC machining, and gives an overview of PCB design with KiCad and EasyEDA. Python programs generate a printable STL part and a laser-cut chassis plate, followed by 25 solved questions and programs.

IN THIS MODULE

24.1 Design process
24.2 CAD software
24.3 3D printing
24.4 Other fabrication methods
24.5 PCB design (overview)
24.6 Practice questions
24.7 Solutions

24.1 Design process

24.1.1 Requirements, sketches, prototypes, testing



DESIGN IS A LOOP: EACH TEST FEEDS BACK INTO A BETTER REQUIREMENT OR MODEL

Step	Questions to answer	Example: line-follower chassis
Requirements	what must it carry, how big, how fast, budget, deadline?	fit 2 N20 motors, 18650 holder, Arduino Nano, 5-sensor IR array; under 15 × 12 cm; under 150 g
Sketch	layout, centre of mass, wiring paths	battery low and central, sensors 3–5 mm above the floor in front
CAD	exact dimensions, clearances, interference check	model the motors and boards as simple blocks first
Prototype	fastest way to get a real part	print the plate at 20 % infill, 3 walls
Test	does it fit, flex, survive a drop?	adjust sensor height, add a rib where it flexed

SHORTCUT TRICK
Download CAD models of the motors, boards and sensors (manufacturer sites, GrabCAD) or model them as simple boxes with their mounting holes. Designing around real parts catches 90 % of fit problems before printing.

24.2 CAD software

24.2.1 Tinkercad for beginners

Tinkercad (free, in the browser) builds parts by combining basic shapes: drag a box, cylinder or text onto the workplane, size it, and mark shapes as "hole" to subtract them. Group shapes to merge or cut. It is perfect for simple brackets, sensor mounts and battery holders, and exports STL for printing.

24.2.2 Fusion 360, Onshape or FreeCAD

Software	Platform	Cost	Strengths
Autodesk Fusion	Windows, macOS (cloud files)	free for personal/hobby and education (limited features)	parametric modelling, assemblies, motion, CAM for CNC, simulation
Onshape	browser, tablets	free plan (public documents), education plan	real-time collaboration, version history, runs anywhere
FreeCAD	Windows, macOS, Linux	free, open source	parametric, offline, Python scripting, workbenches for many tasks
SolidWorks / Inventor	Windows	commercial (student licences)	industry standard in many companies
OpenSCAD	all	free	parts written as code; great for parametric parts

24.2.3 Sketches, extrusions, assemblies and joints

Concept	What it means
Sketch	2-D drawing on a plane (lines, circles, arcs) with **dimensions** and **constraints** (parallel, tangent, equal, coincident)
Extrude	pull a closed sketch into a solid (or cut into one)
Revolve	spin a sketch around an axis: wheels, pulleys, knobs
Fillet / chamfer	round or bevel edges: stronger corners, nicer prints
Pattern / mirror	repeat features: bolt holes, spokes
Parameters	named values (wall = 2.4 mm) that update the whole model when changed
Assembly	several parts positioned together; check interference

Concept	What it means
Joints	how parts move relative to each other (revolute, slider, rigid): animate a gripper before printing it
Drawing	2-D engineering drawing with dimensions for manufacturing

A **parametric** model remembers how it was built. Change the motor diameter from 12 mm to 24 mm and the clamp, holes and plate update automatically. Fully constrain sketches (they turn black in most programs) so nothing moves unexpectedly.

24.3 3D printing

24.3.1 FDM printers and materials (PLA, PETG, ABS)

FDM (fused deposition modelling) printers melt plastic filament (1.75 mm) in a hot nozzle (usually 0.4 mm) and draw the part layer by layer. Popular hobby printers include Bambu Lab, Prusa and Creality models. Resin (SLA/MSLA) printers give finer detail for small gears and figures.

Material	Nozzle / bed (°C)	Strengths	Weaknesses	Robot use
PLA	200–215 / 55–60	easiest to print, stiff, cheap	softens near 55–60 °C (hot car, motors)	prototypes, brackets, chassis
PETG	230–250 / 70–80	tougher, less brittle, some heat and water resistance	strings, sticks too well to PEI	functional parts, outdoor robots
ABS / ASA	240–260 / 100	heat resistant, sandable (ASA: UV resistant)	warps, fumes: needs an enclosure and ventilation	parts near motors, outdoors
TPU (flexible)	220–230 / 50	rubbery, absorbs shock	slow, tricky with Bowden extruders	tyres, bumpers, grippers
Nylon, CF-filled	250–290 / 80+	very strong, wear resistant	absorbs moisture, abrasive (hardened nozzle)	gears, arm links

24.3.2 Slicing software and settings

A **slicer** (PrusaSlicer, Bambu Studio, Orca Slicer, Cura) cuts the STL/3MF model into layers and writes **G-code**: the moves, temperatures and extrusion amounts the printer follows.

Setting	Typical	Effect
Layer height	0.2 mm (0.12 fine, 0.28 draft)	thinner = smoother and slower
Walls (perimeters)	3 for robot parts	walls give most of the strength
Infill	15–25 % (gyroid or grid); 40 %+ for stressed parts	more = stronger, heavier, slower

Setting	Typical	Effect
Top/bottom layers	4–5	solid skins
Supports	only where overhangs exceed ~45–50°	hold up overhangs; leave marks
Brim	5–8 mm for warping materials or small contact area	better bed adhesion
Speed	set by printer (modern printers 150–300 mm/s)	faster may reduce quality

GCODE_BASICS.PY

```
# G-code: the printer's instructions (a few lines from the start of a slice)
gcode = """M104 S210        ; set nozzle to 210 C
M140 S60         ; set bed to 60 C
G28              ; home all axes
G1 Z0.2 F3000    ; move to first layer height
G1 X10 Y10 F6000
G1 X60 Y10 E1.87 F1800   ; print a 50 mm line, extruding 1.87 mm of filament
G1 X60 Y60 E3.74
"""
import math
for line in gcode.splitlines():
    print(line)
# check the extrusion: volume of the printed line = volume of filament pushed in
line_w, layer_h, length, fil_d = 0.45, 0.2, 50, 1.75
e = line_w * layer_h * length / (math.pi * (fil_d / 2) ** 2)
print(f"\nfilament needed for a 50 mm line: {e:.2f} mm (matches E1.87)")
```

OUTPUT

```
M104 S210        ; set nozzle to 210 C
M140 S60         ; set bed to 60 C
G28              ; home all axes
G1 Z0.2 F3000    ; move to first layer height
G1 X10 Y10 F6000
G1 X60 Y10 E1.87 F1800   ; print a 50 mm line, extruding 1.87 mm of filament
G1 X60 Y60 E3.74

filament needed for a 50 mm line: 1.87 mm (matches E1.87)
```

24.3.3 Designing for printability: tolerances, supports, infill

Rule	Why / how
Clearance for fits	0.2–0.3 mm per side for sliding fits, 0.1 mm for press fits; print a tolerance test for your printer
Holes print small	add 0.2–0.4 mm to hole diameters, or drill/ream after printing; M3 clearance hole ≈ 3.4 mm
Overhangs ≤ 45°	chamfer instead of flat overhangs; bridges up to ~20 mm are fine
Orientation = strength	parts are weakest between layers: orient so loads run along layers, not peel them apart

Rule	Why / how
Flat base	a large flat face on the bed avoids supports and warping
Wall thickness	multiples of the line width (0.45 mm): 1.35, 1.8, 2.25 mm
Heat-set inserts	brass M3 inserts melted in with a soldering iron give strong reusable threads
Nut traps	hexagonal pockets (5.7 mm across flats + clearance for M3) hold nuts captive
Fillets on corners	spread stress, reduce cracking

Parts can also be designed in code. This program builds a parametric L-bracket with two M3 holes, writes it as an STL file ready for any slicer, and renders a preview:

MAKE_BRACKET.PY

```
import math
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d.art3d import Poly3DCollection

W, L, H, T, HOLE = 20.0, 30.0, 25.0, 3.0, 3.4      # width, base length, upright height, thickness,
M3 clearance (mm)
tris = []

def box(x0, y0, z0, x1, y1, z1):                    # 12 triangles of a box
    v = [(x, y, z) for x in (x0, x1) for y in (y0, y1) for z in (z0, z1)]
    faces = [(0, 1, 3, 2), (4, 6, 7, 5), (0, 4, 5, 1), (2, 3, 7, 6), (0, 2, 6, 4), (1, 5, 7, 3)]
    for a, b, c, d in faces:
        tris.extend([(v[a], v[b], v[c]), (v[a], v[c], v[d])])

box(0, 0, 0, L, W, T)                               # base plate
box(0, 0, T, T, W, H)                               # upright
box(T, 0, T, T + 6, 2, T + 6)                       # two triangular-ish ribs (as small blocks)
box(T, W - 2, T, T + 6, W, T + 6)

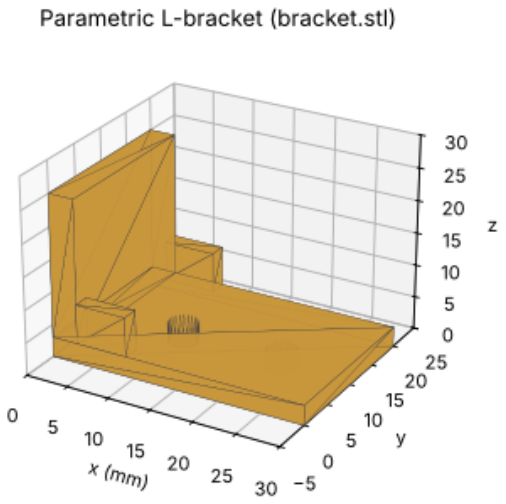
def ring(cx, cy, z0, z1, r, n=24):                  # visual marker for a hole (thin tube)
    for i in range(n):
        a, b = 2 * math.pi * i / n, 2 * math.pi * (i + 1) / n
        p = [(cx + r * math.cos(a), cy + r * math.sin(a)), (cx + r * math.cos(b), cy + r *
math.sin(b))]
        tris.append(((*p[0], z0), (*p[1], z0), (*p[1], z1 + 0.01)))
holes = [(L - 8, W / 2), (L - 20, W / 2)]
for cx, cy in holes:
    ring(cx, cy, 0, T, HOLE / 2)

with open("bracket.stl", "w") as f:                 # ASCII STL: a list of triangles
    f.write("solid bracket\n")
    for t in tris:
        f.write(" facet normal 0 0 0\n  outer loop\n")
        for x, y, z in t:
            f.write(f"   vertex {x:.3f} {y:.3f} {z:.3f}\n")
        f.write("  endloop\n endfacet\n")
    f.write("endsolid bracket\n")

vol = L * W * T + T * W * (H - T) + 2 * 6 * 2 * 6 - 2 * math.pi * (HOLE / 2) ** 2 * T
print(f"triangles: {len(tris)}, solid volume ~ {vol / 1000:.2f} cm^3, PLA mass ~ {vol / 1000 *
1.24:.1f} g (100 % solid)")
fig = plt.figure(figsize=(5, 4)); ax = fig.add_subplot(projection="3d")
ax.add_collection3d(Poly3DCollection(tris, facecolor="#C9973A", edgecolor="#1E2A4A", linewidths=0.2,
alpha=0.9))
ax.set_xlim(0, 30); ax.set_ylim(-5, 25); ax.set_zlim(0, 30); ax.view_init(25, -60)
ax.set_xlabel("x (mm)"); ax.set_ylabel("y"); ax.set_zlabel("z")
ax.set_title("Parametric L-bracket (bracket.stl)"); fig.tight_layout(); fig.savefig("bracket.png",
dpi=80)
```

OUTPUT

```
triangles: 96, solid volume ~ 3.21 cm^3, PLA mass ~ 4.0 g (100 % solid)
```



PREVIEW OF THE GENERATED STL; CHANGE W, L, H OR T AND RERUN FOR A NEW SIZE

Change one number and a new, correctly sized bracket appears: the same idea as parametric CAD, and the way OpenSCAD and CadQuery work. (For real parts, a CAD program or OpenSCAD makes the round holes and fillets properly; here the holes are marked as tubes for the preview.)

24.4 Other fabrication methods

24.4.1 Laser cutting

A **laser cutter** (CO2, 40–100 W) cuts flat sheets of acrylic, plywood and MDF from a 2-D vector drawing (SVG, DXF) in minutes: ideal for chassis plates and arm links. The beam removes a strip about 0.1–0.2 mm wide (the **kerf**), so slots for tab joints are drawn slightly narrower. Never cut PVC or vinyl (releases toxic chlorine gas) or polycarbonate (burns, discolours).

CHASSIS_SVG.PY

```
# generate a 2-D chassis plate as SVG for a laser cutter (units: mm)
W, H, R = 140, 110, 6                          # plate size and corner radius
holes = [(20, 20), (120, 20), (20, 90), (120, 90)]          # M3 standoff holes
motor_slots = [(40, 0, 30, 6), (40, H - 6, 30, 6)]           # x, y, w, h cut-outs for motor
brackets
svg = [f'<svg xmlns="http://www.w3.org/2000/svg" width="{W}mm" height="{H}mm" viewBox="0 0 {W}
{H}">',
       f'<rect x="0" y="0" width="{W}" height="{H}" rx="{R}" fill="none" stroke="red" stroke-
width="0.1"/>']
for x, y in holes:
    svg.append(f'<circle cx="{x}" cy="{y}" r="1.7" fill="none" stroke="red" stroke-width="0.1"/>')
for x, y, w, h in motor_slots:
    svg.append(f'<rect x="{x}" y="{y}" width="{w}" height="{h}" fill="none" stroke="red" stroke-
width="0.1"/>')
svg.append(f'<rect x="{W/2 - 25}" y="{H/2 - 12}" width="50" height="24" rx="3" fill="none"
stroke="red" stroke-width="0.1"/>')  # wire pass-through
svg.append("</svg>")
open("chassis.svg", "w").write("\n".join(svg))
cut = 2 * (W + H) - (8 - 2 * 3.1416) * R + len(holes) * 2 * 3.1416 * 1.7 + sum(2 * (w + h) for *_,
w, h in motor_slots) + 2 * (50 + 24)
print(f"chassis.svg written: {len(svg) - 2} shapes, cut length ~ {cut:.0f} mm")
print(f"at 10 mm/s cutting speed: ~ {cut / 10:.0f} s for 3 mm acrylic")
```

OUTPUT

```
chassis.svg written: 8 shapes, cut length ~ 824 mm
at 10 mm/s cutting speed: ~ 82 s for 3 mm acrylic
```

Laser software treats thin red lines as cuts and filled or other-coloured shapes as engravings (check your machine's colour rules). Because the file is generated, a wider robot is a one-number change.

24.4.2 CNC machining (overview)

A **CNC router or mill** removes material with a spinning cutter following CAM-generated G-code. It machines aluminium, wood, plastics and PCBs, giving strong, precise parts (motor mounts, arm joints) that printers cannot. CAM software (Fusion CAM, FreeCAD Path, Carbide Create) turns the CAD model into toolpaths. Safety: eye protection, clamp the work, never leave it running unattended. For metal parts without a machine, online services cut, bend and machine parts from your files.

Method	Materials	Strengths	Limits
FDM 3D printing	PLA, PETG, ABS, TPU, nylon	complex shapes, cheap, fast iteration	weaker between layers
Resin printing	photopolymer resins	fine detail	brittle, messy chemicals
Laser cutting	acrylic, plywood, MDF, card	very fast flat parts	2-D only, no metal (hobby CO2)
CNC	aluminium, wood, plastics	strong, precise	slower, costly tools, setup time

24.5 PCB design (overview)

24.5.1 KiCad or EasyEDA for custom circuit boards

Once a breadboard circuit works, a **printed circuit board** (PCB) makes it compact, reliable and repeatable: no loose jumper wires vibrating free. The workflow:

1. **Schematic**: place symbols (microcontroller, driver, connectors) and wire them; run the electrical rules check (ERC).
2. **Footprints**: assign the physical pad layout to each symbol (through-hole or SMD).
3. **Layout**: place parts on the board outline and route copper tracks; wider tracks for motor current (about 1 mm per amp on 1 oz copper as a starting point, check a calculator), ground plane, mounting holes.
4. **Design rules check** (DRC): clearances and track widths the factory can make.
5. **Gerber files**: export and upload to a PCB manufacturer (many in India and abroad make 5–10 boards cheaply), optionally with assembly (PCBA).

Tool	Notes
KiCad	free, open source, professional features, 3-D viewer, large libraries; runs offline on all platforms
EasyEDA	browser-based, linked to the LCSC parts catalogue and JLCPCB ordering; quick for beginners
Fritzing	breadboard-style drawings for documentation; basic PCB

WATCH OUT

Keep motor power tracks and returns away from sensor and logic tracks, add decoupling capacitors (100 nF) next to every IC, and put a fuse and reverse-polarity protection on the battery input.

PRACTICE CHECKPOINT

Design a custom robot chassis or gripper in CAD, 3D-print it, and assemble it with your electronics.

24.6 Practice questions

24.6.1 Basic level

Q1 What is the first step of the design process?

(a) print the part
(b) buy filament
(c) paint it
(d) define the requirements

Q2 Which CAD tool runs in the browser and builds parts from basic shapes and holes?

(a) KiCad **(b)** Tinkercad **(c)** Cura **(d)** FreeCAD only

Q3 Which free, open-source CAD program supports Python scripting and works offline?

(a) FreeCAD **(b)** Onshape **(c)** Tinkercad **(d)** Fusion CAM

Q4 What does "extrude" do in CAD?

(a) rotates a part
(b) pulls a 2-D sketch into a 3-D solid
(c) prints a part
(d) deletes a sketch

Q5 Which filament is easiest to print but softens at about 60 °C?

(a) PLA **(b)** ABS **(c)** nylon **(d)** PETG

Q6 Which filament is flexible and suits tyres and bumpers?

(a) PLA **(b)** ABS **(c)** TPU **(d)** acrylic

Q7 What does a slicer produce?

(a) an STL file
(b) a PCB
(c) a schematic
(d) G-code for the printer

Q8 Which software designs custom circuit boards for free and offline?

(a) Tinkercad 3-D **(b)** Cura **(c)** Bambu Studio **(d)** KiCad

24.6.2 Intermediate level

Q9 What is a parametric model?

(a) a model made of triangles only
(b) a photo of a part
(c) a model driven by dimensions and constraints that update when a value changes
(d) a printed part

Q10 Which slicer setting usually adds the most strength to a printed part?

(a) more walls (perimeters)
(b) higher bed temperature only
(c) faster travel moves
(d) thinner first layer

Q11 Why are 3D-printed parts weakest in one direction?

(a) the nozzle is round
(b) infill is always 100 %
(c) G-code limits it
(d) layers bond less strongly to each other than within a layer

Q12 Why should printed holes be designed slightly larger?

(a) to save filament
(b) holes tend to print smaller than modelled
(c) to look nicer only
(d) slicers require it

Q13 Above roughly what overhang angle are supports usually needed?

(a) 5° **(b)** 45–50° **(c)** 20° **(d)** 85° only

Q14 What is the kerf in laser cutting?

(a) the laser power
(b) the sheet thickness
(c) the cutting speed
(d) the width of material removed by the beam

Q15 Which material must never be laser cut because it releases toxic chlorine gas?

(a) PVC **(b)** plywood **(c)** acrylic **(d)** cardboard

Q16 What do you send to a PCB factory?

(a) STL files **(b)** G-code **(c)** Gerber files **(d)** SVG only

Q17 What are heat-set inserts used for?

(a) cooling motors
(b) bed adhesion
(c) filament storage
(d) strong, reusable metal threads in printed parts

24.6.3 Advanced level

Q18 A printed line is 0.45 mm wide, 0.2 mm high and 100 mm long. About how much 1.75 mm filament is used?

(a) about 0.37 mm **(b)** about 3.74 mm **(c)** about 37.4 mm **(d)** about 9 mm

Q19 A part has 12 cm^3 of plastic. With PLA at 1.24 g/cm^3 and filament at ₹1500/kg, what is the material cost?

(a) about ₹22 **(b)** about ₹2 **(c)** about ₹220 **(d)** about ₹1500

Q20 Compute the clearance hole diameters to model for M2, M3 and M4 screws, adding 0.3 mm print compensation to the standard medium-fit clearance (2.4, 3.4, 4.5 mm).

Q21 Estimate print time for a part: 120 cm of perimeter per layer, 80 layers, 60 mm/s average speed, plus 25 % for infill and travel.

Q22 Design a slot for a tab joint: 3 mm acrylic with a kerf of 0.15 mm. What slot width should be drawn so the tab fits snugly?

Q23 Choose a track width for a motor supply of 2 A using the rule of thumb of about 1 mm per amp (1 oz copper, outer layer), with a 50 % safety margin, and compare with a 0.25 mm signal track.

Q24 A printed motor mount cracks along the layer lines when the motor pulls. What is the best fix?

(a) print hotter only
(b) use less infill
(c) re-orient the part so the load runs along the layers, and add walls or a fillet
(d) paint it

Q25 A robot base needs 20 identical, flat 3 mm acrylic plates quickly. Which method fits best?

(a) laser cutting
(b) FDM printing each plate
(c) CNC milling aluminium
(d) resin printing

24.7 Solutions

Q1 **Answer: (d) define the requirements**
Know what it must do.

Q2 **Answer: (b) Tinkercad**
Beginner CAD.

Q3 **Answer: (a) FreeCAD**
Open source.

Q4 **Answer: (b) pulls a 2-D sketch into a 3-D solid**
Sketch → solid.

Q5 **Answer: (a) PLA**
Low glass transition.

Q6 **Answer: (c) TPU**
Rubber-like.

Q7 **Answer: (d) G-code for the printer**
Layers → G-code.

Q8 **Answer: (d) KiCad**
PCB EDA.

Q9 **Answer: (c) a model driven by dimensions and constraints that update when a value changes**
History-based.

Q10 **Answer: (a) more walls (perimeters)**
Walls carry load.

Q11 **Answer: (d) layers bond less strongly to each other than within a layer**

Orient for loads.

Q12 **Answer: (b) holes tend to print smaller than modelled**

Add 0.2–0.4 mm.

Q13 **Answer: (b) 45–50°**

Self-supporting up to ~45°.

Q14 **Answer: (d) the width of material removed by the beam**

0.1–0.2 mm.

Q15 **Answer: (a) PVC**

Chlorine.

Q16 **Answer: (c) Gerber files**

Copper, mask, drill layers.

Q17 **Answer: (d) strong, reusable metal threads in printed parts**

Brass inserts.

Q18 **Answer: (b) about 3.74 mm**

$0.45 \times 0.2 \times 100 = 9$ mm³ ÷ $(\pi \times 0.875^2)$ = **3.74 mm**.

Q19 **Answer: (a) about ₹22**

12 × 1.24 = 14.9 g; 14.9 × 1.5 = **₹22**.

Q20 **Answer: Solution program**

Test on your own printer with a small calibration plate; the right compensation depends on the printer, material and slicer.

```
clear = {"M2": 2.4, "M3": 3.4, "M4": 4.5}
for screw, d in clear.items():
    print(f"{screw}: clearance {d} mm -> model {d + 0.3:.1f} mm")
```

OUTPUT

```
M2: clearance 2.4 mm -> model 2.7 mm
M3: clearance 3.4 mm -> model 3.7 mm
M4: clearance 4.5 mm -> model 4.8 mm
```

Q21 **Answer: Solution program**

1200 × 80 = 96 000 mm ÷ 60 mm/s = 1600 s, × 1.25 = 2000 s ≈ 33 minutes. Slicers estimate this exactly from the G-code.

```
per_layer_mm, layers, speed = 1200, 80, 60
seconds = per_layer_mm * layers / speed * 1.25
print(f"about {seconds / 60:.0f} minutes")
```

OUTPUT

```
about 33 minutes
```

Q22 **Answer: Solution program**

The laser removes about one kerf of extra width from the slot (half on each side), so draw it one kerf narrower than the sheet thickness. Real sheets vary: measure them first.

```
sheet, kerf = 3.0, 0.15
drawn = sheet - kerf                # the beam widens the slot by one kerf in total
print(f"draw the slot {drawn:.2f} mm wide -> it cuts to {drawn + kerf:.2f} mm")
```

OUTPUT

```
draw the slot 2.85 mm wide -> it cuts to 3.00 mm
```

Q23 **Answer: Solution program**

High-current tracks must be much wider than signal tracks to avoid heating. Use an IPC-2221 track-width calculator for real designs.

```
amps, mm_per_amp, margin = 2.0, 1.0, 1.5
width = amps * mm_per_amp * margin
print(f"motor track: {width:.1f} mm wide")
print(f"that is {width / 0.25:.0f} times a 0.25 mm signal track")
```

OUTPUT

```
motor track: 3.0 mm wide
that is 12 times a 0.25 mm signal track
```

Q24 **Answer: (c) re-orient the part so the load runs along the layers, and add walls or a fillet**

Orientation matters most.

Q25 **Answer: (a) laser cutting**

Fast flat parts.

Design loop: requirements → sketch → CAD → prototype → test. CAD: Tinkercad (shapes), Fusion, Onshape, FreeCAD (parametric sketches, constraints, extrude, revolve, fillet, assemblies, joints). FDM: PLA (easy), PETG (tough), ABS/ASA (heat), TPU (flexible); slicer → G-code; walls give strength; holes +0.2–0.4 mm, overhangs ≤ 45°, orient for loads, heat-set inserts. Laser cutting (kerf, never PVC), CNC (metal). PCB: schematic → footprints → layout → DRC → Gerbers (KiCad, EasyEDA).

MODULE 25 · ADVANCED

Industrial robotics, automation, safety and ethics

Minimum duration **2 days** | Level **Advanced**

Industrial robots and automation systems power modern manufacturing. This module introduces the major robot manufacturers (ABB, FANUC, KUKA, Yaskawa), teach pendants and robot programming languages, collaborative robots and robot cells, then PLCs and ladder logic, SCADA and HMI, and Industry 4.0. It covers emergency stops, safety sensors and the main safety standards, and closes with the ethics of robotics: jobs, safety, privacy, responsibility and autonomous decision-making, followed by 25 solved questions and programs.

IN THIS MODULE

25.1 Industrial robots
25.2 Industrial automation
25.3 Robot safety
25.4 Ethics and society
25.5 Practice questions
25.6 Solutions

25.1 Industrial robots

An industrial robot is an automatically controlled, reprogrammable manipulator with three or more axes, used in factories for welding, painting, assembly, palletising, machine tending and packing. Most are 6-axis articulated arms; SCARA, delta and Cartesian robots handle fast pick-and-place.

25.1.1 Major manufacturers: ABB, FANUC, KUKA, Yaskawa

Company	Home	Known for	Programming language
ABB	Switzerland / Sweden	wide range, painting, YuMi and GoFa cobots, RobotStudio simulation	RAPID
FANUC	Japan	very large installed base, CNC controls, yellow robots, CRX cobots	TP (teach pendant) and KAREL
KUKA	Germany	automotive lines, orange robots, LBR iiwa cobot	KRL (KUKA Robot Language)
Yaskawa (Motoman)	Japan	arc welding, high-speed handling, servo drives	INFORM
Others		Universal Robots (cobots), Kawasaki, Epson and Denso (SCARA), Stäubli, Nachi	URScript, etc.

25.1.2 Teach pendants and robot programming

A **teach pendant** is the hand-held terminal used to jog the robot, record positions and edit programs, with an **enabling switch** (three-position "dead-man" switch: the robot moves only while it is held in the middle position) and an emergency stop. Methods of programming:

Method	How	Use
Online teaching	jog the robot to each point with the pendant and record it	simple tasks, small changes on the line
Lead-through / hand guiding	push the robot arm by hand while it records	cobots, painting paths
Offline programming (OLP)	program and simulate in software (RobotStudio, ROBOGUIDE, KUKA.Sim, RoboDK) then download	new lines without stopping production
Code / API	write programs or control from a PC (ROS 2, Python SDKs)	integration with vision and AI

ABB RAPID PICK-AND-PLACE (REFERENCE)

```
MODULE PickPlace
  CONST robtarget pPick  := [[600, -200, 150], [0, 0, 1, 0], [0, 0, 0, 0], [9E9, 9E9, 9E9, 9E9, 9E9,
9E9]];
  CONST robtarget pPlace := [[600,  250, 150], [0, 0, 1, 0], [0, 0, 0, 0], [9E9, 9E9, 9E9, 9E9, 9E9,
9E9]];
  PROC main()
    MoveJ Offs(pPick, 0, 0, 100), v1000, z50, tGripper;   ! joint move above the part
    MoveL pPick, v200, fine, tGripper;                    ! straight line down, stop exactly
    SetDO doGripper, 1;  WaitTime 0.3;                    ! close gripper
    MoveL Offs(pPick, 0, 0, 100), v500, z10, tGripper;
    MoveJ Offs(pPlace, 0, 0, 100), v1000, z50, tGripper;
    MoveL pPlace, v200, fine, tGripper;
    SetDO doGripper, 0;  WaitTime 0.3;                    ! open
    MoveL Offs(pPlace, 0, 0, 100), v500, z10, tGripper;
  ENDPROC
ENDMODULE
```

Every robot language has the same building blocks: **joint moves** (fast, curved path: MoveJ, PTP, J), **linear moves** (straight tool path: MoveL, LIN, L), speed, **zone/blending** (how closely a point must be reached before turning: "fine" stops exactly), the **tool frame** (TCP: tool centre point) and the **work-object frame**, plus digital I/O to grippers and the PLC.

25.1.3 Collaborative robots (cobots)

Cobots (Universal Robots UR series, FANUC CRX, ABB GoFa and YuMi, KUKA LBR iiwa, Techman, Doosan) are designed to work near people. They limit their power and force, detect collisions through joint torque sensing and stop, are light, and can be taught by hand. They suit small-batch tasks such as machine tending, screwdriving, quality inspection and packing.

Collaboration mode (ISO 10218)	How it keeps people safe
Safety-rated monitored stop	the robot stops while a person is in the shared space and resumes when they leave

Collaboration mode (ISO 10218)	How it keeps people safe
Hand guiding	the operator moves the robot directly with an enabling device
Speed and separation monitoring	scanners measure the distance to people; the robot slows down, then stops as they approach
Power and force limiting	contact is allowed but force and pressure stay below limits for each body region

WATCH OUT
A cobot is not automatically safe: a sharp tool, a hot part or a heavy load can make a "collaborative" application dangerous. Every application needs a risk assessment.

25.1.4 Robot cells and safety fences

A traditional industrial robot works inside a **cell**: fixed fences keep people out, interlocked doors stop the robot when opened, and light curtains or laser scanners guard the loading openings. The cell also holds fixtures, conveyors, part feeders and the controller, and a PLC coordinates everything.

25.2 Industrial automation

25.2.1 Programmable logic controllers (PLCs) and ladder logic (overview)

A **PLC** is a rugged industrial computer (Siemens S7, Allen-Bradley, Mitsubishi, Delta, Schneider) with input and output modules for 24 V sensors, buttons, relays, valves and motor drives. It runs a **scan cycle** continuously: read all inputs → execute the program → write all outputs (typically every few milliseconds). IEC 61131-3 defines five languages: Ladder Diagram (LD), Function Block Diagram (FBD), Structured Text (ST), Sequential Function Chart (SFC) and Instruction List (IL, deprecated).

Ladder logic looks like a relay wiring diagram: power flows from the left rail to the right through **contacts** (inputs: normally open --| |--, normally closed --|/|--) to **coils** (outputs --( )--). The classic motor **start/stop seal-in** circuit:

CONVEYOR CONTROL IN LADDER LOGIC (STOP AND E_STOP WIRED NORMALLY CLOSED)

```
     START        STOP       E_STOP      MOTOR
|----| |----+----|/|--------| |--------( )----|      rung 1: seal-in (latching) start/stop
|          |                                  |
|----| |----+                                  |      MOTOR contact in parallel holds it on
|   MOTOR                                     |
|                                             |
|    MOTOR      PART_SENSOR             TON T1  |      rung 2: part detected while running
|----| |----------| |-------------------[2.0 s]-|             -> start a 2 s timer
|                                             |
|    T1.DN                               PUSHER   |      rung 3: timer done -> push the part off
|----| |--------------------------------( )-----|
```

PLC_SCAN.PY

```
# A tiny PLC: scan cycle every 100 ms running the three ladder rungs above
inputs_timeline = {          # time (s): changes to inputs (STOP and E_STOP are NC: True = healthy)
    0.0: dict(START=False, STOP=True, E_STOP=True, PART_SENSOR=False),
    0.5: dict(START=True), 0.7: dict(START=False),          # operator taps START
    1.5: dict(PART_SENSOR=True), 3.9: dict(PART_SENSOR=False),
    4.5: dict(STOP=False), 4.7: dict(STOP=True),            # operator taps STOP
}
I = {}; MOTOR = PUSHER = False; t1_acc = 0.0
prev = None
for scan in range(55):
    t = round(scan * 0.1, 1)
    I.update(inputs_timeline.get(t, {}))                   # 1. read inputs
    MOTOR = (I["START"] or MOTOR) and I["STOP"] and I["E_STOP"]       # 2. rung 1 (seal-in)
    if MOTOR and I["PART_SENSOR"]: t1_acc += 0.1            #    rung 2 (TON accumulates while true)
    else: t1_acc = 0.0
    PUSHER = t1_acc >= 2.0                                 #    rung 3
    state = (MOTOR, PUSHER)                                # 3. write outputs
    if state != prev:
        print(f"t={t:3.1f}s  START={I['START']:d} STOP={I['STOP']:d} PART={I['PART_SENSOR']:d}  ->
MOTOR={MOTOR:d} PUSHER={PUSHER:d}")
        prev = state
```

OUTPUT

```
t=0.0s  START=0 STOP=1 PART=0  ->  MOTOR=0 PUSHER=0
t=0.5s  START=1 STOP=1 PART=0  ->  MOTOR=1 PUSHER=0
t=3.4s  START=0 STOP=1 PART=1  ->  MOTOR=1 PUSHER=1
t=3.9s  START=0 STOP=1 PART=0  ->  MOTOR=1 PUSHER=0
t=4.5s  START=0 STOP=0 PART=0  ->  MOTOR=0 PUSHER=0
```

The motor stays on after START is released because its own contact "seals in" rung 1; pressing STOP breaks the rung. The timer pushes the part off 2 s after it arrives. Wiring STOP and the E-stop as **normally closed** means a broken wire also stops the machine: a fail-safe habit. Free simulators such as OpenPLC (which also runs on a Raspberry Pi), CODESYS and the trial versions of Siemens TIA Portal let you practise ladder logic.

SEALIN.INO

```
// The same seal-in logic as an Arduino "scan cycle" (for learning only, not for real machine
safety)
const int START = 2, STOP = 3, MOTOR = 8;
bool motor = false;
void setup() {
  Serial.begin(9600);
  pinMode(START, INPUT_PULLUP);      // pressed = LOW
  pinMode(STOP, INPUT_PULLUP);       // NC button: normally LOW, pressed (or broken wire) = HIGH
  pinMode(MOTOR, OUTPUT);
}
void loop() {
  bool start = digitalRead(START) == LOW;            // 1. read inputs
  bool stopOk = digitalRead(STOP) == LOW;
  bool next = (start || motor) && stopOk;            // 2. logic (rung)
  if (next != motor) {
    Serial.print(millis()); Serial.println(next ? " ms  MOTOR ON" : " ms  MOTOR OFF");
  }
  motor = next;
  digitalWrite(MOTOR, motor);                         // 3. write outputs
  delay(10);
}
```

SERIAL MONITOR

```
200 ms  MOTOR ON
700 ms  MOTOR OFF
1200 ms  MOTOR ON
1450 ms  MOTOR OFF
```

START at 200 ms latches the motor; STOP at 700 ms drops it; START again at 1200 ms; and at 1450 ms the STOP input goes HIGH as if its wire had broken, so the motor stops: the fail-safe behaviour of a normally closed circuit. Real machines use certified safety relays or safety PLCs for E-stops, never an ordinary microcontroller.

25.2.2 SCADA and HMI (overview)

System	What it is	Example
HMI (human-machine interface)	touch panel at the machine showing status, alarms and buttons	operator starts a batch, sees "jam at station 3"
SCADA (supervisory control and data acquisition)	software in a control room collecting data from many PLCs across a plant or site	water treatment plant, power grid, whole production line
MES (manufacturing execution system)	tracks orders, materials and quality through production	which batch used which parts
Protocols	Modbus (RTU/TCP), PROFINET, EtherNet/IP, OPC UA	OPC UA links machines to IT systems securely

25.2.3 Industry 4.0 and smart factories

Industry 4.0 connects machines, robots and products with sensors and networks so that data drives decisions. Building blocks: industrial IoT sensors, cloud and edge computing, **digital twins** (a live virtual

copy of a machine or line used for simulation and optimisation), predictive maintenance with machine learning, autonomous mobile robots (AMRs) moving materials, additive manufacturing, and augmented reality for maintenance. A key measure of factory performance is **OEE** (overall equipment effectiveness) = availability × performance × quality.

OEE.PY

```
planned_min, downtime_min = 480, 45                # one 8-hour shift
ideal_cycle_s, parts_made, good_parts = 30, 780, 760
availability = (planned_min - downtime_min) / planned_min
performance = parts_made * ideal_cycle_s / 60 / (planned_min - downtime_min)
quality = good_parts / parts_made
oee = availability * performance * quality
print(f"availability {availability:.1%}  performance {performance:.1%}  quality {quality:.1%}")
print(f"OEE = {oee:.1%}")
```

OUTPUT

```
availability 90.6%  performance 89.7%  quality 97.4%
OEE = 79.2%
```

World-class OEE is often quoted at about 85 %. Here availability (downtime) and performance (slow cycles, small stops) each lose about 10 %, so both maintenance and cycle-time work pay off; quality losses are small.

25.3 Robot safety

25.3.1 Emergency stops, safety sensors and standards (overview)

Device	Purpose
Emergency stop (E-stop)	red mushroom button on yellow; latches when pressed; removes power through a safety relay; must be reset deliberately
Enabling switch	three-position switch on pendants: release or squeeze hard = stop
Light curtain	infrared beams across an opening; breaking any beam stops the machine
Safety laser scanner	defines warning and protective zones on the floor (slow, then stop); used on AMRs and cobot cells
Interlocked door / guard switch	opening the cell door stops the robot
Safety mats, two-hand controls	detect a person standing in a zone; keep both hands on buttons away from a press
Safety PLC / safety relay	certified, redundant, self-checking logic for all safety functions

Standard	Covers
ISO 10218-1 / -2	safety of industrial robots and robot systems/cells (revised 2025; includes collaborative applications)
ISO/TS 15066	collaborative robots: force and pressure limits for body regions (merged into the 2025 ISO 10218 revision)

Standard	Covers
ISO 13849 / IEC 62061	safety of control systems: performance levels (PL a–e) and SIL
ISO 13855	positioning safeguards: minimum safety distance
ISO 3691-4	driverless industrial trucks and AMRs
IEC 60204-1	electrical equipment of machines, including stop categories 0, 1, 2

SAFETY_DISTANCE.PY

```
# ISO 13855 minimum distance for a light curtain: S = K x T + C
K = 2000          # approach speed (mm/s) for hand/arm (use 1600 if S > 500 mm)
T = 0.020 + 0.180 # curtain response 20 ms + robot stopping time 180 ms (s)
C = 8 * (14 - 14) # extra distance for a 14 mm-resolution curtain (finger detection)
S = K * T + C
if S > 500:
    K = 1600; S = K * T + C
print(f"stopping time {T * 1000:.0f} ms -> mount the light curtain at least {S:.0f} mm from the
hazard")
```

OUTPUT

```
stopping time 200 ms -> mount the light curtain at least 400 mm from the hazard
```

If the robot needs longer to stop, the curtain must be further away, or the robot must be slowed down. Safety design is a specialised discipline: always follow a formal risk assessment and the standards, and involve a qualified safety engineer for real machines.

25.4 Ethics and society

25.4.1 Robots and jobs

Robots take over tasks that are dull, dirty, dangerous or demand very high precision. History shows automation both removes some jobs and creates others (robot technicians, programmers, integrators, maintenance), and raises productivity. The difficulty is that the people who lose jobs are not always the people who gain the new ones. Responses include retraining and upskilling, designing robots to **assist** workers rather than replace them, and policies that share the gains. Engineers should consider who is affected by what they build.

25.4.2 Safety, privacy and responsibility

Issue	Questions to ask
Safety	What can go wrong, how likely, how bad? Is there a safe state? Has it been tested beyond the happy path?
Privacy	Do the robot's cameras and microphones record people? Is data stored, shared, deleted? Did people consent?
Security	Can someone take control remotely? Are updates signed?

Issue	Questions to ask
Responsibility	If a robot causes harm, who is accountable: designer, manufacturer, operator, owner? Logs help answer this.
Fairness and bias	Does a vision system work equally well for everyone (skin tones, ages, wheelchairs)?
Environment	Energy use, batteries, repairability and recycling.

25.4.3 Autonomous systems and decision-making

As robots decide more by themselves (self-driving cars, delivery robots, drones), people must still be able to **understand**, **oversee** and **override** them. Good practice: keep a human in or on the loop for consequential decisions, make behaviour predictable and explainable, log decisions, test widely including rare situations, and be honest about limitations. Lethal autonomous weapons are the subject of international debate about meaningful human control. Professional codes (IEEE, ACM) and laws such as the EU AI Act, along with India's emerging AI governance guidelines, set expectations for responsible design.

PRACTICE CHECKPOINT

Write a simple ladder-logic program in a PLC simulator for a conveyor, and present a short report on the ethical and social impact of one robotics application.

25.5 Practice questions

25.5.1 Basic level

Q1 Which company makes the orange industrial robots programmed in KRL?

(a) FANUC (b) KUKA (c) ABB (d) Yaskawa

Q2 Which language is used to program ABB robots?

(a) KRL (b) INFORM (c) Ladder only (d) RAPID

Q3 What is a teach pendant?

(a) a type of gripper
(b) a safety fence
(c) a hand-held terminal to jog, teach and program a robot
(d) a PLC module

Q4 What is a cobot?

(a) a robot that only cooks
(b) a PLC
(c) a robot designed to work safely near people
(d) a conveyor

Q5 What does a PLC do in each scan cycle?

(a) prints labels
(b) charges batteries
(c) renders 3-D graphics
(d) reads inputs, executes the program, writes outputs

Q6 Which IEC 61131-3 language looks like relay wiring?

(a) Structured Text (b) Ladder Diagram
(c) Python (d) G-code

Q7 What is the colour scheme of an emergency stop button?

(a) red mushroom button on a yellow background
(b) green on white
(c) blue on black
(d) any colour

Q8 What does an HMI provide?

(a) robot arm torque
(b) a screen for operators to see status and control a machine
(c) Wi-Fi coverage
(d) filament

25.5.2 Intermediate level

Q9 Which robot move gives a straight-line tool path?

(a) linear move (MoveL / LIN)
(b) joint move (MoveJ / PTP)
(c) wait instruction
(d) I/O instruction

Q10 What is the TCP of a robot?

(a) the communication protocol only
(b) the teach pendant case
(c) the tool centre point, the point programmed positions refer to
(d) the robot base

Q11 Why is a STOP button wired normally closed?

(a) it uses less power
(b) it is cheaper
(c) it looks better
(d) a broken wire then stops the machine (fail-safe)

Q12 What does the seal-in contact in a start/stop rung do?

(a) stops the motor
(b) counts parts
(c) measures speed
(d) keeps the motor on after START is released

Q13 Which cobot collaboration mode slows and stops the robot as a person approaches?

(a) hand guiding
(b) monitored stop only
(c) speed and separation monitoring
(d) offline programming

Q14 What is a digital twin?

(a) a live virtual model of a machine or process used for simulation and monitoring
(b) a second identical robot
(c) a backup battery
(d) a printed copy of the manual

Q15 Which standard covers industrial robot safety?

(a) ISO 9001
(b) IEEE 802.11
(c) IEC 61131-3
(d) ISO 10218

Q16 Which protocol connects industrial machines to IT systems securely and is vendor-neutral?

(a) HDMI
(b) OPC UA
(c) I2C
(d) NMEA

Q17 What is offline programming?

(a) programming with the robot unplugged
(b) creating and simulating robot programs in software before loading them
(c) writing code on paper
(d) teach pendant jogging

25.5.3 Advanced level

Q18 Availability 90 %, performance 95 %, quality 98 %. What is the OEE?

(a) about 94.3 %
(b) about 90 %
(c) about 75 %
(d) about 83.8 %

Q19 With K = 1600 mm/s and total stopping time 0.4 s (C = 0), what is the minimum safety distance?

(a) 640 mm **(b)** 400 mm **(c)** 1600 mm **(d)** 64 mm

Q20 Simulate a 2-input PLC interlock: a press may cycle only when both hand buttons are pressed within 0.5 s of each other and the guard is closed.

Q21 Count parts on a conveyor with a PLC-style counter: a sensor gives a list of readings each scan; count rising edges only and switch on a "box full" output at 5 parts.

Q22 Compute a robot cell's parts per hour if each cycle takes 4.2 s of motion, 0.6 s of gripper action and 0.4 s waiting for the conveyor, and how much faster it is if waiting is removed.

Q23 List which ISO 10218 collaboration mode fits each task: a person loads parts while the robot waits; a worker guides a heavy load by hand; a cobot screws parts beside a worker; a robot slows as people pass by.

Q24 A company plans to replace a team of packers with robots. Which approach is most responsible?

(a) announce it on the day of installation

(b) ignore the workers' concerns

(c) plan retraining and redeployment of workers, involve them early, and use robots for the hardest tasks

(d) avoid any risk assessment

Q25 A delivery robot records video on public footpaths. What is the best privacy practice?

(a) store all video forever

(b) share the video publicly

(c) hide the camera

(d) blur faces, keep only what is needed for a short time, and publish a clear notice

25.6 Solutions

Q1 **Answer: (b) KUKA**

KUKA Robot Language.

Q2 **Answer: (d) RAPID**

ABB RAPID.

Q3 **Answer: (c) a hand-held terminal to jog, teach and program a robot**

With E-stop and enabling switch.

Q4 **Answer: (c) a robot designed to work safely near people**

Collaborative robot.

Q5 **Answer: (d) reads inputs, executes the program, writes outputs**

Scan cycle.

Q6 **Answer: (b) Ladder Diagram**

Contacts and coils.

Q7 **Answer: (a) red mushroom button on a yellow background**

Standardised.

Q8 **Answer: (b) a screen for operators to see status and control a machine**

Human-machine interface.

Q9 **Answer: (a) linear move (MoveL / LIN)**

Linear interpolation.

Q10 **Answer: (c) the tool centre point, the point programmed positions refer to**

Tool frame origin.

Q11 **Answer: (d) a broken wire then stops the machine (fail-safe)**

Fail-safe.

Q12 **Answer: (d) keeps the motor on after START is released**

Latching.

Q13 **Answer: (c) speed and separation monitoring**

Scanner zones.

Q14 **Answer: (a) a live virtual model of a machine or process used for simulation and monitoring**

Industry 4.0.

Q15 **Answer: (d) ISO 10218**

Robots and cells.

Q16 **Answer: (b) OPC UA**

Industrial interoperability.

Q17 **Answer: (b) creating and simulating robot programs in software before loading them**

RobotStudio, RoboDK.

Q18 **Answer: (d) about 83.8 %**

0.90 × 0.95 × 0.98 = **0.838**.

Q19 **Answer: (a) 640 mm**

1600 × 0.4 = **640 mm**.

Q20 **Answer: Solution program**

Two-hand controls must be pressed nearly together so an operator cannot tape one down and reach in with the other hand; the guard interlock adds a second condition.

```
events = [  # (left press time, right press time, guard closed?)
    (0.00, 0.20, True), (1.00, 1.80, True), (3.00, 3.10, False), (5.00, 5.45, True)]
for left, right, guard in events:
    ok = abs(left - right) <= 0.5 and guard
    reason = "" if ok else (" (guard open)" if not guard else " (not simultaneous)")
    print(f"left {left:.2f}s right {right:.2f}s guard {'closed' if guard else 'open'} -> {'CYCLE' if
ok else 'BLOCKED'}{reason}")
```

OUTPUT

```
left 0.00s right 0.20s guard closed -> CYCLE
left 1.00s right 1.80s guard closed -> BLOCKED (not simultaneous)
left 3.00s right 3.10s guard open -> BLOCKED (guard open)
left 5.00s right 5.45s guard closed -> CYCLE
```

Q21 **Answer: Solution program**

A part stays in front of the sensor for several scans; counting rising edges (one-shot, like a PLC P-contact) counts each part once.

```
sensor = [0, 1, 1, 0, 0, 1, 0, 1, 1, 1, 0, 1, 0, 0, 1, 1, 0]
count, prev, full = 0, 0, False
for scan, s in enumerate(sensor):
    if s and not prev:                  # rising edge = a new part
        count += 1
        print(f"scan {scan:2d}: part {count}")
    prev = s
    if count >= 5 and not full:
        full = True; print(f"scan {scan:2d}: BOX FULL -> stop conveyor")
```

OUTPUT

```
scan  1: part 1
scan  5: part 2
scan  7: part 3
scan 11: part 4
scan 14: part 5
scan 14: BOX FULL -> stop conveyor
```

Q22 **Answer: Solution program**

Cycle-time analysis shows where to improve: here removing the wait adds 58 parts per hour without touching the robot's speed.

```
motion, grip, wait = 4.2, 0.6, 0.4
cycle = motion + grip + wait
print(f"cycle {cycle:.1f} s -> {3600 / cycle:.0f} parts/hour")
faster = 3600 / (motion + grip)
print(f"without waiting: {faster:.0f} parts/hour (+{faster - 3600 / cycle:.0f})")
```

OUTPUT

```
cycle 5.2 s -> 692 parts/hour
without waiting: 750 parts/hour (+58)
```

Q23 **Answer: Solution program**

The four modes can be combined in one cell; each still needs a risk assessment.

```
tasks = {"robot waits while operator loads": "safety-rated monitored stop",
         "operator moves heavy load by hand": "hand guiding",
         "cobot screwdriving beside worker": "power and force limiting",
         "robot slows as people walk past": "speed and separation monitoring"}
for task, mode in tasks.items():
    print(f"{task:36} -> {mode}")
```

OUTPUT

```
robot waits while operator loads     -> safety-rated monitored stop
operator moves heavy load by hand    -> hand guiding
cobot screwdriving beside worker     -> power and force limiting
robot slows as people walk past      -> speed and separation monitoring
```

Q24 **Answer: (c) plan retraining and redeployment of workers, involve them early, and use robots for the hardest tasks**

Ethics and people.

Q25 **Answer: (d) blur faces, keep only what is needed for a short time, and publish a clear notice**

Data minimisation and transparency.

Manufacturers: ABB (RAPID), FANUC (TP/KAREL), KUKA (KRL), Yaskawa (INFORM). Teach pendant with enabling switch; online, lead-through and offline programming; MoveJ vs MoveL, TCP, zones. Cobots: monitored stop, hand guiding, speed and separation, power and force limiting. PLC scan cycle, ladder contacts/coils, seal-in, timers, NC stops; HMI, SCADA, OPC UA; Industry 4.0, digital twins, OEE. Safety: E-stops, light curtains, scanners, safety PLCs, ISO 10218, ISO 13849, S = K·T + C. Ethics: jobs, safety, privacy, accountability, human oversight.

MODULE 26 · ADVANCED

Competitions, capstone projects and careers

Minimum duration **2 days** | Level **Advanced**

The final module brings everything together in a capstone robot and prepares learners for competitions and careers. It covers line-following, maze, robo-soccer, robo-race and robo-war events and the major programmes (FIRST, WRO, Robocon, e-Yantra, Smart India Hackathon), six capstone projects with architectures, parts and milestones, how to document a project with diagrams, repositories, videos and reports, robotics careers, higher studies and portfolios, and the next steps with Arduino and Raspberry Pi, followed by 25 solved questions and programs.

IN THIS MODULE

26.1 Robotics competitions
26.2 Capstone projects
26.3 Project documentation
26.4 Careers in robotics
26.5 Next steps
26.6 Practice questions
26.7 Solutions

26.1 Robotics competitions

Competitions turn learning into deadlines, teamwork and real testing under pressure. Every event publishes a **rulebook**: read it first, because arena size, robot size and weight limits, power rules and scoring decide the design.

26.1.1 Line-following and maze competitions

Event	Task	Winning ideas
Line follower	follow a black (or white) line with curves, gaps and junctions in the shortest time	IR array (8+ sensors), PID on line position, light chassis, low centre of mass, tuned speed per section
Grid / junction follower	count junctions and reach targets on a grid	reliable junction detection, encoders for turns
Maze solver / micromouse	explore an unknown maze, then run start → centre as fast as possible	wall sensors (IR/ToF), flood-fill algorithm, gyro-assisted turns, save the map
Line maze	learn a line maze on the first run, run the shortest path on the second	left-hand rule while recording turns, then simplify the path

FLOOD_FILL.PY

```
from collections import deque
# 0 = open, 1 = wall cell; find shortest distances to the goal with flood fill (BFS from the goal)
MAZE = ["S.#......",
        ".##.####.",
        "....#....",
        ".##.#.##.",
        "....#..#G"]
H, W = len(MAZE), len(MAZE[0])
goal = next((r, c) for r in range(H) for c in range(W) if MAZE[r][c] == "G")
dist = [[None] * W for _ in range(H)]
dist[goal[0]][goal[1]] = 0
q = deque([goal])
while q:
    r, c = q.popleft()
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        nr, nc = r + dr, c + dc
        if 0 <= nr < H and 0 <= nc < W and MAZE[nr][nc] != "#" and dist[nr][nc] is None:
            dist[nr][nc] = dist[r][c] + 1
            q.append((nr, nc))
for r in range(H):
    print(" ".join(" #" if MAZE[r][c] == "#" else f"{dist[r][c]:2d}" for c in range(W)))
# the robot always moves to the neighbour with the smallest number
pos, path = (0, 0), []
while pos != goal:
    r, c = pos
    pos = min(((r + dr, c + dc) for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1))
               if 0 <= r + dr < H and 0 <= c + dc < W and dist[r + dr][c + dc] is not None),
              key=lambda p: dist[p[0]][p[1]])
    path.append(pos)
print("steps from S to G:", len(path))
```

OUTPUT

```
16 17  #  9  8  7  6  5  4
15  #  # 10  #  #  #  #  3
14 13 12 11  #  5  4  3  2
15  #  # 12  #  6  #  #  1
16 15 14 13  #  7  8  #  0
steps from S to G: 16
```

Flood fill numbers every cell with its distance to the goal; the robot simply rolls "downhill" to smaller numbers. A micromouse updates the numbers every time it discovers a new wall, so it always heads along the best known route.

26.1.2 Robo-soccer, robo-race and robo-wars

Event	What it tests	Tips
Robo-soccer	manual or autonomous robots push or kick a ball into goals	holonomic (omni/mecanum) drive, ball-catching front, fast reliable radio link
Robo-race / terrain	drive a rough track with obstacles, ramps, water, sand	torque over speed, ground clearance, good traction, robust wiring

Event	What it tests	Tips
Robo-wars	remote-controlled combat robots in a weight class	strict safety rules, armour, weapon type (wedge, flipper, spinner), LiPo safety, failsafe

WATCH OUT
Combat and high-power robots can be dangerous. Follow the event's safety rules exactly: removable link or master switch, weapon locks, failsafe on signal loss, and LiPo handling in fire-safe bags.

26.1.3 FIRST, WRO, Robocon, e-Yantra, Smart India Hackathon

Programme	Who	Format
FIRST (LEGO League, Tech Challenge, Robotics Competition)	school students (age bands from ~9 to 18)	season-long team challenges, engineering notebook, core values
WRO (World Robot Olympiad)	school and early university students	robot missions, future innovators, robo-sports; national rounds lead to the international final
Robocon (ABU Robocon; DD Robocon in India)	university engineering teams	a new game every year, built by student teams; national winner represents India
e-Yantra Robotics Competition (IIT Bombay)	engineering students and colleges	themed tasks with kits and online stages; also lab and teacher programmes
Smart India Hackathon	students (hardware and software editions)	solve problem statements from ministries and industry in a non-stop hackathon
College tech fests (Techfest IIT Bombay, Shaastra IIT Madras, etc.)	school and college students	line follower, maze, robo-wars, drone events

Dates, age limits and themes change every year: check each programme's official website for the current season.

26.2 Capstone projects

A capstone combines sensing, actuation, control, software and documentation in one working robot. Choose one that excites you, define a **minimum viable robot** first, then add features. Each project below lists its architecture and milestones.

26.2.1 Autonomous delivery robot with navigation

Part	Choice
Brain	Raspberry Pi 5 (ROS 2, Nav2) + microcontroller for motors and encoders
Sensors	2-D LiDAR, wheel encoders, IMU, bumpers, camera (optional ArUco docking)
Milestones	teleop → odometry → SLAM map → Nav2 goal → multi-stop delivery → auto-docking

Part	Choice
Demo	deliver an item between three rooms on command from a web page

26.2.2 Pick-and-place robotic arm with vision

Part	Choice
Arm	4–6 DOF servo arm with PCA9685, or a stepper arm
Vision	Pi camera above the table, OpenCV colour/ArUco detection, pixel → table coordinates calibration
Milestones	servo control → FK/IK → camera calibration → detect objects → pick by colour → sort into bins
Demo	sort mixed coloured blocks into three bins with no human help

26.2.3 Smart agriculture robot with IoT monitoring

Part	Choice
Platform	rugged 4WD or tracked chassis, ESP32 or Pi
Sensors	soil moisture probe on a servo arm, DHT22/BME280, GPS outdoors
IoT	MQTT to a Node-RED or ThingSpeak dashboard; pump control
Demo	drive along crop rows, measure soil at stops, show a moisture map, water dry spots

26.2.4 Self-balancing robot with PID

A two-wheeled robot balances like an inverted pendulum: the IMU measures the tilt (complementary or Kalman filter), and a PID loop drives the wheels under the falling body. It is the classic test of control skills.

BALANCE_SIM.PY

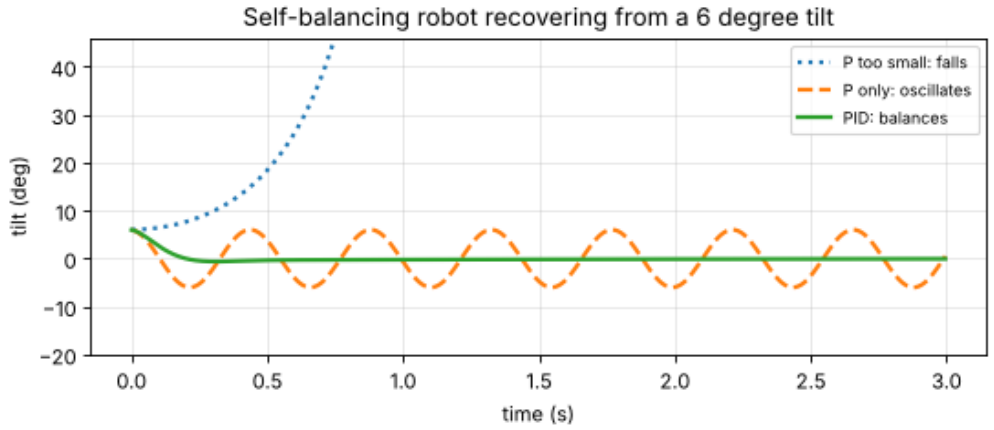
```
import math
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

g, l, dt = 9.81, 0.15, 0.005                    # gravity, pendulum length (m), 200 Hz loop
def run(kp, ki, kd, t_end=3.0):
    th, w, integ, prev, out = math.radians(6), 0.0, 0.0, None, []     # starts tilted 6 degrees
    for k in range(int(t_end / dt)):
        err = th                                 # tilt forward -> drive the wheels forward under
it
        integ += err * dt
        der = 0 if prev is None else (err - prev) / dt
        prev = err
        a = max(-12, min(12, kp * err + ki * integ + kd * der))          # wheel acceleration limit
(m/s^2)
        alpha = (g * math.sin(th) - a * math.cos(th)) / l                  # pendulum on an
accelerating cart
        w += alpha * dt; th += w * dt
        out.append(math.degrees(th))
        if abs(th) > math.radians(45): break                               # fell over
    return out

fig, ax = plt.subplots(figsize=(7, 3.2))
for gains, style, label in [((8, 0, 0), ":", "P too small: falls"), ((40, 0, 0), "--", "P only:
oscillates"), ((40, 20, 3), "-", "PID: balances")]:
    y = run(*gains)
    ax.plot([i * dt for i in range(len(y))], y, style, lw=2, label=label)
    fate = "fell over" if abs(y[-1]) > 40 else f"after 3 s: {y[-1]:+.2f} deg"
    print(f"{label:20} {fate}")
ax.set_ylim(-20, 46); ax.set_xlabel("time (s)"); ax.set_ylabel("tilt (deg)"); ax.grid(alpha=0.3);
ax.legend(fontsize=8)
ax.set_title("Self-balancing robot recovering from a 6 degree tilt"); fig.tight_layout();
fig.savefig("balance.png", dpi=80)
```

OUTPUT

```
P too small: falls   fell over
P only: oscillates   after 3 s: +0.52 deg
PID: balances        after 3 s: -0.06 deg
```



SIMULATED TILT ANGLE FOR THREE SETS OF GAINS

A gain that is too small cannot catch the fall; P alone overshoots and rocks back and forth; adding D damps the motion and a little I removes the leftover lean. Tune on the real robot in the same order (P, then D, then I) with the wheels free to move and a hand ready to catch it.

26.2.5 Voice-controlled assistant robot

Part	Choice
Brain	Raspberry Pi 5 with USB microphone and speaker
Speech	Vosk or Whisper offline recognition, text-to-speech (Piper, espeak-ng)
Skills	drive commands, face following with OpenCV, answering questions, reminders
Demo	"Come here", "follow me", "what is the temperature?", with a safe "stop" at any time

26.2.6 ROS 2 robot with mapping and navigation

Part	Choice
Stack	ROS 2 Jazzy, micro-ROS or serial bridge to the motor controller, slam_toolbox, Nav2, RViz
Model	URDF with correct wheel and LiDAR positions, TF tree verified
Milestones	Gazebo simulation → real teleop → map the lab → navigate to goals → waypoint patrol
Demo	show the same launch files working in simulation and on the real robot

BOM.PY

```
# bill of materials for a capstone robot: total cost and the most expensive items
bom = [("Raspberry Pi 5 4GB", 1, 6000), ("Arduino Nano", 1, 450), ("TB6612FNG driver", 1, 250),
       ("N20 gear motor with encoder", 2, 550), ("RPLIDAR C1", 1, 7500), ("MPU6050 IMU", 1, 150),
       ("18650 cells", 3, 250), ("3S BMS + holder", 1, 350), ("5 V 3 A buck converter", 1, 200),
       ("wheels, caster, screws", 1, 400), ("PLA filament (share)", 1, 300)]
total = sum(q * p for _, q, p in bom)
print(f"{'item':30} qty   cost")
for name, q, p in sorted(bom, key=lambda x: -x[1] * x[2]):
    print(f"{name:30} {q:3d}  {q * p:6,d}")
print(f"{'TOTAL':30}      {total:6,d}  (prices in rupees, approximate)")
print(f"LiDAR + Pi = {(7500 + 6000) / total:.0%} of the budget")
```

OUTPUT

```
item                           qty   cost
RPLIDAR C1                       1   7,500
Raspberry Pi 5 4GB               1   6,000
N20 gear motor with encoder      2   1,100
18650 cells                      3     750
Arduino Nano                     1     450
wheels, caster, screws           1     400
3S BMS + holder                  1     350
PLA filament (share)             1     300
TB6612FNG driver                 1     250
5 V 3 A buck converter           1     200
MPU6050 IMU                      1     150
TOTAL                               17,450  (prices in rupees, approximate)
LiDAR + Pi = 77% of the budget
```

A bill of materials shows where the money goes. Prices change often and vary by seller, so treat these as placeholders and update them from current quotes. Here the LiDAR and the Pi dominate, so a simulation-first plan avoids buying them before the design is proven.

26.3 Project documentation

26.3.1 Circuit diagrams, code repositories, videos and reports

Item	What good looks like
Circuit diagram	a clear schematic (KiCad, EasyEDA or Fritzing) with pin numbers, power ratings and a photo of the real wiring
Code repository	GitHub/GitLab repo with README, folder structure, licence, commits with clear messages, tagged releases
README	what it does, photo/GIF, parts list, wiring, setup steps, how to run, results, known issues
Demo video	1–3 minutes: the problem, the robot working, a close-up of key parts, numbers (speed, accuracy)
Report	problem, requirements, design choices and alternatives, testing data, what failed and why, future work

Item	What good looks like
Engineering notebook	dated log of decisions, sketches and test results (required in FIRST)

README.MD (EXAMPLE STRUCTURE)

```
# Line-Following Robot "Swift"

Fast PID line follower built for the college tech fest (lap time 11.8 s on the 10 m track).

![robot](docs/robot.jpg)

## Hardware
| Part | Qty |
|---|---|
| Arduino Nano | 1 |
| 8-channel IR array (QTR-8A) | 1 |
| N20 1000 rpm motors + TB6612FNG | 2 + 1 |
| 2S 850 mAh LiPo + 5 V regulator | 1 |

Wiring diagram: `docs/schematic.pdf`

## Software
- `firmware/` Arduino sketch (PID on line position, Kp = 0.08, Kd = 1.2)
- `tools/plot_log.py` plots the error log from the serial port

## Results
| Version | Lap time | Change |
|---|---|---|
| v1 | 19.4 s | P only |
| v2 | 13.1 s | added D, lighter chassis |
| v3 | 11.8 s | speed boost on straights |

## Lessons learned
Sensor height of 3 mm gave the cleanest readings; above 8 mm the array could not see the line.
```

RECORDING PROGRESS WITH GIT

```
$ git init && git add . && git commit -m "First working line follower (P control)"
[main (root-commit) 3f2a91c] First working line follower (P control)
 6 files changed, 214 insertions(+)
$ git commit -am "Add D term: lap time 19.4 s -> 13.1 s"
[main 8b7d4e0] Add D term: lap time 19.4 s -> 13.1 s
 1 file changed, 9 insertions(+), 3 deletions(-)
$ git tag v2 && git push origin main --tags
```

26.4 Careers in robotics

26.4.1 Robotics engineer, embedded engineer, automation engineer, computer vision engineer, drone pilot and engineer

Role	What they do	Key skills
Robotics engineer	design, build and integrate complete robots	mechanics, electronics, ROS 2, control, testing
Embedded engineer	firmware for microcontrollers and real-time control	C/C++, RTOS, communication buses, debugging hardware
Automation engineer	factory automation: PLCs, robot cells, SCADA	ladder logic, industrial robots, safety standards, drives
Computer vision / AI engineer	perception: detection, tracking, SLAM, learning	Python, OpenCV, deep learning, maths, data
Drone pilot / engineer	fly survey, inspection or spraying missions; build and maintain drones	DGCA Remote Pilot Certificate, ArduPilot/PX4, GIS
Mechanical / design engineer	CAD, mechanisms, prototypes, manufacturing	CAD, materials, 3D printing, machining
Field / service engineer	install and maintain robots at customer sites	troubleshooting, electrical safety, communication
Robotics educator / entrepreneur	teach, run labs, start a product company	communication, projects, business

26.4.2 Higher studies and research

Degrees that lead into robotics: B.E./B.Tech in mechatronics, robotics and automation, electronics, electrical, mechanical or computer science; diplomas in mechatronics or automation for hands-on roles; M.Tech/M.S. specialising in robotics, control, AI or embedded systems; and PhD research in labs at IITs, IISc and universities abroad. Research areas include legged and humanoid robots, medical and surgical robots, swarm robotics, soft robotics, human-robot interaction, field robotics for agriculture and disaster response, and robot learning.

26.4.3 Building a robotics portfolio

- Three to five well-documented projects beat twenty half-finished ones: show the problem, your design choices, test results and what you learned.
- A GitHub profile with clean repositories, READMEs and commit history.
- Short demo videos (YouTube or LinkedIn) and a one-page portfolio website.
- Competition results, hackathons, internships and open-source contributions (ROS packages, Arduino libraries).
- Certificates and courses add context, but working robots are the strongest evidence.
- Be ready to explain any design in an interview: why this motor, why this controller, what would you change?

26.5 Next steps

26.5.1 Arduino and Raspberry Pi in depth

This course used Arduino for real-time control and the Raspberry Pi for vision, networking and ROS 2. The companion LBF courses **Arduino Complete** and **Raspberry Pi Complete** go much deeper into each platform: every sensor and module, interrupts and timers, power management, Linux, Python, networking, computer vision and IoT. Keep building: pick a capstone, set a demo date, and iterate.

PRACTICE CHECKPOINT

Design, build and demonstrate one capstone robot, document it fully on GitHub with a demo video, and present it as you would in a competition or interview.

26.6 Practice questions

26.6.1 Basic level

Q1 What should you read first before designing a competition robot?

(a) a random forum post
(b) the official rulebook
(c) last year's photos only
(d) nothing

Q2 Which algorithm do micromouse robots commonly use to find the route to the centre?

(a) flood fill
(b) bubble sort
(c) Canny
(d) FFT

Q3 Which university-level competition sets a new game every year and has a national round in India?

(a) FIRST LEGO League
(b) Tinkercad Challenge
(c) Robocon
(d) Wokwi Cup

Q4 Which robotics programme is run by IIT Bombay?

(a) e-Yantra
(b) WRO
(c) FIRST
(d) RoboCup

Q5 What is a minimum viable robot?

(a) the cheapest robot in the shop
(b) the simplest version that does the core task, built first
(c) a robot without code
(d) the final product

Q6 What should a project README contain?

(a) only the licence
(b) only photos
(c) passwords
(d) what it does, parts, wiring, setup, how to run and results

Q7 Which career focuses on PLCs, robot cells and SCADA?

(a) computer vision engineer
(b) drone pilot
(c) automation engineer
(d) web designer

Q8 What does a drone pilot in India need for commercial drones above 2 kg?

(a) a car licence
(b) a passport only
(c) a DGCA Remote Pilot Certificate
(d) nothing

26.6.2 Intermediate level

Q9 Which sensor arrangement suits a fast line follower?

(a) one LDR and a delay
(b) GPS
(c) a microphone
(d) an IR sensor array with PID on the line position

Q10 Which drive is common in robo-soccer for moving sideways quickly?

(a) tank tracks only **(b)** omni or mecanum wheels
(c) a single wheel **(d)** legs

Q11 What is the classic control challenge of a self-balancing robot?

(a) stabilising an inverted pendulum with PID
(b) following a line
(c) path planning
(d) MQTT messaging

Q12 Why record lap times or accuracy for each version of a robot?

(a) to fill space
(b) to prove improvements with data and guide the next change
(c) rules forbid it
(d) it is not useful

Q13 What makes a robotics portfolio strongest?

(a) a few complete, documented projects with videos and results
(b) many unfinished projects
(c) certificates only
(d) a long list of buzzwords

Q14 Which safety feature is essential on combat robots?

(a) bright paint
(b) extra stickers
(c) a removable link or master switch and failsafe on signal loss
(d) louder motors

Q15 In the line-maze challenge, how is the second run made faster?

(a) by driving randomly
(b) by removing sensors
(c) by using a bigger battery only
(d) the turns recorded on the first run are simplified into the shortest path

Q16 Which tool records the history of your code and lets you return to earlier versions?

(a) KiCad **(b)** Cura **(c)** Node-RED **(d)** git

Q17 Which competition invites students to solve problem statements from ministries and industry?

(a) Robocon **(b)** WRO
(c) Smart India Hackathon **(d)** micromouse

26.6.3 Advanced level

Q18 A line follower lap takes 11.8 s on a 10 m track. What is its average speed?

(a) about 0.85 m/s **(b)** about 1.18 m/s **(c)** about 0.12 m/s **(d)** about 8.5 m/s

Q19 A BOM totals ₹17 000 and the LiDAR costs ₹7500. What share is the LiDAR?

(a) about 7.5 % (b) about 17 % (c) about 75 % (d) about 44 %

Q20 Simplify a line-maze path recorded with the left-hand rule (L, R, S = straight, B = U-turn at a dead end) using the rules LBR → B, LBS → R, RBL → B, SBL → R, SBS → B, LBL → S.

Q21 Plan a 6-week capstone schedule: print each milestone with its start and end week, given durations of 1, 1, 2, 1 and 1 weeks.

Q22 Score a line-follower run: base score 100, minus 5 per second over 12 s, minus 10 per line loss, plus 20 bonus for finishing with no line loss.

Q23 Count the robotics skills a job ad asks for that are already in your portfolio, and list the gaps.

Q24 Your team's robot works on the practice track but fails under the bright lights of the venue. What is the best preparation?

(a) change nothing
(b) test and calibrate sensors under different lighting and add shielding or auto-calibration
(c) paint the track
(d) add a bigger battery

Q25 In an interview you are asked about a failed design. What is the best answer?

(a) say nothing ever failed
(b) explain what failed, how you found the cause with tests, and what you changed
(c) blame a teammate
(d) change the subject

26.7 Solutions

Q1 **Answer: (b) the official rulebook**
Rules decide the design.

Q2 **Answer: (a) flood fill**
Distance numbers.

Q3 **Answer: (c) Robocon**
ABU / DD Robocon.

Q4 **Answer: (a) e-Yantra**
e-Yantra.

Q5 **Answer: (b) the simplest version that does the core task, built first**
Iterate from it.

Q6 **Answer: (d) what it does, parts, wiring, setup, how to run and results**
Reproducibility.

Q7 **Answer: (c) automation engineer**

Factory automation.

Q8 **Answer: (c) a DGCA Remote Pilot Certificate**

RPC.

Q9 **Answer: (d) an IR sensor array with PID on the line position**

Line position error.

Q10 **Answer: (b) omni or mecanum wheels**

Holonomic.

Q11 **Answer: (a) stabilising an inverted pendulum with PID**

Unstable system.

Q12 **Answer: (b) to prove improvements with data and guide the next change**

Evidence.

Q13 **Answer: (a) a few complete, documented projects with videos and results**

Show working robots.

Q14 **Answer: (c) a removable link or master switch and failsafe on signal loss**

Safety rules.

Q15 **Answer: (d) the turns recorded on the first run are simplified into the shortest path**

Path simplification.

Q16 **Answer: (d) git**

Version control.

Q17 **Answer: (c) Smart India Hackathon**

SIH.

Q18 **Answer: (a) about 0.85 m/s**

10 ÷ 11.8 = **0.85 m/s**.

Q19 **Answer: (d) about 44 %**

7500 ÷ 17 000 = **0.44**.

Q20 **Answer: Solution program**

Each dead end (B) with the turns around it is replaced by the single turn that skips it; the second run follows the short path.

```
RULES = {"LBR": "B", "LBS": "R", "RBL": "B", "SBL": "R", "SBS": "B", "LBL": "S"}
def simplify(path):
    out = ""
    for move in path:
        out += move
        while len(out) >= 3 and out[-2] == "B" and out[-3:] in RULES:
            out = out[:-3] + RULES[out[-3:]]
    return out
recorded = "LBLLBSRSLBLL"
print("recorded :", recorded, f"({len(recorded)} turns)")
print("shortest :", simplify(recorded), f"({len(simplify(recorded))} turns)")
```

OUTPUT

```
recorded : LBLLBSRSLBLL (12 turns)
shortest : SRRSSL (6 turns)
```

Q21 **Answer: Solution program**

Fixed milestones with dates keep a team on track; leave the last week for testing and documentation, which always take longer than expected.

```
milestones = [("requirements + parts", 1), ("chassis + motors", 1), ("sensors + control", 2),
("integration + testing", 1), ("documentation + video", 1)]
week = 1
for name, d in milestones:
    weeks = f"week {week}" if d == 1 else f"weeks {week}-{week + d - 1}"
    print(f"{weeks:10} {name}")
    week += d
print("demo at the end of week", week - 1)
```

OUTPUT

```
week 1     requirements + parts
week 2     chassis + motors
weeks 3-4  sensors + control
week 5     integration + testing
week 6     documentation + video
demo at the end of week 6
```

Q22 **Answer: Solution program**

Knowing the scoring formula shows the trade-off: here staying on the line is worth more than an extra second of speed.

```
def score(lap_s, losses):
    s = 100 - max(0, lap_s - 12) * 5 - losses * 10
    if losses == 0: s += 20
    return round(s, 1)
for lap, loss in [(11.8, 0), (13.4, 1), (12.0, 2)]:
    print(f"lap {lap:4.1f} s, {loss} losses -> score {score(lap, loss)}")
```

OUTPUT

```
lap 11.8 s, 0 losses -> score 120
lap 13.4 s, 1 losses -> score 83.0
lap 12.0 s, 2 losses -> score 80
```

Q23 **Answer: Solution program**

A simple gap analysis turns a job description into a learning plan: here ROS 2, SLAM and C++ are the next projects to add.

```
job = {"C++", "ROS 2", "Python", "OpenCV", "PID", "Git", "SLAM", "CAD"}
mine = {"Python", "Arduino", "PID", "OpenCV", "Git", "3D printing", "CAD"}
print("matched:", sorted(job & mine), f"({len(job & mine)}/{len(job)})")
print("to learn:", sorted(job - mine))
```

OUTPUT

```
matched: ['CAD', 'Git', 'OpenCV', 'PID', 'Python'] (5/8)
to learn: ['C++', 'ROS 2', 'SLAM']
```

Q24 **Answer: (b) test and calibrate sensors under different lighting and add shielding or auto-calibration**

Expect the venue to differ.

Q25 **Answer: (b) explain what failed, how you found the cause with tests, and what you changed**

Show engineering thinking.

Competitions: read the rulebook; line followers (IR array + PID), maze (flood fill, path simplification), robo-soccer, races, robo-wars (safety). Programmes: FIRST, WRO, Robocon, e-Yantra, SIH, tech fests. Capstones: delivery robot, vision arm, agri-IoT robot, self-balancer, voice assistant, ROS 2 navigator; start with a minimum viable robot, BOM, milestones. Document: schematic, git repo, README, video, report. Careers: robotics, embedded, automation, vision/AI, drones, design; portfolio of finished, documented robots.